

THESE DE DOCTORAT DE L'UNIVERSITE PARIS 6

Spécialité: Informatique

Présentée par:

Pierre Roy

Pour obtenir le grade de

DOCTEUR DE L'UNIVERSITE PARIS 6

SATISFACTION DE CONTRAINTES ET PROGRAMMATION PAR OBJETS

Soutenue le 21 décembre 1998

Devant le jury composé de :

Jean-Paul Allouche, examinateur
Frédéric Benhamou, rapporteur
Philippe Codognet, examinateur
Roland Ducournau, rapporteur,
François Pachet, directeur
Jean-François Perrot, président,
Jean-François Puget, examinateur

TABLE DES MATIERES

INTRODUCTION	8
---------------------------	----------

<u>PREMIÈRE PARTIE UN BREF HISTORIQUE DE LA SATISFACTION DE CONTRAINTES.....</u>	<u>11</u>
---	------------------

CHAPITRE 1 APPROCHE ALGORITHMIQUE : LES CSPS	13
1.1 LE FORMALISME DES CSPS DE MACKWORTH.....	14
1.2 LE LANGAGE ALICE DE JEAN-LOUIS LAURIÈRE	15
1.3 DIRECTIONS DE RECHERCHE SUR LES CSPS.....	18
CHAPITRE 2 INTÉGRATION À DES LANGAGES DE PROGRAMMATION	22
2.1 PROGRAMMATION LOGIQUE AVEC CONTRAINTES	22
2.2 INTÉGRATION DANS DES LANGAGES CLASSIQUES.....	24

<u>DEUXIÈME PARTIE LES ASPECTS TECHNIQUES.....</u>	<u>26</u>
---	------------------

CHAPITRE 1 QU'EST-CE QU'UN PROBLÈME DE SATISFACTION DE CONTRAINTES ?	28
1.1 LE PROBLÈME DES REINES SUR L'ÉCHIQUIER.....	29
1.2 LES CRYPTOGRAMMES	30
1.3 LES PUZZLES NUMÉRIQUES ET LOGIQUES	32
1.4 PLACEMENT EN 2D — LES CARRÉS PARFAITS.....	34
CHAPITRE 2 LE FORMALISME DES CSPS BINAIRES DE MACKWORTH	37
2.1 DÉFINITION	37
2.2 L'EXEMPLE DU PROBLÈME DES REINES.....	39
2.3 CSP EST NP-COMPLET	40
CHAPITRE 3 RÉOLUTIONS DES CSPS BINAIRES.....	42
3.1 L'ÉNUMÉRATION EXPLICITE	42
3.2 LA COHÉRENCE D'ARC	45
3.3 LA COHÉRENCE DE CHEMIN	48
3.4 LES ALGORITHMES DE RÉOLUTION.....	48
CHAPITRE 4 DÉFINITION D'UN FORMALISME PLUS RICHE.....	53
4.1 LES LIMITATIONS DU FORMALISME DE BASE	53
4.2 DÉFINITION D'UN FORMALISME PLUS RICHE.....	55
4.3 EXEMPLES DE PROBLÈMES DÉFINIS AVEC DES CONTRAINTES NON BINAIRES.....	56
CHAPITRE 5 TECHNIQUES DE RÉOLUTION DES PROBLÈMES NON BINAIRES	58
5.1 LIMITATIONS DES PREMIÈRES TECHNIQUES DE RÉOLUTION.....	58
5.2 TECHNIQUES DE RÉOLUTION POUR LES PROBLÈMES NON BINAIRES	63

<u>TROISIÈME PARTIE LES OBJETS POUR IMPLÉMENTER LES CONTRAINTES</u>	<u>68</u>
--	------------------

CHAPITRE 1 QUELQUES SYSTÈMES DE CONTRAINTES IMPLÉMENTÉS AVEC DES OBJETS	69
1.1 LE MODÈLE RÉACTIF — LA PROPAGATION LOCALE DE CONTRAINTES	69
1.2 LES BIBLIOTHÈQUES DE CLASSES.....	71
1.3 LANGAGES HYBRIDES	74
CHAPITRE 2 LE CAHIER DES CHARGES DE BACKTALK.....	76
CHAPITRE 3 LES CLASSES DE BASES DE BACKTALK	78
3.1 LES DOMAINES ET LES VARIABLES.....	78

3.2	LES CLASSES DE CONTRAINTES	82
3.3	LES CLASSES D'ALGORITHMES	88
3.4	LES HEURISTIQUES	93
CHAPITRE 4 IMPLÉMENTATION DU MÉCANISME DE FILTRAGE EN BACKTALK : LES DÉMONS		98
4.1	UNE APPROCHE RÉACTIVE DU FILTRAGE DES CONTRAINTES : LES <i>DÉMONS</i>	99
4.2	DÉFINITION <i>MODULAIRE</i> ET <i>INCRÉMENTALE</i> DES MÉTHODES DE FILTRAGE	100
4.3	DÉFINITION PARTIELLE DES MÉTHODES DE FILTRAGE	108
4.4	IMPLÉMENTATION DES MÉCANISMES DE PROPAGATION DE QUELQUES CONTRAINTES	109
CHAPITRE 5 TECHNIQUES AVANCÉES.....		114
5.1	MÉTA HEURISTIQUES	114
5.2	CONTRÔLE ET TRAÇAGE DE LA RÉOLUTION	117
5.3	CORRECTION ET COMPLÉTUDE.....	121
5.4	LES PROBLÈMES D'OPTIMISATION.....	125
5.5	RAISONNEMENT FORMEL	126
5.6	GESTION DES SYMÉTRIES	127
5.7	DÉFINITION ET RÉOLUTION DE QUELQUES PROBLÈMES EN BACKTALK	139
5.8	PERFORMANCES	144
CHAPITRE 6 CONCLUSION		147
 <u>QUATRIÈME PARTIE LES PROBLÈMES DE CONTRAINTES ET OBJETS.....</u>		<u>148</u>
 CHAPITRE 1 DEUX APPROCHES DIFFÉRENTES		150
1.1	L'APPROCHE PAR ATTRIBUTS	150
1.2	APPROCHE PAR CLASSES	151
1.3	ILLUSTRATION DES DEUX APPROCHES	152
CHAPITRE 2 LANGAGE DE DÉFINITION DES CONTRAINTES.....		155
2.1	LES CONTRAINTES NUMÉRIQUES ET LOGIQUES.....	156
2.2	LES CONTRAINTES SUR LES OBJETS.....	156
 <u>CINQUIÈME PARTIE DEUX APPLICATIONS.....</u>		<u>166</u>
 CHAPITRE 1 LA CRÉATION DE GRILLES DE MOTS CROISÉS.....		167
1.1	LE PROBLÈME.....	167
1.2	LES APPROCHES PRÉCÉDENTES	168
1.3	NOTRE APPROCHE	170
1.4	LA VALEUR DE L'APPROCHE PAR CONTRAINTES.....	187
CHAPITRE 2 L'HARMONISATION MUSICALE AUTOMATIQUE.....		188
2.1	LE SYSTÈME DE PHILIPPE BALLESTA	190
2.2	LA REPRÉSENTATION DES OBJETS MUSICAUX EN MUSES.....	194
2.3	L'HARMONISATION AUTOMATIQUE AVEC MUSES ET BACKTALK : L'APPROCHE PAR CLASSES	196
 <u>CONCLUSION.....</u>		<u>223</u>
 <u>BIBLIOGRAPHIE</u>		<u>226</u>

TABLE DES FIGURES

FIGURE 1 UNE SOLUTION AU PROBLÈME DES HUIT REINES	29
FIGURE 2 UNE SOLUTION DU PROBLÈME DES QUATRE REINES	30
FIGURE 3 DES CARRÉS MAGIQUES D'ORDRE 5, 6 ET 7 TROUVÉS AVEC NOTRE SYSTÈME (BACKTALK)	33
FIGURE 4 UNE SÉQUENCE MAGIQUE D'ORDRE 10	33
FIGURE 5 UN GRAPHE MAGIQUE D'ORDRE 10	34
FIGURE 6 UNE SOLUTION POUR CHACUNE DES QUATRE INSTANCES PRÉCÉDENTES DU PROBLÈME DES CARRÉS PARFAITS	36
FIGURE 7 RÉOLUTION DU PROBLÈME DES QUATRE REINES AVEC L'ALGORITHME DE RETOUR ARRIÈRE CHRONOLOGIQUE. L'ARBRE EST DÉVELOPPÉ DE HAUT EN BAS ET DE GAUCHE À DROITE	44
FIGURE 8 L'ALGORITHME DE REAL FULL LOOKAHEAD	49
FIGURE 9 L'ALGORITHME DE FORWARD CHECKING.....	50
FIGURE 10 EN HAUT, L'ARBRE DE RECHERCHE DÉVELOPPÉ PAR LE REAL FULL LOOK-AHEAD POUR LA RÉSOLUTION DES QUATRE REINES. EN BAS,.....	50
FIGURE 11 L'ARBRE DÉVELOPPÉ PAR LE FORWARD-CHECKING POUR LA RÉOLUTION DU PROBLÈME DES QUATRE REINES.....	50
FIGURE 12 CLASSIFICATION DES ALGORITHMES D'ÉNUMÉRATION POUR LES CSPs BINAIRES. LES ALGORITHMES SONT RANGÉS DE GAUCHE À DROITE SELON L'UTILISATION DES TECHNIQUES DE RETOUR ARRIÈRE "INTELLIGENT". DE HAUT EN BAS, ILS SONT RANGÉS SELON L'UTILISATION DES TECHNIQUES DE RÉDUCTION PAR COHÉRENCE D'ARC	52
FIGURE 13 UN EXEMPLE DE SYSTÈME DE PROPAGATION LOCALE: DEUX JAUGES DONNANT LA TEMPÉRATURE EN DEGRÉS CELSIUS ET FAHRENHEIT. L'UTILISATEUR PEUT AGIR SUR L'UNE OU L'AUTRE DES JAUGES POUR EN MODIFIER LA VALEUR PROVOQUANT UNE MODIFICATION DE L'AUTRE JAUGE PAR LE SYSTÈME AFIN DE MAINTENIR LA RELATION $1,8 \times C = F - 32$	70
FIGURE 14 LE SCÉMA ABSTRAIT DE LA FAMILLES DES ALGORITHMES COMPLETS DE RÉOLUTION DES PROBLÈMES DE SATISFACTION DE CONTRAINTES	89
FIGURE 15 LE DESIGN PATTERN STRATEGY	90
FIGURE 16 LE DESIGN PATTERN TEMPLATE METHOD.....	90
FIGURE 17 LES CLASSES DE VARIABLES, DE CONTRAINTES ET DE DÉMONS, ET LEUR RELATIONS, DANS LE SYSTÈME BACKTALK.....	107
FIGURE 18 AFFICHAGE DYNAMIQUE DES VALEURS DES VARIABLES À TROIS ÉTAPES SUCCESSIVES DE LA RÉSOLUTION DU PROBLÈME DU CARRÉ MAGIQUE D'ORDRE 5. LES POINTS D'INTERROGATION REPRÉSENTENT LES VARIABLES N'AYANT PAS DE VALEUR, C'EST-À-DIRE DONT LE DOMAINE N'A PAS ÉTÉ RÉDUIT À UN SINGLETON	117
FIGURE 19 L'ARBRE DE RECHERCHE CORRESPONDANT À LA RÉOLUTION D'UN CARRÉ MAGIQUE D'ORDRE 4	121
FIGURE 20 L'ARBRE DE RECHERCHE DÉVELOPPÉ POUR LA RÉOLUTION DU PROBLÈME DES 5 PIGEONS DANS SA FORMULATION AVEC DES CONTRAINTES BINAIRES. IL Y A EXACTEMENT 23 BRANCHES, SOIT $4! - 1$	128
FIGURE 21 LE PROBLÈME DE RAMSEY POUR LE GRAPHE COMPLET À QUATRE SOMMETS (K_4).....	132
FIGURE 22 LA MODIFICATION DE LA MÉTHODE BACKWARD POUR L'EXPLOITATION DES RELATIONS DE PERMUTABILITÉ ENTRE VARIABLES	134
FIGURE 23 LA RÉOLUTION DU PROBLÈME DES QUATRE PIGEONS. LES BRANCHES EN POINTILLÉS SONT ÉCONOMISÉES LORS D'UNE RÉOLUTION UTILISANT LA MÉTHODE BACKWARD MODIFIÉE AFIN DE PRENDRE EN COMPTE LES RELATIONS DE PERMUTABILITÉ ENTRE VARIABLES. LES ZONES HACHURÉES E ET F SONT IMAGES L'UNE DE L'AUTRE PAR LA TRANSPOSITION τ_{12} QUI CORRESPOND À DEUX VARIABLES PERMUTABLES. AINSI, APRÈS AVOIR EXPLORÉ E QUI N'A CONDUIT À AUCUNE SOLUTION, ON PEUT ÉCONOMISER L'EXPLORATION DE F.....	134
FIGURE 24 UNE INSTANCE DU PROBLÈME DE DÉCOUPE DE RECTANGLES. LES PLANCHES AYANT LES MÊMES DIMENSIONS SONT PERMUTABLES EN COMPRÉHENSION	135
FIGURE 25 LA STRUCTURE DU PROBLÈME DE DÉCOUPE OPTIMALE. LES VARIABLES P_i REPRÉSENTENT LES POSITIONS DES PLANCHES. LES VARIABLES H_i REPRÉSENTENT LES HAUTEURS ATTEINTES PAR LES RECTANGLES ($H_i = P_i + \text{HAUTEUR}(i)$). LA VARIABLE H DÉSIGNE LA HAUTEUR MAXIMALE	

ATTEINTE ($H = \max (H_i)$). LES CERCLES (SANS FLÈCHE) REPRÉSENTENT LES CONTRAINTES SYMÉTRIQUES, ALORS QUE LES FLÈCHES REPRÉSENTENT LES CONTRAINTES NON SYMÉTRIQUES.	136
FIGURE 26 LA MODIFICATION DE LA BOUCLE DE RÉOLUTION POUR PRENDRE EN COMPTE LES RELATIONS DE PERMUTABILITÉ EN COMPRÉHENSION ENTRE STRUCTURES COMPOSITES.....	138
FIGURE 27 LE PONT À CONSTRUIRE	141
FIGURE 28 ILLUSTRATION DE L'APPROCHE PAR ATTRIBUTS. LES POINTS D'INTERROGATION CERCLES REPRÉSENTENT LES ATTRIBUTS DES OBJETS COMPOSITES, QUI SONT ÉGALEMENT LES VARIABLES D'INSTANCE DU PROBLÈME. LES FLÈCHES SIMPLES REPRÉSENTENT LES CONTRAINTES DE STRUCTURES, QUI PORTENT SUR PLUSIEURS ATTRIBUTS D'UN MÊME OBJET COMPOSITE, ET SERVENT À DÉFINIR SA STRUCTURE. LES FLÈCHES DOUBLES SONT LES CONTRAINTES DU PROBLÈME PROPREMENT DITES. ELLES PORTENT ENTRE LES ATTRIBUTS D'OBJETS COMPOSITES DIFFÉRENTS	151
FIGURE 29 ILLUSTRATION GRAPHIQUE DE L'APPROCHE PAR ATTRIBUTS. LES POINTS D'INTERROGATION CERCLES REPRÉSENTENT LES VARIABLES CONTRAINTES DU PROBLÈME. LA SEULE CONTRAINTES PORTE SUR LES DEUX VARIABLES CONTRAINTES DONT LES DOMAINES CONTIENNENT DES OBJETS STANDARDS (I.E. TOTALEMENT INSTANCIÉS).	152
FIGURE 30 LE PROBLÈME DES RECTANGLES. ICI, (R1, R3) ET (R2, R3) SONT DES SOLUTIONS, ALORS QUE LA PAIRE (R2, R3) N'EN EST PAS UNE.....	152
FIGURE 31 LES RECTANGLES REPRÉSENTÉS PAR LEURS SOMMETS RESPECTIFS (APPROCHE PAR ATTRIBUTS).....	153
FIGURE 32 LE CSP CORRESPONDANT AU PROBLÈME DES DEUX RECTANGLES EXPRIMÉ À L'AIDE DE CONTRAINTES DE TYPE PERFORM. LA CONTRAINTES EXPRIMANT LA RELATION ENTRE LES SURFACES DES DEUX RECTANGLES EST ICI UNE CONTRAINTES ARITHMÉTIQUE ENTRE LES VARIABLES NUMÉRIQUES REPRÉSENTANT LES AIRES.	165
FIGURE 33 UNE GRILLE DIFFICILE CAR ELLE CONTIENT PEU DE CASES NOIRES (6 CASE NOIRES POUR UNE GRILLE DE TAILLE 6x6). SOLUTION TROUVÉE AVEC 77 ÉCHECS EN 1 MINUTE ET 20 SECONDES. LE DICTIONNAIRE EST LE UK ADVANCED CRYPTIC DICTIONARY	183
FIGURE 34 UNE HARMONISATION À QUATRE VOIX DE "LA MARSEILLAISE". CETTE HARMONISATION A ÉTÉ OBTENUE PAR NOTRE APPLICATION EN 0,5 SECONDES ET 30 ÉCHECS SUR PENTIUM 300 MHZ	189
FIGURE 35 LA STRUCTURE D'UNE NOTE DANS LE SYSTÈME DE PHILIPPE BALLESTA. LES CONTRAINTES C_1 ET C_2 SONT DES CONTRAINTES STRUCTURELLES QUI GARANTISSENT LA COHÉRENCE ENTRE LES VALEURS DES DIFFÉRENTS ATTRIBUTS CONSTITUANT LA NOTE.....	191
FIGURE 36 LA DÉCLARATION D'UNE VARIABLE NUMÉRIQUE POUR CHAQUE VARIABLE SYMBOLIQUE. LES DEUX VARIABLES SONT RELIÉES ENTRE ELLES PAR UNE CONTRAINTES DE TYPE CT-ELEMENT QUI ASSURE QU'ELLES SONT ÉQUIVALENTES	193
FIGURE 37 LA STRUCTURE SIMPLIFIÉE DU PROBLÈME D'HARMONISATION AUTOMATIQUE DANS L'APPROCHE PAR CLASSES. IL Y A DEUX CATÉGORIES DE VARIABLES. LES VARIABLES ENCEINTEES SONT CELLES DONT LE DOMAINE CONTIENT DES NOTES, ALORS QUE LES VARIABLES ENCADRÉES CONTIENNENT DES ACCORDS. LES RÈGLES D'HARMONIES EXPRIMANT DES RELATIONS ENTRE NOTES SONT DÉFINIES PAR DES CONTRAINTES POSÉES ENTRE LES VARIABLES DE LA PREMIÈRE CATÉGORIE, ALORS QUE LES RÈGLES CONCERNANT LES ACCORDS SONT DÉFINIES PAR DES CONTRAINTES ENTRE LES VARIABLES DE LA SECONDE CATÉGORIE	198
FIGURE 38 LA MÉLODIE PROPOSÉE PAR TSANG ET AITKEN	199
FIGURE 39 LA STRUCTURE DÉFINIE POUR CHAQUE NOTE DE LA MÉLODIE À HARMONISER. CETTE STRUCTURE COMPREND LES QUATRE VARIABLES NOTES, CORRESPONDANT À CHACUNE DES QUATRE VOIX, AINSI QU'UNE VARIABLE REPRÉSENTANT L'ACCORD ET DEUX VARIABLES ADDITIONNELLES POUR LE DEGRÉ ET LA POSITION. CES DIFFÉRENTES VARIABLES SONT RELIÉES À L'ACCORD POUR DES CONTRAINTES DE TYPE BTPERFORMCT	200
FIGURE 40 À GAUCHE, UNE QUINTE DIRECTE INTERDITE, À DROITE, UNE OCTAVE DIRECTE INTERDITE.	208
FIGURE 41 LA BASSE CHIFFRÉE PROPOSÉE PAR PHILIPPE BALLESTA	212
FIGURE 42 NOTRE PREMIÈRE SOLUTION POUR LA BASSE CHIFFRÉE DE PHILIPPE BALLESTA	212
FIGURE 43 LA PREMIÈRE SOLUTION DE PHILIPPE BALLESTA	213
FIGURE 44 LA SECONDE SOLUTION OBTENUE	213
FIGURE 45 LA SOLUTION DE TSANG ET AITKEN, DONT LES CHIFFRAGES SONT : I, V, I, VI, II, VI, II, V, I, IV, I, IV, V, I	214
FIGURE 46 NOTRE SOLUTION, DONT LES CHIFFRAGES (I, V, I, VI, IV, I, V, VII, V, VII, I, IV, V, I) SONT DIFFÉRENTS, CE QUI EST NÉCESSAIRE PUISQUE NOUS INTERDISONS, PAR EXEMPLES, L'UTILISATION DE L'ACCORD DU SECOND DEGRÉ S'IL N'EST PAS SUIVI D'UN ACCORD DE DOMINANTE ÉVENTUELLEMENT PRÉCÉDÉ D'UN ACCORD DE DEGRÉ IV.....	214

FIGURE 47 UNE SOLUTION, TROUVÉE PAR NOTRE SYSTÈME, EN HARMONISANT LA MÉLODIE DE TSANG ET AITKEN, ET EN IMPOSANT LES DEGRÉS DES ACCORDS. LA SOLUTION TROUVÉE N'EST PAS EXACTEMENT LA MÊME QUE CELLE DE TSANG ET AITKEN.....	215
FIGURE 48 UNE AUTRE SOLUTION POUR LA MÉLODIE DE LA MÉLODIE DE TSANG, OBTENUE AVEC LES DEGRÉS DE TSANG, ET EN AJOUTANT UNE CONTRAINTE POUR LA CONDUITE DES VOIX INTERMÉDIAIRES : ON INTERDIT LES SAUTS PLUS GRANDS QUE LA TIERCE MAJEURE AU TÉNOR ET À L'ALTO.....	215
FIGURE 49 LA PREMIÈRE PHRASE DU CHORAL DE LA CANTATE B.W.V. 67 DE JEAN-SÉBASTIEN BACH. ON REMARQUERA PARTICULIÈREMENT L'ALLURE GÉNÉRALE DE LA LIGNE DE BASSE, AINSI QUE CELLE DE LA VOIX DE SOPRANO, QUI SONT QUASI SYSTÉMATIQUEMENT EN MOUVEMENT CONTRAIRE ...	216
FIGURE 50 UNE PREMIÈRE HARMONISATION DE CETTE BASSE.....	216
FIGURE 51 UNE HARMONISATION DE CETTE BASSE EN IMPOSANT LE DESSIN GÉNÉRAL DE LA VOIX DE SOPRANO	217
FIGURE 52 UNE AUTRE HARMONISATION DE CETTE BASSE AVEC LE MÊME DESSIN POUR LA VOIX DE SOPRANO ET AVEC LES MÊME DEGRÉS QUE L'ORIGINAL DE BACH.....	217
FIGURE 53 SOLUTION OBTENUE AVEC LA CONTRAINTE GLOBALE SUR LE DESSIN DE LA MÉLODIE.....	218
FIGURE 54 LA PREMIÈRE PHRASE DE LA MARSEILLAISE	220
FIGURE 55 UNE PREMIÈRE HARMONISATION DE LA MARSEILLAISE. ON PEUT REMARQUER QUE LA CONDUITE DES VOIX INTERMÉDIAIRES N'EST PAS TRÈS SATISFAISANTE.....	220
FIGURE 56 LA SOLUTION OPTIMALE POUR L'HARMONISATION DE LA MARSEILLAISE AVEC MINIMISATION DES SAUTS AUX VOIX INTERMÉDIAIRES. CETTE SOLUTION EST OBTENUE EN ENVIRON 7 MINUTES, ET SON OPTIMALITÉ EST PROUVÉE EN 15 MINUTES ENVIRON SUR PENTIUM 300 MHZ	221
FIGURE 57 UNE GRILLE DE BIFFE TOUT CONTENANT LES MOTS: AAAI AI ALARM ARE ARS BRA CAR CARE ERA KO LAC LACK OK PAIN REAM RIB SCARE SON SPAIN ET QUELQUES AUTRES	225
TABEAU 1 LES DIFFÉRENCES ESSENTIELLES ENTRE L'APPROCHE TRADITIONNELLE DES CSP ET L'APPROCHE IMPLÉMENTÉE DANS LE SYSTÈME ALICE	16
TABEAU 2 DOMAINES DES VARIABLES AVANT ET APRÈS APPLICATION D'UN ALGORITHME D'ARC COHÉRENCE	61
TABEAU 3 NOMBRE DE CONTRAINTES RENDUES ARC COHÉRENTES LORS DE LA RÉOLUTION D'INSTANCES DU PROBLÈME DES REINES ET DU PROBLÈME DES CARRÉS MAGIQUES	62
TABEAU 4 LES TEMPS DE RÉOLUTION ET LE NOMBRE DE BACKTRACKS POUR LE CALCUL DE LA PREMIÈRE SOLUTION, PUIS LA PREUVE DE L'UNICITÉ, DE LA MULTIPLICATION MAGIQUE. LA RÉOLUTION A ÉTÉ FAITE DANS UN PREMIER TEMPS AVEC L'HEURISTIQUE DE CHOIX DE LA PROCHAINE VARIABLE À INSTANCIER SUIVANT LE PRINCIPE DU PLUS PETIT DOMAINE D'ABORD, APPLIQUÉ À TOUTES LES VARIABLES. DANS UN SECOND TEMPS, LA RÉOLUTION UTILISE L'HEURISTIQUE CONSISTANT À SÉLECTIONNER D'ABORD LES SIX PREMIÈRES VARIABLES, ET ENSUITE LES 14 VARIABLES RESTANTES, ET EN APPLIQUANT ÉGALEMENT LE CRITÈRE DU PLUS PETIT DOMAINE D'ABORD. L'OBTENTION DE LA PREMIÈRE SOLUTION EST DIX FOIS PLUS RAPIDE AVEC CETTE DERNIÈRE STRATÉGIE. EN REVANCHE LA PREUVE DE L'UNICITÉ EST PLUS RAPIDE AVEC LA PREMIÈRE STRATÉGIE.	97
TABEAU 5 TYPOLOGIE DES RÉDUCTIONS DE DOMAINES POSSIBLES LORS DE LA RÉOLUTION D'UN PROBLÈME DE SATISFACTION DE CONTRAINTES	101
TABEAU 6 LA RÉOLUTION DE LA MULTIPLICATION MAGIQUE EN UTILISANT L'HEURISTIQUE QUI CHOISIT LA PROCHAINE VARIABLE À INSTANCIER PARMI LES SIX PREMIÈRES, JUSQU'À OBTENTION DE LA PREMIÈRE SOLUTION, ET ENSUITE CHOISIT LES VARIABLES SUIVANT LE SEUL CRITÈRE DE LA TAILLE DU DOMAINE. CES DONNÉES SONT À COMPARER À CELLES DU TABLEAU 4	114
TABEAU 7 LE TEMPS DE RÉOLUTION DU PROBLÈME DE DÉCOUPE OPTIMALE AVEC ET SANS L'EXPLOITATION DES SYMÉTRIES	138
TABEAU 8 LA COMPARAISON DES PERFORMANCES ENTRE BACKTALK ET CLP (FD) POUR DES PROBLÈMES NUMÉRIQUES CONNUS. LA COMPARAISON EST À L'AVANTAGE DE CLP (FD) QUI EST PLUS RAPIDE QUE BACKTALK D'UN FACTEUR COMPRIS ENTRE 1 ET 10	146
TABEAU 9 LE COÛT DE LA REPRÉSENTATION DANS LE SYSTÈME DE PHILIPPE BALLESTA	192
TABEAU 10 COMPARAISON DES TEMPS DE RÉOLUTIONS ET DU NOMBRE D'ÉCHECS DANS LE SYSTÈME DE PHILIPPE BALLESTA ET LE NÔTRE POUR UN MÊME EXERCICE. LA MACHINE DE BALLESTA EST ENVIRON 8 FOIS PLUS LENTE. *POUR LA MÉLODIE DE 12 NOTES, ON NE CALCULE QUE LES 520 PREMIÈRES SOLUTIONS	219
TABEAU 11 SYNTHÈSE DES DIFFÉRENCES MAJEURES ENTRE QUATRE SYSTÈMES D'HARMONISATION AUTOMATIQUE BASÉS SUR LA PROGRAMMATION PAR CONTRAINTES	222

INTRODUCTION

La *satisfaction de contraintes* (à domaines finis)¹ est un formalisme pour la définition de problèmes combinatoires associé à un ensemble de techniques de résolution performantes. C'est, à ce jour, un des outils les plus performants dont on dispose. La satisfaction de contraintes a notamment permis de résoudre pour la première fois certains problèmes ouverts [Caseau and Laburthe, 1995]. Discipline algorithmique à sa naissance [Mackworth, 1977], la satisfaction de contraintes a été ensuite intégrée à des langages de programmation logique pour donner naissance au paradigme de la *programmation logique avec contraintes* [Jaffar and Lassez, 1987]. Cette intégration a été motivée par le double désir de disposer d'un langage pour la définition des contraintes et d'étendre la déclarativité des langages de logique à des domaines de calcul particuliers comme les nombres entiers. Les langages de logique offrent toutes les structures nécessaires à une programmation des contraintes tout en étant dotés des mécanismes permettant l'implémentation des techniques de résolution. Plus précisément, la notion de variable logique est compatible avec la notion de variable utilisée en satisfaction de contraintes et le mécanisme de résolution des langages de logique est le même que celui des algorithmes de résolution des problèmes de contraintes. D'une certaine manière, la satisfaction de contraintes peut être considérée comme une extension naturelle de l'unification des langages de logique.

Les techniques de satisfaction de contraintes sont maintenant bien maîtrisées et on trouve de nombreux langages ou bibliothèques logicielles proposant des outils efficaces pour la résolution de problèmes numériques. En revanche, la résolution de problèmes impliquant des structures de données complexes n'a pas reçu une attention aussi importante et il existe encore peu d'outils adaptés. En conséquence, lorsque l'on est confronté à un tel problème on se contente en général de l'adapter à l'outil dont on dispose. En d'autres termes, on a recours à une formalisation en termes mathé-

¹ Dans tout ce document par *satisfaction de contraintes*, nous désignerons la *satisfaction de contraintes à domaines finis*.

matiques ou arithmétiques afin de pouvoir faire rentrer le problème dans le moule des systèmes de satisfaction de contraintes. Cette démarche induit un certain nombre de biais sur la représentation du problème et sur sa résolution. Premièrement, cela requiert un travail de formalisation qui est souvent très difficile à mener à bien, surtout quand on sait l'importance de la formalisation sur les performances de la résolution [Fron, 1994]. Deuxièmement, l'aplanissement des structures rend extrêmement difficile l'expression de certaines relations entre les objets du problème [Roy and Pachet, 1997a]. Troisièmement, cela pose un problème de génie logiciel concernant la capacité à réutiliser pour la définition d'un problème des structures logicielles existantes. Finalement, cette approche visant à permettre l'utilisation de techniques efficaces de résolution de contraintes numériques ne conduit pas nécessairement à de bonnes performances de résolution.

Ces quelques remarques montrent l'intérêt d'un paradigme dans lequel on puisse définir et résoudre des problèmes structurés sans avoir recours à une formalisation *ad hoc*. La thèse soutenue dans ce document est que l'intégration des techniques de satisfaction de contraintes dans un langage de *programmation par objets* permet d'apporter une solution satisfaisante à ces problèmes. L'idée est que la combinaison des contraintes et des objets permet d'obtenir un paradigme cumulant leurs avantages respectifs : les objets permettent de représenter et d'organiser des structures complexes ; les contraintes apportent des techniques performantes d'exploration combinatoire.

La première partie de notre travail concerne les aspects techniques. Nous y discuterons de l'implémentation des techniques de satisfaction de contraintes dans un langage à objets, qui pose des difficultés majeures. En effet, afin de réaliser l'intégration des contraintes et des objets il faut réussir à concilier deux visions radicalement différentes de la programmation. À l'instar de la satisfaction de contraintes, l'exécution d'un programme logique consiste à donner des valeurs à des variables par un mécanisme de résolution. En revanche, un langage à objets est un langage impératif dont les programmes sont des suites d'instructions à effectuer selon un ordre défini par les structures de contrôle habituelles. En outre, les variables d'un tel programme ne sont pas manipulées comme des variables contraintes. Elles ont en permanence une valeur qui peut changer de façon arbitraire au cours de l'exécution du programme. Pour ces raisons, l'intégration des contraintes et des objets n'est pas aussi naturelle que celle des contraintes avec les langages de logique. Les systèmes de satisfaction de contraintes implémentés dans des langages à objets se présentent sous la forme de bibliothèques de classes représentant les concepts du domaine (i.e. les problèmes, les contraintes et les variables contraintes) et qui implémentent des algorithmes de résolution. L'utilisateur peut ainsi définir des problèmes de contraintes et en déclencher la résolution. Cette approche conduit à des systèmes "intellectuellement" moins satisfaisants que les langages de programmation logique avec contraintes. Cependant, elle offre certains avantages de conception. En effet, les mécanismes d'héritage de classe et de polymorphisme des objets permettent de définir des architectures logicielles bien adaptés à l'implémentation d'un système de contraintes. Nous approfondirons ce propos avec la présentation de notre système de satisfaction de contraintes, appelé BACKTALK [Roy and Pachet, 1997b].

La seconde partie de notre travail concerne des aspects méthodologiques. Nous y aborderons l'utilisation des techniques de contraintes pour la résolution de problèmes combinatoires structurés. Nous avons souligné plus haut que l'approche de type programmation logique avec contraintes implique une formalisation préalable difficile et pouvant avoir des conséquences importantes sur la résolution. Nous

montrons que l'intégration des contraintes avec les objets permet de s'affranchir de cette formalisation. Plus précisément, nous introduirons une méthodologie qui permet de préserver la déclarativité des contraintes tout en améliorant les performances de la résolution. Cette méthodologie sera illustrée par une application à l'harmonisation musicale automatique.

Ce document comporte les parties suivantes. Dans un premier temps, nous présentons un bref historique de la satisfaction de contraintes depuis ses origines jusqu'aux recherches en cours actuellement. Ensuite, nous abordons la discipline dans ses aspects techniques ce qui nous permet d'introduire les notions nécessaires à la présentation de notre contribution. Les deux parties suivantes sont consacrées à l'étude de l'intégration des contraintes avec les objets. La première traite les aspects techniques concernant l'implémentation des techniques de contraintes dans un langage à objet, en l'occurrence SMALLTALK [INGALLS, 1978]. Nous détaillerons l'implémentation du BACKTALK et nous discuterons des avantages de son architecture. Cette partie sera illustrée par la présentation d'une application à la construction de grilles de mots croisés. La seconde aborde les aspects méthodologiques. Une nouvelle approche pour la résolution de problèmes de combinatoires structurés par la satisfaction de contraintes est présentée. Cette approche sera illustrée par une implémentation en BACKTALK d'une application à l'harmonisation musicale automatique.

PREMIÈRE PARTIE

UN BREF HISTORIQUE DE LA SATISFACTION DE CONTRAINTES

La satisfaction de contraintes est née dans les années 1970 des travaux de Montanari [Montanari, 1974], Waltz [Waltz, 1975] et surtout Mackworth [Mackworth, 1977]. Ces travaux ont conduit à un formalisme, dont la description va suivre, pour une large famille de problèmes combinatoires : les CSPs, pour “*Constraint Satisfaction Problems*”, et à la mise au point de techniques efficaces de résolution : les procédés de cohérences d’arc et de chemin. Néanmoins, le langage ALICE de Jean-Louis Laurière [Laurière, 1978], légèrement antérieur, donnait déjà une implémentation particulière de la satisfaction de contraintes, partageant le même formalisme mais utilisant des mécanismes de résolution différents et plus sophistiqués.

La satisfaction de contraintes est rapidement devenue une discipline importante de la résolution de problèmes combinatoires. C’est en effet le seul formalisme offrant, en plus d’une approche *déclarative*, des *performances excellentes*, qui la distinguent de la programmation logique pure, et une *grande souplesse*, qui est un avantage par rapport aux approches de recherche opérationnelles. La satisfaction de contraintes est un paradigme déclaratif puisqu’elle repose sur une séparation totale de la définition du problème et de sa résolution. En d’autres termes, l’utilisateur définit un problème à l’aide d’un langage particulier, celui des contraintes, et le système se charge de le résoudre sans aucune intervention extérieure. La satisfaction de contraintes est déclarative car il est possible de définir des méthodes efficaces et universelles pour la gestion d’un grand nombre de types de contraintes. On peut alors constituer des bibliothèques de contraintes suffisamment vastes pour permettre de résoudre un grand nombre de problèmes sans avoir besoin d’implémenter des méth-

odes spécifiques de résolution. La satisfaction de contraintes est en outre un paradigme *souple* au sens où les méthodes de résolutions ne sont pas dédiées à des types de problèmes appartenant à un domaine d'application particulier. Il est ainsi possible de résoudre des problèmes non "purs" pour lesquels les algorithmes spécialisés de recherche opérationnelle ne peuvent pas s'appliquer directement.

Après avoir été considérée, dans un premier temps, comme une branche de l'algorithmique, la satisfaction de contraintes a été intégrée dans divers langages de programmation. Pour des raisons techniques essentielles, les premières intégrations ont été réalisées dans le cadre de la programmation logique ce qui a donné naissance à la *programmation logique avec contraintes* ou *CLP (Constraint Logic Programming)*. Plus tard, les techniques de satisfaction de contraintes ont été intégrées dans d'autres langages, notamment des langages à objets.

Les recherches actuelles concernent l'amélioration des algorithmes, dans la tradition de Mackworth, mais également, à l'instar de cette thèse, les questions d'intégration aux langages de programmation.

Cette partie comporte deux chapitres : le premier présente la vision algorithmique des contraintes depuis les premiers travaux jusqu'aux recherches actuelles ; le second traite de l'intégration des contraintes dans les langages de programmation, depuis le paradigme de la programmation logique avec contraintes jusqu'aux systèmes plus récemment intégrés dans des langages de programmation classiques.

Chapitre 1

Approche algorithmique : les CSPs

La satisfaction de contraintes est, à l'origine, une discipline purement algorithmique, motivée par les recherches de Montanari [Montanari, 1974] et Waltz [Waltz, 1975] concernant le traitement et l'analyse d'images. À la fin des années 70, ces travaux ont conduit Alan Mackworth [Mackworth, 1977] à définir le formalisme des “*Constraint Satisfaction Problems*” ou *CSP* et un ensemble de techniques de résolution fondées sur l'exploitation de propriétés locales du problème appelées *cohérence d'arc* et *cohérence de chemin*. À l'heure actuelle, ce formalisme et ces techniques sont toujours au centre des systèmes de satisfaction de contraintes.

On peut cependant considérer que les bases de la satisfaction de contraintes ont été jetées quelques années auparavant avec le langage déclaratif ALICE [Laurière, 1978] de Jean-Louis Laurière. ALICE est un précurseur des systèmes de satisfaction de contraintes actuels dans la mesure où il résout des problèmes similaires aux CSPs à l'aide de techniques utilisant des propriétés locales du problème proches de la cohérence d'arc et de la cohérence de chemin. À ce titre, on peut remarquer que l'algorithme AC4 de Mohr et Henderson [Mohr and Henderson, 1986] est une mise en œuvre, dans le contexte des CSPs, de certaines des techniques exploitées par ALICE. Il existe néanmoins des différences profondes entre l'approche de Laurière et celle de Mackworth, notamment dans la mise en œuvre des techniques de cohérence pour lesquelles ALICE utilise le raisonnement formel alors que Mackworth propose des techniques d'exploration combinatoires. En outre, les stratégies de résolution d'ALICE s'adaptent dynamiquement au problème à résoudre ce qui n'est le cas d'aucune approche de type CSP.

Le formalisme et les techniques des CSPs sont au cœur de la plupart des recherches actuelles en satisfaction de contraintes. On peut distinguer quatre axes principaux de recherche : les techniques de cohérence, les algorithmes complets d'énumération, les heuristiques ou méta-heuristiques de résolution et les extensions du formalisme et des techniques des CSP. Les travaux de Laurière n'ont, en revanche, pas suscité autant de développements ces dernières années. La raison est probablement que les mécanismes de résolution d'ALICE reposent sur des techniques de rai-

sonnement formel, notamment la *réécriture* de contraintes, encore mal maîtrisées aujourd'hui et délicates à mettre en œuvre pour des raisons pratiques et théoriques. On peut néanmoins citer les travaux ultérieurs de Laurière lui-même [Laurière, 1996] et les travaux actuels d'Anne Liret [Liret, et al., 1997a, Liret, et al., 1997b] concernant des extensions d'ALICE aux nombres réels et aux objets.

Dans ce chapitre, nous présentons en premier lieu le formalisme des CSPs défini par Mackworth, ainsi que les techniques de cohérence utilisées pour leur résolution. Nous présentons ensuite le langage ALICE. Finalement, nous présentons quelques-unes des recherches actuelles consacrées aux aspects algorithmiques de la satisfaction de contraintes.

1.1 LE FORMALISME DES CSPs DE MACKWORTH

En 1977, Mackworth a défini le formalisme des CSP, pour "*Constraint Satisfaction Problem*", qui permet de décrire une large classe de problèmes combinatoires. Il a introduit en outre plusieurs notions fondamentales permettant la résolution efficace de ces problèmes et proposé des algorithmes pour leur mise en œuvre. Dans ce formalisme, un CSP est la donnée d'un ensemble de *variables*, représentant les inconnues du problème à résoudre et d'un ensemble de relations entre ces variables, nommées *contraintes*. Chaque variable prend ses valeurs dans un ensemble fini donné *a priori* (son *domaine*). Les contraintes décrivent les propriétés que doivent vérifier les solutions. Une *solution* du problème est une instanciation de chacune des variables, avec une valeur choisie dans son domaine, respectant simultanément toutes les contraintes. Nous donnons de nombreux exemples de problèmes de satisfaction de contraintes dans la suite de ce document (cf. page 28 et suivantes).

Les CSPs étant *finis*, ils peuvent être résolus par exploration d'un espace de recherche fini. Un simple algorithme de recherche arborescente ou d'énumération explicite permet ainsi de résoudre théoriquement tous les problèmes de satisfaction de contraintes. La complexité d'une telle méthode de résolution limite son usage à des problèmes de très petite taille. Mackworth a introduit les notions de *cohérences* de *nœud*, d'*arc* et de *chemin* qui permettent d'améliorer sensiblement l'efficacité de la recherche arborescente. L'idée sous-jacente est d'utiliser activement les contraintes pour couper le plus rapidement possible des branches inutiles de l'arbre de recherche. Plus précisément, la cohérence de nœuds consiste à supprimer *a priori* (i.e. avant le début de la recherche arborescente) les valeurs des domaines des variables qui violent une contrainte (e.g. si une contrainte requiert $x > 0$, on peut supprimer toutes les valeurs négatives ou nulles du domaine de x). L'idée derrière la cohérence d'arc est d'éliminer en cours de résolution les valeurs qui ne peuvent pas satisfaire une contrainte donnée, quelles que soient les valeurs des autres variables. Par exemple, si une contrainte requiert $x = y$, et si le domaine de x contient l'élément 1, alors que le domaine de y ne contient pas 1, on peut supprimer la valeur 1 du domaine de x . Enfin, la cohérence de chemin est une extension de la cohérence d'arc au traitement de plusieurs contraintes à la fois. L'utilisation de cette technique permet de couper un grand nombre de branches de l'arbre de recherche, mais requiert des algorithmes complexes à mettre en œuvre, et coûteux en pratique.

La cohérence de nœuds consiste à reformuler le problème en éliminant des domaines les valeurs inutiles. Des deux autres notions de cohérence, c'est la cohérence d'arc qui est le plus largement utilisée pour la résolution des problèmes de contraintes, ce qui peut s'expliquer par trois raisons essentielles. En effet,

- Cette notion est plus facile à mettre en œuvre. En effet, la cohérence de chemin nécessite l'application d'algorithmes coûteux qui introduisent de nouvelles contraintes dans le problème. En revanche, la cohérence d'arc peut être appliquée sans provoquer de modification de la topologie du problème.
- Contrairement à la cohérence de chemin, la cohérence d'arc apporte un gain quasi systématique dans l'efficacité de la résolution, même lors de la résolution de problèmes faciles.
- Finalement, la cohérence d'arc est une notion *universelle* car elle est applicable de façon systématique à toutes les contraintes qu'on peut définir, indépendamment du problème à résoudre. Elle est en outre *spécifique* au sens où sa mise en œuvre est adaptable en fonction de la nature des contraintes auxquelles on l'applique. Ces deux caractéristiques permettent de créer des bibliothèques de contraintes qu'on peut utiliser à la résolution de problèmes dans des domaines d'applications quelconques.

Ces arguments sont à la base de la construction de la majeure partie des systèmes de résolution de problèmes de contraintes. Nous approfondirons cette idée dans la partie consacrée à l'implémentation du système BACKTALK.

1.2 LE LANGAGE ALICE DE JEAN-LOUIS LAURIÈRE

Le langage ALICE est rarement mentionné dans les articles concernant la satisfaction de contraintes. Cependant, les problèmes que l'on peut définir et résoudre en ALICE sont des problèmes de satisfaction de contraintes définis comme tels. De plus, les techniques mises en œuvre pour la résolution en ALICE reposent sur les notions de cohérences qui sont au cœur de la satisfaction de contraintes. La différence vient, comme nous l'avons écrit dans l'introduction de ce chapitre, de la différence de mise en œuvre des techniques de cohérences. Dans l'approche de type CSP les notions de cohérences sont définies formellement et mises en œuvre de manière systématique et combinatoire. En ALICE, les notions de cohérences employées ne sont pas formellement définies et sont employées de façons différentes suivant le problème et les contraintes. La différence la plus nette étant l'utilisation du raisonnement formel et de la réécriture de contrainte au lieu de l'exploration combinatoire. Le Tableau 1 donne un aperçu des différences essentielles entre les deux approches. Il nous semble que ces divergences sont peu significatives, et que les systèmes modernes de satisfaction de contraintes s'appuient tout autant sur le formalisme des CSP et les techniques de cohérences que sur les idées de Laurière.

Technique	Mackworth	Laurière
Cohérence de nœud	Systématique avant la recherche	Pour certains type de contraintes uniquement
Cohérence d'arc	Définition formelle Raisonnement purement combinatoire	Pas de définition générale Raisonnement mathématique dépendant du type de contrainte Réécriture de contraintes
Cohérence de chemin	Définition formelle Raisonnement purement combinatoire	Pas de définition générale Raisonnement formel pour la combinaison de contraintes
Résolution	Énumération explicite entrelacée avec des phases de cohérence	Énumération explicite ; phases de cohérences ; phases de réécriture

Tableau 1 Les différences essentielles entre l'approche traditionnelle des CSP et l'approche implémentée dans le système ALICE

ALICE est, en premier lieu, un langage permettant de définir des problèmes numériques de façon déclarative en utilisant une syntaxe proche des notations mathématiques, notamment de la théorie des ensembles et des fonctions. On définit un problème en ALICE par un ensemble d'inconnues, qui peuvent être soit des variables numériques, soit des fonctions à valeurs entières, et par un ensemble de relations, les *contraintes*, entre ces inconnues. À titre d'illustration, considérons le simple problème de cryptogramme $SEND + MORE = MONEY$ dont l'énoncé est le suivant : on souhaite associer une valeur entière comprise entre 0 et 9 à chacune des huit lettres apparaissant dans l'équation cryptée $SEND + MORE = MONEY$. On exige en outre, que 1) deux lettres différentes soient associées à deux entiers différents, 2) que les lettres S et M soient non nulles, et 3) que l'addition obtenue en remplaçant chaque lettre par le nombre entier associé soit juste. Voici une définition en ALICE de ce problème. Dans cette déclaration, on a introduit les quatre variables R_1 , R_2 , R_3 et R_4 , qui correspondent aux retenues :

```

SMM
***
SOI ENS
  LET = INT 1 8      "Il y a 8 variables lettres (S, E, N,...),"
  RET = INT 1 4      "et quatre retenues ( $R_1$ ,  $R_2$ ,  $R_3$ ,  $R_4$ )"
  CHI = INT 0 9      "Le domaine des lettres,"
  BOO = INT 0 9      "celui des retenues"
TRO
  INJ F      LET CHI      "Les lettres sont 2 à 2 distinctes,"
  FON G      RET BOO      "mais pas les retenues"
AVE
  = + F 4 F 2 + F 8 * 10 G 1      "D + E = Y + 10. $R_1$ "
  = + G 1 + F 3 F 7 + F2 * 10 G 2      " $R_1 + N + R = E + 10.R_2$ "
  = + G 2 + F 2 F 6 + F 3 * 10 G 3      " $R_2 + E + O = N + 10.R_3$ "
  = + G 3 + F 1 F 5 + F 6 * 10 G 4      " $R_3 + S + M = O + 10.R_4$ "
  = G 4 F 5      " $R_4 = M$ "
  -= F 5 0      " $S \neq 0$ "
  -= F 1 0      " $M \neq 0$ "

```

Les éléments principaux du langage d'ALICE sont les ensembles finis de nombres entiers, les fonctions, qui peuvent être quelconques, injectives, bijectives, multiples, et les opérations arithmétiques (en notation préfixée). La partie définition commence par le mot réservé **SOI**, qui signifie "soit". Dans l'exemple ci-dessus, quatre

ensembles sont définis : les deux ensembles d'indices **LET** et **RET**, qui contiennent les indices des lettres et des retenues respectivement ; Les deux ensembles de valeurs, **CHI** et **BOO**, qui donnent respectivement les domaines des variables lettres et retenues (on peut remarquer que l'on pourrait définir **BOO = INT 0 1** au lieu de **BOO = INT 0 9**). Ensuite, le mot réservé **TRO**, qui signifie "trouver", introduit la partie dans laquelle on définit ce que l'on cherche. Ici, on cherche deux fonctions : la première, entre les lettres et l'ensemble **CHI**, qui est injective, ce qui correspond à une des exigences de l'énoncé ; la seconde fonction, quelconque, est entre les retenues et le domaine des retenues. Enfin, le mot clé **AVE**, qui est une abréviation de "avec", introduit les contraintes du problème.

En plus d'un langage pour la définition de problèmes combinatoires numériques, ALICE est un système permettant de les résoudre. La résolution en ALICE, qui est guidée par de nombreuses heuristiques, est essentiellement fondée sur la recherche arborescente et l'utilisation de plusieurs techniques sophistiquées de raisonnement formel. À titre d'illustration, voici la trace de l'exécution du programme ci-dessus telle que la donne Jean-Louis Laurière dans sa thèse [Laurière, 1976] :

Le programme ne mentionne pas que les retenues ne peuvent valoir que 0 ou 1.

$$F4 + F2 = F8 + 10.G1 \quad (1)$$

$$G1 + F3 + F7 = F2 + 10.G2 \quad (2)$$

$$G2 + F2 + F6 = F3 + 10.G3 \quad (3)$$

$$G3 + F1 + F5 = F6 + 10.G4 \quad (4)$$

$$G4 = F5 \quad (5)$$

$$F5 > 0 \quad (6)$$

$$F1 > 0 \quad (7)$$

L'examen de la contrainte (1) :

$$\begin{array}{ccccccc} F4 & + & F2 & - & (F8 & + & 10.G1) & = & 0 & \Rightarrow & F8 & + & 10.G1 & \leq & 17 \\ 0,9 & & 0,9 & & 0,9 & & 0,90 & & & & & & & & \\ & & & & 0,17 & & 0,99 & & & & & & & & \end{array}$$

donne par raisonnement formel la nouvelle contrainte :

$$\begin{array}{ll} F8 + 10.G1 \leq 17 & \Rightarrow F8 \leq 17 \\ & \Rightarrow 10.G1 \leq 17 \\ 10.G1 \leq 17 & \Rightarrow G1 \leq 1 \end{array}$$

Appliqué à chaque retenue, le même raisonnement montre que chaque retenue est inférieure ou égale à 1.

$$(5) \text{ et } (6) \quad \Rightarrow G4 = F5 = 1 \quad (8)$$

$$(8) \text{ et } (4) \quad \Rightarrow G3 + F1 = F6 + 9 \quad (9)$$

$$G3 + F1 = F6 + 9 \quad \Rightarrow F6 \leq 1$$

$$(8) \text{ et } F6 \leq 1 \quad \Rightarrow F6 = 0$$

$$F6 = 0 \text{ et } (9) \quad \Rightarrow F1 = 9 - G3$$

D'autre part :

$$\begin{array}{ll} G2 + F2 = F3 + 10.G3 & \Rightarrow F3 + 10.G3 \leq 10 \\ 1,10 & 1,19 \end{array}$$

$$\begin{array}{ll} F3 + 10.G3 \leq 10 & \Rightarrow G3 = 0 \text{ et donc } F1 = 9 \\ 1,9 & 0,10 \end{array}$$

$F2 + F3$	$\Rightarrow G2 = 1 \text{ et } F3 = F2 + 1$
$G1 + F7 = 9$	$\Rightarrow G1 = 1 \text{ et } F7 = 8$
Il nous reste : $F4 + F2 = F8 + 10 \text{ et } F3 = F2 + 1$	
Et donc : $4 \leq F2 \leq 7 \text{ et } F4 \geq 5$	
Et la solution unique est donc : $F2=5 \ F3=6 \ F4=7 \text{ et } F8=2$	

Dans cette résolution, on constate que le problème est résolu et que l'unicité de la solution est prouvée sans aucun point de choix, c'est-à-dire sans aucune énumération. Le seul raisonnement formel sur les équations et inéquations permet de résoudre complètement le problème. Bien évidemment, pour des problèmes un peu plus complexes, il est nécessaire de recourir à l'énumération, mais il apparaît clairement que le raisonnement formel permet une résolution efficace et élégante, car proche de la démarche qu'adopterait un être humain confronté au même problème.

Grâce aux techniques de cohérences exposées au paragraphe précédent, les performances des systèmes actuels de satisfaction de contraintes sont bien meilleures que celle d'ALICE. Sur certains problèmes pour lesquels le raisonnement formel est bien adapté, ALICE permet une résolution plus efficace que n'importe quel système de satisfaction de contraintes actuel. Un exemple est la multiplication magique, que nous présentons en détail au Chapitre 1 de la 0, pour laquelle ALICE donne l'unique solution et prouve l'unicité en un temps très inférieur à celui obtenu par les méthodes actuelles de résolution.

Cette constatation motive les recherches en cours d'Anne Liret [Liret, et al., 1997a, Liret, et al., 1998], qui étudie l'intégration des techniques de raisonnement formel avec les techniques actuelles de satisfaction de contraintes. Ces idées nous ont également influencé en nous conduisant à implémenter un module, embryonnaire il est vrai, de raisonnement formel dans notre système BACKTALK de satisfaction de contraintes, qui peut être utilisé pour les contraintes définies par des équations ou des inéquations linéaires.

1.3 DIRECTIONS DE RECHERCHE SUR LES CSPs

Les recherches actuelles dans le domaine de la satisfaction de contraintes portent principalement sur les algorithmes de cohérence et sur les méthodes de résolution. Il suffit pour s'en convaincre de parcourir les actes des conférences spécialisées, comme Principle and Practice of Constraint Programming (CP) ou bien de conférences d'intérêt général comme l'International Joint Conference on Artificial Intelligence (IJCAI) ou sa version européenne : l'European Conference on Artificial Intelligence (ECAI). On peut classer ces recherches algorithmiques selon trois principaux axes : les recherches concernant les techniques de simplification des problèmes ; les algorithmes de résolution à proprement parler ; les heuristiques ou méta-heuristiques de résolution.

1.3.1 LA COHÉRENCE D'ARC

Les algorithmes de simplification, fondés sur les différentes notions de cohérences, ont fait l'objet de nombreux travaux, surtout en ce qui concerne la cohérence d'arc. Les premiers algorithmes de cohérence d'arc, nommés AC1, AC2 et AC3 sont dus à Alan Mackworth. AC3, le meilleur des trois algorithmes de Mackworth, est perfectible à deux points de vue : parce que d'une part, il utilise des *struc-*

tures de données simples mais peu efficaces et, d'autre part, il ne prend pas en compte la nature des contraintes du problème.

L'utilisation de structures de données efficaces est au cœur de l'algorithme AC4 [Mohr and Henderson, 1986] qui utilise un mécanisme de mémorisation des incohérences, inspiré de la gestion du graphe d'ALICE. AC4 affiche une complexité temporelle optimale dans le pire des cas et complexité spatiale moins bonne que celle de AC3. Plus récemment, les travaux sur l'utilisation de structures de données de plus en plus efficaces ont donné naissance aux algorithmes AC6 et AC7 [Bessière, 1994, Bessière and Régin, 1995] dont les performances, sur des problèmes dont on ne connaît pas la nature de contraintes, sont bien supérieures à celle d'AC3.

Les algorithmes qui précèdent ne sont en revanche pas adaptés aux problèmes pour lesquels on dispose d'informations sur la nature des contraintes. C'est cette insuffisance que l'algorithme AC5 de Yves Deville et Pascal Van Hentenryck et [Deville and Hentenryck, 1991] corrige en proposant non pas un algorithme (en dépit du titre de l'article) mais un framework avant la lettre pour l'implémentation d'algorithmes de réduction de domaines par cohérence d'arcs. Les auteurs proposent de définir, *pour chaque type de contraintes*, des méthodes spécifiques pour l'obtention de la cohérence d'arc. Cette adaptation de la méthode de cohérence d'arc en fonction de la nature de la contrainte permet une amélioration spectaculaire des performances par rapport à AC3 ou AC4. L'algorithme AC-Inference [Bessière, et al., 1995] est une autre illustration de ce principe.

Indépendamment des algorithmes de simplification des CSP par utilisation de la cohérence d'arc (la famille des ACn), de nombreuses recherches sont menées pour la mise au point de méthodes d'arc cohérence spécifiques à des types de contraintes particuliers. Par exemple, dans l'article sur AC5, les auteurs présentent une méthode d'obtention de la cohérence d'arc pour les contraintes fonctionnelles [David, 1994] et une pour les contraintes monotones. Les contraintes globales de différence et de cardinalité ont été étudiées par Jean-Charles Régin, qui a mis au point pour chacune d'elles une méthode polynomiale d'arc cohérence [Régin, 1994, Régin 1996].

1.3.2 LA COHÉRENCE DE CHEMINS

La cohérence de chemins est une technique beaucoup plus coûteuse que la cohérence d'arc et elle est plus complexe à mettre en œuvre puisqu'elle implique d'ajouter des contraintes complexes au problème. De plus, contrairement à la cohérence d'arc, il n'est pas évident de dégager pour la cohérence de chemins des propriétés spécifiques à un ensemble donné de contraintes. Pour ces raisons, les recherches sur les techniques de cohérence de chemins sont bien moins nombreuses. Depuis les algorithmes PC1 et PC2 de Mackworth et l'algorithme PC3 de Mohr et Henderson, on citera tout de même une implémentation des techniques de cohérence de chemin dans le système **clp(FD)** qui repose pourtant sur la cohérence d'arc [Codognet and Nardiello, 1994].

1.3.3 LES HEURISTIQUES

On peut suivre différentes stratégies pour la résolution de problèmes de contraintes, notamment concernant l'ordre dans lequel les différents éléments (variables, contraintes ou valeurs) du problèmes sont considérés. Ces stratégies sont contrôlées par ce que l'on appelle des *heuristiques de sélection*.

Depuis les premiers travaux sur la satisfaction de contraintes, les heuristiques de sélection ont fait l'objet de beaucoup de recherche [Nadel, 1988]. Plus récemment, Steve Minton propose une approche qui va au-delà de la simple comparaison expéri-

mentale ou classification. L'idée est de faire des systèmes dont les heuristiques de sélection sont configurées automatiquement en fonction de la résolution d'instances d'essai (plus simples que le véritable problème à résoudre) [Minton, 1996].

1.3.4 LES ALGORITHMES DE RÉOLUTION

Depuis les premiers algorithmes de résolution, fondés sur l'énumération explicite de l'espace de recherche, de nombreuses méthodes plus efficaces ont été imaginées. En ce qui concerne les algorithmes complets on peut distinguer deux types d'amélioration : la gestion du mécanisme de retour arrière et l'utilisation de méthodes de réduction par utilisation de techniques de cohérence.

Les stratégies de retour arrière "intelligentes" ont été étudiées depuis les travaux initiaux de Cox [Cox, 1977] et de Codognet [Codognet, et al., 1986] en programmation logique ou plus récemment de Ginsberg en CSP [Ginsberg, 1993]. Nous mentionnerons également l'utilisation de telles techniques par Kemal Ebcioglu [Ebcioglu, 1992a, Ebcioglu, 1992b] sur les travaux duquel nous reviendrons dans la partie consacrée à l'harmonisation musicale automatique. Ces techniques semblent maintenant abandonnées, ce qui est dû à leur faible efficacité en comparaison des techniques de cohérences, même si les différentes techniques sont compatibles entre elles, comme il a été montré par Patrick Prosser [Prosser, 1993a, Prosser, 1993b, Prosser, 1995] pour un grand nombre d'algorithmes d'énumération.

L'utilisation de techniques de cohérences en cours d'énumération a été, et est encore, un domaine très actif de la recherche en satisfaction de contraintes. Les premiers algorithmes reposant sur ce principe sont le forward-checking de Haralick et Elliott [Haralick and Elliott, 1980] et le full look-ahead [Gaschnig, 1977]. Le premier consiste à réaliser, à chaque pas de la boucle d'énumération, la cohérence d'arc de toutes les contraintes au voisinage de la variable énumérée. Le second consiste à appliquer un algorithme de cohérence d'arc AC n complet à chaque pas [Nadel, 1988]. La recherche s'oriente maintenant vers la mise en œuvre d'algorithmes spécifiques à des domaines d'application particuliers tels que l'ordonnancement [Caseau and Laburthe, 1994, Laburthe, 1998] [Baptiste and Pape, 1995].

Les algorithmes de résolution incomplets sont également employés pour la résolution de problèmes de satisfaction de contraintes. On citera notamment la méta-heuristique GSAT [Kask and Dechter, 1995, Selman, et al., 1992] dont l'application à certains types de problèmes donne des résultats intéressants.

La mise au point d'algorithmes hybrides est très étudiée actuellement. Le but est de combiner diverses stratégies de recherche telles que l'énumération, les techniques de cohérence, les méta-heuristiques de type GSAT [Selman, et al., 1992] ou Tabou [Glover, 1989]. On compte déjà plusieurs langages dans lesquels il est possible de spécifier de tels algorithmes de façon aisée (e.g. le langage SALSA de Yves Caseau et François Laburthe [Laburthe and Caseau, 1998] et le langage LOCALIZER de Michel et Van Hentenryck [Michel and Hentenryck, 1997]).

1.3.5 LES EXTENSIONS DU FORMALISME DES CSPs

Il faut ajouter à cela une recherche très active pour l'extension du formalisme et des techniques de résolution des CSPs. Toutes ces extensions ont pour objets un gain en souplesse que ce soit afin de pouvoir donner des solutions satisfaisantes sur des problèmes trop contraints ou de permettre la définition de problèmes dont les contraintes ne sont pas rigides. Parmi les différents formalismes proposés jusqu'ici, on citera les *Partial CSP* de Eugene C. Freuder et Richard J. Wallace [Wallace, 1993]. La résolution partielle de problèmes consiste simplement à ne pas exiger que *toutes*

les contraintes soient nécessairement satisfaites. L'intérêt d'une telle démarche est de pouvoir proposer certaines propriétés nouvelles pour un système de satisfaction de contraintes, telles que :

- La capacité de trouver des solutions violant peu de contraintes à des problèmes sur-contraints.
- La capacité de résoudre partiellement des problèmes trop complexes pour une résolution complète en fournissant de bonnes solutions, par exemple en maximisant le nombre de contraintes satisfaites (MAX-CSP).
- La capacité à fournir à tout instant une solution au problème. Sachant que plus on alloue de temps de résolution, meilleure sera la solution. Ce qui peut être particulièrement utile dans un contexte de travail en temps réel.

Le formalisme des *Fuzzy CSP* [Fargier, et al., 1995] permet de définir des contraintes qui ne sont pas nécessairement satisfaites ou violées, mais dont la valeur de vérité appartient à un ensemble flou. Techniquement, les contraintes sont définies en associant à chaque n-uplet de valeur pour les variables une valeur, comprise entre 0 (le n-uplet viole la contrainte) et 1 (le n-uplet satisfait la contrainte). La résolution du problème consiste à donner une valeur à chaque variable qui maximise la satisfaction des contraintes, c'est-à-dire qui maximise le minimum des valeurs de chaque contrainte sur cette instanciation.

La *Semiring-based Constraint Satisfaction* [Bistarelli, et al., 1995] [Georget and Codognet, 1998] est un formalisme général pour la définition des problèmes de satisfaction de contraintes flexibles dans lequel on peut définir notamment les problèmes classiques et les problèmes de contraintes floues ou probabilistes. Un *semiring* est un ensemble A contenant au moins deux éléments 0 et 1 et doté de deux opérations $+$ et $*$. L'addition $+$ est une loi de composition interne commutative, associative et idempotente² dont 0 est l'élément neutre et 1 l'élément absorbant. Le produit $*$ est une loi interne associative dont 1 est l'élément neutre et 0 l'élément absorbant. L'ensemble $[0, 1]$ muni des lois définies par $a + b = \max(a, b)$ et $a * b = \min(a, b)$ et dont le zéro est 0 et l'unité est 1 est un *semiring*. Les problèmes classiques sont définis par le *semiring* défini par l'ensemble $\langle \{0, 1\}, \text{et}, \text{ou}, 0, 1 \rangle$. Les *Fuzzy CSPs* sont définis sur $\langle [0, 1], \max, \min, 0, 1 \rangle$.

² Commutativité : $\forall (a, b) \in A \times A, a + b = b + a$.

Associativité : $\forall (a, b, c) \in A \times A \times A, (a + b) + c = a + (b + c)$.

Idempotence : $\forall a \in A, a + a = a$.

Élément neutre x : $\forall a \in A, a + x = a = x + a$.

Élément absorbant x : $\forall a \in A, a + x = x = x + a$.

Chapitre 2

Intégration à des langages de programmation

Après les premiers travaux algorithmes que nous avons rapportés au chapitre précédent, les techniques de satisfaction de contraintes ont été intégrées dans des langages de programmation afin d'en accroître les capacités en les dotant de la puissance de résolution et de la déclarativité des contraintes. Les premières intégrations ont été réalisées avec les langages de programmation logique, pour donner naissance au paradigme de la *programmation logique avec contraintes*.

L'intégration des contraintes dans des langages de programmation classique, que ce soit les langages impératifs, fonctionnels ou à objets, ne peut pas être achevée de façon aussi transparente qu'avec les langages de programmation logique. Il existe cependant de nombreux systèmes de satisfaction de contraintes implémentés dans de tels langages. Nous en présentons quelques-uns au paragraphe 2.2.

2.1 PROGRAMMATION LOGIQUE AVEC CONTRAINTES

La première intégration des techniques de satisfaction de contraintes dans un langage de programmation a été réalisée, dans les années 1980, dans le contexte de la programmation logique. Ce choix s'explique par deux raisons fondamentales.

La première raison est que la programmation logique, avec sa structure de programmes simple et sa structure de données unique est un "métalangage" bien adapté à la programmation de problèmes de contraintes [Codognet, 1995]. De plus, et c'est probablement la raison la plus profonde, la notion de variable logique est parfaitement compatible avec la notion de variable contrainte, ce qui permet une intégration transparente des deux paradigmes. On peut même voir dans les mécanismes de satisfaction de contraintes une extension du mécanisme d'unification des langages de programmation logique. En effet, la notion de variable logique dote les langages de

programmation logique de la capacité à travailler avec des informations partielles selon une stratégie unique : l'accumulation monotone d'information par le truchement du mécanisme *d'unification*. Cette démarche est très proche de celle employée pour la satisfaction de contraintes, ce qui rend possible une intégration profonde des deux paradigmes sans avoir recours à des compromis.

Inversement, la programmation logique souffre d'un profond manque de déclarativité dans les calculs arithmétiques. Le simple programme suivant (voir [Codognet, 1995]) qui calcule la fonction factorielle :

```
fact (0, 1) .  
fact (N, F) :-  
    N > 0,  
    N1 is N-1,  
    fact (N1, F1),  
    F is N*F1.
```

définit une relation **fact** qui ne peut pas être évalué de façon "multi-directionnelle" puisqu'une expression telle que celle qui suit ne sera pas évaluée à cause du prédicat **is** :

```
fact (X, 24) .
```

L'intégration de contraintes linéaires arithmétiques permet de pallier cette impossibilité, et de définir directement des expressions pouvant être évaluées selon plusieurs directions, par exemple, avec la définition suivante pour **fact** :

```
fact (N, F) :-  
    N = 0,  
    F = 1.  
fact (N, F) :-  
    N > 0,  
    F = F1*N,  
    N1 = N-1,  
    fact (N1, F1) .
```

l'expression précédente s'évaluera alors de la façon suivante :

```
fact (N, 24) .  
N = 4  
yes
```

L'intégration de la programmation logique avec les mécanismes de satisfaction de contraintes définit le paradigme de la programmation logique avec contraintes. On compte aujourd'hui de nombreux langages de programmation logique avec contraintes dont les plus célèbres sont PROLOG III [COLMERAUER, 1990], CHIP [Dincbas, et al., 1990] ou **clp (FD)** [Diaz, 1995].

2.2 INTÉGRATION DANS DES LANGAGES CLASSIQUES

Contrairement aux langages de programmation logiques, les langages classiques, par exemple les langages fonctionnels ou les langages à objets, ne disposent pas de la notion de variable logique qui permet de manipuler des expressions partiellement instanciées et qui soit ainsi "compatible" avec la notion de variable contrainte. En effet, une variable dans un langage impératif est nécessairement dotée d'une valeur, éventuellement "inconnue" comme le **()** de LISP, le **nil** de SMALL-TALK ou le **null** de PASCAL ou JAVA. La manipulation des variables dans un langage impératif n'est pas monotone puisque sa valeur peut être arbitrairement changée plusieurs fois au cours de l'exécution d'un programme. Ainsi, il n'est pas envisageable d'intégrer directement, au cœur d'un langage impératif, les techniques de satisfaction de contraintes, comme cela est possible avec les langages de programmation logique. Au contraire, les techniques de satisfaction de contraintes sont généralement fournies sous la forme d'extensions séparées du langage hôte, qui définissent et manipulent leurs propres variables, les variables contraintes, distinctes des variables du langage hôte. En dépit de cette difficulté profonde, il existe quelques réalisations de langages impératifs intégrant en leur cœur des mécanismes de satisfaction de contraintes, que nous présentons à la fin de ce paragraphe.

Parmi les extensions de langages classiques, on peut retenir les systèmes qui suivent. Le système PROSE, de Pierre Berlandier, est une implémentation dans le langage fonctionnel LE_LISP des algorithmes de satisfaction de contraintes [Berlandier, 1992]. PECOS est une bibliothèque de classes d'objets, de fonctions et de structures de contrôle permettant la représentation et la résolution efficace de problèmes de satisfaction de contraintes dans le langage fonctionnel LE_LISP [PUGET, 1992]. ILOG-SOLVER [Puget, 1994] est une version récente et implémentée en C++ de PECOS. ILOG-SOLVER est un système commercial très efficace et qui a été utilisé, et est encore utilisé, dans de nombreuses applications industrielles d'envergure. La bibliothèque commerciale CHIP existe à la fois en une version C++ et une version Prolog. CHIP met à disposition de l'utilisateur de nombreuses contraintes de très haut niveau et des techniques de résolutions issues de la programmation logique avec contraintes, de l'intelligence artificielle ou de la recherche opérationnelle pour la résolution de problèmes complexes [Dincbas, et al., 1990].

Toutes ces bibliothèques participent de la même démarche qui consiste à définir comme des éléments nouveaux les variables contraintes, les contraintes et les problèmes, et à fournir des algorithmes, sous forme de procédures ou méthodes, permettant de résoudre ces problèmes. L'utilisateur définit un problème de satisfaction de contraintes en déclarant les variables contraintes et leur domaine, ainsi que les contraintes. Il exécute ensuite une procédure de résolution pour obtenir la ou les solutions de son problème.

Une autre approche pour l'intégration des contraintes dans un langage de programmation impératif est celle employé dans le langage LAURE de Yves Caseau [Caseau, 1994]. Le langage LAURE, issu de LORE, est un langage à objets disposant des mécanismes de règles de productions et de règles déductives (en chaînage arrière) et de contraintes. Mais, comme le dit Yves Caseau lui-même [Caseau and Laburthe, 1996], l'intégration profonde des contraintes dans un tel langage de programmation est trop complexe à mener à bien dès lors que ces contraintes permettent de définir des problèmes complexes (NP-complets). C'est pour cette raison que le langage CLAIRE, issu de l'expérience de LAURE, ne dispose pas de contraintes, mais propose à la place des mécanismes permettant l'implémentation efficace des contraintes, ce qui a donné lieu à la mise en œuvre de la bibliothèque ECLAIR [Laburthe, 1998], une extension de CLAIRE pour le traitement des contraintes ou au langage SALSA [Laburthe and Caseau, 1998] de spécifications d'algorithmes hybrides de résolutions de problèmes de contraintes.

DEUXIÈME PARTIE

LES ASPECTS TECHNIQUES

La *satisfaction de contraintes* désigne à la fois un formalisme pour la définition de problèmes combinatoires et un ensemble de techniques pour la résolution de ces problèmes. Cette discipline est issue, d'une part, de travaux purement algorithmiques tels ceux de Montanari et de Mackworth et, d'autre part, des idées du langage ALICE de Laurière.

Le succès durable de la satisfaction de contraintes est principalement dû à deux aspects : son caractère “déclaratif” et sa vocation générale. L'aspect déclaratif peut se résumer par la définition suivante : *définir un problème en termes de contraintes revient à caractériser a priori ses solutions*. Cette caractérisation se fait en définissant les propriétés qui doivent être vérifiées par les solutions, et qui sont appelées *contraintes*. Il découle immédiatement de cette caractérisation que la satisfaction de contraintes est une approche très générale pour la définition et la résolution de problèmes combinatoires. En effet, tout problème dont on sait caractériser *a priori* les solutions peut se définir en termes de satisfaction de contraintes. À titre d'illustration, notons que la satisfaction de contraintes est couramment employée pour la résolution de problèmes aussi divers que l'allocation de ressources, l'ordonnancement, la découpe optimale ou encore, comme nous l'illustrerons dans ce document, la composition musicale.

Cette partie est organisée de la façon suivante : après avoir donné, dans le Chapitre 1, une définition intuitive et quelques exemples de problèmes de satisfaction de contraintes nous présentons, au Chapitre 2, le formalisme de Mackworth pour la définition des problèmes de contraintes binaires. Au Chapitre 3 nous présentons les techniques de résolutions développées dans le contexte du formalisme de Mackworth. Au Chapitre 4 nous soulignons les limitations de ce formalisme et proposons son ex-

tension aux contraintes non binaires. Au Chapitre 5, nous montrons que les techniques de résolutions des problèmes binaires ne se transposent pas au contexte des problèmes de contraintes non binaires.

Chapitre 1

Qu'est-ce qu'un problème de satisfaction de contraintes ?

Intuitivement, un problème de satisfaction de contraintes est un problème fini dont on sait décrire *a priori* et précisément les éléments constitutifs *et* les solutions. Plus précisément, un problème de satisfaction de contraintes est un problème qui vérifie les propriétés suivantes :

- 1) les inconnues du problème sont en nombre fini et sont connues *a priori*
- 2) les inconnues du problème prennent leurs valeurs dans un ensemble fini connu *a priori*
- 3) on peut décrire les solutions *a priori* par un ensemble fini de propriétés entre les inconnues

Les inconnues du problème sont appelées *variables* et l'ensemble des valeurs possibles de chaque variable est appelé son *domaine*. Les propriétés des inconnues qui définissent une solution sont appelées *contraintes*.

À titre d'illustration, voici quelques exemples de problèmes pouvant se formaliser par un problème de satisfaction de contraintes dont nous donnerons une définition plus précise ultérieurement : le problème des reines sur l'échiquier, les cryptogrammes (e.g. SEND + MORE = MONEY), la création de carrés magiques, les problèmes d'allocation de ressources, la gestion d'emploi du temps, etc. En effet, dans ces problèmes, on cherche à donner une valeur à des inconnues de façon à satisfaire un certain nombre de propriétés. Par exemple, le problème des reines sur l'échiquier consiste à trouver une case (*valeur*) de l'échiquier pour chaque reine (*variable*) de façon à ce qu'aucune reine n'en attaque aucune autre selon les règles du jeu d'échec (*contrainte*). Le caractère fini de ce problème vient de ce que le nombre de cases d'un échiquier est fini.

Cette définition des problèmes de satisfaction de contraintes est très générale, cependant il existe de nombreux problèmes qui ne sont pas de cette nature. Voici quelques exemples de tels problèmes.

- 1) La résolution de l'équation célèbre $x^n + y^n = z^n$ pour x, y et z des entiers relatifs non tous nuls et n un entier strictement supérieur à 2 (qui n'admet pas de solution selon le grand théorème de Fermat) n'est pas un problème de satisfaction de contraintes puisque les inconnues du problème ne prennent pas leur valeur dans un ensemble fini connu *a priori*.
- 2) La découverte d'une stratégie gagnante au jeu d'échec n'est pas *a priori* un problème de satisfaction de contraintes. En effet, si l'on considère la définition suivante de ce problème : trouver, pour chaque coup possible de l'adversaire, une réplique qui permet de ne pas perdre la partie. Dans ce problème on ignore *a priori* le nombre de coups à trouver, ce qui le rend impossible à exprimer directement sous la forme d'un problèmes de contraintes.

Dans le paragraphe qui suit, nous donnons les énoncés de plusieurs problèmes qui se prêtent naturellement à une définition sous forme de problème de satisfaction de contraintes. Nous décrirons au Chapitre 2 un formalisme pour la définition des problèmes de satisfaction de contraintes.

Il s'agit de problèmes simples et bien connus, qui ont été étudiés dans de nombreuses branches de l'informatique. Nous les présentons ici à des fins "pédagogiques", dans le but d'illustrer certaines notions capitales de la satisfaction de contraintes à l'aide d'exemples simples. Plus loin dans ce mémoire, nous aborderons des applications de plus grande envergure.

1.1 LE PROBLÈME DES REINES SUR L'ÉCHIQUIER

C'est le plus fameux problème de satisfaction de contraintes. Il consiste à disposer 8 reines sur un échiquier traditionnel de 64 cases, de telle façon qu'aucune reine ne soit en prise, sans considérations de couleurs. Voici une solution à ce problème :

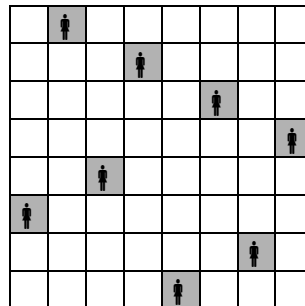


Figure 1 Une solution au problème des huit reines

On peut évidemment définir des problèmes similaires pour des échiquiers de tailles différentes. Dans ce mémoire, on désignera par *problème des n reines* l'énoncé suivant :

Disposer n reines dans un échiquier de taille $n \times n$ de façon à ce que deux reines quelconques ne s'attaquent pas mutuellement.

Problème 1 Le problème des n reines sur un échiquier

Pour $n < 4$ le problème n'admet aucune solution. Voici, en revanche, une solution au problème des 4 reines :

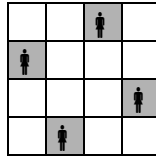


Figure 2 Une solution du problème des quatre reines

1.2 LES CRYPTOGRAMMES

Un cryptogramme est un casse-tête dans lequel on doit remplacer des lettres par des chiffres de façon à satisfaire une équation. L'exemple le plus fameux est le suivant :

On considère l'addition cryptée suivante :

$$\begin{array}{r} \text{SEND} \\ + \text{MORE} \\ \hline \text{MONEY} \end{array}$$

Le problème est d'affecter à chacune des lettres apparaissant dans l'addition suivante un nombre compris entre 0 et 9 afin que :

1. À deux lettres différentes correspondent deux valeurs différentes.
2. Les lettres 'S' et 'M' ne soient pas nulles.
3. L'addition obtenue en remplaçant les lettres par leur valeur soit correcte.

Problème 2 Le cryptogramme $\text{SEND} + \text{MORE} = \text{MONEY}$

Pour les puristes, un cryptogramme admet une unique solution, ce qui est le cas du précédent. Parmi les cryptogrammes du même type, mentionnons "DONALD + GERALD = ROBERT" ou encore "SEND + MORE + GOLD = MONEY", qui admet plusieurs solutions ou l'astucieux "AIN + AISNE + MARNE + DROME = SOMME", qui admet trois solutions.

À titre d'exemple, voici quelques cryptogrammes et leurs solutions :

SEND + MORE = MONEY 9567 + 1085 = 10652	9237 + 1582 + 4507 = 15326 9237 + 1502 + 4587 = 15326 9478 + 1634 + 5628 = 16740 9478 + 1624 + 5638 = 16740 9054 + 1730 + 6724 = 17508 9054 + 1720 + 6734 = 17508 2894 + 1058 + 7034 = 10986 2894 + 1038 + 7054 = 10986 4856 + 1398 + 7326 = 13580 4856 + 1328 + 7396 = 13580 2856 + 1378 + 9346 = 13580 2856 + 1348 + 9376 = 13580 4673 + 1586 + 9503 = 15762 4673 + 1506 + 9583 = 15762 4237 + 1582 + 9507 = 15326 4237 + 1502 + 9587 = 15326 5478 + 1634 + 9628 = 16740 5478 + 1624 + 9638 = 16740 6054 + 1730 + 9724 = 17508 6054 + 1720 + 9734 = 17508
DONALD + GERALD = ROBERT 526485 + 197485 = 723970	
AIN + AISNE + DROME + MARNE = SOMME 190 + 19705 + 26835 + 31605 = 78335 204 + 20648 + 15938 + 32548 = 69338 328 + 32986 + 50716 + 13086 = 97116	
SEND + MORE + GOLD = MONEY 7894 + 1058 + 2034 = 10986 7894 + 1038 + 2054 = 10986 9856 + 1378 + 2346 = 13580 9856 + 1348 + 2376 = 13580 7856 + 1398 + 4326 = 13580 7856 + 1328 + 4396 = 13580 9673 + 1586 + 4503 = 15762 9673 + 1506 + 4583 = 15762	

Il existe aussi des cryptogrammes plus complexes en forme de multiplication. Le problème suivant, dit de la “multiplication magique”, est dû à Jean-Louis Laurière :

Étant donné le “format” d’une multiplication en base dix :

$$\begin{array}{r}
 \text{XXX} \\
 * \text{XXX} \\
 \hline
 \text{XXX} \\
 \text{XXX} \\
 \text{XXX} \\
 \hline
 \text{XXXXX}
 \end{array}$$

on cherche à remplacer les croix par des chiffres en utilisant chaque chiffre exactement deux fois (il y a vingt croix dans le patron), de façon à rendre la multiplication correcte.

Ce problème admet une solution unique, (179 x 224).

Problème 3 La multiplication magique

Le temps de résolution “à la main” d’un tel problème est considérable, probablement plus d’une journée pour un esprit acharné.

Un cryptogramme plus complexe est le problème nommé **alpha** dont l’énoncé est le suivant :

Associer un nombre entier entre 1 et 26 à chacune des lettres de l’alphabet de façon à ce que :

- À deux lettres distinctes correspondent deux nombres distincts
- Les vingt équations linéaires suivantes sont satisfaites :

BALLET	45	GLEE	66
CELLO	43	JAZZ	58

CONCERT	74	LYRE	47
FLUTE	30	OBOE	53
FUGUE	50	OPERA	65
POLKA	59	SONG	61
QUARTET	50	SOPRANO	82
SAXOPHONE	134	THEME	72
SCALE	51	VIOLIN	100
SOLO	37	WALTZ	34

Où il faut comprendre $B + A + L + L + E + T = 45$, $C + E + L + L + O = 43$ etc.

La solution unique de ce système est l'instanciation suivante :

A=5 B=13 C=9 D=16 E=20 F=4 G=24 H=21 I=25 J=17 K=23 L=2 M=8
N=12 O=10 P=19 Q=7 R=11 S=15 T=3 U=1 V=26 W=6 X=22 Y=14 Z=18

Une variante du problème **alpha** est le problème **bêta**, déduit d'**alpha** en remplaçant les deux équations :

BALLET	45
OBOE	53

Par les quatre équations suivantes :

BAND	46
BLUES	54
DOUBLEBASS	113
TUBA	25

Dont l'unique solution est :

A=5 B=13 C=9 D=16 E=20 F=4 G=24 H=21 I=25 J=17 K=23 L=2 M=8
N=12 O=10 P=19 Q=7 R=11 S=15 T=3 U=1 V=26 W=6 X=22 Y=14 Z=18

1.3 LES PUZZLES NUMÉRIQUES ET LOGIQUES

Les problèmes qui suivent sont sans aucun doute des problèmes jouets au même titre que les problèmes de reines. Cependant, s'il est avéré que le problème des reines est un problème facile, certains des problèmes exposés dans cette partie sont difficiles. Par exemple, la multiplication magique est extrêmement combinatoire et les carrés magiques d'ordre 7 ou plus sont quasiment impossibles à résoudre, même pour les techniques de satisfaction de contraintes.

1.3.1 LES CARRÉS MAGIQUES

Le plus fameux problème de ce type est la recherche de carrés *magiques* dont la définition est la suivante :

Un carré magique est une matrice carrée d'ordre n , contenant les nombres entiers de 1 à n disposés de telle sorte que les sommes de chaque ligne, de chaque colonne et des deux diagonales soient identiques.

Problème 4 Le problème des carrés magiques

Voici quelques exemples de carré magique d'ordres divers :

$$\begin{pmatrix} 13 & 24 & 19 & 4 & 5 \\ 21 & 1 & 10 & 17 & 16 \\ 14 & 3 & 23 & 7 & 18 \\ 9 & 12 & 2 & 22 & 20 \\ 8 & 25 & 11 & 15 & 6 \end{pmatrix} \text{ -- } \begin{pmatrix} 1 & 2 & 3 & 34 & 35 & 36 \\ 4 & 18 & 28 & 29 & 5 & 27 \\ 10 & 26 & 30 & 8 & 23 & 14 \\ 31 & 25 & 13 & 21 & 15 & 6 \\ 32 & 16 & 20 & 12 & 22 & 9 \\ 33 & 24 & 17 & 7 & 11 & 19 \end{pmatrix} \text{ -- } \begin{pmatrix} 24 & 26 & 27 & 24 & 33 & 29 & 11 \\ 23 & 22 & 28 & 21 & 30 & 31 & 20 \\ 19 & 32 & 18 & 34 & 16 & 41 & 15 \\ 46 & 48 & 5 & 14 & 9 & 13 & 40 \\ 10 & 2 & 44 & 36 & 37 & 8 & 38 \\ 35 & 42 & 4 & 1 & 7 & 47 & 39 \\ 17 & 3 & 49 & 45 & 43 & 6 & 12 \end{pmatrix}$$

Figure 3 Des carrés magiques d'ordre 5, 6 et 7 trouvés avec notre système (BACKTALK)

1.3.2 LES SÉQUENCES MAGIQUES

Un exemple un peu plus sophistiqué est la création de séquences *magiques* dont la définition est la suivante :

Une séquence magique d'ordre n est une séquence $U_0, U_1, \dots, U_{n-1}$ telle que le nombre i apparaît U_i fois dans la séquence.

Problème 5 Une séquence magique d'ordre 10

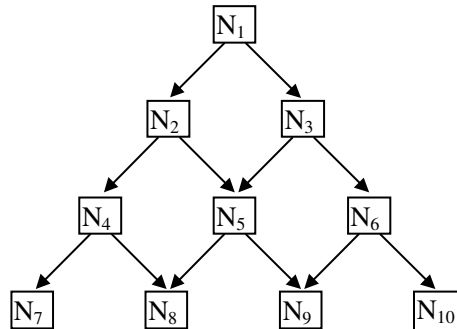
Voici un exemple de séquence magique de taille 10 :

Indice	0	1	2	3	4	5	6	7	8	9
Valeur	6	2	1	0	0	0	1	0	0	0

Figure 4 Une séquence magique d'ordre 10

1.3.3 LES GRAPHES MAGIQUES

Un graphe *magique* est un graphe dont les nœuds contiennent des nombres entiers, et qui est de la forme suivante :



Problème 6 Les graphes magiques

On cherche à donner à chaque nœud une valeur entière de façon à ce que la valeur d'un nœud quelconque soit égale à la différence des valeurs de ses fils en valeur absolue. Une solution est donnée ci-dessous :

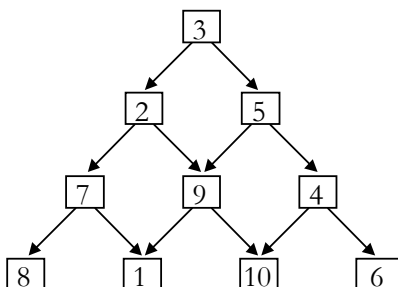


Figure 5 Un graphe magique d'ordre 10

1.3.4 LES PUZZLES LOGIQUES À LA LEWIS CARROLL

Les puzzles logiques sont des énigmes à la manière de Lewis Carroll. Une énigme consiste en un texte contenant des assertions logiques et une question. La réponse à la question peut se déduire *logiquement* des assertions du texte. Le plus fameux exemple est le problème dit du *zèbre* :

Cinq personnes de nationalité différentes habitent les cinq premières maisons d'une rue. Elles exercent cinq professions différentes et ont chacune une boisson et un animal favori. Les maisons sont de cinq couleurs différentes :

L'Anglais habite la maison rouge.
 L'Espagnol possède un chien.
 Le Japonais est peintre.
 L'Italien boit du thé.
 Le Norvégien habite la première maison à gauche.
 Le propriétaire de la maison verte boit du café.
 La maison verte est à droite de la blanche.
 Le sculpteur élève des escargots.
 Le diplomate habite la maison jaune.
 L'habitant de la maison du milieu boit du lait.
 Le Norvégien habite à côté de la maison bleue.
 Le violoniste boit des jus de fruits.
 Le renard est dans la maison voisine de celle du médecin.
 Le cheval à côté de la maison du diplomate.

Qui élève le zèbre ? Qui boit de l'eau ?

Problème 7 Le problème du zèbre

1.4 PLACEMENT EN 2D — LES CARRÉS PARFAITS

Le problème consiste à placer, sans qu'ils ne se chevauchent, des carrés de tailles distinctes dans un rectangle de façon à ce qu'il ne reste aucun espace libre. Voici quatre instances de ce problème avec une solution :

- Tailles des carrés : 18, 15, 14, 10, 9, 8, 7, 4, 1. À placer dans un rectangle de dimensions 32×33

- Tailles des carrés : 25, 24, 23, 22, 19, 17, 11, 6, 5, 3. À placer dans un rectangle de dimensions 47×65
- Tailles des carrés : 50, 42, 37, 35, 33, 29, 27, 25, 24, 19, 18, 17, 16, 15, 11, 9, 8, 7, 6, 4, 2. À placer dans un rectangle de dimensions 112×112
- Tailles des carrés : 81, 64, 56, 55, 51, 43, 39, 38, 35, 33, 31, 30, 29, 20, 18, 16, 14, 9, 8, 5, 4, 3, 2, 1. À placer dans un rectangle de dimensions 175×175

Voici une solution pour chacune de ces quatre instances dans le format suivant : la première collection contient les tailles des carrés (rangées dans l'ordre décroissant), la deuxième liste donne les abscisses et la troisième les ordonnées.

Instance n°1 :

L : (18 15 14 10 9 8 7 4 1)

X : (1 1 19 23 24 16 16 19 23)

Y : (1 19 1 15 25 26 19 15 25)

Instance n°2 :

L : (25 24 23 22 19 17 11 6 5 3)

X : (1 24 1 26 29 1 18 18 24 26)

Y : (1 42 43 1 23 26 26 37 37 23)

Instance n°3 :

L : (50 42 37 35 33 29 27 25 24 19 18 17 16 15 11 9 8 7 6 4 2)

X : (1 71 76 1 80 51 1 51 47 28 53 36 60 36 36 51 28 53 47 76 51)

Y : (1 71 34 51 1 1 86 30 89 94 71 66 55 51 83 55 86 64 83 30 64)

Instance n°4 :

L : (81 64 56 55 51 43 39 38 35 33 31 30 29 20 18 16 14 9 8 5 4 3 2 1)

X : (1 112 1 57 82 133 73 1 141 143 112 82 112 39 39 57 59 64 133 59 60 57 141 59)

Y : (1 112 82 82 1 1 137 138 44 79 81 52 52 156 138 137 162 153 44 157 153 153 79 156)

La Figure 6 donne une représentation graphique de chacune de ces solutions :

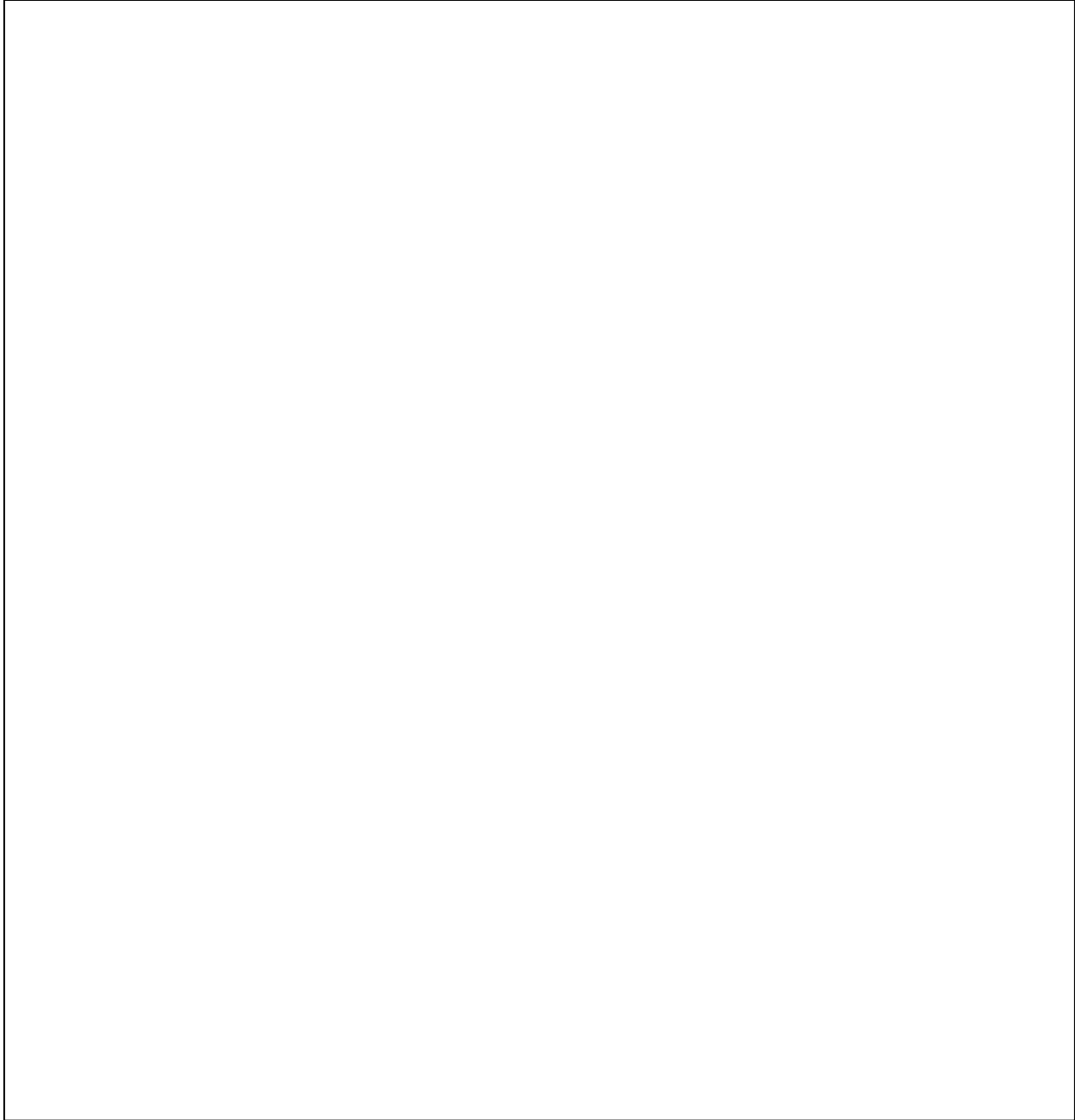


Figure 6 Une solution pour chacune des quatre instances précédentes du problème des carrés parfaits

Chapitre 2

Le formalisme des CSPs binaires de Mackworth

Nous présentons ici le formalisme de la satisfaction de contraintes tel qu'il est défini dans la littérature technique et théorique. Ce formalisme est suffisamment général pour permettre la définition de chacun des problèmes que nous avons présentés dans la partie précédente. En outre, ce formalisme est simple puisqu'on n'y considère que les contraintes binaires, c'est-à-dire les contraintes impliquant les deux variables, définies de façon explicite par la liste des couples de valeurs satisfaisants. Il est ainsi possible, dans ce formalisme, de représenter les problèmes par des graphes, dont les nœuds sont les variables et dont les arêtes sont les contraintes, et d'interpréter les contraintes dans le cadre de la théorie des ensembles.

De nombreuses notions théoriques et techniques de résolution des problèmes de satisfaction de contraintes ont été mises en œuvre dans le cadre de ce formalisme. Nous en présentons les plus importantes, notamment les différentes notions de cohérence définies par Mackworth, et les algorithmes classiques de recherche arborescente qui exploitent ces notions de cohérence.

Il apparaît cependant que pour pouvoir définir et résoudre des problèmes complexes, il est nécessaire d'étendre ce formalisme et de mettre en œuvre des techniques de résolution plus sophistiquées. Nous exposons en détail ces limitations et présentons les extensions qui permettent de s'en affranchir.

Dans cette partie, qui présente le modèle théorique de la satisfaction de contraintes, nous allons définir les CSP binaires, c'est-à-dire les problèmes de satisfaction de contraintes à domaines finis et ne contenant que des contraintes binaires.

2.1 DÉFINITION

Un problème de satisfaction de contraintes à domaines finis, CSP pour *Constraint Satisfaction Problem*, est défini par : 1) un ensemble de *variables contraintes*

(ou plus simplement *variables*) qui représentent les inconnues du problème à résoudre (chaque variable peut prendre ses valeurs dans un ensemble *fini*, défini *a priori* : son *domaine*) ; 2) un ensemble fini de relations entre ces variables, qui caractérisent les solutions du problème : les *contraintes*.

Plus formellement, un CSP est la donnée des éléments suivants :

Définition : un *problème de satisfaction de contraintes* (ou *CSP*) P est défini par

1. Un nombre entier n ;
2. Un ensemble $V(P) = \{V_1, V_2, \dots, V_n\}$, les V_i sont les variables du CSP ;
3. Un ensemble $D(P) = \{D_1, D_2, \dots, D_n\}$ d'ensembles finis. Les D_i sont les domaines des variables ;
4. Un ensemble $C(P) = \{C_{ij} \mid i, j = 1, \dots, n ; i > j\}$, les C_{ij} sont les contraintes du problème. La contrainte C_{ij} porte sur les variables V_i et V_j ;
5. Un ensemble $R(P) = \{R_{ij} \mid i, j = 1, \dots, n ; i > j\}$, avec $R_{ij} \subseteq D_i \times D_j$. Les R_{ij} sont les relations des contraintes (i.e. R_{ij} est l'ensemble des instanciptions des variables V_i et V_j qui satisfont la contrainte C_{ij}).

Définition 1 La définition d'un problème de satisfaction de contraintes ou CSP

Une *instanciation* est une application qui associe à chaque variable du problème un élément de son domaine, qui est appelé la *valeur* de la variable.

Plus formellement, une instanciation est donc une application I définie par

$$\{V_1, V_2, \dots, V_n\} \xrightarrow{I} \bigcup_{i=1}^n D_i$$

telle que

$$\forall i = 1, \dots, n ; I(V_i) \in D_i$$

Une *solution* du problème est une instanciation S qui satisfait simultanément toutes les contraintes :

$$\forall i = 1, \dots, n ; \forall j = 1, \dots, n ; i > j \text{ on a } (S(V_i), S(V_j)) \in R_{ij}$$

2.2 L'EXEMPLE DU PROBLÈME DES REINES

Pour illustrer ces définitions, donnons une première définition du Problème 1 en termes de satisfaction de contraintes. Il s'agit du problème des quatre reines (i.e. $n = 4$) :

On définit 4 variables, une pour chaque reine, dont le domaine est l'ensemble des 16 cases de l'échiquier ; et 6 contraintes qui expriment que deux reines ne doivent pas s'attaquer mutuellement (i.e. ne pas être sur deux cases appartenant à une même ligne, une même colonne ou sur un même diagonale) :

Variables :

Q_1, Q_2, Q_3, Q_4

Domaines :

$D_1 = \{1, 2, \dots, 16\}$

$D_2 = \{1, 2, \dots, 16\}$

$D_3 = \{1, 2, \dots, 16\}$

$D_4 = \{1, 2, \dots, 16\}$

Contraintes :

$C_{1,2}, C_{1,3}, C_{1,4}, C_{2,3}, C_{2,4}, C_{3,4}$

$R_{1,2}=R_{1,3}=R_{1,4}=R_{2,3}=R_{2,4}=R_{3,4}=\{(1,7); (1,8); (1,10); (1,12); (1,14); (1,15); (2,8); (2,9); (2,11); (2,13); (2,15); (2,16); (3,5); (3,10); \dots\}$

CSP 1 Le problème des quatre reines défini dans le formalisme des CSP

Si l'on considère cette définition, la solution donnée par la Figure 2 est, par exemple, l'instanciation suivante des variables : $Q_1=2$; $Q_2=8$; $Q_3=9$; $Q_4=15$.

Remarquons que l'on peut imaginer plusieurs autres formalisations du Problème 1 en termes de contraintes. Par exemple, puisqu'il est évident que dans toute solution il y a une et une seule reine sur chaque ligne de l'échiquier, au lieu de définir le domaine de valeurs d'une variable comme étant l'ensemble des n^2 cases de l'échiquier, on peut considérer que chaque reine est associée à une ligne. Ainsi, les domaines ne sont plus constitués que des n indices de colonnes. Voici une définition du Problème 1 tenant compte de cette remarque :

On définit 4 variables, une pour chaque ligne, dont le domaine est l'ensemble des 4 colonnes de l'échiquier (la valeur j de la i -ème variable indiquant qu'une reine est positionnée à la case de position i, j) ; et 6 contraintes qui expriment que deux reines ne doivent pas s'attaquer mutuellement (i.e. ne pas être sur deux cases appartenant à une même colonne ou sur une même diagonale. La contrainte sur les lignes est implicite dans cette définition du problème) :

Variables :	
Q_1, Q_2, Q_3, Q_4	“la variable Q_i est la reine de la ligne i ”
Domaines :	
$D_1 = \{1, 2, 3, 4\}$	“les valeurs possibles sont les indices des colonnes”
$D_2 = \{1, 2, 3, 4\}$	
$D_3 = \{1, 2, 3, 4\}$	
$D_4 = \{1, 2, 3, 4\}$	
Contraintes :	
$C_{1,2}, C_{1,3}, C_{1,4}, C_{2,3}, C_{2,4}, C_{3,4}$	
$R_{1,2} = \{(1,3);(1,4);(2,4);(3,1);(4,1);(4,2)\}$	“ Q_1 et Q_2 ”
$R_{1,3} = \{(1,2);(1,4);(2,1);(2,3);(3,2);(3,4);(4,1);(4,3)\}$	“ Q_1 et Q_3 ”
$R_{1,4} = \{(1,2);(1,3);(2,1);(2,3);(2,4);(3,1);(3,2);(3,4);(4,2);(4,3)\}$	“ Q_1 et Q_4 ”
$R_{2,3} = \{(1,3);(1,4);(2,4);(3,1);(4,1);(4,2)\}$	“ Q_2 et Q_3 ”
$R_{2,4} = \{(1,2);(1,4);(2,1);(2,3);(3,2);(3,4);(4,1);(4,3)\}$	“ Q_2 et Q_4 ”
$R_{3,4} = \{(1,3);(1,4);(2,4);(3,1);(4,1);(4,2)\}$	“ Q_3 et Q_4 ”

CSP 2 Une autre définition du problème des quatre reines dans le formalisme des CSP

Si l'on considère cette nouvelle définition du problème, la solution donnée par la Figure 2 est, par exemple, l'instanciation suivante des variables : $Q_1=3$; $Q_2=1$; $Q_3=4$; $Q_4=2$.

2.3 CSP EST NP-COMPLET

Le problème général de la satisfaction de contraintes que l'on peut exprimer dans le formalisme des CSPs binaires de Alan K. Mackworth est NP-complet. Pour preuve, considérons le problème 3-SAT de satisfiabilité dans la logique propositionnelle. La définition d'un problème de type 3-SAT est la suivante :

Soient n et p deux entiers naturels.
 Soient $X_1, \dots, X_n$ n variables propositionnelles.
 La proposition logique suivante est-elle satisfiable :

$$\bigwedge_{i=1 \dots p} (X_{j_i} \vee X_{k_i} \vee X_{l_i})$$

Ce problème est le prototype des problèmes NP-complets, et il est facile de voir qu'il peut se réduire à CSP de façon polynomiale. En effet, le CSP suivant est équivalent au problème de satisfiabilité défini ci-dessus :

Soient n et p deux entiers naturels.
 Soient $X_1, \dots, X_n$ n variables contraintes.
 Soient $D_1, \dots, D_n$ les domaines avec $D_i = \{0, 1\}$, $\forall i$.
 Soient $C_1, \dots, C_p$ les contraintes définies par les formules suivantes :

$$\forall i = 1, \dots, p; X_{j_i} + X_{k_i} + X_{l_i} \geq 1$$

Le problème général de la satisfaction de contraintes étant NP-complet, si l'on admet que $P \neq NP$, il est impossible qu'on trouve un algorithme permettant de le résoudre en temps polynomial.

Il existe des sous-classes de problèmes de satisfaction de contraintes qui sont dans P comme celle définie dans [Jeavons, et al., 1995]. Cependant, l'intérêt principal de la satisfaction de contraintes est que c'est une approche générale et flexible, au sens où elle fournit un formalisme simple permettant de définir des problèmes mal catégorisés.

En d'autres termes, les problèmes bien connus et déjà catégorisés pour lesquels il existe des algorithmes efficaces de résolution provenant par exemple de la recherche opérationnelle, e.g. la programmation linéaire en nombres entiers, ne justifient pas une approche par satisfaction de contraintes. En revanche, on utilisera les contraintes pour des problèmes "non purs", par exemple un problèmes de programmation linéaire en nombres entiers incluant quelques contraintes non linéaires, pour lesquels les algorithmes spécialisés ne s'appliquent pas.

Ainsi, nous nous intéresserons aux méthodes générales de résolution des problèmes de satisfaction de contraintes, qui sont nécessairement de complexité exponentielle. Nous allons présenter des procédés qui permettent d'améliorer l'algorithme d'énumération explicite, présenté dans la partie précédente, afin d'obtenir des temps de résolution fortement réduits.

Chapitre 3

Résolutions des CSPs binaires

Les problèmes auxquels nous nous intéressons sont définis par un nombre *fini* de variables et un nombre *fini* de contraintes. De plus, le domaine de chaque variable est un ensemble *fini*. Il est donc possible de résoudre ces problèmes par une exploration exhaustive de l'espace de recherche, c'est la méthode d'énumération explicite.

Cependant, le problème général de la satisfaction de contraintes est NP-complet, et l'on est souvent confronté à des problèmes extrêmement difficiles à résoudre. Ainsi, de nombreuses méthodes ont été développées pour accélérer la résolution, dont la cohérence d'arc.

Nous présentons dans cette partie les méthodes les plus utilisées pour résoudre efficacement les problèmes de satisfaction de contraintes. Ces méthodes sont fondées sur la cohérence d'arc ou des techniques qui en découlent.

3.1 L'ÉNUMÉRATION EXPLICITE

Le procédé le plus simple est la recherche arborescente (dite méthode de *back-track*) présentée ci-dessous. On instancie chaque variable avec une valeur de son domaine. Pour chaque instanciation on teste les contraintes. Si aucune contrainte n'est violée, l'instanciation est une solution. Dans le cas contraire, on change la valeur d'une des variables.

Cet algorithme est bien évidemment trop peu efficace pour résoudre des problèmes non triviaux. Il est cependant à la base de tous les algorithmes complets de résolution des problèmes de satisfaction de contraintes.

```
Choisir une variable V non instanciée
Instancier V avec une valeur X de son domaine
Enregistrer l'état courant du problème
Si toutes les variables sont instanciées alors
    Si toutes les contraintes sont satisfaites alors
        L'instanciation courante est une solution
    sinon
        Rétablir l'état précédent du problème
        Supprimer X du domaine de V
    Fin si
Fin si
Aller en 1)
```

Algorithme 1 Le premier algorithme de résolution de CSP par recherche arborescente exhaustive

Cet algorithme construit pas à pas une instanciation des variables et vérifie, une fois cette instanciation complétée, que toutes les contraintes sont satisfaites. Une amélioration de l'algorithme consiste à anticiper les échecs en testant les contraintes sur des instanciations incomplètes de l'ensemble des variables.

En effet, pour vérifier si une contrainte est violée ou satisfaite, il n'est pas nécessaire d'instancier toutes les variables du problème, mais seulement celles qui sont impliquées dans la contrainte. On remplace pour cela la fin de l'algorithme par les lignes suivantes :

```
Choisir une variable V non instanciée
Instancier V avec une valeur X de son domaine
Enregistrer l'état courant du problème
Si aucune contrainte n'est violée
    Si toutes les variables sont instanciées alors
        L'instanciation courante est une solution
    Fin si
    Aller en 1)
sinon
    Rétablir l'état précédent du problème
    Supprimer X du domaine de V
Fin si
```

Algorithme 2 Une première amélioration de l'algorithme de recherche arborescente exhaustive. Dans ce nouvel algorithme on prend en considération la violation de contraintes durant la construction des instanciations

Ce dernier algorithme est dit de *retour arrière chronologique*, traduction française du terme anglais *backtracking* que nous emploierons en général dans ce mémoire. Nous allons illustrer l'algorithme de backtracking par la résolution du problème des quatre reines :

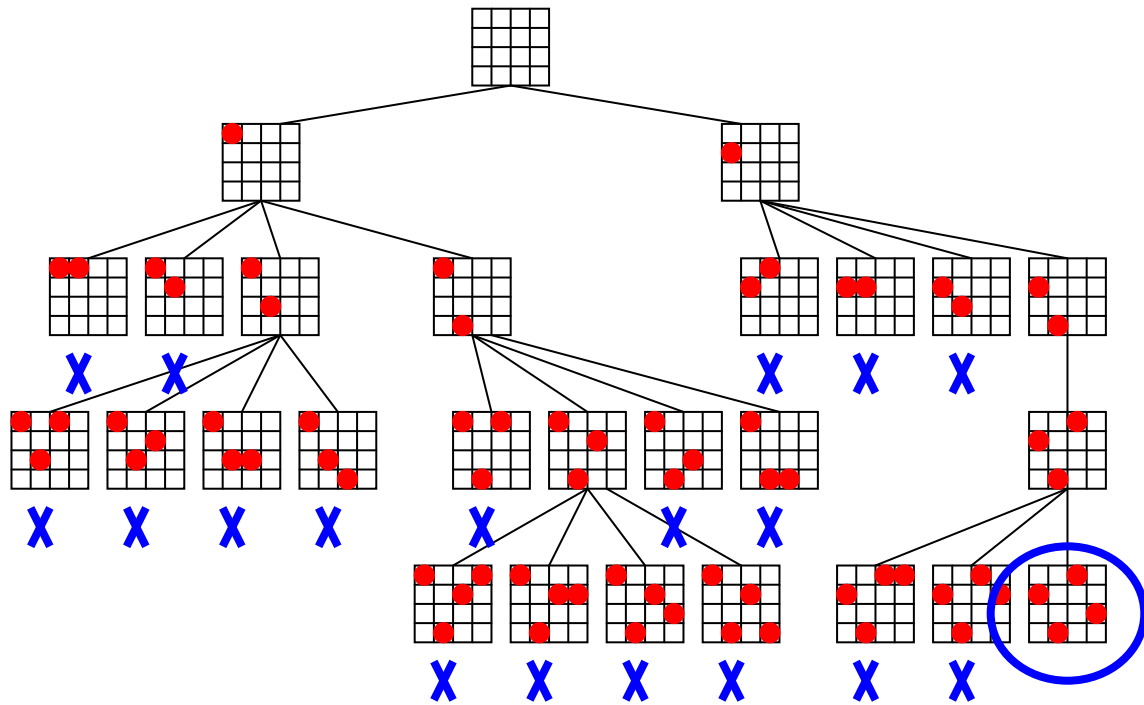


Figure 7 Résolution du problème des quatre reines avec l'algorithme de retour arrière chronologique. L'arbre est développé de haut en bas et de gauche à droite

En observant la résolution du problème des quatre reines (voir Figure 7), il apparaît clairement que certaines branches de l'arbre de recherche sont inutiles. Plus précisément, dans la première partie de la résolution, la variable Q_1 a reçu la valeur 1. L'algorithme teste ensuite, et à plusieurs reprises la valeur 1 pour les autres variables. Aucune de ces instanciations ne peut conduire à une solution puisque les contraintes exigent que les quatre variables aient des valeurs différentes.

Pour éviter ces explorations inutiles, Mackworth propose d'utiliser les contraintes de manière "active", c'est-à-dire de les utiliser pour éliminer *a priori* des valeurs ne pouvant apparaître dans aucune solution. Il est par exemple possible d'utiliser la contrainte C_{12} , qui représente la propriété " $Q_1 \neq Q_2$ ", de la façon suivante : une fois que la valeur 1 a été sélectionnée pour la variable Q_1 , C_{12} "interdit" à la variable Q_2 de prendre la valeur 1. On peut donc supprimer la valeur 1 du domaine de Q_2 pour la suite de la résolution sans risquer d'oublier des solutions. Techniquement, cette utilisation *active* des contraintes est fondée sur la propriété de *cohérence d'arc*, définie dans la section suivante.

3.2 LA COHÉRENCE D'ARC

Dans le paragraphe précédent nous avons montré, sur un exemple simple, que les contraintes peuvent être utilisées *a priori* pour supprimer des valeurs inutiles des domaines des variables. Cela signifie plus précisément que l'on peut utiliser les contraintes avant que toutes leurs variables soient instanciées pour éliminer des valeurs ne pouvant faire partie d'aucune solution.

Définition : la contrainte C_{ij} est dite *arc cohérente au sens de Mackworth* si, et seulement si, on a les deux propriétés suivante :

$$\forall x \in D_i ; \exists y \in D_j \text{ tel que } (x, y) \in R_{ij}$$

et

$$\forall y \in D_j ; \exists x \in D_i \text{ tel que } (x, y) \in R_{ij}$$

Définition 2 La propriété de cohérence d'arc au sens de Mackworth pour une contrainte binaire définie en extension

3.2.1 RÉALISER LA COHÉRENCE D'ARC D'UNE CONTRAINTE

La notion de cohérence d'arc au sens de Mackworth, cf. Définition 2, peut être obtenue de nombreuses manières différentes. On peut par exemple vider les domaines des variables, ce qui rend la contrainte trivialement arc cohérente, on peut aussi rajouter des valeurs dans l'extension de la contrainte afin de rendre chaque valeur cohérente avec la contrainte.

Il existe cependant une manière plus intéressante de rendre une contrainte arc cohérente : *supprimer des domaines des variables les valeurs qui n'appartiennent à aucune instanciation correcte de la contrainte*. Afin de définir de façon plus précise la cohérence d'arc d'une contrainte, nous devons d'abord définir ce que nous appellerons un *produit de domaines arc cohérent* pour une contrainte :

Définition : étant donnée C_{ij} une contrainte le produit de domaines $D'_i \times D'_j$ défini par les propriétés suivantes :

(i) $D'_i \subseteq D_i$

(ii) $D'_j \subseteq D_j$

(iii) $\forall x \in D_i ; \exists y \in D_j \text{ tel que } C_{ij}(x, y)$

est un *produit de domaines arc cohérent* de C_{ij}

Définition 3 Définition d'un produit de domaines arc cohérent pour une contrainte binaire

On peut très facilement démontrer la proposition suivante pour les produits de domaines arc cohérents :

Proposition : si $D'_i \times D'_j$ et $D''_i \times D''_j$ sont deux produits de domaines arc cohérents pour une contrainte C_{ij} alors le produit de domaines $D^3_i \times D^3_j$ défini par :

$$D^3_i = D'_i \cup D''_i$$

$$D^3_j = D'_j \cup D''_j$$

est arc cohérent.

Ce qui permet de donner la définition suivante pour la cohérence d'arc d'une contrainte :

Définition : étant donnée C_{ij} une contrainte, nous appellerons *arc cohérence* de C le produit de domaines arc cohérent $Dac_i \times Dac_j$ défini, de façon unique, par la propriété suivante : pour tout produit $D'_i \times D'_j$ de domaines arc cohérent de C_{ij} on a :

$$D'_i \subseteq Dac_i$$

$$D'_j \subseteq Dac_j$$

Définition 4 La cohérence d'arc pour une contrainte binaire

Cette définition suggère l'algorithme suivant pour rendre une contrainte donnée arc cohérente. La contrainte considérée dans l'algorithme ci-dessous est C_{12} impliquant les variables V_1 et V_2 de domaines D_1 et D_2 .

```

∀x ∈ Di faire
  Garder_x = faux
  ∀y ∈ Dj faire
    Si (x, y) ∈ Rij alors garder_x = vrai
    si non(garder_x) alors Di ← Di \ {x}
∀y ∈ Dj faire
  Garder_y = faux
  ∀x ∈ Di faire
    Si (x, y) ∈ Rij alors garder_y = vrai
    si non(garder_y) alors Dj ← Dj \ {y}

```

Algorithme 3 Algorithme pour rendre une contrainte binaire arc cohérente

La preuve que cet algorithme réalise l'arc cohérence de la contrainte considérée est aisée par l'absurde. De plus, cet algorithme ne supprime des domaines des variables que des valeurs ne pouvant pas satisfaire la contrainte. Pour plus de précisions sur ces notions, on se reportera à l'article de Mackworth [Mackworth, 1977].

3.2.2 RENDRE UN CSP ARC COHÉRENT

La Définition 4 présente la notion d'arc cohérence pour une contrainte. Un problème de satisfaction de contraintes de la façon suivante sera dit arc cohérent si chacune de ses contrainte l'est, ce qui est exprimé par la définition suivante :

Définition : un problème de satisfaction de contraintes P est dit arc cohérent si chacune des contraintes de P est arc cohérente.

Définition 5 La cohérence d'arc d'un problème de satisfaction de contraintes

Cette définition suggère également un algorithme simple pour rendre un CSP arc cohérent sans éliminer aucune de ses solutions.

```
Q = C(P)
Tant que Q ≠ ∅ faire
    Soit C ∈ Q
    Q ← Q \ {C}
    ∀ v ∈ V(C) faire "on mémorise la taille des domaines"
        T(v) ← Card(D(v)) "pour savoir s'il y a eu réduction"
    Rendre C arc cohérente
    ∀ v ∈ V(C) tel que Card(D(v)) < T(v) "on a réduit D(v)"
        alors Q ← Q ∪ C(v) "on ajoute les contraintes de v"
```

Algorithme 4 Algorithme pour rendre un CSP arc cohérent. Cet algorithme est inspiré de AC-3

Nous pouvons donner *a contrario* une bonne compréhension de la propriété d'arc cohérence pour un CSP. Un CSP n'est pas arc cohérent lorsqu'une valeur d'une de ses variables n'est pas susceptible de satisfaire au moins une des contraintes du problème. Autrement dit, rendre un CSP arc cohérent peut se faire en éliminant toutes les valeurs dont on peut déduire, *en étudiant les contraintes individuellement*, qu'elles ne peuvent apparaître dans aucune solution. C'est exactement ce que fait l'algorithme ci-dessus.

Il faut de plus remarquer qu'il n'existe pas de relation directe entre l'existence de solution et la propriété d'arc cohérence pour un CSP. Certains problèmes n'ont pas de solution tout en étant arc cohérents, et inversement, l'existence de solution n'implique pas la cohérence d'arc. Il résulte de cette remarque que les algorithmes utilisés pour rendre arc cohérent un CSP ne sont pas des algorithmes de résolution. Ils sont utilisés pour réduire la taille du problème, mais ne fournissent pas de solution.

La nature NP-complète (cf. 2.3) du problème général de la satisfaction de contraintes a motivé de nombreuses recherches sur les algorithmes de résolution, mais aussi sur les algorithmes de cohérence d'arc qui permettent de réduire la taille des problèmes. Nous ne ferons pas ici une étude détaillée de ces algorithmes, mais voici quelques références importantes dans ce domaine. Le premier algorithme d'obtention de la cohérence d'arc d'un CSP est dû à Mackworth, et est appelé AC-3 [Mackworth, 1977]. Mohr et Henderson [Mohr and Henderson, 1986] ont proposé AC-4, un algorithme de cohérence d'arc dont la complexité au pire des cas est optimale. AC-5, de Deville et Van Hentenryck [Deville and Hentenryck, 1991] est une amélioration de AC-4, qui prend en compte la nature de certaines contraintes pour obtenir une meilleure complexité dans certains cas. Plus récemment, AC-6 [Bessière, 1994], a la même complexité que AC-4, mais améliore la complexité spatiale. Finalement, AC-7

[Bessière and Régin, 1995] permet d'éviter certains tests de satisfaction en utilisant des inférences, fondées sur la bidirectionnalité des contraintes binaires.

3.3 LA COHÉRENCE DE CHEMIN

Mackworth introduit une autre propriété des contraintes, nommée cohérence de chemin. Cette propriété est une généralisation de la cohérence d'arc au cas de plusieurs contraintes. En effet, la cohérence d'arc consiste, comme nous l'avons écrit plus haut, à supprimer les valeurs qui ne sont pas compatibles avec une contrainte donnée. Il est néanmoins fréquent que des valeurs soient compatibles avec chaque contrainte considérée séparément, mais incompatible avec un groupe de contraintes. La définition de la cohérence de chemin, donnée par Mackworth, est la suivante :

Définition : Un chemin de longueur m sur les variables $(V_{i_0}, V_{i_1}, \dots, V_{i_m})$ est chemin cohérent si, et seulement si :

$$\begin{aligned} &\forall z_0 \in Di_0 \text{ et } \forall z_m \in Di_m \text{ tels que } Ci_0i_m(z_0, z_m) \\ &\exists z_1 \in Di_1, \dots, \exists z_{m-1} \in Di_{m-1} \text{ tels que} \\ &\forall k = 1, \dots, m \text{ et } \forall l = 1, \dots, m \text{ on a } Pi_ki_l(z_k, z_l) \end{aligned}$$

Définition 6 La cohérence de chemin pour un chemin de contraintes binaires

Définition : un problème de satisfaction de contraintes P est dit chemin cohérent si chaque chemin de contraintes de P est chemin cohérent.

Définition 7 La cohérence de chemin d'un CSP

3.4 LES ALGORITHMES DE RÉOLUTION

Étant donné que le problème général de la satisfaction de contraintes est NP-complet, il a été produit un effort important pour la mise au point d'algorithmes efficaces pour leur résolution. Depuis les premiers travaux de Mackworth, on dénombre des dizaines d'algorithmes de résolution, que l'on peut classer en plusieurs catégories : les algorithmes incomplets de type "heuristiques gloutonnes" [Selman, et al., 1992] et les algorithmes complets. Nous nous intéressons ici uniquement aux algorithmes complets.

Nous ne ferons pas dans ce document une étude détaillée de l'ensemble de ces algorithmes, mais nous présentons certains d'entre eux qui nous seront utiles par la suite. Ces algorithmes se rangent naturellement en trois familles : ceux qui sont fondés sur la cohérence d'arc, ceux qui sont fondés sur les procédés de retour arrière "intelligent", et les algorithmes hybrides, fondés sur ces deux techniques.

3.4.1 LES ALGORITHMES FONDÉS SUR LA COHÉRENCE D'ARC

Nous avons montré que la propriété d'arc cohérence peut être utilisée pour réduire les domaines des variables d'un CSP en utilisant ses contraintes. Nous avons illustré ce point sur un problème de cryptogramme : une fois le CSP défini,

l'application d'un algorithme d'arc cohérence permet une réduction sensible de la taille du problème.

Pour améliorer encore la complexité de la résolution des CSP, nous pouvons utiliser la cohérence d'arc durant la résolution du problème par une méthode de recherche arborescente. Plusieurs algorithmes sont fondés sur cette idée. Trois d'entre eux sont décrits ci-dessous.

Le premier algorithme est une implémentation de l'idée suivante : durant l'exploration de l'arbre de recherche, après chaque choix d'une valeur, on peut appliquer un algorithme de cohérence d'arc au problème, afin d'en réduire la taille. Cet algorithme s'appelle *full lookahead* [Gaschnig, 1977].

Real Full Look-ahead (RFL) Choisir une variable V non instanciée Instancier V avec une valeur x de son domaine Mémoriser l'état courant du CSP (i.e. les domaines des variables) Appliquer un algorithme de cohérence d'arc à tout le problème Si le domaine d'une des variables est vidé alors Restaurer les domaines d'un état précédent du problème Supprimer x du domaine de V Si toutes les variables sont instanciées, alors une solution est trouvée Retour.
--

Figure 8 L'algorithme de real full lookahead

Cet algorithme explore un espace de recherche aussi petit que possible, au sens de la cohérence d'arcs. Cependant, à chaque pas de la boucle, il requiert l'application à tout le problème d'un algorithme globale de cohérence d'arc. Schématiquement, cet algorithme explore *lentement* un *petit* espace de recherche.

Un autre algorithme, appelé *forward checking* [Haralick and Elliott, 1980], consiste à n'appliquer la cohérence d'arc qu'aux seules contraintes impliquant la dernière variable instanciée. Le coût de l'application de la cohérence d'arc à chaque pas est ici fortement réduit par rapport à l'algorithme précédent, en revanche, les réductions de domaines y sont plus limitées. On peut dire que cet algorithme explore *rapidement* un *grand* espace de recherche.

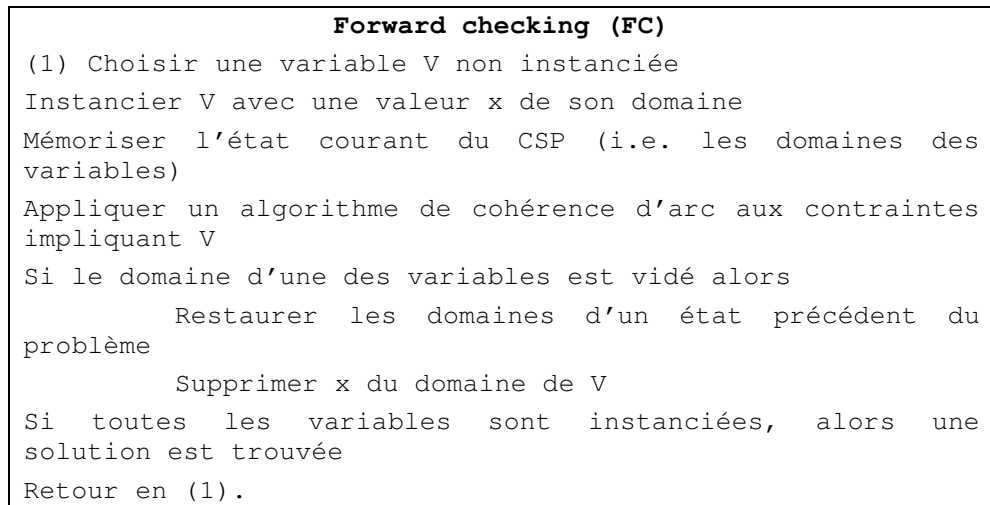


Figure 9 L'algorithme de forward checking

Les Figure 11 et Figure 11 montrent les arbres de recherche développés respectivement par l'algorithme de forward-checking et l'algorithme de real full look-ahead lors de la résolution du problème des quatre reines :

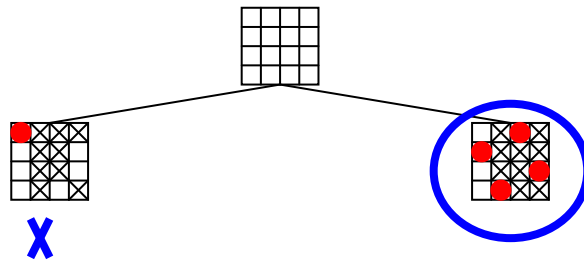


Figure 10 En haut, l'arbre de recherche développé par le real full look-ahead pour la résolution des quatre reines. En bas,

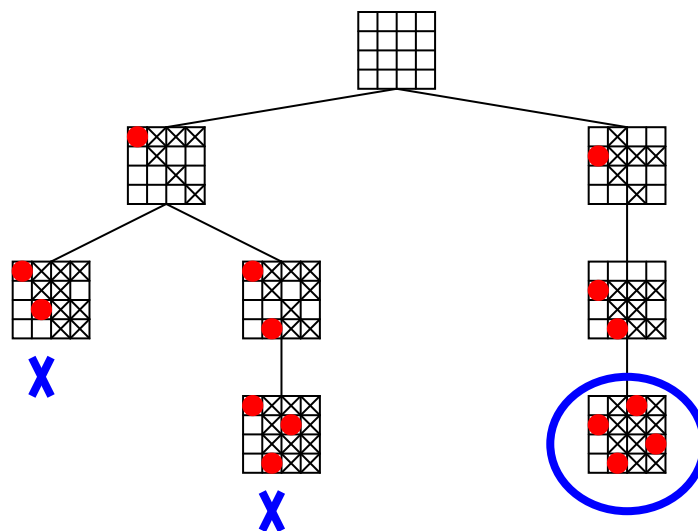


Figure 11 l'arbre développé par le forward-checking pour la résolution du problème des quatre reines

Sur la Figure 11 on voit que l'arbre de recherche développé par l'algorithme de real full look-ahead est très petit, en effet, seulement quatre branches sont dévelop-

pées. L'arbre de recherche développé par le forward-checking comporte deux fois plus de branches. Ces deux nombres sont à comparer aux 26 branches de l'arbre développé par l'algorithme d'énumération explicite. En revanche, à chaque nœud de l'arbre, ces algorithmes requièrent l'application de la cohérence d'arc à certaines contraintes du problème (le real full look-ahead requiert l'application de la cohérence d'arc à tout le problème alors que le forward-checking applique la cohérence d'arc aux seules contraintes impliquant la dernière variable instanciée), ce qui ralentit l'exploration. On voit ainsi apparaître l'une des clefs de la résolution des problèmes de satisfaction de contraintes : trouver le meilleur compromis possible entre la vitesse d'exploration de l'arbre de recherche et la taille de cet arbre. Sur un problème aussi facile que les quatre reines, la solution la plus efficace est l'énumération explicite, mais sur des problèmes plus difficiles, il devient judicieux d'utiliser des algorithmes fondés sur la réduction par cohérence d'arc.

3.4.2 LES ALGORITHMES D'ÉNUMÉRATION FONDÉS SUR LE RETOUR ARRIÈRE "INTELLIGENT"

Nous avons souligné l'une des faiblesses capitales de l'algorithme d'énumération explicite : le fait qu'il explore des branches inutiles de l'arbre de recherche. Nous avons décrit deux algorithmes qui permettent d'éviter de développer de telles branches par utilisation de la propriété de cohérence d'arc.

Cependant, il existe une autre faiblesse de l'algorithme d'énumération explicite : en cas d'échec, il remet en question le *dernier* choix effectué. Souvent, le dernier choix effectué n'est en rien responsable de l'échec, il est alors inutile de le remettre en question. Voici un exemple simple de cette idée :

Considérons trois variables : X, Y, Z de domaine respectifs : $\{1, 2\}, \{1, 2, 3\}$ et $\{2, 3, 4\}$ et trois contraintes : $X = Z, X \leq Y$ et $Y \leq Z$. Si l'on emploie l'Algorithme 2 pour résoudre ce problème, le début de la recherche sera le suivant : $X \leftarrow 1, Y \leftarrow 1$ et $Z \leftarrow 2$, qui provoque un échec à cause de la contrainte $X = Z$. Ensuite, l'Algorithme 2 va tester les valeurs 2, 3, 4 pour la variable Z , sans succès. Puis il remettra en question le choix de $Y \leftarrow 1$ pour choisir $Y \leftarrow 2$, puis $Y \leftarrow 3$ etc. alors que c'est le choix $X \leftarrow 1$ qui est en cause.

L'algorithme appelé *conflict-directed backjumping (CBJ)* permet de ne pas **faire de remise en question inutile**. Il fonctionne de la façon suivante : il maintient en permanence, la liste des variables ayant eu un conflit avec la variable courante (un conflit désigne l'incompatibilité de deux valeurs). En cas d'échec, l'algorithme retourne directement à la dernière variable ayant provoqué un conflit, en ignorant les variables n'ayant provoqué aucun conflit. Illustrons ce fonctionnement sur notre problème simple :

$X \leftarrow 1$ aucun conflit
 $Y \leftarrow 1$ aucun conflit
 $Z \leftarrow 2$ conflit avec X à cause de $X = Z$
 $Z \leftarrow 3$ conflit avec X à cause de $X = Z$
 $Z \leftarrow 4$ conflit avec X à cause de $X = Z$ et **échec** car le domaine de Z est vide
retour directement à X , car il n'y a eu aucun conflit entre Y et Z
 $X \leftarrow 2$ aucun conflit
 $Y \leftarrow 1$ conflit avec X à cause de $X \leq Y$
 $Y \leftarrow 2$ aucun conflit
 $Z \leftarrow 2$ solution

3.4.3 ALGORITHMES FONDÉS SUR LE RETOUR ARRIÈRE “INTELLIGENT” ET LA COHÉRENCE D’ARC

Plusieurs algorithmes ont été imaginés pour combiner les avantages des techniques de retour arrière “intelligent” et de cohérence d’arc. Les travaux dans ce sens, qui sont principalement dus à Prosser [Prosser, 1993a, Prosser, 1993b], ont donné lieu à une famille d’algorithmes hybrides. Un exemple intéressant est l’algorithme FC-CBJ, ou forward checking et conflict-directed backjumping, qui permet une réduction importante de la taille de l’arbre de recherche par utilisation du forward checking, tout en prévenant les retours arrière inutiles par utilisation de la technique du conflict-directed backjumping.

Nous reproduisons ci-dessous une classification, due à Prosser, des algorithmes d’énumération :

	→ Utilisation du retour arrière “intelligent” →		
↓	Énumération explicite	Backjumping BJ	Conflict-directed BJ
Utilisation de la cohérence d’arc	Backmarking BM	BM-BJ	BM-CBJ
↓	Forward checking FC	FC-BJ	FC-CBJ

Figure 12 Classification des algorithmes d’énumération pour les CSPs binaires. Les algorithmes sont rangés de gauche à droite selon l’utilisation des techniques de retour arrière “intelligent”. De haut en bas, ils sont rangés selon l’utilisation des techniques de réduction par cohérence d’arc

Chapitre 4

Définition d'un formalisme plus riche

Le formalisme défini ci-dessus est trop rudimentaire pour permettre de définir aisément des problèmes, même quand ceux-ci sont très simples. À titre d'illustration, si l'on veut définir le problème des huit reines sur l'échiquier, la taille et le nombre des contraintes deviennent rédhibitoires, quid des cent reines ? De plus, il est impossible d'utiliser ce formalisme pour définir un problème de cryptogramme tel que le Problème 2, comme il sera illustré plus loin.

Les principales limitations de ce formalisme concernent d'une part la manière de définir les contraintes et le fait que ces contraintes soient binaires. Nous verrons dans cette partie une première forme étendue du paradigme des CSP, que nous enrichirons encore par la suite.

4.1 LES LIMITATIONS DU FORMALISME DE BASE

Dans le formalisme que nous venons de présenter, les contraintes sont définies par la liste exhaustive des instanciations des variables qui satisfont la contrainte. C'est ce que l'on appelle la définition des contraintes en *extension*. Cette manière est à la fois coûteuse et peu naturelle, et on verra plus loin dans ce document qu'elle limite les performances du système et empêche certaines extensions.

Une meilleure approche consiste à définir les contraintes par une formule de satisfaction, c'est la définition en *compréhension* des contraintes.

Considérons par exemple deux variables x et y ayant pour domaine l'ensemble $\{1, 2, 3\}$ et la contrainte exprimant le fait que les valeurs des variables x et y sont différentes. La définition en extension se fera à l'aide de la liste $\{(1, 2), (1, 3), (2, 3)\}$. La définition en compréhension se fera simplement par la formule $x \neq y$.

Dans notre contexte, puisque les domaines des variables sont des ensembles finis, il est clair que toute contrainte définie en extension peut être définie en compréhension. Plus précisément, pour une contrainte C , portant sur un ensemble

$\{V_1, V_2, \dots, V_n\}$ de variables, et dont l'extension est l'ensemble R , la formule est la suivante : $(V_1, V_2, \dots, V_n) \in R$.

Il est inversement possible de définir en extension toute contrainte définie en compréhension, et ceci en appliquant la formule à tous les éléments du produit cartésien des domaines des variables. Cette représentation est très coûteuse à calculer, elle est également encombrante en mémoire, et de surcroît elle est souvent moins efficace.

Nous adopterons dorénavant la définition des contraintes en compréhension.

Dans le formalisme présenté précédemment, les contraintes portent systématiquement sur deux variables, ce qui constitue un handicap rédhibitoire pour la définition de problèmes un tant soit peu complexes. Considérons par exemple le cryptogramme $SEND + MORE = MONEY$ défini au Problème 2, et donnons une en définition possible avec des contraintes binaires :

Variables: s, e, n, d, m, o, r et y dont le domaine est $\{0, 1, \dots, 9\}$.
 Contraints: $s \neq e ; s \neq n ; s \neq d ; \dots etc... r \neq y$
 Variables: se, nd, mo, re, on et ey dont le domaine est $\{0, 1, \dots, 99\}$
 Contraints: $nd \equiv d (10)$ " nd est congru d modulo 10"
 $[nd / 10] = n$ "où $[.]$ désigne la partie entière d'un nombre réel"
 $se \equiv e (10)$ et $[se / 10] = s$
 $re \equiv e (10)$ et $[re / 10] = r$
 $mo \equiv o (10)$ et $[mo / 10] = m$
 $on \equiv n (10)$ et $[on / 10] = o$
 $ey \equiv y (10)$ et $[ey / 10] = e$
 Variables: $send, more$ et $oney$ dont le domaine est $\{0, \dots, 9\,999\}$
 Contraints: $send \equiv nd (100)$ et $[send / 100] = se$
 $more \equiv re (100)$ et $[more / 100] = mo$
 $oney \equiv ey (100)$ et $[oney / 100] = no$
 Variable: $money$ dont le domaine est $\{0, \dots, 99\,999\}$
 Contraints: $money \equiv oney (1,000)$ et $[money / 1,000] = m$
 Variable: $sendmore \in \{(0,0), (0,1), \dots, (0,9999), (1,1), \dots, (1,9999), \dots, (9999,0), \dots, (9999,9999)\}$
 Contraints: $sendmore1 = send$ et $sendmore2 = more$
 Variable: $sendplusmore$ dont le domaine est $\{0, \dots, 19\,998\}$
 Contraint: $sendplusmore = (sendmore1 + sendmore2)$

CSP 3 La définition du cryptogramme $SEND + MORE = MONEY$ dans le formalisme des CSP binaires

Cette définition présente le double inconvénient d'être encombrante et très éloignée de l'énoncé mathématique du problème. Plus précisément, l'utilisation des seules contraintes binaires oblige à définir de nombreuses contraintes et variables intermédiaires. Finalement, une telle approche requiert la définition de 49 contraintes binaires, et elle nécessite environ 100.000.000 de valeurs pour la constitution des domaines des variables. En outre, cette approche oblige à déclarer des contraintes qui ne font aucunement partie de l'énoncé mathématique du problème, et qui ne sont pas naturelles.

Cette restriction du formalisme aux seules contraintes binaires est due à plusieurs raisons. D'une part, les problèmes binaires sont directement interprétables dans

la théorie des graphes, ce qui présente de nombreux avantages : on peut facilement les représenter graphiquement, et on peut utiliser la théorie des graphes pour imaginer des résultats théoriques.

D'autre part, les problèmes binaires sont un contexte de travail plus simple, et il est facile d'imaginer pour les résoudre de nombreux algorithmes qui sont faciles à concevoir et à implémenter.

Enfin, il est prouvé que tout CSP est équivalent à un CSP binaire au sens où il est possible de trouver une bijection entre les ensembles de solutions des deux problèmes.

Ces trois raisons font que de nombreux résultats et algorithmes sont exposés dans le cadre des CSP binaires, et que la généralisation aux problèmes non binaires est généralement considérée comme triviale et laissée à la charge du lecteur.

La définition donnée ci-dessus pour le cryptogramme SEND + MORE = MONEY donne une idée de la difficulté de définition d'un problème en termes de contraintes binaires. Nous verrons en outre que les techniques employées pour la résolution des CSP ne s'appliquent pas de la même façon sur les problèmes binaires et sur les problèmes généraux. Et finalement, il est évident que l'adaptation d'un algorithme quelque peu complexe pour les problèmes binaires au contexte des problèmes non binaires n'est généralement pas envisageable des raisons topologiques.

4.2 DÉFINITION D'UN FORMALISME PLUS RICHE

Afin de surmonter les limitations soulignées dans la partie précédente, nous proposons pour la satisfaction de contraintes un formalisme plus riche que celui présenté précédemment. Ce formalisme permet la définition de contraintes en compréhension, ainsi que la création de contraintes non binaires.

Un problème de satisfaction de contraintes, dans ce formalisme, est défini par la liste des éléments suivants :

1. Un ensemble fini de *variables contraintes*, chacune associée à un ensemble fini de valeurs, son *domaine*.
2. Un ensemble fini de contraintes. Une contrainte est définie par l'ensemble des variables qu'elle implique, et par une formule de satisfaction, c'est-à-dire en compréhension.

On utilisera dans ce document les notations suivantes pour désigner les différents éléments d'un problème de satisfaction de contraintes :

Pour un CSP P , on notera $V(P)$ l'ensemble des variables de P , et $C(P)$ l'ensemble de ses contraintes. Pour une contrainte C , on notera de la même manière $V(C)$ l'ensemble des variables impliquées par C , et pour une variable V , on notera $C(V)$ l'ensemble des contraintes impliquant V . Le domaine d'une variable V sera désigné par la notation $D(V)$.

Pour simplifier les définitions, étant donnée une contrainte C , nous noterons souvent $V_1, \dots, V_n$ les variables qu'elle implique, et D_i le domaine de V_i .

4.3 EXEMPLES DE PROBLÈMES DÉFINIS AVEC DES CONTRAINTES NON BINAIRES

Ce formalisme enrichi permet une définition simple des problèmes que nous avons présentés précédemment, ce qui n'était pas le cas du formalisme original. Nous donnons ici une définition de ces problèmes.

- Le problème des reines sur un échiquier

Variables :	$Q_1, Q_2, \dots, Q_n$
Domaines :	$D_i = \{1, 2, \dots, n\}$
Contraintes :	$C_{ij} = (Q_i \neq Q_j) \wedge (i - j \neq Q_i - Q_j)$

CSP 4 Le problème des n reines dans le formalisme étendu. Cette définition est à comparer avec celle donnée aux CSP 1 et CSP 2

- Le cryptogramme "SEND+MORE=MONEY"

Variables :	S, E, N, D, M, O, R, Y
Domaines :	$\{0, 1, \dots, 9\}$ pour toutes les variables
Contraintes :	$S \neq 0 ; M \neq 0$ S, E, N, D, M, O, R, Y sont 2 à 2 distinctes $1.000 \times S + 100 \times E + 10 \times N + D$ $+ 1.000 \times M + 100 \times O + 10 \times R + E$ $= 10.000 \times M + 1.000 \times O + 100 \times N + 10 \times E + Y$

CSP 5 Le cryptogramme SEND+MORE=MONEY dans le formalisme étendu. Cette définition est à comparer à celle donnée dans le contexte des contraintes binaires au CSP 5

- Le problème des carrés magiques

Variables : V_{ij} pour i et j allant de 1 à n Domaines : $\{1, 2, \dots, n^2\}$ pour toutes les variables Contraintes : Les V_{ij} sont deux à deux distincts $\forall i \sum_j V_{ij} = S$, les lignes ont même somme $\forall j \sum_i V_{ij} = S$, les colonnes ont même somme $\sum_i V_{ii} = S$, pour la première diagonale $\sum_i V_{i(n-i)} = S$, pour la seconde diagonale avec $S = 1 + 2 + \dots + n$
--

CSP 6 Les carrés magiques dans le formalisme étendu

Chapitre 5

Techniques de résolution des problèmes non binaires

Dans ce chapitre, nous allons montrer les limitations des premières techniques de résolutions que nous avons présentées (i.e. les procédés de résolution fondés sur l'énumération et les notions de cohérence d'arc et de chemin). Ensuite, nous présenterons des techniques plus sophistiquées qui permettent de résoudre efficacement les problèmes de satisfaction de contraintes. Ces techniques sont celles qui sont utilisées en pratique dans les système de satisfaction de contraintes.

5.1 LIMITATIONS DES PREMIÈRES TECHNIQUES DE RÉOLUTION

Au chapitre précédent, nous avons prôné l'utilisation des contraintes non binaires et définies en compréhension. Ce formalisme, plus riche, permet, comme nous l'avons montré plus haut, de définir des problèmes aisément. En revanche, la résolution de problèmes exprimés dans ce formalisme nécessite d'adapter les techniques de résolution pour permettre la manipulation des contraintes non binaires. Dans cette partie, nous soulignons certaines limitations des techniques de résolution présentées dans les chapitres précédents.

5.1.1 LA COHÉRENCE D'ARC POUR LES CONTRAINTES NON BINAIRES EST TROP COÛTEUSE EN PRATIQUE

La notion de cohérence d'arc a été, comme son nom l'indique, inventée dans le contexte des contraintes binaires. Si sa définition s'étend facilement aux contraintes non binaires, comme nous le faisons au paragraphe suivant, son application pose alors des problèmes d'efficacité, que nous soulignons dans les paragraphes qui suivent. Des solutions à ces problèmes seront proposées au chapitre suivant.

A) EXTENSION DE LA COHÉRENCE D'ARC AUX CONTRAINTES NON BINAIRES

Nous avons défini la propriété de cohérence d'arc pour des contraintes binaires (cf. Définition 2). On peut immédiatement généraliser cette définition aux contraintes non binaire définies en compréhension comme suit :

Définition : une contrainte C (avec $V(C) = \{V_1, \dots, V_n\}$) est dite *arc cohérente au sens de Mackworth* si, et seulement si, on a les deux propriétés suivante :

$$\begin{aligned} & \forall i = 1, \dots, n ; \forall x \in D_i ; \\ & \exists (x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n) \in D_1 \times \dots \times D_{i-1} \times D_{i+1} \times \dots \times D_n \\ & \text{tel que} \\ & C(x_1, \dots, x_n) \end{aligned}$$

Définition 8 La propriété de cohérence d'arc au sens de Mackworth pour une contrainte quelconque binaire définie en compréhension

De la même façon, la notion de *produit de domaines arc cohérent* pour une contrainte peut être définie ainsi pour une contrainte non binaire :

Définition : étant donnée C une contrainte impliquant les $V(C) = \{V_1, \dots, V_n\}$ le produit de domaines $D'_1 \times \dots \times D'_n$ défini par les propriétés suivantes :

$$\begin{aligned} (i) & \quad \forall i = 1, \dots, n ; D'_i \subseteq D_i \\ (ii) & \quad \forall i = 1, \dots, n ; \forall x \in D_i ; \\ & \quad \exists (x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n) \in D_1 \times \dots \times D_{i-1} \times D_{i+1} \times \dots \times D_n \\ & \quad \text{tel que} \\ & \quad C(x_1, \dots, x_n) \end{aligned}$$

est un *produit de domaines arc cohérent* de C

Définition 9 Définition d'un produit de domaines arc cohérent pour une contrainte non binaire

Et finalement, voici la définition de la cohérence d'arc d'une contrainte non binaire :

Définition : étant donnée C une contrainte, nous appellerons *arc cohérence* de C le produit de domaines arc cohérent $Dac_1 \times \dots \times Dac_n$ défini, de façon unique, par la propriété que pour tout produit $D'_1 \times \dots \times D'_n$ de domaines arc cohérent de C on a :

$$\forall i = 1, \dots, n ; D'_i \subseteq Dac_i$$

Définition 10 La cohérence d'arc pour une contrainte

L'algorithme suivant permet d'obtenir la cohérence d'arc pour une contrainte C non binaire impliquant les variables $V_1, \dots, V_n$ de domaine respectifs $D_1, \dots, D_n$:

```

 $\forall i \in \{1, \dots, n\}$ 
   $\forall x \in D_i$  faire
    si  $\forall (x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n) \in D_1 \times \dots \times D_{i-1} \times D_{i+1} \times D_n \neg C(x_1, \dots, x_n)$ 
    alors  $D_i \leftarrow D_i \setminus \{x\}$ 

```

Algorithme 5 Algorithme pour rendre une contrainte non binaire arc cohérente

Cet algorithme, qui est une généralisation directe de l'Algorithme 3, a une complexité exponentielle au pire des cas et très mauvaise en pratique. Plus précisément, il est par exemple inutilisable pour des contraintes impliquant plus de 8 variables ayant un domaine contenant dix éléments. Voici une version plus performante de cet algorithme, dont la complexité au pire des cas est également exponentielle, mais dont la complexité en pratique est bien meilleure :

```

Soient  $D'_1 = \emptyset, D'_2 = \emptyset, \dots, D'_n = \emptyset$ 
 $\forall i \in \{1, \dots, n\}$ 
   $\forall x \in D_i$  faire
    Si
       $\exists (x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n) \in D_1 \times \dots \times D_{i-1} \times D_{i+1} \times D_n$ 
      tel que  $C(x_1, \dots, x_n)$ 
    alors
       $\forall j = 1, \dots, n$ 
         $D'_j \leftarrow D'_j \cup \{x_j\}$ 
      sinon  $D_i \leftarrow D_i \setminus \{x\}$ 
 $\forall j = 1, \dots, n$ 
   $D_j \leftarrow D'_j$ 

```

Algorithme 6 Un algorithme plus performant en pratique pour rendre une contrainte non binaire arc cohérente

B) ILLUSTRATIONS DE L'ARC COHÉRENCE APPLIQUÉE À UN PROBLÈME NON BINAIRE

Afin de donner une idée des effets de l'arc cohérence sur un problème simple, considérons l'exemple de l'addition cryptée présentée dans le Problème 2. Le Tableau 2 suivant donne les domaines des différentes variables du problème avant et après application d'un algorithme d'arc cohérence à tout le problème.

Variable	Domaines avant arc cohérence	Domaines après arc cohérence
S	0...9	9
E	0...9	5...6
N	0...9	6...7
D	0...9	6...7
M	0...9	1
O	0...9	0
R	0...9	8
Y	0...9	2...3

Tableau 2 Domaines des variables avant et après application d'un algorithme d'arc cohérence

Dans le tableau précédent les réductions de domaines conduisent à une diminution importante de la taille du problème, et donc de sa complexité. En effet, dans le problème initial, le nombre de branches de l'arbre de recherche développé par la méthode de recherche arborescente exhaustive est 10^8 , soit cent millions. Dans le problème réduit par arc cohérence, le nombre de branches n'est plus que de 16.

L'application d'un algorithme d'arc cohérence ne donne pas nécessairement des résultats intéressants. À titre d'illustration, il ne découle aucune réduction de domaines lorsque l'on applique un algorithme d'arc cohérence au problème des reines. En d'autres termes, dans la formulation présentée à la 1.1, le problème des reines est déjà arc cohérent. En effet, la propriété d'arc cohérence ne concerne que les contraintes considérées individuellement. Il est clair que chaque contrainte du problème est cohérente par arc, puisque si on choisit une place pour une seule reine sur l'échiquier, il reste toujours possible de placer une seconde reine sans violer de contrainte.

C) LA COHÉRENCE D'ARC EST COÛTEUSE

Nous venons de montrer que l'application d'un algorithme de cohérence d'arc à un problème permet, dans certains cas, une réduction sensible de la complexité. Afin d'apprécier l'intérêt réel de cette réduction des domaines, il faut évaluer le coût de l'application d'un tel algorithme de cohérence d'arc.

La complexité des algorithmes qui réalisent la cohérence d'arc d'un problème peut se décomposer en deux parties. D'une part, le nombre de contraintes rendues arc cohérentes par l'algorithme, et d'autre part, le coût de l'obtention de la cohérence d'arc chaque contrainte.

Le nombre de contraintes rendues cohérentes par arc dépend de l'algorithme utilisé. Pour le forward-checking, et en considérant le cas d'un problème binaire, le nombre de contraintes considérées à chaque étape est le nombre de contraintes impliquant la dernière variable instanciée, qui est en pratique assez petit. Pour l'algorithme de full look-ahead, ce nombre est plus difficile à évaluer puisqu'on applique la cohérence d'arc à toutes les contraintes jusqu'à obtention d'un point fixe (i.e. toutes les

contraintes sont arc cohérentes). Selon le problème, ce point fixe peut être plus ou moins difficile à obtenir.

Le Tableau 3 ci-dessous montre le nombre de contraintes rendues arc cohérentes lors de la résolution de deux types de problèmes avec l'algorithme de real full look-ahead. Ce nombre augmente rapidement avec la taille des instances. De plus, une analyse de la résolution d'un problème montre que la majeure partie du temps est passée à la réalisation de la cohérence d'arc. Plus précisément, même en employant des méthodes très efficaces, qui sont exposées dans la suite de cette section, 80% du temps de résolution sont employés de cette manière.

Il est donc crucial d'avoir des méthodes efficaces d'obtention de la cohérence d'arc.

Problème des reines sur l'échiquier		Problème des carrés magiques	
Taille	Nombre de cohérences	Taille	Nombre de cohérences
4 reines	23	3×3	184
16 reines	607	4×4	999
32 reines	2.346	5×5	37.902

Tableau 3 Nombre de contraintes rendues arc cohérentes lors de la résolution d'instances du problème des reines et du problème des carrés magiques

En revanche, il est plus facile d'évaluer le coût de l'obtention de la cohérence d'arc pour une contrainte. L'Algorithme 6, qui permet de rendre une contrainte quelconque arc cohérente, a la complexité exponentielle suivante au pire des cas : il requiert un nombre d'opérations égal à la *taille du produit cartésien des domaines des variables*. Cela signifie que pour une contrainte telle que la contrainte :

$$1.000.S+100.E+10.N+D+1.000.M+100.O+10.R+E = \\ 10.000.M+1.000.O+100.N+10.E+Y$$

du cryptogramme SEND+MORE=MONEY il faudra au pire des cas, en considérant les domaines initiaux qui contiennent 10 valeurs, 10^8 opérations d'évaluations de la formule et de manipulation de domaines, ce qui s'avère très coûteux en pratique ($10^8 = 100$ millions).

En conclusion, il s'avère impossible d'employer en pratique les méthodes classiques d'obtention de la cohérence d'arc puisque, d'une part, le nombre d'opération de cohérence d'arc pour une contrainte est très important (cf. Tableau 3), et que, d'autre part, rendre une contrainte non binaire arc cohérente est très coûteux. Au paragraphe 5.2, nous allons présenter des méthodes qui permettent d'obtenir efficacement la cohérence d'arc d'une contrainte et nous présenterons une alternative à la cohérence d'arc : le *filtrage* de contrainte.

5.1.2 LA COHÉRENCE DE CHEMINS EST TROP COÛTEUSE À METTRE EN ŒUVRE

La cohérence de chemin est une technique de réduction des domaines plus forte que la cohérence d'arc. Réaliser la cohérence d'arc d'un problème consiste à s'assurer qu'aucune valeur n'est incompatible avec chaque contrainte. Les contraintes sont dans ce cas considérées individuellement. La cohérence de chemin est une extension de la cohérence d'arc à des groupes de contraintes. À l'évidence, d'un point de vue combinatoire, cette technique est plus coûteuse à mettre en œuvre que la cohérence d'arc. Cela explique en partie le manque de succès de cette technique. Mais il existe une raison plus essentielle nous présentée plus loin 5.2.2 de cette partie.

5.1.3 LES ALGORITHMES CLASSIQUES NE SE GÉNÉRALISENT PAS AUX PROBLÈMES NON BINAIRES

Nous avons présenté au paragraphe 3.4 des algorithmes de résolution pour des problèmes définis dans le formalisme de Mackworth, c'est-à-dire pour des problèmes de satisfaction de contraintes binaires. Certains de ces algorithmes, notamment ceux qui exploitent à la fois les notions de cohérences et de retour arrière "intelligent" sont très efficaces. Cependant, ils ne se transposent pas (du moins pas facilement) au formalisme enrichi que nous avons présenté, notamment à cause de la topologie des problèmes qui est bien plus complexe lorsque les contraintes ne sont pas toutes binaires (i.e. leur structure est un hypergraphe au lieu d'un graphe). Il est par exemple assez délicat de mettre en œuvre un algorithme de type FC-CBJ [Prosser, 1993b] (forward-checking associé à un retour arrière intelligent basé sur les conflits) efficace pour le cas général.

5.2 TECHNIQUES DE RÉOLUTION POUR LES PROBLÈMES NON BINAIRES

Aux paragraphes précédents nous avons mis en évidence certaines des limitations des techniques de résolutions que nous avons présentées. Nous avons notamment souligné le coût de l'obtention de la cohérence d'arc pour une contrainte. Dans cette partie, nous présenterons d'une part des techniques plus efficaces de mise en œuvre de la cohérence d'arc, et d'autre part une alternative intéressante à la cohérence d'arc : le *filtrage* de contraintes.

Nous avons d'autre part montré que les algorithmes classiques, définis dans le contexte des problèmes de satisfaction de contraintes binaires, ne se généralisent pas aisément au contexte des problèmes non binaires. Nous présentons dans cette partie un algorithme abstrait plus général, au sens où il permet de spécifier les algorithmes classiques, dont nous aborderons la mise en œuvre pratique dans la partie suivante.

5.2.1 LE FILTRAGE DE CONTRAINTES

Nous avons écrit plus haut que la cohérence d'arc est fortement exploitée par la plupart des algorithmes de résolution de CSP. Nous avons présenté une méthode systématique pour l'obtention de la cohérence d'arc (cf. Algorithme 5), dont le coût est clairement exponentiel en fonction du nombre de variables impliquées dans la chaque contrainte.

Cette complexité rend impraticable l'utilisation de cette méthode dans le cadre de problèmes non binaires, d'autant plus que les algorithmes présentés à la 3.4 y font fréquemment appel. Nous allons montrer ici deux voies pour l'amélioration de cette situation.

Dans cette section nous présentons quelques idées pour améliorer le coût d'obtention de l'arc cohérence. Enfin, nous présentons une alternative à l'arc cohérence : le *filtrage* de contraintes. Cette dernière notion est à l'heure actuelle au centre de la plupart des systèmes de satisfaction de contraintes.

A) MÉTHODES DE COHÉRENCE D'ARC SPÉCIFIQUES

La complexité exponentielle de la méthode par défaut la rend inexploitable pour des contraintes impliquant plusieurs variables dont les domaines sont assez grands. À titre d'illustration, pour une contrainte portant sur 10 variables dont les do-

maines contiennent 10 éléments chacun, la complexité est de l'ordre de 10^{10} créations de vecteurs de taille 10. ($10^{10} = 10$ milliards.)

Cependant, pour de nombreuses contraintes, on peut utiliser des méthodes spécifiques qui sont bien plus efficaces. À titre d'exemple, considérons la contrainte de différence $X \neq Y$. Il est facile de montrer la proposition suivante :

Proposition : la contrainte $X \neq Y$ est arc cohérente si, et seulement si, l'une des propriété suivante est vérifiée :

(i) $|D(X)| > 1$ et $|D(Y)| > 1$

(ii) $D(X) = \{x\}$ et $x \notin D(Y)$

(iii) $D(Y) = \{y\}$ et $y \notin D(X)$

ainsi, afin de rendre arc cohérente cette contrainte, il suffit d'appliquer l'algorithme suivant, dont la complexité est en temps constant :

“cohérence d'arc de $X \geq Y$ ”

si $D(X) = \{x\}$ alors $D(Y) \leftarrow D(Y) \setminus \{x\}$

si $D(Y) = \{y\}$ alors $D(X) \leftarrow D(X) \setminus \{y\}$

“sinon, ne rien faire”

Un autre exemple, du même type, est donné par la contrainte de différence globale. Il a été montré par J.-Ch. Régim [Régim, 1994] que cette contrainte peut être rendue cohérente en appliquant un algorithme polynomial de théorie des graphes. (Nous ne présentons pas ici cet algorithme relativement complexe.) La complexité polynomiale de cet algorithme permet de réaliser, en un temps raisonnable, la cohérence de contraintes de différences portant sur un assez grand nombre de variables.

Ainsi, pour de nombreux types de contraintes, il est possible de trouver des algorithmes réalisant la cohérence d'arc en un temps polynomial, ce qui permet d'utiliser cette propriété pour des problèmes non binaires. Mais, pour certaines contraintes ; de tels algorithmes n'existent pas, il est alors nécessaire de remplacer la notion de cohérence d'arc par un autre concept : le *filtrage* de contrainte, présenté dans le paragraphe suivant.

B) UNE ALTERNATIVE À LA COHÉRENCE D'ARC : LE FILTRAGE DE CONTRAINTES

Deux raisons font que l'on ne réalise pas systématiquement la cohérence d'arc pour les contraintes. D'une part, pour de nombreuses contraintes, il n'existe pas de méthode polynomiale pour obtenir la cohérence. C'est par exemple le cas des contraintes aussi simples que la suivante : $U + V + W = 0$. D'autre part, même s'il existe une méthode polynomiale pour aboutir à la cohérence d'arc d'une contrainte, elle peut s'avérer coûteuse en pratique, ce qui est le cas pour les contraintes de différence globale [Régim, 1994].

De plus, certaines contraintes sont “peu contraignantes”, dans le sens où elles sont satisfaites par la majorité des instanciations possibles de leurs variables. Pour de telles contraintes, il n'est pas judicieux d'appliquer une méthode de cohérence d'arc coûteuse pour n'obtenir, le plus souvent, aucune réduction de domaine. Un bon exemple est la contrainte de différence globale pour laquelle il existe un algorithme polynomial de cohérence d'arc [Régim, 1994].

Pour ces deux raisons, il est souvent préférable de ne pas obtenir la cohérence d'arc, mais simplement une propriété plus faible, ce que nous appellerons un *filtrage* de la contrainte, ce que l'on peut définir de la façon suivante :

Définition : soit C une contrainte, un *filtrage* de la contrainte C est un produit $F_1 \times \dots \times F_n$ de domaines contenant le produit de domaines arc cohérents de la contrainte, c'est-à-dire :

$$\forall i = 1, \dots, n ; D_{ac_i} \subseteq F_i$$

Définition 11 Le filtrage d'une contrainte est une réduction de domaines plus faible que la cohérence d'arc

L'idée essentielle du filtrage est de se contenter de supprimer les valeurs dont on peut rapidement déterminer qu'elles sont incohérentes avec la contrainte. Plus précisément, on cherche à trouver un compromis aussi judicieux que possible entre la réduction des domaines et la vitesse d'exécution. Considérons par exemple la contrainte de différence globale pour laquelle un filtrage intéressant consiste à s'assurer que si une variable est instanciée, sa valeur est absente du domaine des autres variables. L'algorithme suivant réalise ce filtrage particulier :

```

filtrer-all-diff(V1, V2, ..., Vn)
    "les Di sont les domaines des Vi"

    continuer ← true
    tant que (continuer) faire
        continuer ← false
        ∀i = 1..n faire
            si Di = {x} alors      "i.e. si Di est un singleton"
                ∀j = 1..n tel que i ≠ j
                    si x ∈ Dj alors
                        continuer ← true
                        Dj ← Dj \ {x}

```

Algorithme 7 Algorithme de filtrage de la contrainte globale de différence. Cet algorithme ne réalise pas la cohérence d'arcs, mais une propriété plus faible

Tant que l'une des variables a pour domaine un singleton $\{x\}$, alors on supprime x des domaines des autres variables. On applique ceci jusqu'à ce qu'aucune réduction supplémentaire ne soit plus possible. La complexité de cette méthode est excellente en pratique. De plus, elle provoque dans la plupart des cas des réductions de domaines équivalentes à la cohérence d'arc. Il existe cependant quelques situations où cette méthode "oublie" de supprimer des valeurs incohérentes. Il suffit de considérer le problème suivant : trois variables X , Y et Z dont le domaine est l'ensemble $\{1, 2\}$. La contrainte de différence globale entre X , Y et Z n'est alors pas arc cohérente, mais le filtrage défini à l'Algorithme 7 ne supprimera pourtant aucune valeur. Si ceci est rédhibitoire pour certains problèmes extrêmement particuliers, comme le problème des pigeons qui est constitué d'une seule contrainte globale de différence, cela n'est en pratique jamais gênant.

Donnons un autre exemple de méthode de filtrage systématiquement utilisé en pratique à la place de la cohérence d'arc : le filtrage des contraintes arithmétiques linéaires. Le compromis le plus généralement retenu consiste à ne considérer que les valeurs minimales et maximales du domaine de chaque variable. C'est ce que l'on appelle, en anglais, **l'arc-B-consistency (cohérence d'arc aux bornes)**. Illustrons cette idée sur un exemple simple. Considérons deux variables U et V dont le domaine est $\{1, \dots, 99.999, 100.001, \dots, 1.000.000\}$, et liées par la contrainte $U - 3.V = 0$. En considérant tour à tour les formules suivantes : $V = U/3$ et $U = 3.V$, on déduit dans un premier temps que $V \leq 1.000.000/3$ et donc que $V \leq 333.333$. dans un second temps on en déduit que $U \leq 999.999$. Cette méthode ne réalise pas la cohérence d'arc car la valeur 300.000 n'est pas cohérente. Cependant, le coût de calcul de la véritable cohérence d'arc est ici rédhibitoire. Voici un algorithme qui permet de réaliser ce filtrage (i.e. arc-B-consistency) des contraintes arithmétiques linéaires. Pour plus de simplicité, nous le donnons ici dans la version pour les équations dont tous les coefficients sont égaux à un. La généralisation aux équations avec coefficients et aux inéquations est relativement aisée.

```

"Filtrage des contraintes de type  $X_1 + X_2 + \dots + X_n = C$ , où  $C$  est
une constante et les  $X_i$  sont les variables contraintes de
domaine  $D_i$ "

Répéter jusqu'à ce qu'un point fixe soit atteint
     $\forall i = 1, 2, \dots, n$ 
         $\min(X_i) \geq C - \sum_{j \neq i} \max(X_j)$ 
         $\max(X_i) \leq C - \sum_{j \neq i} \min(X_j)$ 

ou plus formellement,

continuer  $\leftarrow$  true
tant que (continuer)
    continuer  $\leftarrow$  false
     $\forall i = 1, 2, \dots, n$ 
         $m \leftarrow C - \sum_{j \neq i} \max(X_j)$ 
         $M \leftarrow C - \sum_{j \neq i} \min(X_j)$ 
         $D \leftarrow D_i \setminus \{x \in D_i \mid x \leq m \text{ ou } x \geq M\}$ 
    Si  $(D \neq D_i)$  alors continuer  $\leftarrow$  true

```

Algorithme 8 Algorithme de filtrage des contraintes représentant des équations linéaires dont tous les coefficients sont égaux à un. Le résultat de ce filtrage n'est pas l'obtention de la cohérence d'arc, mais la cohérence aux bornes (i.e. arc-B-consistency)

5.2.2 COHÉRENCE D'ARC ET COHÉRENCE DE CHEMIN

La notion de filtrage est essentielle car elle permet de gestion efficace de la propagation des informations données par les contraintes. Le point crucial est que la cohérence d'arc fournit un mécanisme général de gestion des contraintes, considérées

individuellement, qui peut être redéfini par une propriété plus faible mais plus facile à obtenir pour certains types de contraintes.

La raison essentielle du manque de systèmes fondés sur la cohérence de chemin est précisément qu'il n'existe pas de concept équivalent au filtrage pour cette notion. Le de filtrage de chemin de contraintes consisterait à redéfinir une notion plus faible que la cohérence de chemin pour certains types de chemins (ou groupes) de contraintes. Dans la pratique, l'utilisation de contraintes globales est une manière de regrouper plusieurs contraintes en une seule dont le filtrage revient à faire le filtrage d'un groupe de contraintes.

5.2.3 ALGORITHMES DE RÉOLUTION

Nous avons écrit au paragraphe 3.4 qu'il existe de nombreux algorithmes complets pour la résolution de problèmes de contraintes, et que ces algorithmes sont pour la plupart définis dans le contexte du formalisme des CSPs binaires de Mackworth. Certains de ses algorithmes s'avèrent très efficaces sur de nombreux problèmes, notamment les plus sophistiqués comme le FC-CBJ ou conflit directed backjumping. Cependant, la plupart de ces algorithmes, et c'est le cas des plus sophistiqués, ne peut pas être adaptés au contexte des problèmes de satisfaction de contraintes non binaires. En effet, ces algorithmes sont conçus pour des problèmes dont la structure est un graphe. L'adaptation à des problèmes dont la structure est un hypergraphe est périlleuse, voire impossible.

L'expérience montre qu'en pratique, la technique la plus efficace pour résoudre les problèmes complexes non binaires consiste à propager toutes les contraintes, à la manière de l'algorithme de real full look-ahead. Cette constatation est exprimée dans [Hentenryck, 1989] et est étayée par les meilleurs systèmes existants (ILOGSOLVER, clp(FD), chip). Cette approche était déjà au cœur d'Alice [Laurière, 1976, Laurière, 1978] où la propagation est plus systématique encore, puisque le système procède à des combinaisons de contraintes. Ainsi, en dépit des résultats souvent exprimés concernant les algorithmes de résolution de problèmes de satisfaction de contraintes, il apparaît qu'en pratique, il est judicieux d'adopter une méthode de type real full look-ahead.

Cependant, comme il est dit par Yves Caseau dans [Caseau, 1991], il est fréquemment nécessaire d'avoir recours à des stratégies de résolutions spécifiques pour résoudre certains types de problèmes :

“no constraints solver to our knowledge holds all the techniques that we have found necessary to solve [particular problems]”

TROISIÈME PARTIE

LES OBJETS POUR IMPLÉMENTER LES CONTRAINTES

Dans le premier chapitre de cette partie nous présentons quelques systèmes combinant objets et contraintes. Historiquement, les premiers systèmes de contraintes et objets sont nés des travaux d'Alan Borning sur les problèmes de constructions d'interfaces graphiques interactives. Cette problématique, que nous présentons dans la partie qui suit, est radicalement différente de l'approche "satisfaction de contraintes" qui nous a intéressés. Ensuite, de nombreux systèmes de satisfaction de contraintes ont été implémentés dans des langages de programmation par objets. C'est principalement le souhait de pouvoir bénéficier de la puissance des langages à objets, notamment de C++, pour l'implémentation des techniques de satisfaction de contraintes et pour le confort d'utilisation ainsi que de leur popularité grandissante qui a conduit à la réalisation de tels systèmes.

Le second chapitre est consacrée à la description du système de satisfaction de contraintes BACKTALK, implémenté dans le langage à objets SMALLTALK. BACKTALK est, à l'instar de CHIP ou ILOGSOLVER, une extension du langage hôte par une bibliothèque de classes. Les classes de BACKTALK représentent les domaines, les variables contraintes et les contraintes mais également les algorithmes de résolution et les heuristiques de sélection. Cette approche fonde une architecture de type *framework* qui permet de définir un système qui soit ouvert et robuste. Nous détaillerons les différentes classes constituant BACKTALK, ainsi que son architecture et donnerons des exemples et des temps de résolution [Roy and Pachet, 1998].

Chapitre 1

Quelques systèmes de contraintes implémentés avec des objets

Il existe maintenant de nombreux systèmes combinant diverses techniques de contraintes avec les objets. Contrairement à ce qui s'est passé avec l'intégration des contraintes dans les langages de programmation logique et pour des raisons que nous avons expliquées, il n'y a pas dans ce domaine une approche dominante qui se dégage de ces travaux. La différence essentielle de vision de la programmation qui existe entre le paradigme de la satisfaction de contraintes et celui des objets a donné naissance à des approches fort différentes. Nous présenterons ici des systèmes de contraintes implémentés dans des langages de programmation par objet et nous ne parlerons pas des approches fondées sur la représentation par objets. On pourra pour cela se reporter à l'étude de Jérôme Gensel [Gensel, 1998].

Nous distinguons trois types d'approches de l'intégration des contraintes avec les objets. Il y a d'une part les systèmes de *propagation locale* qui sont principalement utilisés pour la construction de d'interfaces graphiques interactives. On trouve également plusieurs bibliothèques de classes pour la satisfaction de contraintes. Une approche intéressante est celle des langages hybrides qui combinent les techniques de programmation de plusieurs paradigmes différents, dont les techniques de satisfaction de contraintes.

1.1 LE MODÈLE RÉACTIF — LA PROPAGATION LOCALE DE CONTRAINTES

Les systèmes de propagation locale de contraintes ne sont pas fondés sur le modèle de la satisfaction de contraintes mais sur un modèle dit *réactif*. En d'autres termes, il ne s'agit pas de résoudre des problèmes combinatoires en satisfaisant un certain nombre de contraintes, mais plutôt de *maintenir* au cours du temps des relations entre les objets d'un système subissant des modifications locales.

Un exemple simple d'un tel système est constitué de deux jauges donnant la température ambiante, l'une en degrés Celsius, l'autre en degrés Fahrenheit. Les deux jauges sont liées par l'équation de conversion des températures en degrés Celsius et Fahrenheit :

$$1,8 \times c = f - 32$$

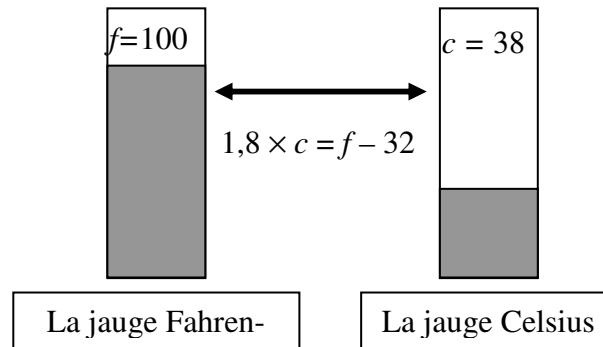


Figure 13 Un exemple de système de propagation locale: deux jauges donnant la température en degrés Celsius et Fahrenheit. L'utilisateur peut agir sur l'une ou l'autre des jauges pour en modifier la valeur provoquant une modification de l'autre jauge par le système afin de maintenir la relation $1,8 \times c = f - 32$

Le domaine d'application idéal des techniques de propagation locale est la construction d'interfaces graphiques interactives. Divers objets graphiques sont disposés sur l'écran, et certaines relations entre ces objets doivent être maintenues au cours du temps. Par exemple, "deux fenêtres doivent avoir la même taille", "une boîte de dialogue doit être à gauche d'un éditeur", "les cases à cocher OUI et NON sont exclusives", "le texte et le fond d'écran sont de couleurs différentes" etc.

Les techniques mises en œuvre pour l'implémentation des systèmes de propagation locales répondent aux exigences suivantes, définies par Alan Borning [Borning and Freeman-Benson, 1995] :

1. Il faut des performances suffisantes pour que l'es effets d'une perturbation soient immédiats afin de ne pas dérouter l'utilisateur.
2. Il faut gérer les hiérarchies de contraintes, de façon à pouvoir spécifier différents degrés d'importance pour les contraintes, afin de permettre de transgresser une contrainte si une contrainte plus importante l'impose.
3. Les variables contraintes doivent pouvoir prendre comme valeurs des objets quelconques du langage, afin de permettre de définir des interfaces comportant des objets complexes éventuellement définis par l'utilisateur.
4. Le comportement du système doit être aussi peu "surprenant" que possible pour ne pas être déroutant.

Toute une génération d'algorithmes a été développée pour permettre de satisfaire ces exigences [Borning, et al., 1996, Sannella, 1994]. Leur domaine d'application privilégié est la construction d'interfaces graphiques interactives (ThingLab, [Borning, 1981], OTI Constraint Solver, [Borning and Freeman-Benson, 1995], INGRAS [Goubier, 1997]).

Les techniques de propagation locale de contraintes ne sont pas limitées à la production d'interfaces graphiques interactives. On citera la recherche de Olivier Dellerue sur le système MIDISPACE. MIDISPACE est une interface de contrôle pour la spatio-

alisation du son qui utilise la propagation locale de contraintes afin de maintenir la cohérence du mixage [Pachet and Delerue, 1998].

Les techniques mises en œuvre dans les systèmes de propagation locale sont radicalement différentes de celles que l'on utilise pour la satisfaction de contraintes.

1.2 LES BIBLIOTHÈQUES DE CLASSES

CHIP [Dincbas, et al., 1990], distribué par la société COSYTEC, est à l'origine un système de programmation logique avec contraintes. Il est maintenant disponible dans une implémentation "objets" dans le langage C++. CHIP est constitué par différents modules, qui étendent un langage hôte, qui permettent de créer et résoudre des problèmes de contraintes. CHIP existe en deux versions, une pour le langage de programmation logique Prolog, et l'autre pour le langage C/C++. Comme nous l'avons déjà souligné en général au paragraphe 2.1, l'intégration de CHIP dans Prolog est plus profonde et transparente. La version C/C++ se présente comme une bibliothèque de classes contraintes que l'on peut utiliser dans les programmes.

Les contraintes *usuelles* (e.g. contraintes linéaires ou logiques) sont bien évidemment disponibles dans CHIP. En outre, CHIP fournit des contraintes plus complexes, dites *contraintes globales*, qui permettent de représenter des relations complexes entre de nombreuses variables. Les contraintes globales présentent les avantages suivants :

- Elles peuvent être utilisées dans plusieurs domaines d'applications très différents
- Elles sont associées à des mécanismes de filtrage très performants qui permettent des réductions de domaines importantes et dans "toutes les directions"
- Elles peuvent, grâce à l'efficacité de leur gestion, être appliquées à des problèmes de grande taille

Voici quelques exemples de contraintes globales définies en CHIP : la contrainte *cumulative*, qui permet d'exprimer des propriétés sur la gestion des ressources ; la contrainte *diffn* qui est une généralisation de la contrainte de différence à n dimensions, et qui permet d'exprimer des propriétés de non-recouvrement ; la contrainte *cycle* qui trouve des circuits dans des graphes [Beldiceanu and Contejean, 1994].

La "philosophie" de l'utilisation de CHIP est l'approche "contraindre et résoudre", qui consiste à d'abord chercher les contraintes nécessaires à la définition du problème, et à chercher les paramétrages correspondants. Une fois les contraintes identifiées et correctement paramétrées, le système prend la résolution complètement en charge de façon très efficace.

Voici, à titre d'exemple, le problème SEND+MORE=MONEY codé en CHIP :

```
money([S,E,N,D,M,O,R,Y]) :-
    ranges([S,E,N,D,M,O,R,Y]),
    state_constraint([S,E,N,D,M,O,R,Y]),
    labelling([S,E,N,D,M,O,R,Y]).

state_constraint([S,E,N,D,M,O,R,Y]) :-
    alldifferent([S,E,N,D,M,O,R,Y]),
    S ## 0,
    M ## 0,
    1000*S + 100*E + 10*N + D + 1000*M + 100*O + 10*R + E
    #= 10000*M + 1000*O + 100*N + 10*E + Y.

ranges([]).
ranges([X|Xs]) :-
    X :: 0..9,
    ranges(Xs).

labelling([]).
labelling([H|T]) :-
    delete(V,[H|T],R,0,first_fail),
    indomain(V),
    labelling(R).
```

Le système ILOGSOLVER [Puget, 1994, Puget and Leconte, 1995] est un des systèmes commerciaux de satisfaction de contraintes les plus rapides et les plus utilisés. ILOGSOLVER se présente comme une bibliothèque C++ contenant plusieurs classes de variables et de nombreuses classes de contraintes. La résolution se fait en utilisant un algorithme d'énumération avec maintien de cohérence. Ce système permet à l'utilisateur de définir ses propres classes de contraintes et d'utiliser des méthodes de propagation spécifiques. La stratégie de résolution est paramétrable par des heuristiques de sélections de valeurs et de variables. En revanche, le mécanisme d'énumération n'est pas accessible à l'utilisateur.

Voici le problème SEND+MORE=MONEY codé en ILOGSOLVER :

```
#include <fstream.h>
#include <ilsolver/ctint.h>

CtInt    dummy = CtInit( );

CtIntVar    S(1, 9), E(0, 9), N(0, 9), D(0, 9),
            M(1, 9), O(0, 9), R(0, 9), Y(0, 9);
CtIntVar*    AllVars[]={&S, &E, &N, &D, &M, &O, &R, &Y};

int main(int, char**)
{
    CtAllNeq(8, AllVars);
    CtEq( 1000*S + 100*E + 10*N + D + 1000*M + 100*O + 10*R + E,
        10000*M + 1000*O + 100*N + 10*E + Y);
    CtSolve(CtGenerate(8, AllVars));

    for (CtInt i = 0; i < 8; i++)
        AllVars[i]->setName("");
    cout << " " << S << E << N << D << endl;
    cout << " + " << M << O << R << E << endl;
    cout << " = " << M << O << N << E << Y << endl ;
    CtEnd( );
    return 0;
}
```

La démarche que nous avons employée pour l'implémentation du système BACKTALK est très proche de celle d'ILOGSOLVER. Cependant, nous sommes allés plus loin dans la représentation par des classes des outils de la satisfaction de contraintes. Nous avons notamment réifié les heuristiques, mais surtout les algorithmes de résolution. Nous montrerons dans la partie consacrée à BACKTALK que cette démarche offre la possibilité de définir de nouvelles stratégies de résolution tout en préservant la fiabilité et les performances du système.

Mentionnons également les bibliothèques PROSE [Berlandier, 1992] et CSPOO [Kökeny, 1993, Kökeny, 1994]. PROSE n'est pas à proprement parler implémenté dans un langage à objets, mais est implémenté, selon les mots de son auteur, selon des principes permettant son exploitation dans un tel langage. Outre les classes de variables et de contraintes, PROSE dispose d'un mécanisme de gestion de méta-contraintes. CSPOO est une bibliothèque de contraintes, implémentée dans l'environnement de représentation par objets Y3 [Ducourneau and Quinqueton, 1986]. Dans CSPOO, les algorithmes de résolution sont représentés par une classe. De plus, les domaines des variables sont des extensions de classes ce qui permet de définir des contraintes portant sur des classes. Ainsi, la résolution peut bénéficier des relations de spécialisation et de généralisations. Plus précisément, si une classe satisfait une contrainte, alors la contrainte est également satisfaite par toutes ses spécialisations.

1.3 LANGAGES HYBRIDES

À l'opposé de l'approche de CHIP, CLAIRE [Caseau, 1994, Laburthe and Caseau, 1996] est un langage de programmation à objets qui intègre des mécanismes de gestion des règles de productions, mais qui ne dispose en standard d'aucune classe de contrainte. Il faut voir CLAIRE non pas comme un système de satisfaction de contraintes, mais comme un langage pour la programmation *d'algorithmes* de résolution de problèmes, notamment dans le domaine de la satisfaction de contraintes. L'exemple typique d'utilisation de CLAIRE est le langage SALSA [Laburthe, 1998, Laburthe and Caseau, 1998], qui est implémenté en CLAIRE et qui permet de définir simplement des algorithmes efficaces de résolution de problèmes. Voici quelques exemples d'algorithmes définis en SALSA :

“la résolution par énumération explicite des valeurs des variables sans utiliser d'ordre particulier pour le choix des variables et des valeurs”

```
L :: Choice(  
  moves post(FD, x == v)  
  with x = some(x in Variable | unknown ? (value, x)),  
       v ∈ domain(x) )
```

“la résolution d'un problème par réduction des domaines en utilisant une séparation dichotomique”

```
R :: Choice(  
  moves post(FD, x <= mid) or post(FD, x > mid)  
  with x = some(x in Variable | unknown?(value, x) & card(x) > 4),  
       mid = (x.sup + x.inf / 2 )
```

“la résolution du problème des n reines en choisissant les variables par taille croissante de leur domaine, et les valeurs en commençant par le milieu de la colonne et en s'en écartant pas à pas”

```
Q :: Choice(  
  moves post(FD, queens[i] == j)  
  with i = some(i in {i in (1 .. n) | unknown?(value, queens[i])}  
               minimizing card(domain(queens[i])) )  
       j in domain(i)  
  ordered by increasing abs(n / 2 - j) )
```

Ainsi, la philosophie de CLAIRE est très différente de celle de CHIP, plutôt que “contraindre et résoudre”, il s'agit ici de développer à la fois les classes de contraintes, les algorithmes de résolution et de définir les problèmes, ce qu'on pourrait résumer à l'extrême par la formule : “faites-le vous-même !”. Une telle approche peut sembler complexe et décourageante pour l'utilisateur, cependant, elle présente plusieurs avantages. D'une part, CLAIRE est un langage relativement simple à appréhender, par rapport à son prédécesseur LAURE, dont il ne reprend que les éléments essentiels (par exemple, les règles déductives et les contraintes de LAURE ont été abandonnées dans CLAIRE). D'autre part, le pouvoir expressif de CLAIRE est celui d'un langage complet, ce qui permet de programmer facilement des techniques de résolu-

tion adaptées à des problèmes très divers. Finalement, cette approche permet d'obtenir du code d'une efficacité impressionnante. Par exemple, le difficile problème MT10 est résolu en CLAIRE en 79 secondes (incluant la preuve de l'optimalité de la solution).

En outre, il existe maintenant la bibliothèque ECLAIR [Laburthe, 1998], implémentée au-dessus de CLAIRE, qui fournit des classes dédiées à la satisfaction de contraintes. L'utilisateur de CLAIRE peut ainsi adopter une approche de type "contraindre et résoudre" au sein même de CLAIRE. Voici comme illustration le code, écrit en ECLAIR du cryptogramme SEND+MORE=MONEY :

```
S :: Var(inf = 1, sup = 9)
E :: Var(inf = 0, sup = 9)
N :: Var(inf = 0, sup = 9)
D :: Var(inf = 0, sup = 9)
M :: Var(inf = 1, sup = 9)
O :: Var(inf = 0, sup = 9)
R :: Var(inf = 0, sup = 9)
Y :: Var(inf = 0, sup = 9)

Letters :: set(S,E,N,D,M,O,R,Y)
word1 :: Var(sup = 1000000)
word2 :: Var(sup = 1000000)
word3 :: Var(sup = 1000000)

// solution (S,E,N,D,M,O,R,Y) = (9,5,6,7,1,0,8,2)
[send()
 -> AllDifferent(Letters),
  list(1000,100,10,1) scalar list(S,E,N,D) == word1,
  list(1000,100,10,1) scalar list(M,O,R,E) == word2,
  list(10000,1000,100,10,1) scalar list(M,O,N,E,Y) == word3,
  list(1,1) scalar list(word1,word2) == word3,
  list(1000,91,1,10) scalar list(S,E,D,R)
  == (list(90,1,9000,900) scalar list(N,Y,M,O))]
```

Chapitre 2

Le cahier des charges de BACKTALK

La mise en œuvre d'un système de satisfaction de contraintes procède d'une démarche pratique, ce qui impose des choix et des compromis. Il n'est, par exemple, pas envisageable d'implémenter tous les algorithmes de résolution existants, d'autant que leur nombre augmente régulièrement chaque année. Il n'est pas non plus envisageable de proposer les techniques de traitement de contraintes les plus sophistiquées, car elles ne sont pas nécessairement compatibles entre elles et parce que leur domaine d'application est souvent restreint à des types de problèmes ou de contraintes qui se rencontrent rarement en pratique. Par exemple, l'utilisation de contraintes définies en extension est peu fréquente ; les problèmes purs, par exemple ne contenant que des contraintes binaires, fonctionnelles ou bijectives ou ayant une structure très particulière (e.g. arborescentes) sont également fort rares. Enfin, que peut-on espérer, pour la résolution de problèmes complexe et structurés, de techniques qui n'ont fait leurs "preuves" que sur des problèmes binaires engendrés aléatoirement ? De même, il n'est pas possible de fournir une implémentation de toutes les contraintes étudiées jusqu'ici : chaque domaine d'application implique un grand nombre de contraintes spécifiques dont certaines nécessitent des techniques de filtrage très sophistiquées (e.g. contraintes globales de CHIP).

Pour ces raisons, le logiciel BACKTALK, que nous présentons ici, n'est pas un système universel disposant de tous les outils pour définir et résoudre directement tous les problèmes de satisfaction de contraintes dans tous les domaines d'application. BACKTALK est un système de petite taille, dont le noyau, qui comporte moins de 70 classes et de 1400 méthodes, fournit les objets nécessaires au développement d'applications simples.

L'architecture de BACKTALK est celle d'un *framework*, c'est-à-dire une bibliothèque de classes, organisées en plusieurs hiérarchies (e.g. les variables, les contraintes), et dont le schéma de fonctionnement (i.e. la collaboration entre les différentes hiérarchies) est prédéfini. Les choix d'implémentation de BACKTALK ont été principalement dirigés par les exigences suivantes :

- **Efficacité** : la nature particulièrement difficile des problèmes de satisfaction de contraintes rend nécessaire l'utilisation de techniques sophistiquées, comme le filtrage de contraintes, implémentées de façon efficace. La mise en œuvre de démons, pour l'implémentation des méthodes de filtrage, et d'un algorithme de résolution très général, mais que l'on peut spécialiser, permet d'obtenir une bonne efficacité tout en autorisant une programmation simplifiée. Finalement, les performances de BACKTALK sont comparables aux performances des meilleurs systèmes existants (e.g. ILOGSOLVER, **clp (FD)**).
- **Extensibilité** : la richesse du formalisme que nous avons adopté pour les problèmes de satisfaction de contraintes permet de définir des problèmes dans des domaines d'application très variés. Il est donc nécessaire de pouvoir adapter les techniques de résolution afin d'obtenir des performances comparables aux algorithmes dédiés. Il résulte de cette nécessité qu'un système de satisfaction de contraintes, pour être pleinement utilisable, doit être facilement extensible pour permettre de définir de nouveaux algorithmes ou de nouvelles classes de contraintes. Nous avons adopté une architecture de type *framework*, ce qui permet d'obtenir de bonnes capacités d'extensions. Il a par exemple été facile d'adapter BACKTALK à la résolution de problèmes temporels [Ramaux and Fontaine, 1996] ou de développer des classes de contraintes spécifiques (e.g. musique, ordonnancement).
- **Intégration au langage hôte** : nous avons souligné auparavant la difficulté de réaliser une intégration transparente d'un système de satisfaction de contraintes dans un langage à objets. Avec BACKTALK, comme avec toute autre bibliothèque de contraintes, l'utilisateur doit créer et manipuler des objets spécifiques comme les variables contraintes. Afin de ne pas accentuer l'hiatus entre le langage hôte et le système de contraintes, l'utilisation de BACKTALK se fait par le truchement des mécanismes classiques de la programmation par objets (i.e. instanciation de classes et envoi de message) et de la même manière, l'extension du système se fait par héritage (création de sous-classes) et polymorphisme de SMALLTALK (i.e. redéfinition de méthodes dans des sous-classes). De plus, pour des raisons de portabilité et de transparence, nous avons implémenté BACKTALK sans avoir recours à aucune modification de la machine virtuelle, à la manière du système de backtracking de [Lalonde and Gulik, 1988].

Dans cette partie, nous décrivons le système BACKTALK en commençant par présenter une à une des différentes hiérarchies de classes : les variables, les contraintes, les heuristiques et les algorithmes. Ensuite, nous décrivons les classes de démons, qui sont au cœur du système et définissent le mécanisme prédéfini de collaboration entre les différentes classes. Nous présentons également quelques techniques avancées mises en œuvre dans BACKTALK. Finalement, nous discutons les performances sur des problèmes combinatoires classiques.

Chapitre 3

Les classes de bases de BACKTALK

Dans cette partie nous présentons les principales classes composant BACKTALK. Ces classes représentent les notions principales de la satisfaction de contraintes, à savoir : 1) les domaines et les variables, 2) les contraintes 3) les problèmes et les algorithmes de résolution et 4) les heuristiques de résolution. Chacune de ces notions est représentée par plusieurs classes, organisée dans une hiérarchie d'héritage simple. Il y a quatre hiérarchies de classes, les variables, les contraintes, les algorithmes de résolution et les heuristiques.

Mais BACKTALK n'est pas seulement une bibliothèque de classes, mais plutôt un *framework* au sens de [Fayad and Schmidt, 1997, Johnson and Foote, 1988], c'est-à-dire une application "semi-complète", définie par un ensemble de hiérarchies de classes collaborant entre elles selon un schéma prédéfini de façon à offrir une architecture réutilisable pour une famille d'applications proches.

3.1 LES DOMAINES ET LES VARIABLES

Dans le système BACKTALK, les domaines et les variables sont représentés par des classes SMALLTALK. La réification des domaines est nécessaire pour l'obtention d'une efficacité que les classes standard de collection ne permettent pas d'obtenir. La réification des variables entre dans la conception globale du système, et permet une plus grande expressivité.

3.1.1 LES DOMAINES

Les domaines sont réifiés, en BACKTALK, sous forme de classe SMALLTALK pour des raisons d'efficacité. En effet, les domaines sont des ensembles finis dont on ne fait que supprimer des éléments, éventuellement en modifiant les bornes pour les domaines ordonnés. On pourrait utiliser l'une des classes de collection standards de SMALLTALK, par exemple la classe **OrderedCollection** pour les domaines quelconques et la classe **SortedCollection** pour les domaines numériques. Mais ces

deux classes implémentent les collections dans une grande généralité, c'est-à-dire les collections auxquelles on peut ajouter ou soustraire des éléments de façon arbitraire. Pour cette raison leur mise en œuvre n'est pas optimale pour toutes les opérations qui nous intéressent ; notamment la suppression d'éléments et les tests d'appartenance. De plus les collections standard de SMALLTALK représentent explicitement tous leurs éléments, et sont donc encombrantes et coûteuses à copier.

La réalisation d'un système de satisfaction de contraintes nécessite une gestion efficace des domaines. Plus précisément, on veut pouvoir supprimer un élément, modifier les bornes et sauvegarder ou restaurer les domaines le plus rapidement possible. Pour ce faire nous pouvons bénéficier de la propriété fondamentale des domaines qui est de ne jamais "grossir" en cours de résolution. En d'autres termes, les domaines des variables sont, durant la résolution, toujours inclus dans le domaine initialement déclaré à la définition des variables du problème.

Nous avons expérimenté deux approches différentes pour l'implémentation des domaines. Étudions d'abord le cas des domaines contenant des nombres entiers, par exemple le domaine $\{1, 2, 3, 4, 5, 6\}$. La première approche consiste à représenter un tel domaine par l'intervalle $[1, 6]$, et la seconde à le représenter par le couple de nombres $(1, 63)$, où 1 désigne le premier élément du domaine, et 63 la valeur $2^0 + 2^1 + 2^2 + 2^3 + 2^4 + 2^5$. Si l'on supprime l'élément 4 du domaine, alors selon la première représentation, sa valeur devient $[1, 3] \cup [5, 6]$ et $(1, 47)$ selon la seconde.

Selon les cas, l'une ou l'autre des deux approches est la plus compacte. Par exemple, pour les domaines "creux" tels $\{1, 2, 10.000.000, 10.000.001\}$, la première approche est plus compacte, alors que la seconde l'est pour les domaines fortement "troués" tels $\{2, 4, 6, 8, 10, \dots, 128\}$.

Pour ce qui est des opérations élémentaires, la seconde approche autorise le test d'appartenance, la suppression d'un élément et l'intersection de deux domaines en temps constant pour des domaines contenant moins de 64 éléments, ce qui est la taille d'un mot machine. Pour des domaines de tailles supérieures, ces opérations nécessitent des décompositions en base 2 qui peuvent devenir coûteuses pour les très grandes tailles. Les primitives sont donc fort efficaces pour des tailles raisonnables, mais deviennent inutilisables au-delà de quelques centaines d'éléments. La première approche permet en revanche une définition des primitives dont l'efficacité est moins sensible à la taille. Le test d'appartenance peut être implémenté efficacement par dichotomie. En revanche, la suppression d'un élément nécessite, s'il se trouve à l'intérieur d'un des intervalles, de le scinder en deux. Ceci nécessite d'augmenter le nombre d'éléments de la liste des intervalles, ce qui peut parfois nécessiter une recopie explicite de cette liste.

Pour ce qui concerne la sauvegarde et la restauration des domaines, la première approche nécessite une recopie de liste, qui peut être assez coûteuse pour les grandes tailles (i.e. les domaines troués). Il en va de même concernant la seconde approche.

En dépit des apparences, la première approche donne de bien meilleurs résultats en pratique. Plus précisément, pour de très petits problèmes comme celui des 8 reines, la seconde approche donne des résultats légèrement meilleurs, soit environ 10% plus rapide. En revanche, pour les problèmes de taille supérieure la première approche est nettement plus efficace. Ceci est en partie dû aux possibilités d'optimisations de l'implémentation de la première approche concernant la représentation des intervalles, des listes d'intervalles et des mécanismes de sauvegarde et de restauration. Ces optimisations sont les suivantes :

- 1) Les intervalles de type $[x, x]$ peuvent être représentés par la seule valeur x , ce qui diminue l'espace mémoire alloué, et donc le temps de création ;
- 2) Les listes d'intervalles peuvent être implémentées efficacement en n'utilisant pas directement la classe standard **SortedCollection** mais en utilisant une classe de collection dont certaines primitives sont améliorées, et dans laquelle la disposition des intervalles est étudiée pour éviter la recopie systématique en cas d'ajout d'éléments ;
- 3) Lors de la sauvegarde d'un domaine, il n'est pas nécessaire de recopier en profondeur la liste des intervalles (c'est-à-dire en créant une copie de chaque intervalle), mais on peut se contenter d'une copie superficielle, dans laquelle les éléments sont les mêmes que ceux que le système continue d'utiliser. Il en est de même pour la restauration des domaines. En revanche, lors de la résolution, lorsqu'un des intervalles d'un domaine est modifié, on doit le recopier afin que la modification ne se propage pas aux différentes sauvegardes.

Ainsi, le domaine $\{1, 2, 3, 4, 5, 6\}$ sera représenté par la liste d'intervalles $([1, 6])$. Si l'élément 4 est supprimé, cette liste devient $([1, 3], [5, 6])$. Si le domaine est sauvegardé, on crée une autre liste, contenant les mêmes instances, soit $([1, 3], [5, 6])$. Si maintenant on supprime la valeur 1 du domaine, on crée une nouvelle instance de l'intervalle $[1, 3]$, notons la $[1, 3]'$, dont on modifie la borne inférieure en $[2, 3]'$ et la liste des intervalles devient $([2, 3]', [4, 6])$. Ce mécanisme permet d'avoir des temps de sauvegarde et de restauration de contextes négligeables comparés aux autres opérations nécessaires à la résolution.

La gestion des domaines contenant des valeurs non entières se fait par l'intermédiaire des listes d'intervalles. On utilise pour cela le fait que les intervalles ne "grossissent" pas durant la résolution. Ainsi, un domaine $D = \{a, b, c, d\}$ contenant des objets quelconques, peut se représenter *in fine* par l'intervalle $[1, 4]$, sachant qu'un lien est conservé vers l'ensemble D . On bénéficie ainsi, pour les domaines quelconques, des primitives de gestion des domaines d'entiers qui sont optimisées. Le seul coût supplémentaire concerne les conversions que l'on doit faire entre les objets du domaine et leur indice. Cette conversion se fait en utilisant une table de "hashage" pour les domaines quelconques et une recherche dichotomique pour les domaines totalement ordonnés.

3.1.2 LES VARIABLES

Les variables contraintes sont représentées en BACKTALK par des classes organisées dans une hiérarchie d'héritage. Chaque classe correspond en fait à un type de domaine particulier, mais on peut également utiliser cette représentation pour implémenter dans les variables des mécanismes différents de ceux définis par défaut.

La hiérarchie des classes de variables est la suivante :

```

BTVariable ('label' 'value' 'domain' 'constraints'...)
  BTBooleanVariable
  BTIntegerVariable ('min' 'max')
    QueenVariable ('column')
  BTObjectVariable
    BTOrderedObjectVariable ('min' 'max')

```

La classe abstraite **BTVariable** dispose des variables d'instances représentant son nom, pour l'affichage, sa valeur éventuelle, initialement cette variable vaut **nil**, son domaine et la liste de ses contraintes. Les trois sous classes concrètes men-

tionnées ci-dessus représentent les variables dont le domaine contient des valeurs respectivement booléennes, entières ou quelconques.

On peut utiliser cette hiérarchie pour définir de nouvelles classes de variables. Ainsi, la classe **BOrderedObjectVariable**, définie comme une sous-classe de **BObjectVariable** et dont le domaine contient des objets triés, ce qui permet de leur faire supporter des contraintes concernant leur ordre et de définir des filtrages sur les bornes du domaine. On peut également définir des sous-classes qui disposent de variables d'instances annexes, ainsi la classe **QueenVariable** utilisé dans la définition du problème des n reines dispose d'une variable d'instance **column** qui représente la colonne de la variable, ce qui est utilisé pour la définition des contraintes.

De plus, les variables implémentent une méthode choisissant un élément de leur domaine, ce qui est utilisé par le système pour faire des choix lors de la résolution. Cette méthode donne le premier élément du domaine, qui est le plus petit pour les domaines ordonnés. Si l'on souhaite redéfinir cette méthode pour la résolution de certains problèmes, il suffit de faire une sous-classe de l'une des classes existantes.

3.1.3 LES OPÉRATIONS SUR LES DOMAINES DES VARIABLES

Les variables répondent à un petit nombre de primitives de modification de leur domaine. Ces primitives concernent la suppression d'éléments, la modification des bornes pour les variables dont le domaine est ordonné ou encore la modification du domaine. L'action de ces primitives est de provoquer la modification effective du domaine, de la mémoriser (ce qui sera utile à la résolution, comme nous le verrons plus loin dans cette partie) et qui déclenche une exception si le domaine se trouve vidé. Cette exception est une instance de **DomainWipedOutException**, qui est une sous-classe de la classe standard **Exception** de SMALLTALK. Les primitives sont les suivantes :

- **value: unObjet.** Provoque l'instanciation de la variable avec la valeur passée en argument. Si l'argument **unObjet** n'est pas dans le domaine, une exception **DomainWipedOutException** est déclenchée.
- **remove: unObjet.** Supprime l'élément **unObjet** du domaine. Si **unObjet** est la seule valeur du domaine, alors une exception est levée.
- **domain: uneListeOuUnDomaine.** Remplace les éléments du domaine par ceux de la liste passée en paramètre. Pour les domaines des variables de type **BObjectVariable**, on ne conserve que les éléments de la liste en argument qui sont dans le domaine initial. Ceci est nécessaire à cause de notre implémentation très particulière des domaines d'objets, décrite au premier paragraphe de ce chapitre.
- **min: unObjetTrié.** Supprime du domaine de la variable toutes les valeurs inférieures à **unObjetTrié**. Par exemple, si une variable **v** a pour domaine {1, 2, 3, 4}, alors **min: 7** ne provoque aucune modification, mais **min: 0** lève une exception **DomainWipedOutSignal**.
- **max: unObjetTrié.** Similaire à **min** :

3.2 LES CLASSES DE CONTRAINTES

Les contraintes fournissent un langage permettant de définir de manière déclarative et parcellaire des problèmes combinatoires. Lors de la résolution de ces problèmes, les contraintes subissent de façon fréquente des opérations de filtrage. Les contraintes sont les briques de base pour la construction des problèmes. Chaque type de contrainte est susceptible d'être utilisée dans nombre de problèmes très divers. Par exemple, des contraintes de différence apparaissent aussi bien dans des problèmes de découpe optimale, d'allocation de ressources, de carrés magiques ou d'ordonnancement.

Nous pouvons faire deux remarques capitales concernant le filtrage des contraintes. Nous les énoncerons comme suit :

Le filtrage d'une contrainte dépend de sa nature Le filtrage d'une contrainte est indépendant du domaine d'application

La première propriété résulte de ce qu'on a exposé sur le mécanisme de filtrage dans la 5.2.1.

Il existe quelques contre-exemples à la seconde remarque. Le problème des pigeons comporte une unique contrainte de différence globale. Si l'on utilise le filtrage défini en 5.2.1a), ce problème est très difficile à résoudre. En revanche, si l'on utilise la méthode de filtrage définie par J.-Ch. Régim [Régim, 1994], le problème devient polynomial. Cependant, le problème des pigeons est atypique car il ne comporte qu'une seule contrainte, ce qui n'est pas représentatif des problèmes de contraintes. En pratique, un filtrage efficace l'est indépendamment du type de problème à résoudre.

Il résulte de ces deux remarques que l'on peut organiser les contraintes de façon définitive dans une bibliothèque de classes. La première remarque impose que chacune des contraintes de cette bibliothèque implémente son propre mécanisme de filtrage alors que la seconde remarque suggère qu'une telle bibliothèque est universelle au sens où elle est utilisable pour des domaines d'application divers.

3.2.1 LA HIÉRARCHIE DE CONTRAINTES

Dans le cadre de la programmation par objets il est naturel de donner à cette bibliothèque la forme d'une hiérarchie de classes. Les différentes classes implémentent les mécanismes de filtrage sous la forme de méthodes d'instances, chaque classe représentant un type de contrainte. Cette approche permet d'obtenir plusieurs propriétés intéressantes : d'une part, en faisant des contraintes des classes, on peut, grâce au polymorphisme, les doter d'un langage commun, permettant de les manipuler indifféremment les unes des autres ; d'autre part, le mécanisme d'héritage des langages à objets (héritage simple en ce qui concerne SMALLTALK) permet de créer de nouvelles classes de contraintes qui bénéficient des mécanismes de filtrages de contraintes existantes sans avoir à les récrire.

La hiérarchie de contraintes de BACKTALK comporte des classes de contraintes définies en extension et en compréhension, incluant les relations arithmétiques et logiques usuelles, ainsi que des contraintes particulières qui seront abordées dans la suite de ce document. Voici un extrait de cette hiérarchie :

object ()

```
BTConstraint ('isPersistent' 'isStatic' 'owner')
  BTBinaryCt ('x' 'y')
    BTArrayIndexCt ('array')
    BTBinaryExtensionCt ('relation')
    BTBinaryQueensCt ()
    BTComparatorCt ()
      BTGreaterOrEqualCt ()
      BTGreaterThanOrCt ()
    BTEqualCt ()
    BTUnaryOperatorCt ()
      BTAbsCt ()
      BTOppositeCt ()
      BTPlusConstCt ('constant')
      BTSquareCt ()
      BTTimesConstCt ('constant')
  BTGeneralCt ('variables')
    BTAllDiffCt ()
    BTBlockCt ('block')
    BTCardinalityCt ('expression' 'card' ...)
    BTCaseOfCt ('caseBlock' 'actionBlocks')
    BTIfThenCt ('ifBlock' 'thenBlock')
      BTIfThenElseCt ('elseBlock')
    BTIncreasingCt ('varsIndexDictionary')
      BTStrictlyIncreasingCt ()
    BTLinearCt ('expression' 'constant')
      BTLinearEqualCt ('min' 'max' 'remainingArity')
      BTSumEqualCt ()
      BTLinearGreaterOrEqualCt ('max' 'maxList')
    BTMaxOfCt ('maxVar')
    BTMetaXOrCt ()
    BTMinOfCt ('minVar')
    BTNaryLogicalCt ('expression' 'leftVariablesNumber')
      BTAndCt ('trueVariablesNumber')
      BTOrCt ('falseVariablesNumber')
      BTXOrCt ('falseVariablesNumber')
    BTTemporalDisjunctionCt ('durations' 'eventList')
  BTternaryCt ('x' 'y' 'z')
    BTMinusVarCt ()
    BTPlusVarCt ()
    BTTimesVarCt ()
  BTUnaryCt ('x')
    BTDynamicDomainViewCt ('view')
    BTDynamicValueViewCt ('view')
```

Cette hiérarchie est dominée par la classe abstraite **BTConstraint** dont la hiérarchie immédiate comprend les sous-classes suivantes :

```
BTConstraint ('isPersistent' 'isStatic' 'owner')
  BTUnaryCt ('x')
  BTBinaryCt ('x' 'y')
  BTternaryCt ('x' 'y' 'z')
  BTGeneralCt ('variables' 'arity')
```

La classe **BTUnaryCt** est une classe abstraite et représente les contraintes qui n'impliquent qu'une variable et elle a les deux sous-classes concrètes suivantes :

```
BTUnaryCt ('x')
  BTDynamicDomainViewCt ('view')
  BTDynamicValueViewCt ('view')
```

qui sont utilisées pour l'affichage dynamique des variables durant la résolution du problème. Ces deux classes seront détaillées à la partie suivante.

La classe abstraite **BTBinaryCt** définit deux variables d'instance **x** et **y**, qui représentent les deux variables impliquées par la contrainte. Ses sous-classes sont les suivantes :

```
BTBinaryCt ('x' 'y')
  BTArrayIndexCt ('array')
    BTAllDiffArrayIndexCt ()
  BTBinaryExtensionCt ('relation')
  BTComparatorCt ()
    BTGreaterOrEqualCt ()
    BTGreaterThanCt ()
  BTEqualCt ()
  BTPerformCt ('expression')
  BTUnaryOperatorCt ()
    BTAbsCt ()
    BTOppositeCt ()
    BTPlusConstCt ('constant')
    BTSquareCt ()
```

La classe **BTEqualCt** représente la relation d'égalité entre variables. La classe **BTBinaryExtensionCt** représente les contraintes définies en extensions, par la liste des couples la satisfaisant. Les différentes classes de comparateurs, sous-classes de **BTComparatorCt** implémentent les contraintes $X \geq Y$ et $X > Y$. Les classes d'opérateurs unaires, sous-classes de **BTUnaryOperatorCt** implémentent les contraintes binaires correspondant aux opérateurs unaires suivants : $|X|$, $-X$, $X + Cste$, X^2 et $X \times Cste$. Les contraintes correspondantes sont : $Y = |X|$, $Y = -X$, $Y = X + Cste$, $Y = X^2$ et $Y = X \times Cste$. La classe **BTArrayIndexCt** permet de contraindre l'indice d'un tableau. Cette contrainte exprime la relation $Y = T[X]$ où T est un tableau SMALLTALK, et X une variable entière et Y une variable contrainte de type quelconque. Le problème suivant illustre son fonctionnement :

```
BTArrayIndexCt
  array: #(a b c b a)
  index: (BTIntegerVariable from: 1 to: 5).
```

Les instanciations cohérentes de cette contrainte sont:

⇒ (1 #a) (5 #a) (2 #b) (4 #b) (3 #c)

La classe abstraite **BTternaryCt** a trois sous-classes qui représentent respectivement les relations : $X - Y = Z$, $X + Y = Z$ et $X \times Y = Z$

```
BTternaryCt ('x' 'y' 'z')
  BTminusVarCt ()
  BTplusVarCt ()
  BTimesVarCt ()
```

La classe **BTGeneralCt** représente les contraintes d'arité quelconque. Elle dispose d'une variable d'instance **variables** qui contient la liste de ses variables.

```
BTGeneralCt ('variables')
  BTAllDiffCt ()
  BTBlockCt ('block')
  BTCardinalityCt ('expression' 'card' ...)
  BTNaryLogicalCt ('expression' 'leftvariablesNumber')
    BTAndCt ('truevariablesNumber')
    BTOrcT ('falsevariablesNumber')
    BTXOrCt ('falsevariablesNumber')
  BTIncreasingCt ('varsIndexDictionary')
  BTLinearCt ('expression' 'constant' 'mustBeInitialized')
    BTLinearEqualCt ('min' 'max' 'remainingArity')
    BTSumEqualCt ()
    BTLinearGreaterOrEqualCt ('max' 'maxList')
  BTMaxOfCt ('maxVar')
  BTMinOfCt ('minVar')
  BTTemporalDisjunctionCt ('durations' 'eventList')
  BTMetaXOrCt ()
  BTCaseOfCt ('caseBlock' 'actionBlocks')
  BTIfThenCt ('ifBlock' 'thenBlock')
    BTIfThenElseCt ('elseBlock')
```

La sous-classe **BTAllDiffCt** représente la différence deux à deux. La classe **BTBlockCt** permet de définir des contraintes à l'aide de n'importe quelle expression booléenne. La classe **BTCardinalityCt** implémente les contraintes de cardinalité [Régis, 1996], que l'on utilise par exemple dans la multiplication magique, pour exprimer que parmi les variables deux exactement doivent prendre la valeur 0, deux la valeur 1, etc. Les sous-classes de la classe abstraite **BTNaryLogicalCt** implémentent les relations logiques usuelles entre variables booléennes. La classe **BTIncreasingCt** est une généralisation au cas de plus de deux variables de la contrainte $X \geq Y$. Les trois sous-classes de la classe **BTLinearCt** représentent re-

spectivement les équations linéaires quelconques, par exemple $3 \times X + 2 \times Y - 6 \times Z = 18$, les sommes sans coefficient, par exemple $X + Y + Z = 18$, et les inégalités du type de $3 \times X + 2 \times Y - 6 \times Z \geq 18$. La classe **BTMinCt** et **BTMaxCt** représente respectivement les relations $X = \max(Y_1, \dots, Y_n)$ et $X = \max(Y_1, \dots, Y_n)$.

3.2.2 LA CRÉATION DES CONTRAINTES

La méthode standard de création des contraintes en BACKTALK est l'utilisation du mécanisme usuel d'instanciation des classes de contraintes. Cette méthode présente l'avantage d'être parfaitement intégrée au langage hôte SMALLTALK et de ne pas introduire de syntaxe particulière. Cependant, pour certaines classes de contraintes, telles les contraintes arithmétiques, il est plus naturel pour l'utilisateur d'employer une syntaxe plus habituelle. Par exemple, pour créer une contrainte exprimant la relation $3 \times X + 2 \times Y - 6 \times Z = 18$ en suivant la première méthode, on doit écrire le code SMALLTALK suivant :

```
BTLinearEqualCt
  on: (Array with: x with: y with: z)
  coefs: #(3 2 -6)
  constant: 18
```

Cependant, on préfère généralement écrire directement l'expression arithmétique dont on souhaite faire une contrainte. Ceci est possible en BACKTALK grâce à l'utilisation de classes spécifiques : les expressions BACKTALK. Ces deux méthodes sont présentées dans ce qui suit.

A) CRÉATION PAR INSTANCIATION DES CLASSES DE CONTRAINTES

La méthode la plus directe pour la création de contraintes en BACKTALK est d'instancier l'une des classes de contraintes en utilisant l'un des messages d'instanciation de la classe. Le message d'instanciation de classe par défaut de SMALLTALK est **new**, qui crée une nouvelle instance. Pour les contraintes, la méthode **new** n'est pas implémentée puisqu'on doit préciser au minimum la les variables sur lesquelles porte une contrainte.

La méthode par défaut de création de contraintes a pour sélecteur **on:** et prend en argument une collection de variables contraintes. Cette méthode n'est pas implémentée par les classes abstraites telles **BTConstraint** ou **BTBinaryCt**. Voici quelques exemples de création de contraintes utilisant le message **on:**.

```
BTAllDiffCt      "x, y et z sont deux à deux distinctes"
  on: (Array with: x with: y with: z).

BTIncreasingCt   "x > y > z > t"
  on: (Array with: x with: y with: z with: t).

BTEqualCt        "a = b"
  on: (OrderedCollection with: a and: b)
```

La première expression SMALLTALK crée une contrainte exprimant que les variables **x**, **y** et **z** ont des valeurs deux à deux distinctes. La deuxième spécifie que la valeur de la variable **x** est inférieure à celle de la variable **y**, qui est inférieure à celle de **z**, elle-même inférieure à celle de **t**. enfin, la troisième requiert l'égalité des valeurs des variables **a** et **b**. Il est à noter que SMALLTALK n'étant pas un langage typé, le type de la collection en argument de la méthode **on:** importe peu.

Pour plus de commodité, et pour éviter de créer des collections inutiles, on peut utiliser les méthodes de sélecteurs **on:**, **on:and:** ou encore **on:and:and:**. Par exemple, dans les expressions suivantes :

BTEqualCt	on: a and: b.	"x = y"
BTGreaterCt	on: x and: y.	"x > y"
BTPlusVarCt	on: x and: y and: z.	"x + y = z"
BTOppositeCt	on: x and: y.	"x = - y"

Pour les contraintes plus complexes, il est nécessaire de donner des arguments supplémentaires lors de la création. Ainsi, pour les contraintes linéaires, on doit préciser à la fois les coefficients et la constante :

```

BTLinearEqualCt          "3.x + 2.y - z = 0"
  on: (Array with: x with: y with: z)
  coefs: #(3 2 -1)
  constant: 0.

BTTimesConstCt           "x = 3.y"
  on: x
  and: y
  constant: 3.

```

B) CRÉATION PAR L'UTILISATION DES EXPRESSIONS

L'utilisation des classes d'expressions BACKTALK permet de déclarer des contraintes en utilisant une syntaxe plus naturelle pour la création de certaines contraintes, notamment les contraintes arithmétiques et logiques, mais aussi quelques contraintes plus spécifiques à la programmation par objets, que nous aborderons dans la partie "contraintes et objets".

La notion d'expression est purement syntaxique, et permet de dispenser l'utilisateur de la création explicite des contraintes. En pratique, on envoie aux variables des messages qui créent des expressions. Ces expressions sont ensuite transformées automatiquement en contraintes. Par exemple, les messages correspondant aux opérateurs arithmétiques usuels (e.g. +, -, *) sont implémentés dans les classes des variables contraintes, et provoquent la création d'expressions BACKTALK, auxquelles on peut également appliquer les opérateurs arithmétiques, ce qui construit de nouvelles expressions plus complexes. Finalement, les expressions arithmétiques sont utilisées pour la création de contraintes à l'aide des messages " $\leq$ ", " $\geq$ ", "<", ">" ou "=".

Afin d'illustrer ce mécanisme, considérons la contrainte suivante : " $x + 2.y - 5.z \geq 0$ ", où **x**, **y** et **z** sont des variables contraintes entières (i.e. des instances de la classe **BTIntegerVariable**). Comme nous l'avons écrit plus haut, on peut créer une telle contrainte en instanciant la classe **BTLinearGreaterOrEqualCt** avec le message suivant :

```
BTLinearGreaterOrEqualCt
  on: (Array with: x with: x with: z)
  coefficients: #(1 2 -5)
  value: 0
```

L'utilisation des expressions permet d'utiliser directement la syntaxe SMALL-TALK traditionnelle pour la création des expressions arithmétiques. Il suffit d'évaluer l'instruction suivante :

$$(E) \quad x + (2*y) - (5*z) \geq 0$$

Plus précisément, l'évaluation de l'expression **(E)** en BACKTALK déclenche la série d'évaluations suivantes :

2*y	Crée une expression linéaire BACKTALK.
x+(2*y)	Crée une nouvelle expression à partir de la précédente.
x+(2*y)-(5*z)	Crée une dernière expression arithmétique BACKTALK.
x+(2*y)-(5*z)>=0	Crée une contrainte BTLinearGreaterOrEqualCt portant sur les variables x , y et z .

Les expressions sont des classes, organisées dans la hiérarchie d'héritage suivante :

```
BTExpression ()
  BTEumericExpression ('variables')
    BTCardinality ('targetValue')
    BTLinearCombination ('coefs')
  BTLogicalExpression ()
    BTAnd ()
    BTOr ()
    BTXor ()
  BTPerform ('messages' 'injective' 'linkedVariable')
    BTBooleanPerform ()
    BTPerformArg ('arg')
```

La classe **BTEexpression** est abstraite et dispose de trois sous-classes, également abstraites, qui désigne respectivement les expressions numériques (instances de la classe **BTNumericExpression**), logiques (instances de la classe **BTLogicalExpression**) et des expressions spécifiques aux objets (e.g. **BTPerformCt**, que nous présenterons plus loin).

3.3 LES CLASSES D'ALGORITHMES

Nous avons prôné l'utilisation d'un algorithme de type full look-ahead pour la résolution des problèmes de satisfaction de contraintes. Ainsi, l'algorithme de résolu-

tion de BACKTALK est fondé sur cette stratégie consistant à propager toutes les contraintes à chaque pas de boucle d'énumération. Nous avons également souligné le fait que cette stratégie ne soit pas universellement efficace, mais nécessite d'être adaptée pour la résolution de problèmes spécifiques.

3.3.1 ARCHITECTURE DES ALGORITHMES

Nous proposons dans BACKTALK une architecture qui permet à l'utilisateur de spécifier relativement simplement tous les algorithmes qui partagent la structure suivante : une procédure de recherche arborescente est entrelacée avec un procédé de réduction des domaines des variables par application de méthodes de cohérence d'arc ou de filtrage de contrainte. Plus précisément, ces algorithmes se décomposent naturellement en quatre points essentiels : une phase de *réduction de domaine* par filtrage ou cohérence d'arc ; une stratégie de *choix de la prochaine variable* à instancier ; une stratégie de *choix de la valeur* de cette variable ; une stratégie de *retour arrière*. On peut schématiser cette famille d'algorithmes par la figure suivante :

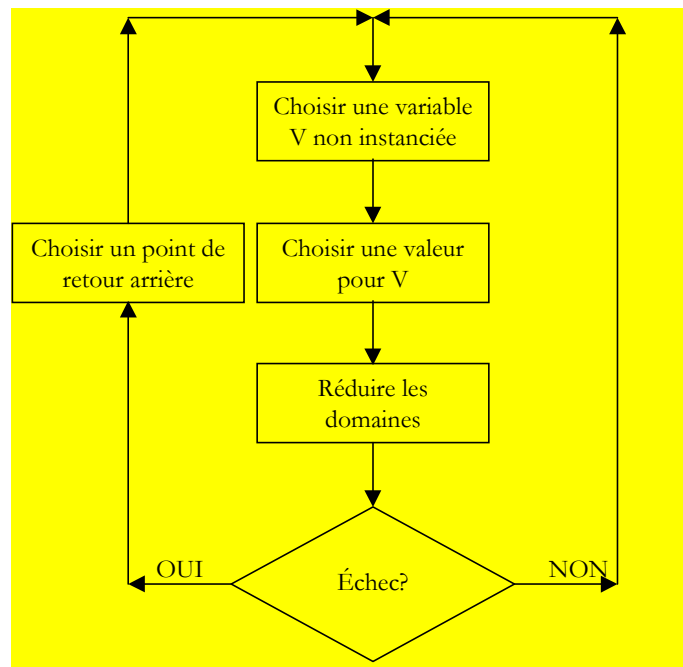


Figure 14 Le schéma abstrait de la famille des algorithmes complets de résolution des problèmes de satisfaction de contraintes

Ce schéma abstrait permet d'instancier, en spécifiant les différents points évoqués ci-dessus, tous les types d'algorithmes fondés sur le filtrage de contrainte et la recherche arborescente.

Pour mettre en œuvre ce schéma d'algorithmes, nous avons suivi les design patterns *Strategy* et *Template Method*. Le pattern *Strategy* consiste à encapsuler des méthodes de résolutions dans une classe afin d'en faire une famille d'algorithmes interchangeables. Le pattern *Template Method* consiste à écrire dans une classe des méthodes qui définissent une suite d'opérations à effectuer selon un certain ordre. Chacune de ces opérations est implémentée par une méthode de la classe ou d'une de ses sous-classes. La combinaison de ces deux patterns consiste à définir des *Strategy* (i.e. classes qui encapsulent un algorithme) dont certaines méthodes sont des *Template Methods*. Il est ainsi possible de redéfinir le contenu de chaque étape de

l'algorithme dans des sous-classes, alors que le schéma général d'exécution reste inchangé par rapport à la super-classe.

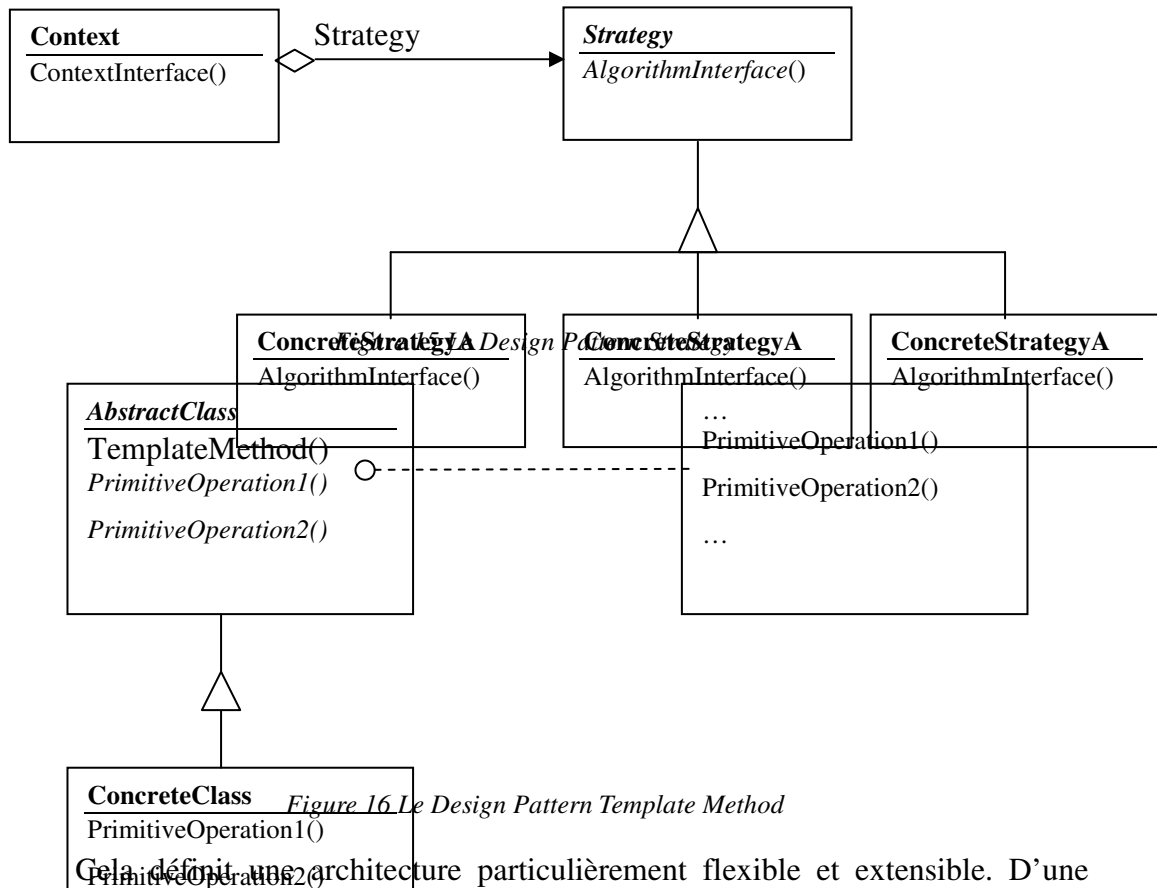


Figure 16. Le Design Pattern Template Method

Cela définit une architecture particulièrement flexible et extensible. D'une part, les algorithmes sont interchangeables au sens où, bien qu'ils n'aient pas exactement le même comportement, ils disposent de la même interface. Ainsi, l'utilisateur n'a pas à connaître le détail du fonctionnement des algorithmes, mais seulement la manière de les exécuter. D'autre part, pour définir un nouvel algorithme, il suffit d'identifier les étapes qui diffèrent entre cet algorithme et l'une des Strategy déjà implémentées. Ensuite, il suffit de redéfinir les Template Methods associées à ces étapes.

3.3.2 IMPLÉMENTATION

Concrètement, un algorithme, dans BACKTALK, est constitué des éléments suivants :

- Un problème (i.e. un ensemble de contraintes et un ensemble de variables) ;
- Une heuristique de choix de la variable à instancier en priorité ;
- Une heuristique de choix du meilleur point de retour arrière ;
- Une pile de contextes (pour la recherche arborescente) ;

l'implémentation des classes d'algorithmes est constituée par les méthodes suivantes :

- Une méthode **forward** pour commander un pas en avant à la boucle d'énumération ;
- Une méthode **backward** pour provoquer un retour arrière après un échec ;
- Une méthode **propagateDemons** qui déclenche le filtrage des contraintes par l'intermédiaire des démons ;

- Une méthode **makeAChoice** qui spécifie la manière dont on choisit une nouvelle instanciation à effectuer ;
- Une méthode **saveContext** qui spécifie comment mémoriser l'état courant du problème ;
- Une méthode **restoreContext** qui spécifie comment rétablir l'état précédent du problème.

À titre d'illustration, voici l'implémentation de quelques-unes de ces méthodes pour l'algorithme de real full look-ahead, implémenté par la classe **BTSolver** de BACKTALK :

moveForward

```
DomainWipedOutSignal
  handle: [:ex | self triggerBackward]
  do: [[self forward] repeat]
```

forward

“sauvegarde le contexte courant, et fait un nouveau choix”

```
self saveContext; makeAChoice
```

makeAChoice

“fait un nouveau choix : choisit une variable, et une valeur dans son domaine ; ensuite, déclenche la propagation de cette nouvelle instanciation”

```
self chooseAVar.
currentVal := currentVar nextValue.
self propagateDemons “filters the constraints”
```

propagateDemons

```
[| k |
(k := self bestVarToPropagate) == nil ifTrue: [^self].
k chooseAndPropagateADemon] repeat
```

backward

“initialise les démons, et si un échec est détecté, alors, provoque un retour arrière”

```
self resetDemons.
self domainWipedOut
  handle: [:ex | self backward]
  do: [self chooseAndRestoreAContext]
```

A) LA MARCHÉ AVANT

Les méthodes **moveForward**, **forward**, **makeAChoice** et **propagateDemons** servent à faire progresser l'algorithme, c'est-à-dire à prolonger la solution partielle qu'il est en train de construire. La méthode **moveForward** effectue simplement la répétition de **forward** jusqu'à ce qu'un signal d'exception de type **DomainWipedOutSignal** soit déclenché. La méthode **forward** empile le contexte courant, afin de pouvoir y revenir en cas d'échec, et appelle la méthode

makeAChoice qui choisit, selon des heuristiques, une variable et une valeur du domaine de cette variable, et ensuite déclenche le filtrage des contraintes.

B) LE RETOUR ARRIÈRE

Dans la méthode **backward** écrite ci-dessus, apparaît l'instruction suivante :

```
self domainwipedOut
  handle: [:ex | self backward]
  do: [self chooseAndRestoreAContext]
```

Cette instruction est une gestion d'exception SMALLTALK, avec un appel récursif à la méthode **backward**. Cet appel récursif est déclenché par la levée de l'exception **BTDomainWipedOut** qui indique qu'un des domaines est vide. Ainsi, il faut restaurer un nouveau contexte.

C) L'INTERFACE UTILISATEUR

Voici, finalement l'interface utilisateur implémentée dans la classe **BTSolver**. Ces méthodes ne sont pas susceptibles d'être redéfinies dans les sous-classes, afin de préserver la même interface pour tous les algorithmes qui sont supposés être interchangeables.

```
firstSolution
| solved solution |
solved := true.
self addResolutionTime:
  (Time millisecondsToRun:
    [NoSolutionFoundSignal
      handle: [:ex | solved := false]
      do: [solution := self basicFirstSolution]]).
solved ifTrue: [^solution].
Dialog warn: 'no solution found'.
^nil
```

```
nextSolution
| return |
self addResolutionTime:
  (Time millisecondsToRun:
    [NoSolutionFoundSignal
      handle: [:ex | return := nil]
      do:
        [contexts removeLast.
         return := self backward;
        basicFirstSolution]]).
^return == nil
  ifTrue: [^NoSolutionFoundSignal raise]
  ifFalse: [return]
```

allSolutions

```
| list |  
list := BTCollection new.  
self allSolutionsDo: [:s | list add: s].  
^list
```

allSolutionsDo: aDoBlock

```
NoSolutionFoundSignal  
handle:  
    [:ex |  
        BTConstraint initialize.  
        ObjectMemory garbageCollect]  
do:  
    [aDoBlock value: self firstSolution.  
    [aDoBlock value: self nextSolution]  
    repeat]
```

3.3.3 DÉFINIR DE NOUVEAUX ALGORITHMES DE RÉOLUTION

Si l'on veut implémenter un nouvel algorithme de résolution, il suffit de créer une sous-classe de la classe **BTSolver**, et d'y redéfinir les méthodes correspondant aux changements voulus. Donnons immédiatement un exemple simple. La stratégie de résolution la plus simple, c'est-à-dire l'énumération explicite, définie par l'Algorithme 1 défini au paragraphe 3.1, s'implémente simplement en redéfinissant la méthode **propagateDemons** de la façon suivante :

propagateDemons

“Si une contrainte est violée, i.e. `issatisfied` rend `false`, alors, on signale un échec”

```
constraints do: [:ct |  
    ct issatisfied ifFalse: [self domainwipedOut]].
```

Un exemple plus complexe est l'implémentation d'un algorithme de retour arrière intelligent pour la résolution d'un problème très particulier : la construction de grilles de mots croisés. Nous présenterons cet algorithme dans la partie consacrée aux mots croisés.

3.4 LES HEURISTIQUES

À l'instar des variables, des contraintes et des algorithmes de résolution, nous avons choisi de réifier les heuristiques et d'en faire des classes organisées dans une hiérarchie d'héritage. Une autre approche aurait consisté à “diluer” les heuristiques dans les différentes classes constituant BACKTALK. La réification des heuristiques présente l'avantage d'isoler les heuristiques des autres mécanismes de résolution, et ainsi de clarifier la structure du système. Cela permet en outre de les manipuler aussi simplement que n'importe quel objet SMALLTALK, ce qui rend possible la construction d'*heuristiques composées* et de *méta-heuristiques*.

3.4.1 LES HEURISTIQUES “SIMPLES”

La classe la plus générale d’heuristiques est une classe abstraite, nommée **BTHeuristic**. Cette classe comporte trois variables d’instances : **elements** qui est la liste des objets que l’on souhaite sélectionner ; **filteringBlock** qui est block SMALLTALK dont l’évaluation rend un booléen et qui permet de sélectionner des éléments suivant un critère quelconque ; **solver** qui pointe sur le problème propriétaire de l’heuristique.

La définition SMALLTALK de la classe **BTHeuristic** est la suivante :

```
Object subclass: #BTHeuristic
  instanceVariableNames: 'elements filteringBlock solver '
  classVariableNames: ''
  poolDictionaries: ''
  category: 'BT-Heuristics'
```

Cette classe abstraite implémente une méthode de sélecteur **next** qui renvoie un élément de la liste **elements**. C’est cette méthode qui est au cœur de l’heuristique, et dont l’implémentation permet de prendre en compte les critères que l’on souhaite appliquer à la sélection des éléments.

La hiérarchie des heuristiques prédéfinies en BACKTALK est la suivante :

```
BTHeuristic ('elements' 'filteringBlock' 'solver')
  BTDynamicHeuristic ('sortCriterion' 'optimalValue')
    BTMixedHeuristic ('firstElements')
  BTMetaHeuristic ('threshold' 'backtrackDictionary')
  BTMinSize ()
  BTMinSizeMinValue ()
  BTMultiCriterionHeuristic ('sortCriteria')
  BTMultiHeuristic ('sortCriteria')
  BTStaticHeuristic ('sortCriterion')
```

A) ORDRE STATIQUE SUR LES VARIABLES

Une heuristique couramment utilisée en satisfaction de contraintes est le choix de la variable la plus contrainte d’abord, l’idée sous-jacente est que la variable la plus contrainte est celle qui apporte le plus d’information sur le problème et donc dont l’instanciation permettra de réduire le plus efficacement la taille de l’arbre de recherche.

La classe utilisée pour créer de telles heuristiques est nommée **BTStaticHeuristic** et dispose d’une variable d’instance supplémentaire, nommée **sortCriterion**, qui pointe sur un block SMALLTALK permettant de comparer deux à deux les éléments de la liste.

```
BTHeuristic subclass: #BTStaticHeuristic
  instanceVariableNames: 'sortCriterion '
  classVariableNames: ''
  poolDictionaries: ''
  category: 'BT-Heuristics'
```

La création d'une heuristique de type **BTStaticHeuristic** se fait par l'intermédiaire du message d'instanciation suivant, envoyé à la classe **BTStaticHeuristic**.

```
BTStaticHeuristic
  on: variables
  filteringBlock: [:v | v instantiated not]
  criterionBlock: [:v1 :v2 | v1 connectivity < v2 connectivity]
```

Enfin, la méthode **next**, utilisée pour sélectionner une nouvelle variable est implémentée comme il suit :

```
next
  ^elements
    detect: [:x | filteringBlock value: x]
    ifNone: [self solutionFound]
```

B) PLUS PETIT DOMAINE D'ABORD (FIRST FAIL PRINCIPLE)

Dans ce paragraphe, nous allons détailler une des classes de la hiérarchie, la classe **BTMinSize**, qui fournit une implémentation d'une heuristique fréquemment utilisée en satisfaction de contraintes pour la sélection des variables. Cette heuristique consiste à sélectionner, parmi les variables qui ne sont pas encore instanciées, celle ayant le plus petit domaine. L'idée de cette méthode est de conduire rapidement à des échecs et on la nomme pour cela *first fail principle* en anglais.

Pour instancier une heuristique de type **BTMinSize**, on envoie le message suivant à la classe **BTheuristic** :

```
BTheuristic
  on: variables
  filteringBlock: [:v | v instantiated not]
```

La seule méthode redéfinie dans cette sous-classe de la classe **BTheuristic** est la méthode **next** dont l'implémentation suit :

```
next
  "renvoie la variable ayant le plus petit domaine parmi celles
  de la collection <elements> qui ne sont pas déjà instanciées"

  | bestElement bestSize |
  bestSize := SmallInteger maxVal. "initialisation suffisante"
  Elements
    do: [:v |
      ((filteringBlock value: v) and: [v size < bestSize])
      ifTrue:
        [bestSize := v size.
         bestElement := v]].
  BestElement isNil ifTrue: [self solutionFound].
  ^bestElement
```

La méthode **solutionFound** appelée dans la méthode **next** précédente est implémentée dans la classe **BTheuristic** comme suit :

solutionFound

“informe le solver qu’il ne reste pas de variable
non instanciée”

solver solutionFound

3.4.2 HEURISTIQUES “MULTICRITÈRES”

Il est fréquent de devoir appliquer plusieurs critères successifs à la sélection d’un élément par une heuristique. BACKTALK fournit une classe nommée **BTMultiHeuristic** qui permet de définir ce genre de stratégies en fournissant les différents critères sous la forme de blocks SMALLTALK.

Par exemple, dans un problème numérique, si l’on souhaite appliquer la stratégie qui consiste à choisir la variable ayant le plus petit domaine et, en cas d’égalité, de choisir celle dont la valeur minimum est la plus petite, on créera une instance de la classe **BTMultiHeuristic** de la façon suivante :

BTMultiHeuristic

on: variables

filteringBlock: [:v | v instantiated not]

criterionBlocks: (Array

with: [:v1 :v2 | v1 size < v2 size]

with: [:v1 :v2 | v1 min < v2 min])

Il est à noter que l’adjectif multicritère est ici entre guillemets car les différents critères sont totalement ordonnés suivant leur importance. Autrement dit, le $i^{\text{ème}}$ est appliqué uniquement pour séparer les ex æquo éventuels pour le $j^{\text{ème}}$ critère. Il n’y a donc aucun raisonnement relevant de l’analyse multicritère.

3.4.3 HEURISTIQUES “COMPOSÉES”

Tous les problèmes ne sont pas aussi homogènes que le placement de reines sur un échiquier (cf. Problème 1), la construction de carrés, de séquences ou encore d’arbres magiques (cf. Problème 4, Problème 5 et Problème 6). Par exemple, dans le problème dit de la multiplication magique (cf. Problème 3) toutes les variables ne sont pas similaires car elles sont sujettes à des contraintes diverses. Pour résoudre un tel problème il peut être utile d’accorder plus d’importance à certaines variables plutôt qu’à d’autres. Dans ce cas précis, il est évident que les six premières variables définies au **Erreur ! Source du renvoi introuvable.**, et nommées A, B, C, D, E et F apportent plus d’information que les autres, et c’est d’ailleurs par elles qu’un humain, confronté à ce problème, commence sa résolution.

Pour utiliser ce type de stratégies, on peut définir en BACKTALK des heuristiques dites *composées* qui permettent de grouper les variables en plusieurs listes et d’appliquer à chaque collection des critères de choix différents. Dans le cas précédent, on souhaite ranger les variables dans deux listes, la première contient les six premières variables A, B, C, D, E et F, et la seconde contient les autres variables du problème. On souhaite ensuite appliquer à chaque liste le principe du plus petit domaine d’abord.

La définition en BACKTALK d'une telle stratégie se fait par le message d'instanciation suivant envoyé à la classe **BTCompositeHeuristique** :

BTCompositeHeuristic

```
on: (Array
  with: ((OrderedCollection new) "la 1ère liste"
    add: A; add: B; add: C; add: D;
    add: E; add: F; yourself)
  with: ((OrderedCollection new) "la 2nde liste"
    add: G; add: H; add: I; add: J;
    add: K; add: L; add: M; add: N;
    add: O; add: P; add: Q; add: R;
    add: S; add: T; yourself)
filteringBlocks: (Array
  with: [:v | v instantiated not] "filtrage de la liste 1"
  with: [:v | v instantiated not]) "filtrage de la liste 2"
criterionBlocks: (Array
  with: [:x :y | x size < y size] "critère de la 1ère liste"
  with: [:x :y | x size < y size]) "critère de la 2nde liste"
```

Dans ce cas, la résolution du problème se fait de la façon suivante : BACKTALK instancie les variables de la première liste qui ne sont pas déjà instanciées en commençant par celles de plus petit domaine. Une fois chacune des variables de la première liste instanciée, BACKTALK sélectionne les variables non instanciées de la seconde liste en privilégiant également les petits domaines. Le tableau suivant en donne une illustration :

Multiplication magique	<i>1ère solution</i>	<i>Unicité</i>
<i>Plus petit domaine d'abord</i>	1.300 ms	20.000 ms
	101 backtracks	1769 backtracks
<i>A → F, puis G → T (plus petit domaine d'abord)</i>	100 ms	33.000 ms
	7 backtracks	2978 backtracks

Tableau 4 Les temps de résolution et le nombre de backtracks pour le calcul de la première solution, puis la preuve de l'unicité, de la multiplication magique. La résolution a été faite dans un premier temps avec l'heuristique de choix de la prochaine variable à instancier suivant le principe du plus petit domaine d'abord, appliqué à toutes les variables. Dans un second temps, la résolution utilise l'heuristique consistant à sélectionner d'abord les six premières variables, et ensuite les 14 variables restantes, et en appliquant également le critère du plus petit domaine d'abord. L'obtention de la première solution est dix fois plus rapide avec cette dernière stratégie. En revanche la preuve de l'unicité est plus rapide avec la première stratégie.

Chapitre 4

Implémentation du mécanisme de filtrage en BACKTALK : les démons

Dans ce chapitre nous allons décrire l'implémentation du mécanisme de filtrage des contraintes en BACKTALK. Ce mécanisme permet, sous le contrôle de l'algorithme de résolution, d'utiliser les contraintes afin de réduire les domaines des variables. Ce mécanisme est au cœur du système et il nécessite la collaboration des principales hiérarchies de classes de BACKTALK. Nous avons choisi de le mettre en place de façon explicite en le représentant par des objets particuliers : les *démons*. Un démon est un objet qui met en relation une variable avec les contraintes qui l'impliquent. Un démon sert à mémoriser les différentes réductions de domaine subies par la variable lors de la résolution du problème. Cette mémoire est ensuite utilisée par l'algorithme de résolution afin de déclencher les filtrages des contraintes lors de la phase de propagation.

L'implémentation des mécanismes de filtrage à l'aide des démons présente plusieurs avantages essentiels. D'une part, l'utilisation des démons permet d'implémenter les méthodes de filtrage des contraintes de façon *modulaire* et *partielle* : au lieu d'écrire une méthode générale de filtrage pour chaque type de contrainte qu'il définit, l'utilisateur/programmeur peut définir une méthode de filtrage élémentaire pour chaque type de réduction de domaine subie par la variable. Cela simplifie l'écriture des méthodes de filtrage et améliore leur fiabilité. De plus, cette approche permet de définir les méthodes de filtrage de façon *réactive*, le filtrage étant déclenché en réaction à une réduction de domaine particulière, et *incrémentale* car la méthode de filtrage exécutée a pour objet de supprimer les valeurs rendues incohérentes par cette réduction de domaine. La définition incrémentale des méthodes de filtrage permet une amélioration importante de l'efficacité de la phase de propagation.

4.1 UNE APPROCHE RÉACTIVE DU FILTRAGE DES CONTRAINTES : LES *DÉMONS*

Nous avons présenté un schéma abstrait pour les algorithmes de résolution qui consiste essentiellement en une boucle d'énumération entrelacée avec une phase de filtrage des contraintes qui a pour objet de réduire les domaines des variables. En outre, nous avons précisé que durant cette phase de filtrage, la stratégie la plus efficace consiste à filtrer toutes les contraintes jusqu'à obtention d'un point fixe, c'est-à-dire, jusqu'à ce le filtrage ne produise aucune réduction de domaine supplémentaire. Mais nous n'avons proposé aucune implémentation pour cette phase de l'algorithme.

Dans l'algorithme de full look-ahead, la phase de filtrage est constituée d'un algorithme de type AC_n qui réalise le filtrage, originellement la cohérence d'arc, de toutes les contraintes. Cette approche procède d'une vision "centralisée" de la phase de filtrage : un algorithme manipule les contraintes de façon à obtenir un résultat, puis "rend la main" à la boucle d'énumération.

Au contraire, notre approche se fonde sur une vision essentiellement *réactive* du filtrage, au sens où nous avons employé ce terme concernant les systèmes de propagation locale. Plutôt que de réaliser le filtrage avec un algorithme centralisé, il est effectué grâce à une collaboration entre les contraintes et les variables. Concrètement, chaque variable mémorise les différentes réductions de domaine qu'elle subit. Cette mémoire est ensuite utilisée pour envoyer aux contraintes des messages indiquant les réductions de domaines subies, ce qui en déclenche le filtrage. Chaque filtrage provoque d'autres réductions de domaines qui sont également mémorisées par les variables concernées, et sont utilisées à leur tour pour déclencher de nouveaux filtrages. Ce processus s'arrête une fois que la mémoire de chaque variable est épuisée.

La mémorisation des réductions de domaines subies par les variables ainsi que l'envoi de messages aux contraintes n'est pas directement géré par les variables mais se fait par l'intermédiaire d'objets particuliers appelés *démons*. Un démon associe une variable à une liste de contraintes impliquant cette variable, et possède un état qui permet de mémoriser une réduction de domaine de la variable. Voici, décrit grossièrement, le processus de filtrage :

filtrer

“Méthode implémentée au sein de l’algorithme de résolution”

$\forall V$ une variable

$\forall D$ un démon actif associé à V
déclencher(D)

déclencher(D)

“Méthode implémentée dans la classe des démons”

$\forall C$ une contrainte associée à D
filtrer(C)

filtrer(C)

“Méthode implémentée dans les classes de contraintes :
cette méthode provoque des réductions des domaines
de ses variables, ce qui active de nouveaux démons”

<code dépendant de la contrainte C >

La mise en œuvre de ce processus est en fait un peu plus raffinée puisqu’il existe plusieurs classes de démons, ce qui permet une plus grande richesse et facilité pour la définition du filtrage d’une contrainte.

4.2 DÉFINITION *MODULAIRE* ET *INCRÉMENTALE* DES MÉTHODES DE FILTRAGE

Jusqu’à maintenant, nous avons défini le filtrage d’une contrainte par un algorithme, comme par exemple dans les algorithmes de la 5.2.1b) (cf. Algorithme 7 et Algorithme 8). Cependant, il peut s’avérer difficile de définir le filtrage de certaines contraintes à l’aide d’un unique algorithme. Nous allons utiliser la structure des algorithmes de résolution que nous employons pour permettre la définition des filtres de façon *modulaire* et *incrémentale*, c’est-à-dire en utilisant plusieurs méthodes élémentaires, qui permettent de maintenir la cohérence des contraintes après chaque type de réduction de domaine subie par les variables.

Les algorithmes que nous utilisons pour la résolution procèdent à une recherche arborescente par instantiation des variables. Chaque instantiation est ensuite propagée *via* les contraintes afin de réduire les domaines des autres variables. Il est intéressant de constater que l'on peut classer les différentes réductions de domaines subies durant cette phase selon les six catégories suivantes : 0) suppression de tous les éléments du domaine ; 1) réduction du domaine à un singleton ; 2) suppression d'un des éléments du domaine ; 3) diminution de la borne supérieure ou 4) accroissement de la borne inférieure du domaine (pour les domaines ordonnés) ou bien 5) autre réduction, par exemple la suppression de plusieurs éléments d'un coup. Voici une illustration de chacune de ces réductions possibles sur un même domaine initial :

Type de réduction	Domaine initial	Domaine final
Vidage complet	{0, 1, 2, 3, 4, 5, 6}	$\emptyset$
Réduction à un singleton	{0, 1, 2, 3, 4, 5, 6}	{4}
Suppression d'un élément	{0, 1, 2, 3, 4, 5, 6}	{0, 1, 3, 4, 5, 6}
Augmentation de la borne inférieure	{0, 1, 2, 3, 4, 5, 6}	{5, 6}
Diminution de la borne supérieure	{0, 1, 2, 3, 4, 5, 6}	{0, 1, 2, 3}
Suppression de plusieurs éléments	{0, 1, 2, 3, 4, 5, 6}	{1, 2, 5, 6}

Tableau 5 Typologie des réductions de domaines possibles lors de la résolution d'un problème de satisfaction de contraintes

4.2.1 MÉTHODES ÉLÉMENTAIRES DE FILTRAGE

Cette typologie des réductions de domaine conduit naturellement à définir le filtrage d'une contrainte en spécifiant des filtres "élémentaires" pour chaque type de réduction et pour chaque variable impliquée. En d'autres termes, le processus de filtrage des contraintes, exposé au paragraphe précédent, devient ceci :

```

filtrer
    "Méthode implémentée au sein de l'algorithme de
    résolution"

    ∀V une variable
        ∀D un démon actif associé à V
            déclencher(D)

déclencher(D)
    "Méthode implémentée dans la classe des démons"

    T ← type de réduction subi par V
    ∀C une contrainte associée à D
        filtrer(C, T, V)

```

filtrer(C, T, V)

“Méthode implémentée dans les classes de contraintes :
cette méthode provoque des réductions des domaines
de ses variables, ce qui active de nouveaux démons”

<code dépendant à la fois
- de la contrainte C
- du type de réduction T
- de la variable V>

Ainsi, la variable ayant subi une réduction de domaine, ainsi que le type de cette réduction sont pris en compte au moment de déclencher le filtrage. Illustrons ceci avec la contrainte simple $X \geq Y$, où X et Y sont des variables contraintes entières. Considérons qu’initialement le domaine de chacune des deux variables est l’ensemble $\{0, 1, 2, 3, 4\}$. La contrainte est dans ce cas arc cohérente. La méthode de filtrage la plus judicieuse pour cette contrainte consiste à réaliser la cohérence d’arc car cela peut être fait facilement en ne considérant que les bornes des deux domaines. En effet, par définition de la cohérence d’arc, on a les équivalences suivantes :

$$X \geq Y \text{ est arc cohérente} \quad (1)$$

$$\Leftrightarrow \forall x \in X ; \exists y \in Y \text{ t.q. } x \geq y \text{ et } \forall y \in Y ; \exists x \in X \text{ t.q. } x \geq y \quad (2)$$

$$\Leftrightarrow \forall x \in X ; x \geq \min(Y) \text{ et } \forall y \in Y ; y \geq \max(X) \quad (3)$$

$$\Leftrightarrow \min(X) \geq \min(Y) \text{ et } \max(Y) \geq \max(X) \quad (4)$$

Ainsi, en vertu de (4), on peut implémenter la méthode filtrage suivante, qui réalise la cohérence d’arc :

Soit yMin le minimum de Y
Soit xMax le maximum de X
Supprimer de X tous les éléments plus petits que yMin
Supprimer de Y tous les éléments plus grands que xMax

Mais cette méthode peut être implémentée de façon modulaire et incrémentale par le jeu de deux méthodes suivant :

filtrer1

“méthode exécutée quand $V = X$ et quand le type T
de réduction de domaine de V est la diminution
de la borne supérieure de Y”

Y setMax: X max

filtrer2

“méthode exécutée quand $V = Y$ et quand le type T
de réduction de domaine de V est l’augmentation
de la borne inférieure de X”

X setMin: Y min

Cette définition du filtrage est *modulaire* au sens où deux méthodes élémentaires correspondant à deux événements différents sont implémentées, au lieu d’une

méthode globale assurant le filtrage de la contrainte dans le cas général. De plus, cette définition est *incrémentale* car chacune des deux méthodes élémentaires ne réalise qu'une partie du filtrage, tout en assurant d'obtenir le même résultat. En d'autres termes, si l'on considère la formule (4) comme un invariant que l'on veut rétablir après chaque réduction de domaine, chacune des deux méthodes de filtrage rétablit une seule des deux clauses de cet invariant.

$$(\max(Y) \geq \max(X)) \wedge (\min(X) \geq \min(Y))$$

La méthode **filtrer1** rétablit la première clause de l'invariant sans considérer la seconde, qui est supposée vraie, alors que **filtrer2** rétablit la seconde sans considérer la première. Ainsi, lorsque seule la borne supérieure de la variable X est modifiée, l'exécution de la méthode **filtrer1** suffit à rétablir l'invariant. Cette approche permet une amélioration sensible de l'efficacité de la phase de propagation des contraintes.

Nous avons implémenté ce mécanisme de filtrage de façon systématique. Les classes de contraintes en BACKTALK implémentent systématiquement les cinq méthodes suivantes correspondant aux cinq types de réduction de domaine donnés au Tableau 5 (N.B. la suppression de tous les éléments du domaine n'est pas considérée ici puisqu'elle provoque un échec, traité par une exception comme nous l'avons mentionné au chapitre précédent) :

value: avariable
min: avariable
max: avariable
remove: avalue from: avariable
domain: avariable

La première méthode, **value:**, est celle qui est évaluée lorsqu'une des variables impliquées par la contrainte est instanciée. La deuxième (resp. troisième) est déclenchée lorsque la borne inférieure (resp. supérieure) du domaine d'une variable a été diminuée (resp. augmentée). La quatrième est exécutée lorsqu'une valeur unique a été supprimée du domaine d'une variable. La dernière, **domain:**, est déclenchée lorsque le domaine d'une des variable a subi une réduction non identifiée comme étant de l'un des types précédemment considérés.

Dans tous les cas la variable dont le domaine a subi une réduction est envoyée en paramètre de la méthode. De plus, dans le cas de la suppression d'un unique élément, cet élément est passé en paramètre à la contrainte.

Voici l'implémentation complète de ces cinq méthodes pour la contrainte $X \geq Y$:

```
value: uneVariable
    uneVariable == X
        ifTrue: [self max: X]
        ifFalse: [self min: Y]

min: uneVariable
    "uneVariable est nécessairement Y"

    X setMin: Y min

max: uneVariable
    "uneVariable est nécessairement X"

    Y setMax: X max

remove: aValue from: aVariable
    "aucune action"

domain: aVariable
    "aucune action"
```

On pourrait ajouter d'autres réductions de domaines à cette classification. Par exemple, on pourrait considérer la suppression de tous les éléments pairs d'un domaine contenant des nombres entiers. Cependant, pour la plupart des contraintes usuelles, il n'est pas intéressant de considérer un tel type de réduction, mais il est possible de définir de nouveaux type de réduction si l'on souhaite utiliser des contraintes qui le nécessitent.

4.2.2 LES DIFFÉRENTES CLASSES DE DÉMONS

Nous venons de proposer de définir les filtrages de contraintes par une série de méthodes élémentaires de filtrage correspondant chacune à un type donné de réduction de domaine. Nous avons aussi écrit que le filtrage d'une contrainte est déclenché par un démon attaché à une variable. Afin d'implémenter ces idées nous avons opté pour la définition de cinq classes de démons, une pour chaque type de réduction de domaine. Ces cinq classes sont organisées dans la hiérarchie suivante :

```
BTAAbstractDemon (variable, idle, constraints)
    BTValueDemon ()
    BTRemoveDemon (removedValues)
    BTDomainDemon ()
    BTMinDemon ()
    BTMaxDemon ()
```

Chaque variable est associée, selon sa classe, à plusieurs démons. Par exemple, les instances de la classe **BTVariable** sont associées à trois démons, instances des classes **BTValueDemon**, **BTRemoveDemon** et **BTDomainDemon**. Les variables dont le domaine est ordonné, comme les instances des classes

BTIntegerVariable et **BTOrderedVariable**, sont associées en plus à un démon de type **BTMinDemon** et à un démon de type **BTMaxDemon**. Voici l'implémentation de la classe abstraite **BTAbstractDemon** :

```
Object subclass: #BTAbstractDemon
  instanceVariableNames: 'variable idle constraints '
  classVariableNames: ''
  poolDictionaries: ''
  category: 'BT-Demons'
---
trigger
  “déclenche le filtrage des contraintes associées à ce démon”

  idle ifTrue: [^self].
  self switchOff; basicTrigger

basicTrigger
  “méthode abstraite”

  self subclassResponsibility

addConstraint: ct
  “on associe une nouvelle contrainte à ce démon”

  constraints add: ct

switchOff
  “le démon est désactivé”

  idle := true

switchOn
  “le démon est réactivé”

  idle := false
```

Voici ci-dessous l'implémentation, dans les cinq sous-classes concrètes de la classe abstraite **BTAbstractDemon**, de la méthode abstraite **basicTrigger** :

```
basicTrigger
  “pour la classe BTValueDemon”

  constraints do: [:ct | ct value: variable]
---
basicTrigger
  “pour la classe BTDomainDemon”

  constraints do: [:ct | ct domain: variable]
---
```

basicTrigger

“pour la classe **BTRemoveDemon**”

constraints

do:

```
[:ct |  
v1 notNil ifTrue: [ct remove: v1 from: variable].  
v2 notNil ifTrue: [ct remove: v2 from: variable].  
v3 notNil ifTrue: [ct remove: v3 from: variable].  
v4 notNil ifTrue: [ct remove: v4 from: variable].  
v5 notNil ifTrue: [ct remove: v5 from: variable]]
```

basicTrigger

“pour la classe **BTMaxDemon**”

constraints do: [:ct | ct max: variable]

basicTrigger

“pour la classe **BTMinDemon**”

constraints do: [:ct | ct min: variable]

L’implémentation de la méthode **basicTrigger** dans la classe **BTRemoveDemon** est un peu complexe. En effet, les démons de type **remove**, ont cinq variables d’instance supplémentaires, nommées **v1**, **v2**, **v3**, **v4** et **v5** qui contiennent les cinq dernières valeurs supprimées du domaine de la variable. Si plus de cinq valeurs ont été supprimées, alors le démon de type **remove** est désactivé, et on active à sa place le démon de type **domain**.

Plus généralement, il y a une hiérarchisation des démons : le démon de type **value** est privilégié par rapport à tous les autres, puisqu’il est le plus informant. Techniquement, lorsque le démon de type **value** est activé, cela provoque la désactivation de tous les autres démons, ce qui évite de déclencher des opérations de filtrage redondantes.

4.2.3 LES DÉMONS ET LES VARIABLES

Selon sa classe une variable est associée à différents démons. Par exemple, dans la classe abstraite **BTVariable** trois démons sont définis : le démon de type **value**, celui de type **remove** et celui de type **domain**. Les classes de variables dont le domaine est ordonné (e.g. **BTIntegerVariable**, **BTOrderedObjectVariable**) définissent en plus un démon de type **min** et un de type **max**.

Chaque démon pointe sur la liste des contraintes, parmi celles qui impliquent la variable concernée, que l’on doit filtrer après la réduction de domaines associée au démon. Les démons sont inactifs par défaut, mais sont activés, selon leur classe, lorsqu’une réduction de domaine est subie par la variable. Plus précisément, si la variable est instanciée, le démon de type **value** sera activé. Si une valeur est supprimée, alors le démon de type **remove** sera activé. Cependant, si la valeur supprimée est une des bornes du domaine le démon correspondant à cette borne sera activé (i.e. **min** ou

bien **max**). De la même façon, si la suppression provoque l'instanciation de la variable, alors le démon de type **value** sera activé.

Ainsi, les démons des variables permettent de mémoriser les réductions de domaines ayant eu lieu lors d'une étape de la résolution. Il est capital de remarquer que cette mémorisation inclut le type de la réduction. Cette information sera utilisée pour déclencher la méthode de filtrage ad hoc au moment du filtrage des contraintes associées à la variable.

4.2.4 LES DÉMONS ET LES CONTRAINTES

Les contraintes ne connaissent pas directement les démons qui leurs sont associés car ce n'est pas nécessaire. En revanche, elles implémentent les méthodes de filtrage qui sont susceptibles d'être exécutées par les démons. Plus précisément, la méthode **value** : implémentée par une contrainte sera exécuté à la demande du démon de type **value** d'une de ses variables. De la même façon, l'évaluation de la méthode **min** : sera commandé par le démon de type **min**.

4.2.5 LES DÉMONS ET LES ALGORITHMES

La collaboration entre les démons et l'algorithme de résolution se situe à deux niveaux différents. D'une part, l'algorithme, par l'intermédiaire des variables, déclenche l'exécution des démons. Les démons qui sont activés déclenchent alors l'exécution de la méthode de filtrage leur correspondant dans chacune des contraintes qui lui sont associées.

D'autre part, il est parfois nécessaire, notamment après un retour arrière, d'initialiser les démons. En effet, lorsqu'un échec est rencontré suite au filtrage d'une contrainte, la propagation des démons n'a pas nécessairement été achevée. Il convient alors de désactiver tous les démons, afin de ne pas déclencher de propagation dont la cause a disparu après l'opération de retour arrière.

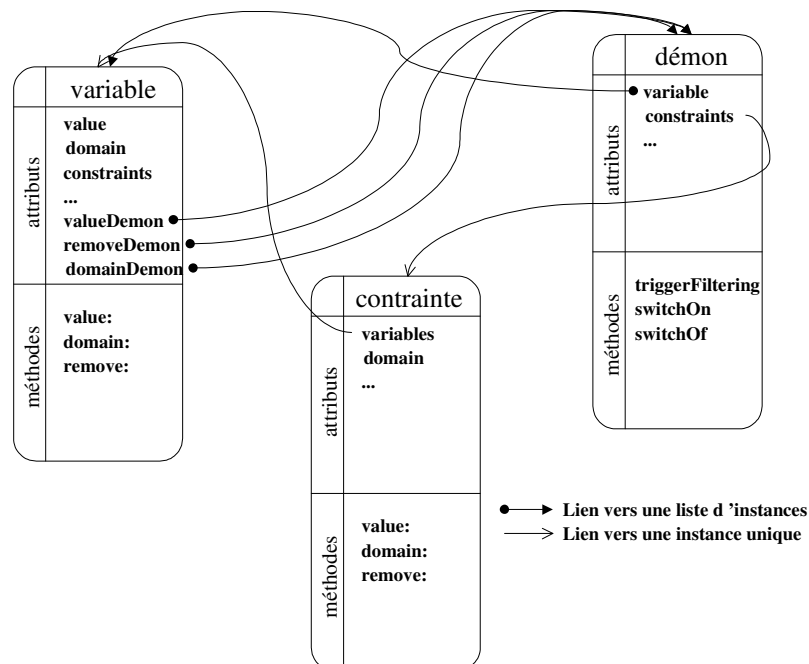


Figure 17 Les classes de variables, de contraintes et de démons, et leur relations, dans le système BACKTALK

4.3 DÉFINITION PARTIELLE DES MÉTHODES DE FILTRAGE

Le mécanisme de filtrage des contraintes que nous avons présenté dans cette partie permet de définir le filtrage des contraintes de façon modulaire par un ensemble de méthodes élémentaires. De plus, il permet de définir les méthodes de filtrage de façon partielle ou incomplète, tout en obtenant les mêmes résultats. Plus précisément, prenons un exemple concret simple : le filtrage des contraintes globales de différence dont nous avons donné un algorithme de filtrage (cf. Algorithme 7). Dans cet algorithme, on exécute une boucle jusqu'à ce que plus aucune réduction de domaine ne soit possible. On obtient la propriété suivante après exécution de la méthode :

$$\forall i, \forall j \text{ avec } i \neq j, \text{ si } D_i = \{x\} \text{ alors } x \notin D_j$$

Autrement dit, si une des variables est instanciées avec la valeur x , aucune autre variable de la contrainte ne possède la valeur x dans son domaine. Il est *a priori* nécessaire de procéder à une boucle dans l'écriture de la méthode de filtrage car la méthode de filtrage elle-même peut instancier des variables.

Cependant, grâce au mécanisme de propagation par les démons exposé dans cette partie, il n'est pas nécessaire de faire cette boucle. En effet, si l'on implémente simplement la méthode `SMALLTALK` suivante (à comparer avec l'Algorithme 7) :

```
value: aVariable
  x := aVariable value.
  variables
    do: [:v | v ~= aVariable
      ifTrue: [v remove: x]]
```

En effet, si la méthode provoque l'instanciation de nouvelles variables par effet de bord, lesquels seront mémorisés dans les démons correspondants, et seront donc pris en compte ultérieurement pour déclencher à nouveau l'évaluation de cette méthode. Par exemple, si l'on considère le jeu de variables suivant : X , Y et Z ayant pour domaine initial respectivement $\{1, 2\}$, $\{1, 2\}$ et $\{1, 2, 3\}$. Si la variable X est instanciée avec la valeur 1, l'exécution de la méthode ci-dessus provoquera l'instanciation de Y avec la valeur 2 et la réduction du domaine de Z à $\{2, 3\}$. Le démon de type **BTValueDemon** de Y déclenchera ultérieurement le filtrage de la même contrainte pour l'instanciation de Y avec la valeur 2, ce qui aura pour effet de réaliser l'instanciation Z avec la valeur 3, comme l'aurait fait l'application de l'Algorithme 7.

De façon similaire, le filtrage des contraintes représentant les équations linéaires à coefficients unitaires, qui consiste à assurer la cohérence des bornes du domaine, peut se définir plus simplement que nous ne l'avons fait avec l'Algorithme 8 que nous reproduisons ici :

```

continuer ← true
tant que (continuer)
  continuer ← false
  ∀i = 1, 2, ..., n
    m ← C - ∑j≠i max(Xj)
    M ← C - ∑j≠i min(Xj)
    D ← Di \ {x ∈ Di | x ≤ m ou x ≥ M}
  Si (D ≠ Di) alors continuer ← true

```

En effet, pour les raisons que nous venons d'évoquer, il est inutile de s'assurer au sein de la méthode de filtrage de la cohérence complète. On peut s'abstenir de faire une boucle jusqu'à obtention de la cohérence des bornes, et se contenter d'une seule passe. Le filtrage dû aux réductions de domaines provoquées par la méthode elle-même sera pris en charge par les démons correspondants. Ainsi, l'implémentation de ces méthodes de filtrage devient la suivante :

```

value: v
  self min: x; max: x

min: var
  | c |
  c := constant - variables sumOfMin.
  variables do: [:vi | vi min: c + vi max]

max: var
  | c |
  c := constant - variables sumOfMin.
  variables do: [:vi | vi max: c + vi min]

```

4.4 IMPLÉMENTATION DES MÉCANISMES DE PROPAGATION DE QUELQUES CONTRAINTES

- **La classe abstraite** `BTConstraint`

La classe *abstraite* **BTConstraint** est la racine de la hiérarchie des contraintes prédéfinies en BACKTALK. C'est une classe importante puisqu'elle fournit un certain nombre de méthodes clés pour la définition de nouvelles classes de contraintes. Parmi ces méthodes, certaines ne doivent pas être redéfinies, d'autres fournissent des mécanismes par défaut, qui peuvent être redéfinis si besoin est, et certaines méthodes, dites *abstraites*, doivent nécessairement être redéfinies dans les sous-classes.

1) La création

À la création d'une contrainte, trois méthodes sont nécessairement exécutées par le système : la méthode **prePost**, la méthode **post** et la méthode **postPost**. La première sert à initialiser les variables d'instances spécifiques à la contrainte. Dans la

classe **BTConstraint**, elle ne fait rien, mais elle peut être redéfinie. La deuxième, **post**, crée un démon correspondant à l'instanciation pour chaque variable qui est nécessaire à la correction de la résolution, cette méthode ne doit donc pas être redéfinie. La méthode **postPost** sert à la définition éventuelle de démons additionnels, par exemple correspondant à la suppression d'un élément, et permet aussi de définir un filtrage à exécuter juste après la création de la contrainte. Cette méthode est généralement redéfinie dans les sous-classes, même si cela n'est pas obligatoire. Dans la classe **BTConstraint**, la méthode **postPost** ne fait rien.

2) La formule de satisfaction

La formule de satisfaction d'une contrainte est définie dans une méthode de sélecteur **isSatisfied** qui ne prend aucun argument. Cette méthode renvoie vrai ou faux selon que l'instanciation courante de ses variables est satisfaisante ou non. Cette méthode doit être redéfinie dans toutes les sous-classes de **BTConstraint**, même si les méthodes de filtrage correspondant aux démons peuvent suffire à assurer la complétude de la résolution.

Pour le même argument de complétude du système, les sous-classes de **BTConstraint** doivent implémenter la méthode **isSatisfied**: qui prend en argument une collection d'objets, et qui renvoie vrai ou faux selon que la contrainte est satisfaite ou non par les valeurs contenues dans la liste. L'ordre des éléments de la liste correspondant par défaut à l'ordre des variables définies par la contrainte considérée.

3) Le filtrage

La classe **BTConstraint** n'implémente aucune méthode de filtrage, les méthodes de filtrage doivent donc nécessairement être implémentées dans les sous-classes. Ces méthodes sont les suivantes : 1) **values**: qui prend en argument une variable BACKTALK. Cette méthode doit être définie dans chaque sous-classe, puisqu'un démon correspondant à l'instanciation est défini pour chacune des variables par la méthode **post**, et que ce démon appellera nécessairement la méthode **value**; 2) **min**: et **max**: qui prennent en argument une variable BACKTALK et correspondent au démon associé aux modifications des bornes du domaine de la variable passée en argument. Ces méthodes ne doivent être définies que si au moins un démon leur correspondant est défini par la contrainte; 3) **remove:from**: qui prend en argument un objet quelconque et une variable BACKTALK et qui est appelée par le démon correspondant à la suppression d'une valeur; 4) **domain**: prend en argument une variable BACKTALK et qui est appelée par le démon correspondant à la suppression de plusieurs valeurs du domaine de la variable en argument.

• L'implémentation de quelques contraintes

Voici le détail de l'implémentation des méthodes de filtrage de quelques contraintes usuelles.

1) Une contrainte binaire simple : $X \geq Y$

Les méthodes définies dans la classe de la contrainte $X \geq Y$ sont les suivantes :

postDemos

```
x addMaxDemonOn: self.  
y addMinDemonOn: self
```

Les démons correspondant à l'instanciation d'une variable de la contrainte sont automatiquement posés par la super classe. En suivant le principe d'héritage, on ne définit dans la classe elle-même que les démons qui lui sont propres.

issatisfiedBy: values

“values est une collection contenant deux objets”

```
values first >= (values at: 2)
```

value: v

```
v == x  
    ifTrue: [self max: x]  
    ifFalse: [self min: y]
```

min: v

“v est nécessairement == à y”

```
x min: y min
```

max: v

“v est nécessairement == à x”

```
y max: x max
```

2) La contrainte globale de différence BTAllDiffCt

La contrainte globale de différence est filtrée en considérant simplement le cas où l'une des variables est instanciées. Dans ce cas on supprime sa valeur du domaine des autres variables. Ce filtrage ne réalise pas la cohérence d'arc mais il est très peu coûteux.

postDemos

^self

issatisfied

```
| vals |  
vals := OrderedCollection new: arity.  
variables do: [:v | v instantiated ifTrue: [vals add: v  
value]].  
^vals size = vals asSet size
```

issatisfiedBy: values

^values size = values asSet size

value: var

```
| val |  
val := var value.  
variables do: [:v | v == var ifFalse: [v remove: val]]
```

min: v

^self

max: v

^self

3) Les contraintes linéaires

Voici l'exemple de la contrainte linéaire la plus simple, i.e. la contrainte linéaire d'égalité dont tous les coefficients sont égaux à un. Les méthodes qui la caractérisent sont les suivantes :

postDemos

```
variables do: [:v | v addMinDemonOn: self; addMaxDemonOn:  
self]
```

La contrainte doit être filtrée chaque fois que l'une des bornes d'une variable est modifiée.

isSatisfiedBy: values

“values est une collection contenant n objets”

```
acc := 0.  
values do: [:v | acc := acc + v value].  
^acc == constant
```

value: v

```
self min: x; max: x
```

min: var

```
| c |  
c := constant - variables sumOfMin.  
variables do: [:vi | vi min: c + vi max]
```

max: var

```
| c |  
c := constant - variables sumOfMin.  
variables do: [:vi | vi max: c + vi min]
```

Chapitre 5

Techniques avancées

Cette partie est consacrée à la présentation de quelques techniques avancées mises en œuvre dans BACKTALK ou en cours de développement, comme l'introduction de mécanismes de raisonnement formel pour la résolution.

5.1 MÉTA HEURISTIQUES

Il est à noter qu'en BACKTALK les heuristiques sont modifiables dynamiquement, durant la résolution du problème, ce qui permet de définir des méta heuristiques, c'est-à-dire des stratégies de sélection des heuristiques de résolution.

Un exemple simple de méta heuristique consiste à appliquer une stratégie donnée pour le début de la résolution d'un problème (disons jusqu'à instanciation de la moitié des variables) et d'en changer pour la fin de la résolution. Un autre exemple consiste à utiliser une heuristique donnée pour la recherche de la première solution, et à en changer pour la fin de la résolution. Le Tableau 4 montre que l'heuristique qui consiste à choisir d'abord parmi les six premières variables du problème accélère la découverte de la première solution, alors que cette stratégie est défavorable à la suite de la résolution. À titre d'illustration, voici les temps de résolutions pour la multiplication magique en utilisant l'heuristique privilégiant les six premières variables jusqu'à la découverte de la première solution et en adoptant l'heuristique dite du plus petit domaine d'abord après avoir trouvé la première solution.

Multiplication magique	<i>1^{ère} solution</i>	<i>Unicité</i>
<i>Méta Heuristique</i>	100 ms	20.000 ms
	7 backtracks	1769 backtracks

Tableau 6 La résolution de la multiplication magique en utilisant l'heuristique qui choisit la prochaine variable à instancier parmi les six premières, jusqu'à obtention de la première solution, et ensuite choisit les variables suivant le seul critère de la taille du domaine. Ces données sont à comparer à celles du Tableau 4

De la même façon, lors de la résolution d'un problème d'optimisation on peut utiliser une heuristique permettant de trouver rapidement une première solution, sans aucune considération de la fonction économique, et de changer ensuite pour une heuristique favorisant la diminution du critère d'optimisation. Concrètement, on débutera la résolution en utilisant la stratégie du plus petit domaine d'abord jusqu'à l'obtention de la première solution, puis on utilisera ensuite une heuristique de type "regret minimal" pour favoriser la décroissance rapide de la fonction économique. La stratégie du regret minimal consiste à choisir en priorité la variable dont les deux premières valeurs sont le plus proche possible.

Un exemple plus complexe de méta-heuristique consiste à choisir les variables en fonction de *l'histoire* de la résolution. En effet, certains problèmes sont difficiles à résoudre parce qu'ils contiennent une zone de difficulté importante. Une stratégie efficace est de s'attacher en priorité aux variables constituant cette zone. Pour illustrer cette idée, considérons le problème suivant :

```
x := v arrayLabel: 'X' size: 5 from: 1 to: 5.  
y := v arrayLabel: 'Y' size: 6 from: 1 to: 5.  
BTAlldiffct on: x.  
BTAlldiffct on: y.
```

Les cinq premières variables, les $\mathbf{X}_i$, ne posent pas de difficulté particulière, en revanche, les $\mathbf{Y}_i$ rendent le problème insoluble, puisqu'elles constituent un sous problème équivalent à un problème de pigeon (trouver une injection d'un ensemble dans un ensemble plus petit). La résolution utilisant l'heuristique du plus petit domaine d'abord va commencer par instancier les variables $\mathbf{X}_i$ avant de s'attaquer aux $\mathbf{Y}_i$ et de trouver une impossibilité. Ensuite, les valeurs des $\mathbf{X}_i$ seront remises en question avant de considérer encore les $\mathbf{Y}_i$. En d'autres termes, la résolution va développer un arbre de recherche contenant $5!$ branches pour chaque instantiation possible des $\mathbf{X}_i$. Chacune de ces branches donne naissance à également $5!$ branches pour l'énumération des $\mathbf{Y}_i$. En tout, 14.400 branches seront développées pour prouver l'absence de solution.

Dans cette résolution, toutes les erreurs sont provoquées par les variables $\mathbf{Y}_i$, ce que nous allons utiliser pour définir un méta-heuristique permettant de choisir une stratégie d'instanciation des variables en tenant compte des échecs observés auparavant en cours de résolution. L'idée est simple, à chaque échec rencontré lors de la résolution, on mémorise dans une table la variable qui en est responsable, c'est-à-dire celle dont le domaine vient d'être réduit à l'ensemble vide. Ensuite, de temps en temps, on va changer l'heuristique de choix de la prochaine variable à instancier en optant pour une heuristique composée, dont les premiers éléments sont les variables qui ont provoqué le plus d'échecs. Techniquement, voici le code qui correspond à la création de cette heuristique composée :

“T est la table contenant le nombre d’échecs dû à chaque variables, et maxT est la valeur maximum de la table.

On range les variables dans deux listes : firstVars qui contient celles qui ont provoqué un nombre d’échec égal à au moins R% de maxT, et lastVars qui contient les autres”

```
firstVars := vars select: [:v | (T at: v) > (maxT * (R/100))].  
lastVars := vars reject: [:v | firstVars includes: v].
```

“On crée l’heuristique composée privilégiant les variables responsables du plus grand nombre d’échec”

```
solver heuristicForVariables: (BTCompositeHeuristic  
  on: (Array  
    with: firstVars  
    with: lastVars  
  filteringBlocks: (Array  
    with: [:v | v instantiated not] “filtrage de la liste 1”  
    with: [:v | v instantiated not]) “filtrage de la liste 2”  
  criterionBlocks: (Array  
    with: [:x :y | x size < y size] “critère de la 1ère liste”  
    with: [:x :y | x size < y size]) “critère de la 2nde liste”
```

“Réinitialisation de la table T”

```
T keysDo: [:var | T at: var put: 0].
```

Deux paramètres sont nécessaires à la mise en œuvre de cette méta-heuristique, d’une part le pourcentage **R**, qui sert à discriminer les variables des deux listes, et d’autre part la fréquence de mise à jour de l’heuristique. Voici quelques résultats sur le problème précédent avec R=50 et différentes fréquences de mise à jour :

Pas de méta-heuristique (plus petit domaine)	⇒	21.241s ; 14399
bt Mise à jour tous les 1.000 backtracks	⇒	5.506s ; 3551
bt		
Mise à jour tous les 100 backtracks	⇒	2.992s ; 1791
bt		
Mise à jour tous les 50 backtracks	⇒	3.744s ; 2377
bt		
Mise à jour tous les 5 backtracks	⇒	18.958s ; 10665
bt		

Nous donnerons une illustration de cette méta-heuristique sur un problème concret, l’harmonisation musicale automatique.

5.2 CONTRÔLE ET TRAÇAGE DE LA RÉOLUTION

Le mécanisme des démons peut être utilisé à des fins de contrôle de la résolution, par exemple en permettant de visualiser dynamiquement certains éléments du problème. Pour ce faire, on peut créer des contraintes, ne faisant pas à proprement parler partie du problème, mais permettant, grâce aux démons qui lui sont associés, de déclencher des actions quelconques en fonction des événements de résolution qui se produisent.

C'est ce principe que l'on utilise pour réaliser un affichage dynamique de la résolution dans une fenêtre de texte. La Figure 18 donne un aperçu de la visualisation d'un problème de carrés magiques à trois étapes successives de sa résolution.



Figure 18 Affichage dynamique des valeurs des variables à trois étapes successives de la résolution du problème du carré magique d'ordre 5. Les points d'interrogation représentent les variables n'ayant pas de valeur, c'est-à-dire dont le domaine n'a pas été réduit à un singleton

Pour mettre à jour cette vue on utilise des contraintes additionnelles, ne faisant pas partie du problème, instances de la classe **BTDynamicValueDisplayCt**, pour laquelle seul le démon correspondant à l'instanciation est activé. Ainsi, dès que la variable concernée reçoit une valeur, la contrainte est informée et déclenche la méthode associée au démon correspondant à l'instanciation. Cette méthode déclenche une mise à jour de la fenêtre d'affichage.

On peut de la même façon déclencher des opérations d'affichage correspondant à d'autres démons, par exemple la suppression d'une valeur ou bien encore la modification d'une des bornes du domaine. De façon plus générale, ce type de con-

traintes permet de visualiser ou de réaliser un suivi (traçage) de la résolution d'un problème sans avoir recours à aucune modification du système lui-même. Voici par exemple deux traces textuelles de la construction d'un carré magique d'ordre 4. La première ne mentionne que les instanciations de variables, alors que la seconde mentionne toutes les modifications subies par le domaine de la variable **e**, c'est-à-dire la première case de la deuxième ligne du carré.

Cette trace mentionne toutes les instanciations de variables durant la construction d'un carré magique d'ordre 4. Le décalage par rapport à la marge de gauche représente la profondeur atteinte dans l'arbre de recherche.

```
a := 1
> b := 2
> > c := 15
> > d := 16
> > > e := 8
> > > m := 11
> > > j := 4
> > > e := 9
> > > > f := 13
> > > f := 14
> > > j := 5
> > > p := 7
> > > e := 10
> > > > m := 9
> > > > i := 14
> > > m := 11
> > > j := 4
> > > e := 11
> > > > f := 12
> > > > f := 13
> > > > > i := 12
> > > > i := 14
> > > > m := 8
> > > f := 14
> > > > g := 3
> > > > h := 6
> > > > j := 5
> > > g := 4
> > > m := 9
> > > e := 12
> > > > f := 10
> > > > f := 11
> > > > k := 13
> > > > f := 13
> > > > > i := 11
> > > > i := 14
> > > > m := 7
> > > > n := 11
> > > > o := 6
> > > > h := 5
> > > f := 14
> > > > i := 11
> > > > m := 10
> > > > j := 5
> > > i := 13
> > > m := 8
> > > n := 11
> > > j := 7
```

```
> > > g := 3
> > > o := 6
> > > h := 5
> > > k := 10
> > > l := 4
> > > p := 9
```

Cette trace mentionne toutes les réductions de domaines subies par la variable e durant la résolution du problème. >= correspond à une augmentation de la borne inférieure du domaine, <= à une diminution de la borne supérieure, <> à une suppression de valeur et := à une instanciation. À la fin de chaque ligne, le domaine de la variable est affiché. Le décalage par rapport à la marge de gauche représente la profondeur atteinte dans l'arbre de recherche.

```
e >= 2 => [(2..16)]
> e >= 4 => [(4..16)]
> > e <> 15 => [(4..14) 16]
> > e >= 8 => [(8..14)]
> > e <= 14 => [(8..14)]
> > > e := 8
> > e >= 9 => [(9..14)]
> > > e := 9
> > e >= 10 => [(10..14)]
> > > e := 10
> > e >= 11 => [(11..14)]
> > > e := 11
> > e >= 12 => [(12..14)]
> > > e := 12
```

La figure suivante montre une vue de l'arbre de recherche correspondant à la résolution d'un carré magique d'ordre 4 :

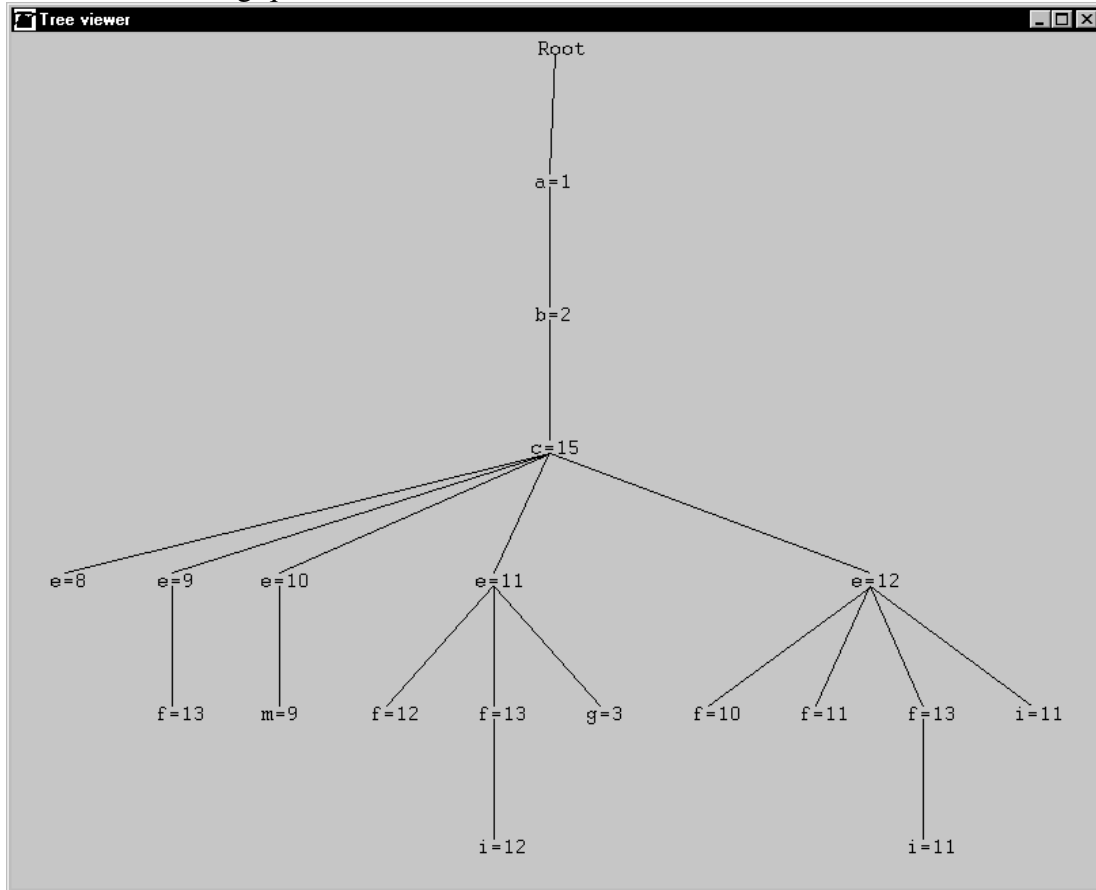


Figure 19 L'arbre de recherche correspondant à la résolution d'un carré magique d'ordre 4

5.3 CORRECTION ET COMPLÉTUDE

De façon générale, pour un système de résolution de problèmes, la recherche de l'efficacité n'est pas l'unique objectif poursuivi. En effet, deux notions sont tout aussi importantes : la *complétude* et la *correction* du système. Un système de résolution possède la propriété de complétude s'il trouve nécessairement une solution lorsqu'il en existe. Autrement dit, un système complet *n'oublie* pas de solution. La correction est la propriété duale de la complétude. Un système possède la propriété de correction si les solutions qu'il calcule sont effectivement des solutions. Intuitivement, un système est correct s'il *n'invente* pas de solution.

Dans la conception de BACKTALK nous avons essayé, autant que possible, de garantir à l'utilisateur la correction et la complétude de la résolution. Bien évidemment, il n'est pas possible de garantir ces deux propriétés de façon systématique et théorique. Simplement, nous avons doté BACKTALK de mécanismes de contrôle qui favorise la correction. De plus, la philosophie d'utilisation de BACKTALK permet de faciliter l'obtention de la complétude.

A) COMPLÉTUDE DE LA RÉOLUTION

Pour garantir la complétude de la résolution, il suffit de respecter la définition d'un filtrage que nous avons donnée à la 5.2.1b), cf. Définition 1, c'est-à-dire que le filtrage d'une contrainte est une réduction de domaines qui n'excède pas la cohérence d'arc. Dans ce cas, les valeurs supprimées du domaine d'une variable étaient toutes en contradiction avec au moins une contrainte, ainsi aucune solution n'est rendue inaccessible. On peut bien évidemment remarquer que cette condition est suffisante mais pas nécessaire. En résumé :

Pour assurer la complétude de la résolution, il est suffisant que chacune des méthodes de filtrage associées à chaque démon ne supprime aucune valeur arc cohérente avec la contrainte

S'assurer qu'une méthode de filtrage ne supprime aucune valeur cohérente est impossible en pratique. Le seul moyen est de comparer la réduction de domaine produite par la méthode concernée avec celle obtenue en réalisant la cohérence d'arc avec l'Algorithme 6 qu'on peut considérer comme prouvé. Cependant, cette approche rend inutile l'utilisation de méthodes de filtrage spécifique, puisqu'on annihile ainsi le gain en performance. On peut tout au plus utiliser cette approche, sur des problèmes simples, à des fins de débogage.

En pratique, la validité des méthodes de filtrage n'est pas vérifiable, et c'est à celui qui les implémente de le faire le plus précautionneusement possible, sous peine d'engendrer des incomplétudes. On peut remarquer que la définition modulaire des méthodes de filtrage permet de simplifier leur écriture et donc de réduire le risque d'erreur.

B) CORRECTION DE LA RÉOLUTION

Pour assurer la correction, il faut au contraire garantir un minimum de réductions de domaines, afin de ne pas sélectionner *in fine* des valeurs incohérentes. Ainsi, le besoin de correction conduit à fixer une espèce de filtrage minimal. Une condition suffisante triviale est la suivante :

Pour assurer la correction de la résolution, il est suffisant que chacune des méthodes de filtrage associées à chaque démon rende la contrainte arc cohérente

Nous ne souhaitons pas employer une telle méthode, puisque nous avons justement défini la notion de filtrage pour éviter de réaliser systématiquement la cohérence d'arc des contraintes. Une condition suffisante plus raisonnable en pratique est la suivante :

Pour assurer la complétude de la résolution, il est suffisant que la contrainte soit satisfaite dès que toutes ses variables sont instanciées

On peut aisément réaliser cette vérification de façon plus ou moins précoce dans le cours de la résolution. On peut par exemple vérifier *a posteriori* que l'instanciation courante des variables est satisfaisante, et si ce n'est pas le cas déclencher une erreur. On peut aussi, lors de l'activation d'un démon de type **value**, et s'il ne reste qu'une seule variable non instanciée, vérifier *a priori* que toutes les valeurs de son domaine sont cohérentes avec les autres variables (qui elles sont déjà instanciées).

La première méthode présente l'avantage de n'engendrer qu'un surcoût négligeable car elle n'implique qu'une seule évaluation de la contrainte, en revanche son "pouvoir prédictif" est faible. Inversement, la seconde approche est plus efficace en termes de prédictions, mais implique une évaluation de la contrainte pour l'ensemble des valeurs d'un domaine, ce qui peut s'avérer coûteux.

Nous avons implémenté une approche intermédiaire, qui consiste à employer la vérification *a priori* lorsque le domaine est petit, alors que l'on réalise systématiquement la vérification *a posteriori*. L'implémentation de cette vérification est très simple en BACKTALK et est simplement fondée sur le mécanisme d'héritage de SMALLTALK. Pour cela, nous avons modifié la méthode **basicTrigger** de la classe **BTValueDemon**, afin qu'avant d'envoyer le message **value:** à la contrainte, elle procède à la vérification voulue :

"L'ANCIENNE IMPLÉMENTATION"

basicTrigger

"pour la classe BTValueDemon"

constraints do: [:ct | ct value: variable]

"EST REMPLACÉE PAR..."

basicTrigger

"pour la classe BTValueDemon. S'il reste moins d'une variable non instanciée pour une contrainte, alors on vérifie qu'on ne laisse aucune valeur incohérente"

constraints do:

[:ct |

n := ct numberOfUnknownVariables.

n = 0 ifTrue:

[^ct isSatisfied ifFalse:

[self domainwipedOut]].

n = 1 ifTrue: [| var |

var := ct unknownVariable.

var size <= ct thresholdForCorrectionChecking

ifTrue: [ct arcConsistencyForVariable: var]]

ct value: variable]

“AVEC...”

arcConsistencyForVariable: var

“réalise la cohérence d’arc pour la variable var,
sachant que cette méthode ne s’exécute que quand var
est la seule variable non instanciée de self.

crtValues est la liste des valeurs
index est l’indice de var dans la liste des variables

on teste la valeur de la contrainte pour chaque valeur
du domaine de var et on supprime celles qui ne sont pas
cohérentes”

```
| crtValues index |  
crtValues:= variables collect: [:eachVar | eachVar value].  
index := variables indexOf: var.  
var domainValuesReverseDo: [:eachValue |  
    “on met crtValues à jour avec la une nouvelle valeur  
    du domaine de var (eachValue)”  
  
    crtValues at: index put: eachValue.  
    (self issatisfied: crtValues)  
    iffFalse: [var remove: eachValue]]
```

Un tel mécanisme dégrade les performances du système mais améliore sensiblement la correction du système dans le sens où il permet d’affirmer que les solutions produites sont véritablement des solutions au problème posé, cette vérification étant menée en cours de résolution, et non *a posteriori*. De plus, le coût de cette vérification est le plus souvent négligeable. Il peut cependant devenir sensible si l’évaluation de la méthode **issatisfied**: est difficile. Pour cela, la taille maximale du domaine de la variable autorisé pour procéder à la vérification est spécifique à chaque type de contraintes. Plus précisément, la méthode **thresholdForCorrectionCheck** est implémentée dans la classe **BTConstraint** où elle renvoie la valeur 10, ce qui constitue une taille par défaut. Elle peut être redéfinie dans les sous-classes, avec une valeur plus grande (resp. petite) si la méthode **issatisfied**: est peu coûteuse (resp. très coûteuse). Par exemple, cette valeur est fixée à 0 dans la classe **BAllDiffCt** parce que la méthode **issatisfied**: correspondante est coûteuse et que le filtrage de cette contrainte standard peut-être considéré comme fiable.

thresholdForCorrectionCheck

“Retourne 10 dans la classe BTConstraint”

^10

thresholdForCorrectionCheck

“Retourne 0 dans la classe BAllDiffCt (pas de vérification)”

^0

5.4 LES PROBLÈMES D'OPTIMISATION

BACKTALK permet naturellement de définir et de résoudre des problèmes d'optimisation. Un problème d'optimisation se définit de la même manière qu'un problème de décision. Il faut simplement définir le critère d'optimisation comme étant une variable contrainte. Par exemple, si l'on souhaite résoudre le problème suivant :

maximiser $Y \times X$
avec
 $X \in \{1, \dots, 1\,000\}$
 $Y \in \{1, \dots, 1\,000\}$
 $X + Y = 1\,000$

Il suffit de le définir comme un problème de décision. Voici la déclaration en BACKTALK de ce problème :

```
p := self new.  
X := v from: 1 to: 1000.  
Y := v from: 1 to: 1000.  
x + y @= 1000.  
p variablesToInstantiate: (Array with: X with: Y)
```

Il reste à définir la fonction économique comme une variable contrainte, ici appelée **produit** :

```
...  
produit := X * Y.  
...
```

La résolution se fait en utilisant le message **minimize:** au lieu de **firstSolution**. Ce message prend un argument, qui est la variable représentant le critère à minimiser. Dans notre cas, puisque l'on souhaite maximiser le critère, il suffit de minimiser son opposé. Le message utilisé pour déclencher l'appel de la méthode d'optimisation sera donc :

```
P minimize: produit opposite
```

La technique utilisée pour la résolution de problèmes d'optimisation se fait selon les étapes suivantes :

- 1) Le système calcule une première solution, sans considération sur le critère d'optimisation **C**.
- 2) La valeur **V** du critère d'optimisation est évaluée pour la dernière solution trouvée.
- 3) On ajoute la contrainte **C < V** au problème. Il faut ici remarquer que cette contrainte ne doit pas être supprimée en cas de retour arrière, elle est définitive. Ainsi elle est ajoutée en utilisant une syntaxe particulière afin qu'elle ne soit pas manipulée par la gestion dynamique des contraintes.
- 4) On retourne à l'étape 1)

Cette méthode assure que l'on calcule la solution optimale du problème. En revanche, elle n'est pas forcément très efficace. Nous n'avons pas apporté beaucoup

d'attention à ce problème. Signalons que la résolution de quelques problèmes d'optimisation, comme le problème du pont, se fait avec ce mécanisme et donne des temps de résolution assez satisfaisants.

5.5 RAISONNEMENT FORMEL

Un problème d'arithmétique simple peut être très difficile à résoudre par des techniques de satisfaction de contraintes. Voici un tel exemple en BACKTALK :

```
x := v from: -A to: A.  
y := v from: -A to: A.  
(x * x) - y @= 0.  
y <= 0.
```

Le code ci-dessus définit une famille de problèmes qui peuvent être résolus en temps constant. En effet, qu'elle que soit la valeur de **A**, il n'y a qu'une solution : **x = y = 0**.

Cette propriété peut se montrer facilement par le raisonnement formel suivant : La première contrainte implique **y = x²**, et donc **y ≥ 0**. La seconde contrainte impliquant **y ≤ 0**, il s'en suit que **y = 0**, et donc **x = 0**. Voici cependant les temps d'exécution de BACKTALK pour trois valeurs de **A** :

```
Cas ou A = 10      => 0.027s ; 47 bt  
Cas ou A = 100     => 0.426s ; 491 bt  
Cas ou A = 1000    => 29.114s ; 4927 bt
```

On remarque que le nombre d'échecs est directement proportionnel à la valeur de **A**. Ceci est dû au fait que la contrainte **((x * x) - y @= 0)** n'est pas équivalente à **(x² = y)**.

Voici un autre exemple de problème simple qui ne se résout pas bien avec des techniques de satisfaction de contraintes :

```
x + y + z @= 0.  
x + y + t @= 0.  
z < t.
```

Ce problème est sans solution puisque les deux premières contraintes impliquent la relation d'égalité entre **z** et **t** alors que la troisième contrainte implique leur différence.

Ces deux exemples montrent l'inaptitude des techniques de contraintes à procéder à un raisonnement formel sur les contraintes.

Nous avons introduit dans BACKTALK un certain nombre de règles de réécriture simples qui permettent de résoudre le problème souligné par le premier exemple. Ces règles de réécritures sont implémentées dans les classes d'expressions. Ainsi, à la déclaration des contraintes, certaines expressions sont simplifiées. Par exemple, dans notre premier exemple, la contrainte **((x * x) - y @= 0)** sera auto-

matiquement réécrite ($\mathbf{x}^2 = \mathbf{y}$), ce qui implique immédiatement la suppression des valeurs négatives du domaine de $\mathbf{y}$. La résolution de ce problème se fait donc sans échec. De la même manière, $(\mathbf{x} - \mathbf{x})$ se réécrit en 0.

Le problème souligné dans le second exemple est plus complexe à traiter en général. Nous avons mis en œuvre, dans les classes d'expressions, un mécanisme de composition linéaire qui permet en combinant deux expressions d'en déduire de nouvelles. Cependant, il est difficile de mettre au point une stratégie efficace de composition des contraintes arithmétiques. Il existe plusieurs compositions possibles pour chaque paire de contraintes. De plus, si l'on conserve toutes les nouvelles relations issues de ces compositions le nombre de contraintes devient très important.

Anne Liret [Liret, et al., 1997a] mène actuellement des expérimentations dans ce sens avec le système BACKTALK.

5.6 GESTION DES SYMÉTRIES

En dépit de sa remarquable efficacité, il existe de nombreux problèmes que la satisfaction de contraintes ne permet pas de résoudre, ce qui est inhérent au caractère NP-complet des problèmes traités. Cependant, certaines classes de problèmes *faciles* posent des difficultés importantes. Nous avons déjà vu un tel exemple au paragraphe 5.5, dû à l'absence de raisonnement formel dans les systèmes de contrainte. On peut exhiber une autre carence des techniques de satisfaction de contraintes à l'aide du problème générique des pigeons dont l'énoncé est le suivant : on souhaite accueillir n pigeons dans un pigeonnier disposant de $n-1$ trous, sachant qu'un trou ne peut pas héberger plus d'un pigeon. À l'évidence, ce problème n'admet aucune solution quelle que soit la valeur de n , puisqu'il est équivalent à trouver une fonction injective d'un ensemble dans un ensemble plus petit. Cependant, les techniques "usuelles" de satisfaction de contraintes ne sont pas adaptées à la résolution de ce problème. Donnons deux formulations et trois méthodes de résolution différentes pour ce petit problème :

Formulation n°1 : Chaque pigeon est représenté par une variable p_i dont le domaine est $\{1, \dots, n-1\}$. Nous représentons le fait qu'un trou ne puisse héberger qu'un pigeon à la fois par une série de contraintes binaires de différence :

$$\forall i, \forall j \text{ tels que } i \neq j, (C_{ij}) \Leftrightarrow (p_i \neq p_j)$$

Formulation n°2 : Les variables sont les mêmes que dans la première formulation, mais on pose une seule contrainte globale de différence.

La résolution du problème déclaré dans la première formulation conduit à l'exploration d'un espace de recherche dont la taille est la factorielle de $(n-1)$. En effet, une incohérence n'est détectée que lorsqu'il ne reste que deux variables à instancier, ce que l'on peut voir sur la Figure 20. En ce qui concerne la seconde formulation, il nous faut distinguer deux éventualités selon le type de filtrage adopté pour la contrainte globale de différence. En effet, cette contrainte n'étant pas cohérente, si le filtrage adopté consiste à vérifier la cohérence d'arc, alors l'insolubilité du problème est détectée immédiatement, du moins si l'on adopte un algorithme efficace pour le filtrage de cette contrainte [Régis, 1994]. En revanche en utilisant le filtrage que nous avons exposé plus tôt, alors la résolution est équivalente à celle de la première formulation. En conclusion, en l'absence de filtrage spécialisé pour la contrainte globale de différence, ce problème fort simple, est extrêmement difficile à résoudre (il y a environ factorielle $(n-1)$ solutions à énumérer).

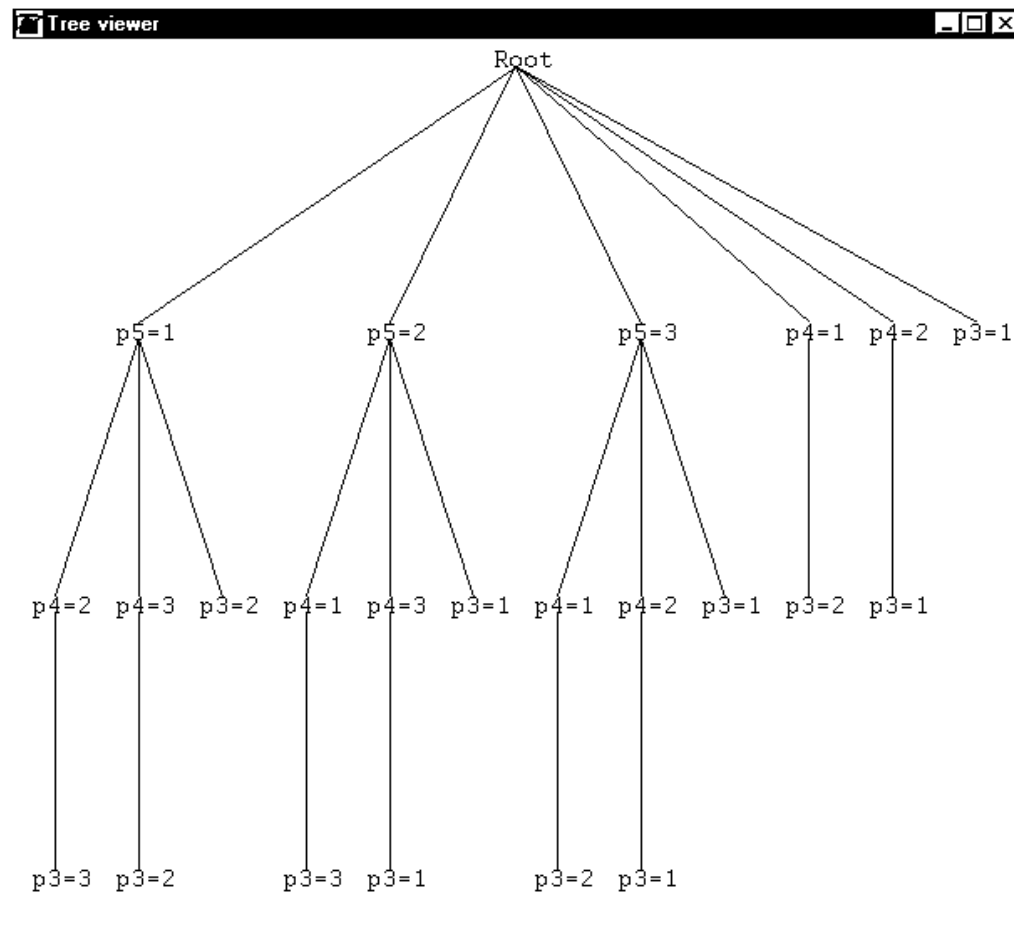


Figure 20 L'arbre de recherche développé pour la résolution du problème des 5 pigeons dans sa formulation avec des contraintes binaires. Il y a exactement 23 branches, soit $4! - 1$

1

Ce problème est étonnamment difficile à résoudre en raison de sa nature symétrique. Plus précisément, dans ce problème, chaque trou du pigeonier est identique à tous les autres. De la même façon chaque pigeon est identique aux autres pigeons. La première remarque signifie que, dans le problème de contraintes correspondant, les valeurs des variables sont interchangeableables, c'est-à-dire qu'il n'importe aucunement en soi qu'un pigeon soit dans le trou n°1 ou dans le n°2. La seconde remarque se traduit par l'interchangeabilité des variables contraintes, c'est-à-dire qu'il est indifférent que le premier trou soit occupé par le premier ou par le deuxième pigeon. La symétrie entre les trous du pigeonier est une symétrie entre les différentes valeurs des variables du problème. Ce type de symétrie a déjà été étudié afin d'améliorer la résolution de certains problèmes de satisfaction de contraintes [Freuder and Sabin, 1995]. La symétrie entre les pigeons est, dans notre représentation du problème, une symétrie entre les variables contraintes du problème qui a été étudiée également dans le but de l'amélioration des techniques de résolution [Crawford, et al., 1996, Puget, 1993].

Dans ce paragraphe nous allons présenter une nouvelle méthode pour la détection des symétries entre les variables. Cette méthode est fondée sur l'étude de la structure des contraintes définies en compréhension plutôt que sur leur définition en extension et elle permet une détection très économique de certaines symétries. Nous montrerons également comment il est possible, par une simple modification de

l'algorithme de résolution, exploiter cette information sur les symétries entre variables pour accélérer la recherche de solutions.

5.6.1 ÉTAT DE L'ART

Dans ce paragraphe nous faisons un résumé des principaux résultats présentés par Jean-François Puget [Puget, 1993] concernant les symétries entre variables. Dans ce travail, la structure des contraintes du problème n'est pas considérée. Jean-François Puget introduit les notions suivantes. Pour plus de simplicité dans les définitions, nous considérons ici que les contraintes sont définies en extension.

On notera $V_1, \dots, V_n$ les variables, et $D_1, \dots, D_n$ leur domaine et D le produit cartésien des domaines. Les contraintes sont notées $C_1, \dots, C_m$ et leur extension est notée $R(C_i)$.

Une *instanciation* est un n -uplet $inst$ de valeurs avec

$$inst \in D = D_1 \times \dots \times D_n.$$

Une *solution* est une instanciation sol qui satisfait toutes les contraintes :

$$\forall i=1, \dots, m ; sol \in R(C_i)$$

On notera dorénavant S l'ensemble des solutions du problème. Le groupe Π des permutations de l'ensemble $\{1, \dots, n\}$ opère naturellement sur l'ensemble des instanciations. Cette opération à gauche est définie par :

$$\forall (\sigma, s) \in \Pi \times D ; \sigma.s = (s_{\sigma(1)}, s_{\sigma(2)}, \dots, s_{\sigma(n)})$$

Une *permutation cohérente* est une permutation qui transforme toute solution en une solution et l'ensemble des permutations cohérentes est un sous-groupe Γ de Π . Ce sous-groupe Γ laisse stable l'ensemble des solutions du problème, et il définit une relation d'équivalence³ $\equiv$ sur cet ensemble. Quand le groupe Γ n'est pas trivial (i.e. réduit à la seule identité), le problème est dit symétrique. Dans ce cas, il est intéressant de calculer l'ensemble quotient $S/\equiv$ au lieu de S en entier, afin de ne pas calculer de solutions équivalentes à une permutation cohérente près.

Le résultat central de l'article de Jean-François Puget est que pour chaque problème P de satisfaction de contraintes, il existe un problème non symétrique P' , qui est obtenu en ajoutant des contraintes à P , et dont l'ensemble de solutions est exactement $S/\equiv$. Cependant, dans le cas général, il est probablement aussi difficile de construire ce problème non symétrique P' que de résoudre P .

5.6.2 RAISONNER SUR LA STRUCTURE DES CONTRAINTES

La méthode que nous présentons est fondée sur deux idées simples. La première idée est que la symétrie entre les variables du problème peut être détectée à partir de l'étude de chaque contrainte séparément. La seconde est que les propriétés de symétrie d'une contrainte peuvent être déduites directement de sa définition en compréhension, sans passer par l'étude, forcément combinatoire, de son extension.

Par exemple, dans la contrainte de différence globale, les variables sont deux à deux interchangeables, et c'est une propriété intrinsèque de la contrainte, qui ne dépend pas d'une instance particulière, ni même du nombre de variables impliquées

³ Quand un groupe G opère sur un ensemble E , la relation R définie par $\forall (x, y) \in E \times E ; x R y$ si, et seulement si, $\exists g \in G$ tel que $g.x = y$

dans la contrainte ou de leur domaine de valeur. Nous appellerons cette propriété la *permutabilité en compréhension*. Pour une contrainte donnée, les variables permutable en compréhension forment une partition de l'ensemble des variables du problème. Nous appellerons PC-classes de la contrainte les éléments de cette partition. Voici quelques exemples de contraintes usuelles, avec leurs PC-classes. (On peut remarquer que pour chaque contrainte, les variables qui ne sont pas explicitement impliquées dans la contrainte forment une PC-classe additionnelle.)

- La contrainte de différence globale est symétrique. Toutes les variables de la contrainte sont dans une unique PC-classe, les variables non impliquées formant une PC-classe supplémentaire.
- Les contraintes linéaires ne sont en général pas symétriques. Par exemple, la contrainte suivante : $\sum \alpha_i . x_i$ possède une PC-classe pour chaque valeur différente des coefficients α_i . Dans le cas de la contrainte $2.x - 3.y + 2.z = 4$, il y a trois PC-classes : $\{x, y\}$; $\{z\}$ et l'ensemble des variables du problème qui ne sont pas impliquées dans la contrainte.
- Les contraintes d'ordre, comme $x > y$ possèdent trois PC-classes : $\{x\}$; $\{y\}$ et le reste.
- La contrainte de cardinalité, définie par la formule suivante : $|\{i \in \{1, \dots, n\} \mid V_i = V\}| = c$ possède quatre PC-classes : $\{V_1, \dots, V_n\}$; $\{V\}$; $\{c\}$ et les variables restantes.

Les PC-classes des différentes contraintes du problème sont ensuite combinées pour donner les classes de variables permutable en compréhension pour le problème global. Cette combinaison consiste simplement à faire l'intersection des différentes PC-classes. Cette détection, qui peut être réalisée une seule fois avant de commencer la résolution proprement dite peut être ensuite utilisée durant la résolution, à la volée, afin d'éviter l'exploration de régions similaires de l'espace de recherche. Nous allons maintenant introduire quelques définitions. Dans la suite P désigne le problème à résoudre.

Une contrainte peut être considérée comme un sous problème susceptible d'être symétrique. Nous appellerons *sous problème engendré par une contrainte C* le problème de contraintes $P(C)$ contenant les mêmes variables et les mêmes domaines que P , mais contenant la seule contrainte C .

Nous dirons que deux variables contraintes u et v de P sont *fortement permutable* si pour toute contrainte C de P , la transposition $\tau_{u,v}$, qui consiste à échanger les valeurs des deux variables u et v , est cohérente pour le problème $P(C)$. Dans ce cas on dira également de $\tau_{u,v}$ qu'elle est une transposition *fortement permutable*.

Le groupe Σ , engendré par les transpositions fortement permutable est un sous-groupe de Γ . Par définition, vérifier la forte permutabilité de deux variables contraintes u et v implique d'étudier l'extension de chaque contrainte impliquant ces deux variables. Ce processus peut être très coûteux en pratique. Afin de l'éviter, nous proposons d'utiliser la notion de PC-classe, ce qui nous conduit à la notion de *permutabilité en compréhension*, définie ci-dessous, que nous comparons à la notion de forte permutabilité.

Deux variables contraintes u et v sont dites *permutable en compréhension* si elles sont dans la même PC-classe pour chaque contrainte du problème. Par extension, si deux variables contraintes u et v sont permutable en com-

préhension, la transposition $\tau_{u,v}$ sera également dite permutable en compréhension.

Cette relation de permutabilité en compréhension peut être calculée *a priori* pour chaque variable en calculant simplement l'intersection de toutes les PC-classes auxquelles elle appartient.

Une remarque importante est que le groupe Ψ engendré par toutes les transpositions permutable en compréhension *n'opère pas* sur l'ensemble S des solutions de P . En effet, si l'on considère le problème défini par deux variables u et v de domaine respectif $\{1, 2\}$ et $\{1, 2, 3\}$ et une unique contrainte $u \neq v$. Bien que la contrainte de différence soit symétrique, c'est-à-dire que u et v sont dans une même PC-classe, la transposition $\tau_{u,v}$ transforme la solution $(1, 3)$ en $(3, 1)$ qui n'est pas une solution, puisque 3 n'est pas un élément du domaine de u . Ceci revient à dire que $\tau_{u,v}$ n'est pas cohérente au sens de Jean-François Puget. Ainsi, Ψ n'opère pas sur S .

Cette remarque montre qu'il est nécessaire de considérer les domaines des variables pour définir le groupe Ψ . La solution la plus simple consiste à considérer ajouter une contrainte au problème, dont les PC-classes sont les ensembles de variables ayant le même domaine. Par exemple, pour un problème ayant trois variables u, v et w dont les domaines sont respectivement $\{0, 1\}$, $\{0, 1\}$ et $\{1, 2, 3\}$, les PC-classes de cette contrainte sont $\{u, v\}$ et $\{w\}$. Grâce à cette contrainte supplémentaire, le groupe Ψ opère sur l'ensemble S , sur lequel il induit une relation d'équivalence $\approx$. Puisque calculer $S/\equiv$ est jugé trop difficile, nous allons nous contenter de calculer $S/\approx$.

Avant de montrer comment on peut travailler dans $S/\approx$ en pratique, nous allons faire deux remarques qui permettent d'évaluer en théorie le gain apporté par notre approche. Nous avons défini jusqu'ici quatre groupes de permutations de l'ensemble des variables. Nous avons les relations suivantes d'inclusion entre groupes : $\Psi \subset \Sigma \subset \Gamma \subset \Pi$. Nous allons montrer que ces relations d'inclusions sont, pour certains problèmes, des inclusions strictes.

1) $\Psi \neq \Sigma$

Cette formule d'inclusion stricte signifie qu'il existe des problèmes pour lesquels le groupe Σ des permutations cohérentes, au sens de Jean-François Puget, n'est pas réduit à l'élément neutre alors que le groupe des permutations fortement cohérentes l'est. En d'autres termes, il existe des problèmes symétriques au sens de Jean-François Puget pour lesquels aucune paire de variables n'est fortement permutable. Un exemple est le problème de Ramsey. Ce problème consiste à colorer les arêtes d'un graphe complet en utilisant au plus deux couleurs et de façon à ce qu'aucun triangle ne soit monochrome. Voici par exemple le cas de K_4 , le graphe complet à quatre sommets. En utilisant les notations de la Figure 21, on voit clairement que la permutation $(X \rightarrow Y \rightarrow Z \rightarrow X ; U \rightarrow V \rightarrow T \rightarrow U)$ est cohérente. En revanche, aucune transposition de deux arêtes ne préserve toutes les solutions.

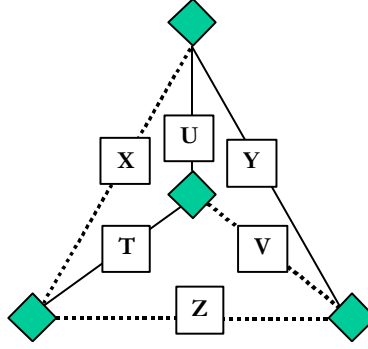


Figure 21 Le problème de Ramsey pour le graphe complet à quatre sommets (K_4)

2) $\Sigma \neq \Gamma$

Cette inclusion stricte signifie qu'il y a des exemples de problèmes dans lesquels deux variables sont fortement permutable, tout en n'étant pas permutable en compréhension. Par exemple, considérons le problème ayant deux variables u et v de domaine $\{0, 1\}$, et une contrainte de cardinalité exprimant la relation suivante : la valeur de u est le nombre de variables appartenant à l'ensemble $\{v\}$ ayant pour valeur 1. Cette relation de cardinalité conduit, dans cas "dégénéré" à la propriété suivante :

$$[(u = 0) \Rightarrow (v = 0)] \text{ et } [(u = 1) \Rightarrow (v = 1)]$$

Cette contrainte de cardinalité est équivalente à la contrainte d'égalité $u = v$, laquelle est symétrique, alors que la contrainte de cardinalité, comme nous l'avons déjà souligné, n'est pas symétrique. Ces deux variables sont fortement permutable sans l'être en compréhension. Bien évidemment, de telles relations sont rares à la définition du problème, mais elles peuvent surgir en cours de résolution lors de la réduction de certains domaines. Considérer de tels cas nécessiterait d'opérer une reformulation dynamique des contraintes du problème, au fur et à mesure de la résolution, ce que nous ne traitons pas ici.

Ainsi, la relation de permutabilité en compréhension, que nous savons détecter facilement en faisant l'intersection des PC-classes du problème, représente un affaiblissement sensible par rapport à la relation de cohérence définie par Jean-François Puget. Cependant ce compromis est intéressant dans la mesure où il peut être mis en œuvre à un coût négligeable tout en permettant d'améliorer de façon spectaculaire la résolution de certains problèmes fortement symétriques.

5.6.3 EXPLOITATION DES RELATIONS DE PERMUTABILITÉ

Nous avons montré comment détecter les relations de permutabilité en compréhension. Ces relations peuvent être utilisées facilement pour éviter de calculer des solutions symétriques par rapport aux variables permutable en compréhension, c'est-à-dire pour travailler dans $S/\approx$ au lieu de S . Il y a deux approches différentes pour exploiter ces informations. On peut, d'une part, utiliser des contraintes pour casser la symétrie comme le font Jean-François Puget [Puget, 1993] ou James Crawford [Craw-

ford, et al., 1996]. On peut aussi, et c'est ce que nous allons montrer, modifier légèrement l'algorithme de résolution pour tirer parti de ces relations.

A) L'AJOUT DE CONTRAINTES POUR CASSER LA SYMÉTRIE

L'ajout de contraintes pour casser la symétrie a été utilisée avec succès par Crawford dans le contexte de la logique propositionnelle. De la même façon, Jean-François Puget propose tout simplement d'utiliser des contraintes d'ordre entre les variables permutable. Par exemple, dans le problème du pigeonnier, il suffit d'ajouter les contraintes $p_i \geq p_{i+1} + 1$ pour que les symétries entre variables disparaissent. Cependant, cette approche présente deux inconvénients majeurs dans le contexte qui nous intéresse.

D'une part, l'ajout de telles contraintes modifie la structure du problème, ce qui peut conduire à une dégradation sensible des performances de l'algorithme de résolution. Nous pouvons donner un exemple de cette perturbation causée par l'adjonction de contraintes d'ordre. Dans le problème des carrés magiques, afin de réduire le nombre de solutions symétriques, il est courant d'ajouter des contraintes d'ordre entre les coins du carré. Ces contraintes permettent d'éviter de calculer des solutions similaires, mais en revanche, si l'on utilise une heuristique de type plus petit domaine d'abord, elles peuvent conduire à rendre plus difficile l'obtention de la première solution. Par exemple, si l'on ajoute les contraintes suivantes : $A \leq B$; $A \leq C$; $A \leq D$; $C \leq B$, où A, B, C et D représentent les quatre coins du carré, dans l'ordre supérieur gauche, supérieur droit, inférieur gauche et inférieur droit, alors la première solution est trouvée en plus de 2.000 backtracks au lieu de 791. D'autre part, cette approche implique que l'on peut comparer deux à deux les éléments des domaines des variables. Bien évidemment, ceci peut être fait de façon générale dans le contexte des problèmes de satisfaction de contraintes à domaines finis, dans la mesure où tout ensemble fini peut être muni d'un ordre total. Cependant, en pratique, les objets manipulés ne sont pas ordonnés, c'est par exemple le cas dans l'application d'harmonisation automatique que nous détaillerons ultérieurement dans ce document. Munir *a priori* ces objets d'un ordre implique de les modifier, par exemple afin d'ajouter une méthode de comparaison dans la classe considérée, ce qui ne peut pas nécessairement être fait. Ainsi, cette méthode ne fonctionne que pour les problèmes numériques.

B) MODIFICATION DE LA BOUCLE D'ÉNUMÉRATION

L'approche que nous allons présenter, en revanche, est applicable à tous les types de problèmes de satisfaction de contraintes, quelle que soit la nature des éléments des domaines des variables. En outre, cette approche ne modifie pas la stratégie de résolution. En clair, elle garantit simplement que certaines branches de l'arbre de recherche ne seront pas explorées, si celles-ci sont l'image par une symétrie de Ψ d'une région déjà explorée. En revanche, elle ne provoque aucune modification de l'ordre dans lequel les branches de l'arbre de recherche sont explorées par le système.

Cette modification consiste simplement à tenir compte, lors d'un retour arrière, des relations de permutabilité afin de ne pas explorer des régions correspondant à une même valeur pour deux variables permutable. Cette approche, tout comme celle qui consiste à ajouter des contraintes d'ordre, est compatible avec tous les algorithmes de résolution fondés sur l'énumération et le maintient de cohérence (e.g. forward-checking, full look-ahead).

Techniquement, après chaque retour arrière la valeur de la variable courante, c'est-à-dire la dernière variable instanciée avant le déclenchement du retour arrière, est supprimée du domaine de chaque variable qui est permutable en compréhension

avec la variable courante. Ainsi, on ajoute l'instruction suivante à la méthode **backward** de la classe de l'algorithme :

```

backward
  (...)
  variables do: [:v |
    ((v isPermutableWith: currentVar)
    and: [v instantiated not])
    ifTrue: [v remove: currentVal]]

```

Figure 22 La modification de la méthode **backward** pour l'exploitation des relations de permutabilité entre variables

Il faut souligner que cette réduction de domaine ne doit s'effectuer que pour les variables n'ayant pas été instanciée par le système. Plus généralement, elle ne doit pas se faire pour les variables ayant eu leur domaine réduit par le système. En effet, sinon, les relations de permutabilité sont détruites car les domaines des variables ne sont plus identiques.

Voici une illustration sur le problème des quatre pigeons dans un pigeonnier à trois emplacements. La Figure 23 illustre la différence entre l'arbre de recherche développé avec la méthode de recherche standard et avec le traitement des relations de permutabilité.

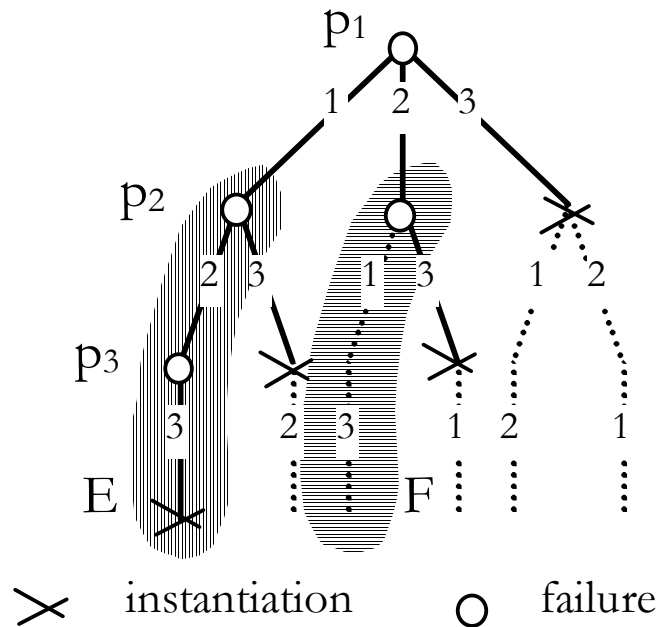


Figure 23 La résolution du problème des quatre pigeons. Les branches en pointillés sont économisées lors d'une résolution utilisant la méthode **backward** modifiée afin de prendre en compte les relations de permutabilité entre variables. Les zones hachurées E

et F sont images l'une de l'autre par la transposition $\mathcal{T}_{12}$ qui correspond à deux variables permutable. Ainsi, après avoir exploré E qui n'a conduit à aucune solution, on peut économiser l'exploration de F

Il est aisé de voir que cette réduction de domaine supplémentaire permet de ne calculer qu'une seule solution par élément de $S/\approx$, et de ne pas en omettre.

5.6.4 IMPLÉMENTATION DANS BACKTALK

L'implémentation de la méthode exposée dans le système BACKTALK se fait en trois étapes. Premièrement, il s'agit d'implémenter, dans les classes des variables, la méthode qui construit les PC-classes. Deuxièmement, il faut, avant le lancement de la résolution, calculer la PC-classe de chaque variables en faisant l'intersection des PC-classes des différentes contraintes. Finalement, il faut modifier la méthode **backward** de l'algorithme d'énumération (ce qui a été montré ci-dessus).

5.6.5 RÉSULTATS EXPÉRIMENTAUX

Sur le problème des pigeons, cette méthode permet d'avoir une complexité de l'ordre de 2^n au lieu de $O(n!)$. Ainsi, quand la résolution standard développe 40.000 nœuds, l'utilisation de la méthode permet de ne développer que 2.000 nœuds.

Sur le problème linéaire suivant, si les domaines des variables sont $\{1, 2, \dots, 20\}$, la résolution standard nécessite 1.228 backtracks, alors que la détection des symétries permet de constater que les variables x, y et z sont permutable en compréhension, ce qui ramène le nombre d'échecs à 199. Les temps de résolution évoluent dans la même proportion.

$$\begin{aligned}x + y + z + t &= 20 \\x + y + z + u &= 20 \\x + y + z + v &= 20\end{aligned}$$

Considérons un exemple de découpe en deux dimensions. Il s'agit de disposer des planches rectangulaires de diverses tailles dans un espace rectangulaire. On considère que toutes les dimensions sont des nombres entiers. Ainsi, le problème peut être défini comme en termes de satisfaction de contraintes. On considère une définition du problème dans laquelle la position de chaque planche dans le rectangle est une variable contrainte. Il y a alors une unique contrainte de non-recouvrement qui implique toutes les variables. Cette contrainte est évidemment symétrique. Ainsi, la méthode présentée ici détecte la permutabilité des variables ayant le même domaine, c'est-à-dire les mêmes dimensions.

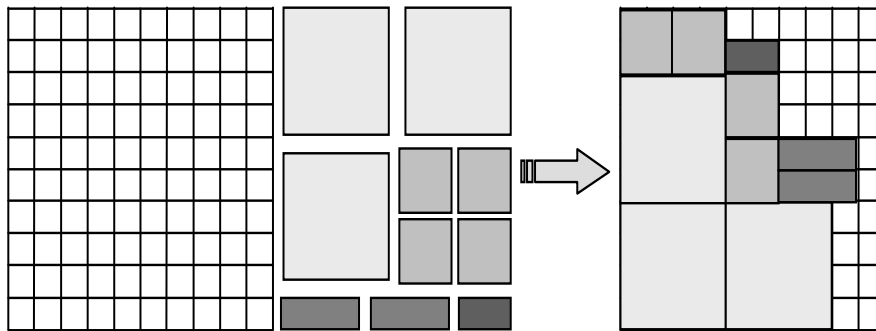


Figure 24 Une instance du problème de découpe de rectangles. Les planches ayant les mêmes dimensions sont permutable en compréhension

L'exemple Data4 du User's Manual de ILOGSOLVER est représenté Figure 24. Dans cet instance, on doit disposer trois planches de taille 4×4, quatre de taille 2×2, deux de taille 3×1 et une de taille 2×1. L'espace global est un carré de largeur 10. Les planches qui figurent avec la même teinte sur la figure sont permutable.

5.6.6 GÉNÉRALISATION À DES GROUPES DE VARIABLES

Cette méthode est très restrictive dans la mesure où elle ne détecte que les paires de variables qui jouent le même rôle dans chacune des contraintes du problème. C'est une approche efficace pour des problèmes dans lesquels il y a peu de contraintes et si ces contraintes sont globales. Cette sorte de symétrie est brisée dès que l'on ajoute des contraintes locales. Par exemple, le problème de découpe que nous venons d'exposer se rencontre le plus souvent sous sa forme de problème d'optimisation : on doit disposer les planches de façon à utiliser le moins de hauteur possible du rectangle global. Pour définir ce problème à partir du précédent, il suffit d'ajouter pour chaque planche une variable numérique qui représente sa hauteur. Résoudre le problème d'optimisation revient alors à minimiser la borne supérieure de toutes ces hauteurs. La structure du problème devient alors celle-ci :

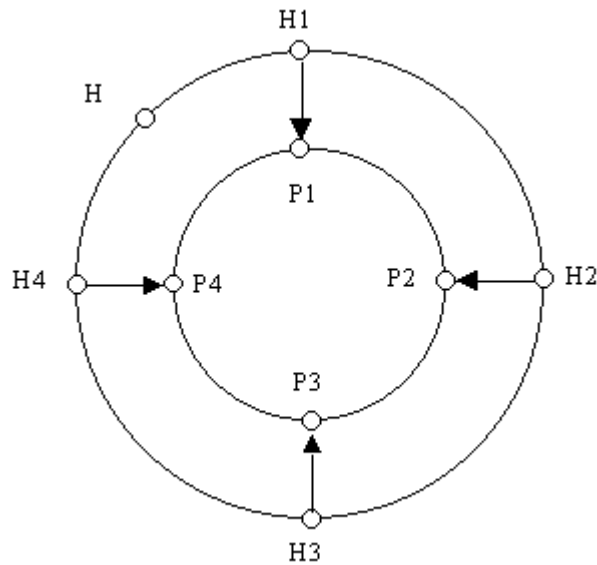


Figure 25 La structure du problème de découpe optimale. Les variables P_i représentent les positions des planches. Les variables H_i représentent les hauteurs atteintes par les rectangles ($H_i = P_i + \text{Hauteur}(i)$). La variable H désigne la hauteur maximale atteinte ($H = \max(H_i)$). Les cercles (sans flèche) représentent les contraintes symétriques, alors que les flèches représentent les contraintes non symétriques

Les contraintes supplémentaires, qui expriment les relations entre la position d'une planche, sa taille et la hauteur qu'elle atteint, ne sont pas symétriques. Ainsi, elles brisent la symétrie générale du problème. Néanmoins, les planches ayant les mêmes dimensions semblent être encore permutable. En fait, cette intuition vient de ce que les variables représentant les planches ne sont pas permutable, mais les structures composites qui représentent chaque planche et qui sont constituées par une variable position et une variable hauteur, sont permutable. Cependant, ces structures ne sont pas représentées explicitement par des variables contraintes, mais par des groupes de variables. Nous allons montrer comment étendre notre méthode de détection des relations de permutable aux structures composites. Nous montrerons également comment on peut exploiter les relations de permutable entre structures composites pour améliorer la résolution.

A) PERMUTABILITÉ DES STRUCTURES COMPOSITES

Nous définissons une structure composite comme étant un sous problème particulier. Plus précisément, une structure composite C est définie par un ensemble

$V(C)$ de variables et un ensemble $C(C)$ de contraintes impliquant des variables dans $V(C)$. Étant donnée une structure composite C , les contraintes de $C(C)$ seront dites *internes*, alors que les autres contraintes seront dites *externes*.

À partir d'un problème P de satisfaction de contraintes et une liste de structures composites de P , nous pouvons construire un problème P' , appelé problème *condensé*, dont les variables contraintes correspondent à la fois aux variables et aux structures composites de P . le problème P' est construit de façon à ce que les variables permutable de P' correspondent aux structures et variables permutable de P . Voici la définition de P' :

- Les variables de P' sont les structures composites de P et les variables restantes de P (i.e. les variables de P qui n'appartiennent à aucune structure composite) ;
- Les contraintes de P' sont les contraintes de P qui ne sont internes à aucune structure composite de P . Pour des raisons d'homogénéité, on considérera que ces contraintes sont reformulées pour porter sur ces nouvelles variables.

Deux structures composites C_1 et C_2 de P' sont dites *isomorphes* si les deux bijections suivantes existent :

$$\Phi : V(C_1) \longrightarrow V(C_2) \text{ et } \Psi : C(C_1) \longrightarrow C(C_2)$$

telles que pour tout $c \in C(C_1)$ on a les propriétés suivantes :

- (i) $\forall c \in C(C_1) ; \Phi(V(c)) = \Phi(V(\Psi(c)))$,
- (ii) $\forall c \in C(C_1) ; \Psi(c) \Leftrightarrow c$; (i.e. c and $\Psi(c)$ are similar)
- (iii) $\forall v \in V(C_1) ; D(v) = D(\Phi(v))$.

Deux structures composites sont dites permutable en compréhension si 1) elles sont isomorphes et 2) les variables qui leur sont associées sont permutable en compréhension dans P' . Ainsi, la détection des structures composites permutable revient à la détection des variables permutable dans le problème condensé.

B) MODIFICATION DE LA MÉTHODE DE RÉOLUTION

Si l'on résout directement le problème condensé, la modification de la boucle de résolution est la même que celle que nous avons présentée plus haut. Cependant, en général, on résout le problème tel qu'il est posé. La boucle de résolution peut être modifiée de façon à prendre en compte les relations de permutable entre les structures composites. La modification consiste simplement à ajouter les lignes suivantes à la méthode **backward** :

```

backward
(...)
variables do: [:v |
    ((v isPermutableWith: currentVar)
    and: [v instantiated not])
    ifTrue: [v remove: currentVal]].
c := currentVar compositeStructure.
c isNil
    ifFalse:
        [compositeStructures do: [:cs | | imageVar |
            ((cs isPermutableWith: c)
            and: [cs isNotPartiallyInstantiated])
            ifTrue:
                [imageVar := cs imageOfVariable:
currentVar.
                    imageVar remove: currentValue]]]

```

Figure 26 La modification de la boucle de résolution pour prendre en compte les relations de permutabilité en compréhension entre structures composites

La réduction de domaine consiste ici à supprimer la valeur **currentVal** de la dernière variable instanciée (**currentVar** dans le code) du domaine de toute variable image de **currentVar** dans une structure composite permutable avec la structure composite de **currentVar**. Cette réduction de domaine n'est effectuée que si la structure composite en question n'a pas encore été instanciée partiellement par le système. Cette précaution est nécessaire pour la même raison que celle que nous avons invoquée pour le cas des variables permutable.

5.6.7 ÉVALUATION DE LA MÉTHODE

Nous avons évalué cette méthode sur quatre instances du problème de découpe optimale en deux dimensions qui figure dans le User's Manual de ILOGSOLVER [ILOGSOLVER, 1996]. Ces instances sont numérotées Data 4, Data 5, Data 6 et Data 9. Le Tableau 7 donne les différents temps de résolution obtenus avec et sans la méthode pour trouver la solution optimale et prouver l'optimalité.

Instance [ILOG-SOLVER, 1996]	CPU sans la méthode	CPU avec la méthode
Data #4	0,004 sec.	0,003 sec.
Data #5	17,1 sec.	0,8 sec.
Data #6	3,2 sec.	0,23 sec.
Data #9	0,2 sec.	0,11 sec.

Tableau 7 Le temps de résolution du problème de découpe optimale avec et sans l'exploitation des symétries

5.7 DÉFINITION ET RÉOLUTION DE QUELQUES PROBLÈMES EN BACKTALK

Dans toutes les définitions, le caractère **V** est un raccourci pour la classe **BTVariable**.

5.7.1 LES CRYPTOGRAMMES

Voici une définition possible du cryptogramme SEND + MORE = MONEY en BACKTALK. Cette formulation ne construit pas de variables additionnelles pour les retenues, elle comporte en fait deux contraintes globales : une contrainte linéaire qui exprime la correction de l'addition, et une contrainte de différence globale.

sendmory

```
| letters s e n d m o r y send more money pbm |
pbm := self new: 'send + more = money'.
(letters := BTFixedHeadCollection new: 8)      "The variables"
    add: (s := V label: 's' from: 1 to: 9);
    add: (e := V label: 'e' from: 0 to: 9);
    add: (n := V label: 'n' from: 0 to: 9);
    add: (d := V label: 'd' from: 0 to: 9);
    add: (m := V label: 'm' from: 1 to: 9);
    add: (o := V label: 'o' from: 0 to: 9);
    add: (r := V label: 'r' from: 0 to: 9);
    add: (y := V label: 'y' from: 0 to: 9).

    "trois formules intermédiaires (cf. expressions)"
    send := (1000*s + (100*e) + (10*n) + d).
    more := (1000*m + (100*o) + (10*r) + e).
    money := (10000*m + (1000*o) + (100*n) + (10*e) + y).

    (send + more - money) @= 0. "l'addition est correcte"

    BTAllDiffCt on: letters.    "les lettres sont 2 à 2
différentes"

    Pbm "pour l'affichage"
        print: s; print: e; print: n; print: d;
        print: '+'; print: m; print: o; print: r; print: e;
        print: '='; print: m; print: o; print: n; print: e; print:
y.

    pbm variablesToInstantiate: letters.
    ^pbm
```

L'intégration de BACKTALK à SMALLTALK permet d'utiliser SMALLTALK comme un métalangage de définition des contraintes et des problèmes. Il est ainsi possible de définir des problèmes génériques sous forme de classes ou de méthodes SMALLTALK. Par exemple, nous avons implémenté une classe qui permet de définir et résoudre des cryptogrammes à l'aide d'une simple chaîne de caractères. Cette classe,

nommée **BTCryptogram** définit un problème de cryptogramme, avec les retenues, à l'aide de l'addition cryptée. Voici quelques exemples d'exécutions :

```
(BTCryptogram fromString: 'send+more=money')
printFirstSolutionWithInformation
(send+more=money) 0.002s ; 1 bt
9567+1085=10652

(BTCryptogram fromString: 'donald+gerald=robert')
printFirstSolutionWithInformation
(donald gerald robert) 0.051s ; 71 bt ; 73 ch
526485 197485 723970

(BTCryptogram fromString: 'ain+aisne+drome+marne=somme')
printAllSolutions
SOL 1: (ain+aisne+drome+marne=somme) 0.006s ; 9 bt ; 14 ch
190+19705+26835+31605=78335
SOL 2: (ain+aisne+drome+marne=somme) 0.038s ; 83 bt ; 86 ch
204+20648+15938+32548=69338
SOL 3: (ain+aisne+drome+marne=somme) 0.98s ; 178 bt ; 180 ch
328+32986+50716+13086=97116
```

5.7.2 LE PROBLÈME DU PONT : “BRIDGE PROBLEM”

Le *bridge problem* est un benchmark pour les applications d'ordonnancement disjonctif. Le but est de trouver un ordonnancement des tâches nécessaires à la construction d'un pont composé de 5 segments distincts qui minimise le temps total requis. Le projet comporte 44 tâches soumises à diverses contraintes qui se partagent en trois classes : 1) les contraintes de *précédence*, qui indiquent qu'une tâche doit être achevée avant qu'une autre ne commence, par exemple, les travaux de maçonnerie (M1—M6 sur le dessin) doivent être finis avant que l'on ne commence le tablier (tâches T1—T5) les contraintes de *ressources* : plusieurs tâches peuvent nécessiter une ressource commune, par exemple les tâches B1—B6 requièrent un mixeur. Puisqu'il n'y a qu'un mixeur, alors ces tâches ne peuvent pas se dérouler en même temps ; 3) les contraintes de *distances* qui contraignent la durée séparant la fin d'une tâche et le début d'une autre. Par exemple, les fondations (B1—B6) doivent être coulées au plus quatre jours après la fin de la construction des coffrages des piliers (S1—S6).

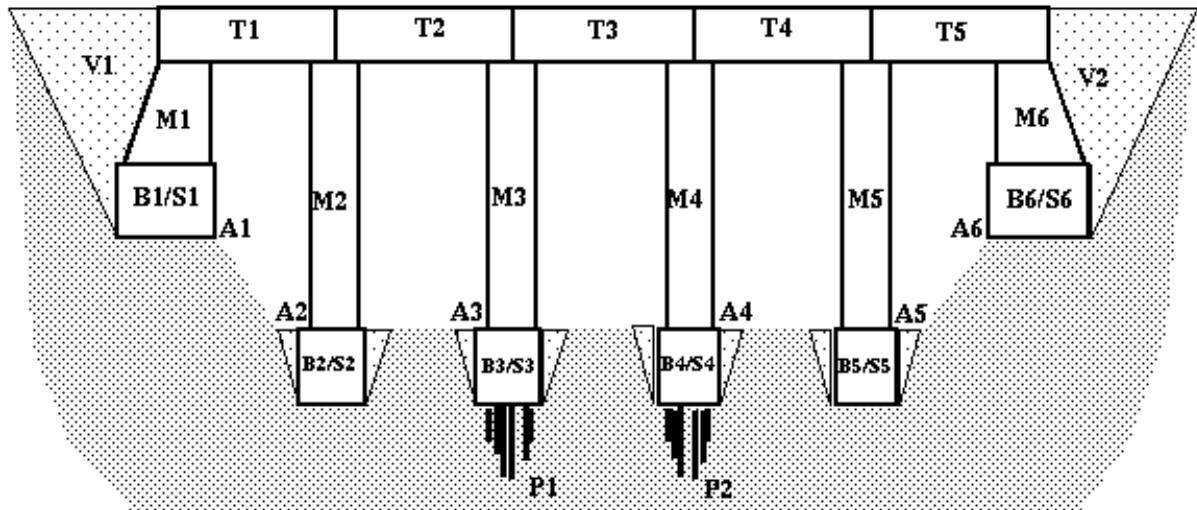


Figure 27 Le pont à construire

Ce problème est rendu difficile pour les techniques de résolution de contraintes à cause de la présence des ressources. En effet, les ressources imposent des contraintes de non-recouvrement entre tâches qui entraînent la création de nombreuses disjonctions :

$$5! (\text{tâches T1—T5}) \times 6! (\text{tâches M1—M6}) \times \dots = 10^{17}$$

ce qui signifie qu'il y a environ 10^{17} possibilités pour l'ordre des tâches.

En pratique, si l'on définit ce problème de façon directe, c'est-à-dire définissant deux variables entières pour chaque tâche (sa date de début et sa date de fin), et en posant les contraintes entre ces variables, la résolution par énumération et maintien de cohérence n'est pas réalisable en un temps raisonnable.

Cependant, le problème présente une propriété intéressante, que l'on va utiliser pour permettre une résolution efficace. En effet, en dehors des contraintes de disjonction (non-recouvrement), il n'y a que des contraintes de précédence et de distances, pour lesquelles la cohérence d'arc est complète. Plus précisément, si l'on rend arc cohérent un problème ne comprenant que ce type de contraintes, si aucun domaine n'est vidé, chaque valeur apparaît dans au moins une solution.

Cette remarque permet d'inventer une stratégie plus efficace pour résoudre le problème. Cette stratégie consiste à choisir uniquement l'ordre de déroulement des tâches qui partagent une ressource commune, et à réaliser le filtrage par cohérence d'arc sur tout le problème. Ainsi, on énumère non pas sur les variables correspondant aux dates de début et de fin de tâche, mais sur les ordres relatifs entre les tâches partageant une ressource.

L'implémentation de cette stratégie se fait de façon élégante en utilisant la possibilité que donne BACKTALK de pouvoir manipuler des variables dont les domaines contiennent des objets SMALLTALK quelconques. En l'occurrence, il suffit de définir pour chaque ressource, une série de variables additionnelles, dont le domaine contient l'ensemble des tâches utilisant la ressource. Plus précisément, pour la ressource "mixeur", on crée cinq variables Mixeur₁, Mixeur₂, ..., Mixeur₅ dont le domaine contient les cinq tâches S1, S2, ..., S6. La valeur de la variable Mixeur₁ donne alors la tâche qui sera exécutée en i^{ème} position. La résolution du problème consiste simplement à énumérer sur ces variables, et à propager les contraintes de précédence et de distance après chaque choix. Cette approche peut être comparée à celle adoptée

dans le système **clp(FD)**. La stratégie mise en œuvre pour la résolution de ce problème consiste à définir une série de disjonctions pour chaque ressource et à propager uniquement les contraintes de précédence et de distance. Les disjonctions sont prises en compte directement par PROLOG. Cette méthode est plus simple que la nôtre, mais elle repose sur les capacités de PROLOG à gérer les disjonctions, ce qui n'est pas possible en SMALLTALK, et plus généralement en programmation procédurale. Cependant, notre approche apporte une plus grande richesse expressive, dans la mesure où l'utilisation explicite de variables contraintes pour représenter les disjonctions permet d'en contrôler finement la gestion. Il est par exemple possible d'utiliser des heuristiques, de type plus petit domaine d'abord, pour le choix d'un ordre sur les variables.

Cette approche est suffisamment efficace pour la résolution du bridge problem, mais pour des problèmes d'ordonnancement plus complexes il est nécessaire de disposer de mécanismes de filtrage plus efficaces comme les techniques de "cutting edge" [Baptiste and Pape, 1995] ou "d'intervalles de tâches" [Caseau and Laburthe, 1994]. Nous pouvons naturellement utiliser BACKTALK pour implémenter simplement de telles techniques. Voici par exemple l'implémentation des intervalles de tâches en BACKTALK.

Les intervalles de tâche servent à déduire rapidement la position d'une tâche par rapport à un ensemble de plusieurs tâches partageant la même ressource. Par exemple, considérons les quatre tâches suivantes. (On donne le domaine de la date de début de la tâche, qui est un intervalle, suivi de la durée de la tâche.)

- ['T1' int[(36..87)] duration=12]
- ['T2' int[(76..99)] duration=12]
- ['T3' int[(80..99)] duration=12]
- ['T4' int[(80..99)] duration=12]
- ['T5' int[(64..87)] duration=12]

Un intervalle de tâche est défini par deux tâches A et B et par l'ensemble des tâches qui commencent nécessairement après le début le plus anticipé possible de A, et qui finissent nécessairement avant la fin la plus tardive possible pour B. dans notre exemple, les tâches T5 et T2, ont pour intervalle de tâche l'ensemble {T5, T3, T4, T2} (car T3 et T4 commencent nécessairement après la date 64, et finissent nécessairement avant 99+12).

L'intérêt des intervalles de tâches est que la durée cumulée des tâches permet de faire des déductions sur l'ordre des tâches n'appartenant pas à l'intervalle par rapport à celles qui lui appartiennent. Sur notre exemple, la tâche T1 ne peut pas commencer après la fin de l'ensemble des tâches T2, T3, T4 et T5. En effet, la durée cumulée des tâches de l'intervalle est $4 \times 12 = 48$ et la première tâche de l'intervalle commence au plus tôt à la date 64. Ainsi, la dernière tâche de l'intervalle finira au plus tôt à la date $64 + 48 = 112$, or T1 commence au plus tard à la date 99. Ainsi, T1 est nécessairement achevée avant le début de toute tâche appartenant à l'intervalle.

L'implémentation de ce mécanisme en BACKTALK peut se faire simplement en définissant une nouvelle classe de contraintes, appelée **BTTemporalDisjunctionCt**, que l'on instancie pour chaque ressource du problème. Une contrainte, instance de cette classe, porte sur l'ensemble des tâches qui nécessitent la ressource, et son filtrage consiste à calculer les intervalles de tâches intéressants, et à en déduire de nouvelles contraintes de précédence. Ces nouvelles contraintes sont créées dynamiquement, et seront donc supprimées en cas de retour arrière, ce qui permet de remettre en question l'ordre des tâches partageant une même ressource. Voici l'implémentation des principales méthodes de cette contrainte :

value: aTask (idem **min: aTask** et **max: aTask**)

“la méthode de filtrage déclenchée par le démon d’instanciation”

```
self handleTaskIntervalsForTask: aTask
```

handleTaskIntervalsForTask: i

“cette méthode construit la liste des intervalles de tâche intéressants”

```
| taskIntervals |
taskIntervals := BTCollection new.
1 to: arity do:
    [:j |
    | Tij Tji |
    i == j
        ifFalse:
            [Tij := self taskIntervalBetween: i and: j.
             Tji := self taskIntervalBetween: j and: i.
             Tij == nil ifFalse: [taskIntervals add: Tij].
             Tji == nil ifFalse: [taskIntervals add: Tji]]].
```

“pour chaque intervalle de tâche construit, on exécute une méthode pour déduire la position des tâches extérieures à l’intervalle”

```
taskIntervals do: [:ti | self deduceFromTaskInterval: ti]
```

deduceFromTaskInterval: ti

“méthode qui essaye de déduire si les tâches extérieures à ti sont

avant ou après toutes les tâches de ti”

```
| a b |
a := ti earlierTask.
b := ti laterTask.
1 to: arity do:
    [:c | (ti includesTask: c)
        ifFalse:
            [b latestEndTime - c earliestEndTime - ti
             cumulatedDuration < 0
                 ifTrue: [c cannotBeBefore: ti].
             c latestStartTime - a earliestStartTime - ti
             totalDuration < 0
                 ifTrue: [c cannotBeAfter: ti]]]
```

cannotBeBefore: aTaskInterval

“pose autant de contraintes de précédences qu’il y a de tâches dans aTaskInterval”

```
aTaskInterval tasks do: [:t | self isAfter: t]
```

cannotBeAfter: aTaskInterval

“pose autant de contraintes de précédences qu’il y a de tâches dans aTaskInterval”

aTaskInterval tasks do: [:t | self isBefore: t]

5.8 PERFORMANCES

Dans le cahier des charges, nous avons souligné l’importance de l’efficacité d’un système de satisfaction de contraintes. Dans ce chapitre nous présentons les performances de BACKTALK sur des problèmes combinatoires classiques. Nous comparons ces résultats avec ceux obtenus en **clp(FD)** et en ILOGSOLVER. Les temps d’exécution sont donnés pour une machine à base de processeur Pentium 300 MHz pour BACKTALK et **clp(FD)** et sur un Pentium Pro 200 MHz. Notre système tournait sous WINDOWS 95 avec la version 2.5 de VISUALWORKS. Les temps pour **clp(FD)** sont ceux fournis par Daniel Diaz, ceux de ilogsolver par Jean-François Puget et Jean-Charles Régin.

Certains résultats sont difficilement comparables. Le carré magique de d’ordre 6 est plus facile à résoudre que celui de taille 5 en **clp(FD)** ce qui est probablement dû à un effet de chance dans la sélection des variables. Il en est de même pour le problème des 99 reines pour lequel la résolution en BACKTALK et ILOGSOLVER est extrêmement longue alors qu’en **clp(FD)** et en ILOGSOLVER elle est pratiquement instantanée. En revanche, pour le problème des 100 reines, la résolution en BACKTALK et en ILOGSOLVER est instantanée, alors qu’elle requiert plusieurs secondes en **clp(FD)**. Sur les problèmes pour lesquels la comparaison est envisageable, **clp(FD)** et ILOGSOLVER sont plus rapides d’un facteur oscillant entre 1 et 10. En faisant la moyenne arithmétique du rapport entre les temps d’exécution de BACKTALK et des deux autres systèmes pour les problèmes significatifs (i.e. en supprimant les 99 reines pour la comparaison avec les deux systèmes et le carré magique d’ordre 6 pour la comparaison avec **clp(FD)**) on obtient les rapports moyens suivants :

- 1) **clp(FD)** est environ 4 fois plus rapide que BACKTALK
- 2) ILOGSOLVER est 3 fois et demi plus rapide sur une machine 30% plus lente, soit plus de 5 fois plus rapide

Ces benchmarks sont des problèmes assez faciles qui ne mettent pas en évidence le comportement toutes les caractéristiques des systèmes comme la gestion de la mémoire pour de gros problèmes. Cela suffit cependant à mesurer les vitesses relatives d’exploration, ce qui dépend de la stratégie de recherche et de la propagation. On constate que le rapport entre les temps de résolution de BACKTALK et d’ILOGSOLVER est du même ordre de grandeur que la différence de performance entre SMALLTALK et C++. La comparaison avec **clp(FD)** s’explique pour partie par les optimisations réalisées dans l’implémentation de ce système [Diaz, 1995] et pour partie par des stratégies de résolution distinctes dues à la conception différente des deux systèmes.

Problème	BACKTALK (CPU sec. / échecs) 300 MHz	clp(FD) (CPU sec.) 300 MHz	ILOG- SOLVER (CPU sec.) 200 MHz
Le problème du pont (Bridge)	0,42 / 204	0,06	0,09
Carré magique 3 ff	0,002 / 1	0	0,016
Carré magique 4 ff	0,013 / 15	0,01	0,015
Carré magique 5 ff	0,86 / 791	0,21	0,157
Carré magique 6 ff	96,0 / 74 597	0,1	13,375
SEND + MORE = MONEY ff	0,001 / 1	0	0,015
DONALD + GERALD = ROBERT ff	0,023 / 67	0,01	0
Alpha	5,0 / 3 306	0,85	0,875
Alpha ff	0,03 / 10	0,01	0,016
Bêta	12,1 / 7	4,23	2,8
Bêta ff	0,026 / 7	0,03	0,030
La multiplication magique	5,1 / 1620	1,5	0,87
La multiplication magique ff	0,34 / 95	3,35	0,57
Série magique 10 ff	0,008 / 7	0	0,015
Série magique 50	0,78 / 69	0,39	0,110
Série magique 50 ff	0,08 / 7	0,07	0,31
Série magique 100	5,8 / 144	3,51	0,515
Série magique 100 ff	0,29 / 7	0,28	0,063
8 reines ff	0,01 / 23	0	0
16 reines ff	0,007 / 7	0	0
40 reines ff	0,04 / 19	0,01	0,015
99 reines ff	2,8 / 454	0,03	0,922
100 reines ff	0,265 / 22	5,46	0,094
102 reines ff	0,28 / 19	0,03	0,094
Carrés parfaits #1	0,07 / 3	0,01	?
Carrés parfaits #2	0,11 / 1	0,02	?
Carrés parfaits #3	9,1 / 346	0,78	?
Carrés parfaits #4	9,8 / 289	1,290	?

*Tableau 8 La comparaison des performances entre BACKTALK et **clp (FD)** pour des problèmes numériques connus. La comparaison est à l'avantage de **clp (FD)** qui est plus rapide que BACKTALK d'un facteur compris entre 1 et 10*

Chapitre 6

Conclusion

Les objectifs que nous nous sommes fixés dans le cahier des charges ont été atteints. Le système est suffisamment efficace pour permettre la comparaison avec les meilleurs logiciels existants malgré la relative lenteur du langage hôte. Hormis l'inévitable hiatus entre les notions de variables du système et du langage, l'intégration à SMALLTALK est complète : pas d'extension syntaxique, pas de nouveaux mécanismes ni de modification de la machine virtuelle.

L'architecture de BACKTALK en fait un système résolument extensible. Notre proposition repose sur la conviction que la meilleure approche pour la résolution de problèmes de contraintes est l'utilisation d'algorithmes combinant l'énumération complète et le filtrage de contraintes. L'architecture de BACKTALK permet de définir de nouvelles classes de contraintes, à la fois par héritage et par composition en créant des sous-classes ou en composant des contraintes existantes. En outre, BACKTALK offre les outils nécessaires pour spécifier aisément des stratégies de résolution rentrant dans le schéma énumération + propagation. Nous avons implémenté des mécanismes qui garantissent, autant que possible et sans perte d'efficacité, la correction et la complétude la résolution.

Enfin, l'objectif d'efficacité du système a été atteint. Il est sensiblement moins rapide que les meilleurs systèmes actuels ce qui était inévitable en utilisant la langage SMALLTALK.

QUATRIÈME PARTIE

LES PROBLÈMES DE CONTRAINTES ET OBJETS

La majeure partie des problèmes traités par la satisfaction de contraintes est constituée de problèmes provenant de la recherche opérationnelle comme les problèmes d'ordonnancement, d'allocation de ressources, de découpe optimale ou encore de gestion d'emploi du temps. Ces problèmes sont généralement bien formalisés mathématiquement.

Lorsqu'on les résout par la satisfaction de contraintes, on les définit comme des problèmes numériques ou logiques, c'est-à-dire des problèmes dont les domaines des variables contiennent des nombres ou des valeurs booléennes, et dont les contraintes expriment des relations arithmétiques ou logiques. Cette méthodologie s'avère efficace et a permis la résolution de problèmes difficiles parfois avec de meilleures performances que les algorithmes spécialisés de recherche opérationnelle [Laburthe, 1998]. L'application de cette approche à des problèmes portant sur des structures de données complexes, que nous appellerons des objets, nécessite "l'aplanissement" des structures ainsi que la reformulation des contraintes en termes mathématiques ou logiques.

Dans cette partie, nous montrons que cette approche conduit à plusieurs écueils lorsqu'elle est appliquée à des problèmes dont la structure n'est pas aisément représentable numériquement et pour lesquels il n'existe pas de bonne formalisation mathématique. Plus précisément, cela conduit à une représentation des structures du problème qui est néfaste à la fois pour la définition des problèmes et l'efficacité de la résolution.

Nous introduisons une méthodologie pour la définition de problèmes structurés en termes de contraintes *et* d'objets : *l'approche par classe*. Nous montrons que cette approche permet une représentation des structures du problème qui présente plusieurs avantages. Elle permet l'utilisation d'un langage plus riche pour l'expression des contraintes. En outre, cette méthodologie autorise la réutilisation de classes d'objets prédéfinis pour la définition des valeurs des variables contraintes. Finalement, l'approche par classes conduit à une amélioration spectaculaire de l'efficacité de résolution.

Chapitre 1

Deux approches différentes

Certains problèmes portant sur des structures de données complexes qu'il peut être intéressant de représenter par des objets. Nous appellerons objets composites ces structures de données complexes. Deux approches diamétralement opposées sont possibles pour la définition de problèmes de contraintes impliquant des objets composites. L'une des deux approches, appelée *approche par attributs*, consiste à se ramener à un problème numérique en "aplanissant" les objets composites du problème. Dans ce cas les variables du problème contiennent des entités atomiques (e.g. des nombres) et les contraintes du problème sont exprimées comme des relations arithmétiques entre ces valeurs. La seconde approche, dite *approche par classes*, consiste au contraire à manipuler des variables dont les domaines contiennent des objets composites du problème et à définir les contraintes directement entre ces objets. Les parties qui suivent sont consacrées à la présentation de ces deux approches et à l'étude de leurs différences.

1.1 L'APPROCHE PAR ATTRIBUTS

Une façon naturelle de procéder à la définition d'un problème de contraintes portant sur des objets composites est de considérer les attributs de ces objets comme les inconnues du problème. Ainsi les variables contraintes du problème représenteront les attributs des objets composites du problème à résoudre. On considérera dans la suite de ce document que les attributs des objets composites sont des entités atomiques (i.e. des nombres ou des valeurs booléennes ou symboliques).

Ceci conduit à la notion d'*objet partiellement instancié* dont les attributs sont des variables contraintes au lieu d'être des valeurs. Ainsi, dans l'approche par attributs, les contraintes portent sur les attributs des objets composites. Dans cette approche, les contraintes sont de deux types. Il y a d'une part les contraintes servant à définir les relations entre objets composites, qui sont les véritables contraintes du problème. D'autre part, il est nécessaire de définir des contraintes entre les différents

attributs d'un objet composite donné, afin de représenter sa structure. Nous appellerons ces dernières des *contraintes de structures*. La Figure 28 donne une illustration graphique de l'approche par attributs. Dans cette figure, on représente un problème impliquant deux objets composites, nommés O_1 et O_2 . Les objets O_1 et O_2 sont des objets partiellement instanciés : i.e. leurs attributs sont des variables contraintes du problème.

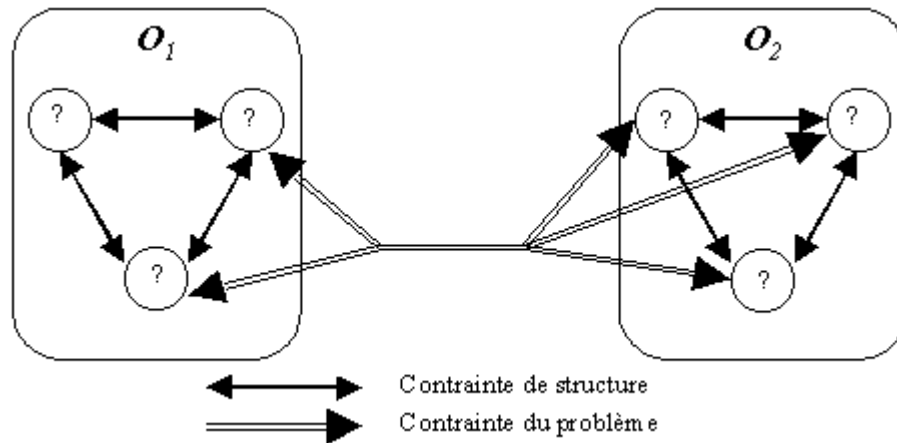


Figure 28 Illustration de l'approche par attributs. Les points d'interrogation cerclés représentent les attributs des objets composites, qui sont également les variables d'instance du problème. Les flèches simples représentent les contraintes de structures, qui portent sur plusieurs attributs d'un même objet composite, et servent à définir sa structure. Les flèches doubles sont les contraintes du problème proprement dites. Elles portent entre les attributs d'objets composites différents

1.2 APPROCHE PAR CLASSES

L'approche orthogonalement opposée à l'approche par attributs consiste à faire porter les contraintes du problème directement entre objets composites. Dans ce cas, on ne manipule plus d'objets partiellement instanciés, mais uniquement des objets *standards* ou *totalement instanciés*, c'est-à-dire des objets dont les attributs ne sont pas des variables contraintes mais des valeurs. L'idée sous-jacente est de considérer les *classes* du langage comme des domaines potentiels pour les variables contraintes du problème. La Figure 29 illustre graphiquement l'approche par classe.

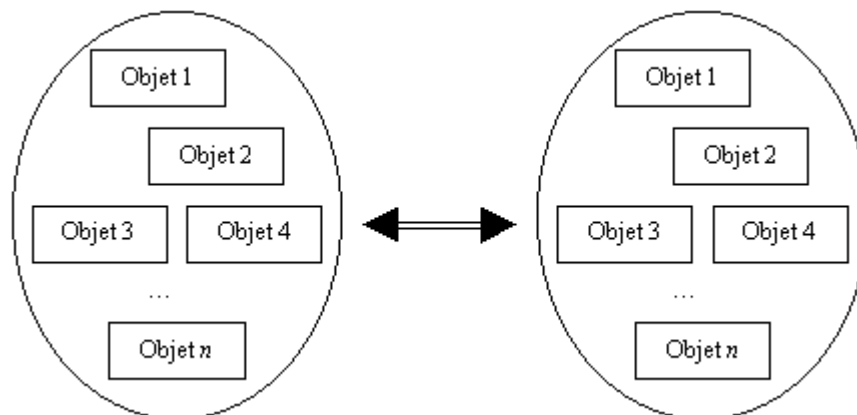


Figure 29 Illustration graphique de l'approche par attributs. Les points d'interrogation cerclés représentent les variables contraintes du problème. La seule contrainte porte sur les deux variables contraintes dont les domaines contiennent des objets standards (i.e. totalement instanciés).

1.3 ILLUSTRATION DES DEUX APPROCHES

Afin d'illustrer les deux approches précédentes pour la définition des problèmes portant sur des objets composites, considérons un problème simple de géométrie plane. (P) Trouver toutes les paires de quadrilatères vérifiant les propriétés suivantes :

1. Les sommets ont des coordonnées entières dans $\{1, \dots, n\}$
2. Les deux quadrilatères sont des rectangles ayant leurs côtés parallèles aux axes
3. Les deux quadrilatères ne se rencontrent pas

Voici une illustration de ce problème :

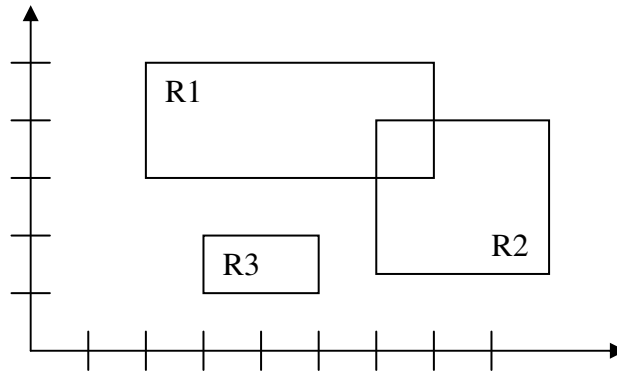


Figure 30 Le problème des rectangles. Ici, $(R1, R3)$ et $(R2, R3)$ sont des solutions, alors que la paire $(R2, R3)$ n'en est pas une

1.3.1 APPROCHE PAR ATTRIBUTS

Dans la formulation suivant l'approche par attributs, on définit une variable contrainte pour chaque attribut. Ainsi, les quatre sommets de chaque rectangles sont représentés par des variables contraintes. Nous les appellerons a, b, c, d et a', b', c', d' . Les propriétés de l'énoncé sont représentées par des contraintes entre ces variables. Ces variables ont pour domaine l'ensemble des points (i.e. $\{(0,0), \dots, (n,n)\}$). Selon notre définition, l'approche par attributs consiste à décomposer chaque point en deux variables contraintes : une pour l'abscisse et l'autre pour l'ordonnée. nous le ferons pas ici pour que l'exemple reste simple.

La propriété 4 sera représentée par la contrainte suivante :

$x(b) < x(a')$ "R1 est à gauche de R2"
 ou $x(a) < x(b')$ "R1 est à droite de R2"
 ou $y(c) < y(a')$ "R1 est au dessus de R2"
 ou $y(a) < y(c')$ "R1 est au dessous de R2"

où les $x()$ et $y()$ représentent les fonctions coordonnées de chaque point.

Selon cette formulation le problème comporte 8 variables dont le domaine est de cardinalité n^2 et 9 contraintes. La relation 1 de l'énoncé sont représentées dans les domaines. La relation 2 est représentée par un série de 8 contraintes binaires. Les relation 3, donnée ci-dessus, est représentée par une contrainte globale (i.e. d'arité 8).

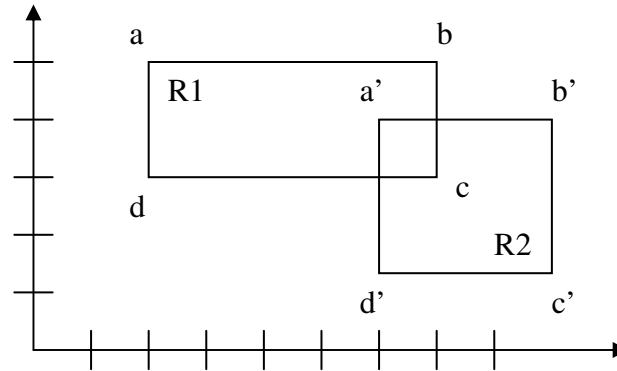


Figure 31 Les rectangles représentés par leurs sommets respectifs (approche par attributs)

1.3.2 APPROCHE PAR CLASSES

En utilisant l'approche par classes, les variables contraintes représentent les rectangles. Il y a alors deux variables contraintes dont le domaine contient des objets rectangles. Dans ce cas, la relation 3 est représentée par une contrainte définie par la formule de satisfaction : $(R1 \text{ intersects } R2) \text{ not}$ où intersects est une méthode implémentée dans la classe des rectangles.

Dans ce cas le problème est défini par deux variables dont le domaine contient $N!/(N-4)!$ avec $N=n^2$ rectangles et une contrainte binaire.

1.3.3 COMPARAISON DES DEUX FORMULATIONS

La seconde formulation pose une difficulté majeure. Elle nécessite de créer tous les rectangles possibles, afin de construire les domaines. Leur nombre est très important : c'est le nombre de points à la puissance 4.

Pour éviter cette difficulté, il est nécessaire de décomposer la résolution du problème en deux phases distinctes. Dans la première phase, on ne considère que les objets simples du problème (i.e. les points) et on réduit leur nombre par application d'un algorithme de réduction de domaines (arc cohérence). Dans la seconde phase, on construit le problème impliquant les objets complexes, dont le nombre est réduit grâce au travail fourni à la première phase.

Plus précisément, dans la première phase, on considère, pour chaque rectangle, les variables a, b, c et d . Les contraintes entre ces objets sont les suivantes : $x(a)=x(c)$; $y(a)=y(b)$; $x(b)=x(d)$; $y(b)=y(d)$.

L'énumération des solutions de ce problème donne l'ensemble des rectangles à considérer dans la seconde phase de la résolution. Leur nombre est $n!/(n-4)!$ Qui est bien inférieur à $N!/(N-4)!$

La complexité des deux approches est différente. Dans la première, il y a 8 variables dont le domaine contient n^2 points. Considérant que le traitement d'une contrainte consiste à calculer le produit cartésien des domaines de ses variables (ce qui est le cas si l'on utilise la méthode d'arc cohérence par défaut), la complexité totale est

donc de l'ordre de n^{16} à cause de la contrainte d'arité 8. Dans la seconde approche, le premier problème comporte huit variables, dont le domaine contient n^2 valeurs, reliées entre elles par des contraintes binaires. La complexité de ce premier problème est donc de l'ordre de $(n^2)^4$, soit n^8 . Le second problème comporte deux variables de taille $n!/(n-4)!$ Reliées par une contraintes binaire. La complexité est donc de l'ordre de n^8 pour ce second problème.

Finalement, la première approche conduit à une complexité de l'ordre de n^{16} , alors que la seconde a une complexité de l'ordre de n^{16} . Il y a un rapport quadratique entre les complexités des deux approches.

Ce résultat n'est valable que parce que le nombre de rectangles ayant leurs côtés parallèles aux axes est relativement faible devant le nombre de quadrilatères. Il y a entre ces deux quantités un rapport quadratique, qui est directement à l'origine du rapport entre les complexités des deux approches.

Dans le cas général, le rapport de complexité entre les deux approches, pour un problème donné, dépend directement de la "densité" des objets composites dans l'espace des n -uplets d'objets atomiques. Dans la partie consacrée à l'harmonisation automatique, nous verrons un problème pour lequel les objets composites (i.e. les accords musicaux) sont très peu denses dans l'ensemble des quadruplets de notes, ce qui rend une approche par classes beaucoup plus efficace qu'une approche par attributs.

Chapitre 2

Langage de définition des contraintes

Définir un problème combinatoire en termes de satisfaction de contraintes nécessite d'identifier les variables et les domaines qui leur sont associés, ainsi que les contraintes qui définissent une solution (cf. page 14). Pour cette dernière étape il est nécessaire de disposer d'un langage pour la définition des contraintes.

Dans la partie précédente, nous avons présenté deux approches différentes pour la définition de problèmes de contraintes portant sur des objets complexes. L'utilisation de l'une ou l'autre des deux approches modifie à la fois la nature des valeurs contenues dans les domaines des variables, mais également la nature des contraintes. En effet, les contraintes expriment, dans l'approche par attributs, des relations entre les attributs des objets composites, qui sont des objets atomiques. Dans l'autre cas, suivant l'approche par classes, les contraintes expriment des relations entre les objets composites eux-mêmes. Le langage de définition des contraintes dépend du type des variables sur lesquelles on les fait porter (i.e. le type de valeurs se trouvant dans leurs domaines). Pour les variables atomiques (i.e. numériques ou logiques), il est facile de définir un langage adapté pour la définition des contraintes, à partir des expressions arithmétiques usuelles, ce qui permet d'en donner *a priori* une implémentation efficace et générale sous forme de contraintes prédéfinies. De façon similaire, pour les problèmes logiques, le langage de définition des contraintes est constitué par les relations logiques usuelles.

En revanche, pour les variables dont les valeurs sont des objets quelconques, le langage nécessaire à la définition des contraintes est impossible à définir *a priori*. En effet, l'ensemble des relations que l'on souhaite pouvoir exprimer sous forme de contraintes est potentiellement infini et dépend du type des objets considérés. Par exemple, si l'on manipule des figures géométriques, les relations que l'on veut pouvoir exprimer portent sur les propriétés géométriques des objets : leur surface, le nombre de leurs côtés, leur positions relatives, etc. Si, en revanche, on manipule des objets représentant des adresses postales, les contraintes que l'on veut exprimer porteront sur le code postal, le nom de la personne, sa situation géographique etc. Mettre à la disposition de l'utilisateur un langage suffisamment puissant pour l'expression de

telles contraintes oblige à définir une méthodologie et des outils particuliers, que nous présentons dans les parties qui suivent.

2.1 LES CONTRAINTES NUMÉRIQUES ET LOGIQUES

Nous appelons numérique (resp. logique) un problème dont les domaines des variables contiennent des nombres (resp. des valeurs booléennes). Nous avons déjà donné plusieurs exemples de tels problèmes (cf. 1.2 et 1.3). La majeure partie des contraintes nécessaires à la définition de tels problèmes expriment des relations arithmétiques ou logiques composées à l'aide d'opérateurs de base : l'*addition*, la *soustraction*, la *multiplication*, les opérations de *reste* et de *quotient*, l'*égalité* et la *différence* et les relations d'ordre, le *et*, le *ou* et la *négation*.

Ces relations sont d'usage courant depuis longtemps et leurs propriétés sont bien connues. Ceci permet de constituer des bibliothèques de contraintes prédéfinies. L'implémentation de ces contraintes est très efficace, puisqu'il est facile de trouver de bons compromis pour la définition de méthodes de filtrages, notamment en utilisant le raisonnement sur les bornes des domaines. À titre d'illustration, voici la liste des classes de contraintes prédéfinies du système BACKTALK, et quelques renseignements sur l'implémentation et l'efficacité de leurs méthodes de filtrages. Ces quelques classes de contraintes permettent la définition de la plupart des problèmes numériques.

2.2 LES CONTRAINTES SUR LES OBJETS

Dans les problèmes impliquant des objets, c'est-à-dire dont les domaines des variables contiennent des objets quelconques, les relations que l'on souhaite exprimer ne sont pas connues *a priori*. On ne peut donc pas fournir en toute généralité de bibliothèque de classes de contraintes prédéfinies pour ces problèmes. On peut bien entendu réaliser de telles bibliothèques pour des domaines d'application très particuliers comme la géométrie ou le graphisme en deux dimensions. Cette possibilité est intéressante, même si elle est par définition limitée à des domaines d'application particuliers bien connus.

Le langage dont on souhaite disposer pour les contraintes dans de tels problèmes doit être suffisamment riche pour permettre l'expression de toutes les relations possibles entre les objets manipulés. Il doit de plus permettre une implémentation efficace des contraintes, c'est-à-dire de leurs mécanismes de filtrage.

Nous avons implémenté dans le système BACKTALK deux classes générales, qui permettent de définir des contraintes exprimant des relations arbitrairement complexes entre des objets quelconques. Ces deux classes de contraintes sont respectivement la classe des contraintes de type "block" et la classe des contraintes de type "perform". La première, nommée **BTBlockCt**, est la plus générale, et permet de définir une contrainte à partir de toute expression booléenne du langage SMALLTALK. La seconde, nommée **BTPerformCt**, correspond à la méthode perform du langage SMALLTALK. Elle permet de définir une contrainte exprimant que la valeur d'une variable est le résultat de l'application d'une méthode quelconque à la valeur d'une autre variable. L'implémentation de ces deux classes de contraintes est détaillée ci-dessous à l'aide de plusieurs d'exemples d'utilisation.

2.2.1 LES CONTRAINTES DE TYPE “BLOCK”

Lors de la définition de problèmes de contraintes impliquant des objets, il est fréquent que l’on souhaite déclarer des contraintes exprimant des propriétés arbitraires entre ces objets. Par exemple, on souhaite exprimer le fait que la surface d’un rectangle *R1* soit le double de la surface d’un rectangle *R2*. Dans l’approche par attributs, on *réifierait* la surface de chaque rectangle comme une variable contrainte, disons *S1* et *S2*, et on déclarerait la contrainte exprimant la relation :

$$S1 = 2 \times S2$$

Dans l’approche par classes, nous devons exprimer cette contrainte en termes de relation entre rectangles. Si l’on suppose que la classe des rectangles implémente une méthode, disons **surface**, donnant la surface d’une instance, la relation à exprimer (dans la syntaxe SMALLTALK) est :

$$R1 \text{ surface} = 2 \times (R2 \text{ surface})$$

La classe des contraintes de type block permet de déclarer une contrainte directement à partir d’une telle expression du langage hôte. Pour ce faire, il suffit de créer un block SMALLTALK (un block encapsule du code, similaire à une méthode, dont l’évaluation n’est pas automatique) qui encapsule cette expression, ce qui se fait avec la syntaxe suivante :

```
[ :R1 :R2 | R1 surface = 2*(R2 surface) ]
```

Ensuite, on crée une contrainte de type block par instanciation de la classe **BTBlockCt** avec comme paramètres les variables de la contrainte et le block définissant la relation qu’on souhaite exprimer. Dans l’exemple précédent, pour créer une telle contrainte entre deux variables **Rect1** et **Rect2** dont les domaines contiennent des instances de la classe des rectangles, on utilisera la syntaxe suivante :

```
BTBlockCt  
  on: Rect1  
  and: Rect2  
  block: [ :R1 :R2 | R1 surface = 2*(R2 surface) ]
```

Remarque : Les paramètres du block ne sont pas les variables contraintes, mais des arguments propres. Lors de la résolution, ces paramètres seront remplacés par des valeurs du domaine des variables **Rect1** et **Rect2**.

A) FILTRAGE DES CONTRAINTES DE TYPE BLOCK

La classe des contraintes de type block est implémentée comme une sous-classe de la classe abstraite **BTConstraint**. La classe **BTBlockCt** implémente une seule méthode de filtrage qui est déclenchée par chaque démon. Cette méthode réalise la cohérence d’arc. L’algorithme le plus simple pour réaliser la cohérence d’arc d’une contrainte de ce type est le suivant :

Variables: $x_1, \dots, x_n$
 Domaines: $D_1, \dots, D_n$
 Block: B

Soient: $E_1 = \emptyset, \dots, E_n = \emptyset$ "les nouveaux domaines"

```

∀i ∈ {1, ..., n} {
  ∀xi ∈ Di \ Ei {
    si ∃(x1, ..., xi-1, xi+1, ..., xn) ∈ D1 × ... × Di-1 × Di+1 × ... × Dn
    tel que B(x1, ..., xn) = vrai
      alors Ei ← Ei ∪ {xi}
  }
}

```

```

∀i ∈ {1, ..., n} {      "mise à jour des domaines"
  Di ← Ei
}

```

Algorithme 9 L'algorithme le plus simple pour réaliser la cohérence d'arc d'une contrainte de type block. Pour chacun des éléments x_i du domaine D_i de chaque variable V_i , cet algorithme cherche une instantiation cohérente contenant x_i . La complexité de cet algorithme est systématiquement exponentielle.

On peut améliorer cet algorithme sans perdre aucune généralité en y apportant les modifications qui figurent en gras dans l'algorithme suivant :

Variables: $x_1, \dots, x_n$
 Domaines: $D_1, \dots, D_n$
 Block: B

Soient: $E_1 = \emptyset, \dots, E_n = \emptyset$ "les nouveaux domaines"

```

∀i ∈ {1, ..., n} {
  ∀xi ∈ Di \ Ei {
    si ∃(x1, ..., xi-1, xi+1, ..., xn) ∈ D1 × ... × Di-1 × Di+1 × ... × Dn
    tel que B(x1, ..., xn) = vrai
      alors {
        ∀j ∈ {1..n} {
          Ej ← Ej ∪ {xj}
        }
      }
  }
  Di ← Ei      "mise à jour des domaines"
}

```

Algorithme 10 L'algorithme amélioré pour le filtrage des contraintes de type block. Cet algorithme réalise la cohérence d'arc de la contrainte. Il procède par énumération implicite du produit cartésien des domaines des variables. Sa complexité est bien évidemment exponentielle dans le pire des cas.

Cet algorithme a une complexité exponentielle dans le pire des cas puisqu'il peut réclamer l'exploration complète du produit cartésien des domaines des variables

de la contrainte. Cependant, les trois lignes écrites en gras dans l'algorithme constituent des améliorations sensibles par rapport à l'Algorithme 9.

La première modification est fondée sur la remarque suivante : si une instantiation $(x_1, \dots, x_n)$ est cohérente, alors pour tout i la valeur x_i doit être maintenue dans le domaine D_i . Cette remarque permet de justifier n valeurs au lieu d'une à la fois.

La seconde amélioration consiste à mettre à jour le domaine D_i de la variable V_i avant de justifier les valeurs de D_{i+1} ce qui permet de chercher les instantiations cohérentes dans un ensemble de plus en plus réduit d'instanciations.

À titre d'exemple, si l'on considère la contrainte de type block suivante :

```
BTBlockCt  
on: Rect1  
and: Rect2  
block: [:R1 :R2 | R1 surface = 2*(R2 surface)]
```

et si les domaines des variables **Rect1** et **Rect2** contiennent les valeurs suivantes (chaque rectangle est ici défini par les coordonnées gauche, bas, droite et haut) :

{A=(0,0,3,2), B=(3,2,5,3), C=(1,5,4,6), D=(0,0,2,3), E=(5,7,6,11)}

Dans ce cas, l'Algorithme 9 procédera de la façon suivante :

```

X1 = A  X2 = A; (A,A) non cohérent
        X2 = B; (A,B) cohérent => E1 ← {A}
X1 = B  X2 = A; (B,A) non cohérent
        X2 = B; (B,B) non cohérent
        X2 = C; (B,C) non cohérent
        X2 = D; (B,D) non cohérent
        X2 = E; (B,E) non cohérent
X1 = C  X2 = A; (C,A) non cohérent
        X2 = B; (C,B) non cohérent
        X2 = C; (C,C) non cohérent
        X2 = D; (C,D) non cohérent
        X2 = E; (C,E) non cohérent
X1 = D  X2 = A; (D,A) non cohérent
        X2 = B; (D,B) non cohérent
        X2 = C; (D,C) cohérent => E1 ← E1 ∪ {D}
X1 = E  X2 = A; (E,A) non cohérent
        X2 = B; (E,B) cohérent => E1 ← E1 ∪ {E}

X2 = A  X1 = A; (A,A) non cohérent
        X1 = B; (B,A) non cohérent
        X1 = C; (C,A) non cohérent
        X1 = D; (D,A) non cohérent
        X1 = E; (E,A) non cohérent
X2 = B  X1 = A; (A,B) non cohérent
        X1 = B; (B,B) non cohérent
        X1 = C; (C,B) non cohérent
        X1 = D; (D,B) non cohérent
        X1 = E; (E,B) cohérent => E2 ← {B}
X2 = C  X1 = A; (A,C) cohérent => E2 ← E2 ∪ {C}
X2 = D  X1 = A; (A,D) non cohérent
        X1 = B; (B,D) non cohérent
        X1 = C; (C,D) non cohérent
        X1 = D; (D,D) non cohérent
        X1 = E; (E,D) non cohérent
X2 = E  X1 = A; (A,E) non cohérent
        X1 = B; (B,E) non cohérent
        X1 = C; (C,E) non cohérent
        X1 = D; (D,E) non cohérent
        X1 = E; (E,E) non cohérent

```

D1 ← E1 = {A,D,E}

D2 ← E2 = {B,C}

38 tests de cohérence sont nécessités avant de réaliser la cohérence d'arc de la contrainte.

L'Algorithme 10 fera un nombre sensiblement plus réduit de tests :

X1 = A	X2 = A; (A,A) non cohérent	
	X2 = B; (A,B) cohérent => E1 ← {A} et E2 ← {B}	
X1 = B	X2 = A; (B,A) non cohérent	
	X2 = B; (B,B) non cohérent	
	X2 = C; (B,C) non cohérent	
	X2 = D; (B,D) non cohérent	
	X2 = E; (B,E) non cohérent	
X1 = C	X2 = A; (C,A) non cohérent	
	X2 = B; (C,B) non cohérent	
	X2 = C; (C,C) non cohérent	
	X2 = D; (C,D) non cohérent	
	X2 = E; (C,E) non cohérent	
X1 = D	X2 = A; (D,A) non cohérent	
	X2 = B; (D,B) non cohérent	
	X2 = C; (D,C) cohérent => E1 ← E1 ∪ {D} et E2 ← E2 ∪ {C}	
X1 = E	X2 = A; (E,A) non cohérent	
	X2 = B; (E,B) cohérent => E1 ← E1 ∪ {E} et E2 ← E2 ∪ {B}	

D1 ← E1 = {A,D,E}

X2 = A	X1 = A; (A,A) non cohérent
	X1 = D; (D,A) non cohérent
	X1 = E; (E,A) non cohérent
X2 = D	X1 = A; (A,D) non cohérent
	X1 = D; (D,D) non cohérent
	X1 = E; (E,D) non cohérent
X2 = E	X1 = A; (A,E) non cohérent
	X1 = D; (D,E) non cohérent
	X1 = E; (E,E) non cohérent

D2 ← E2 = {B,C}

26 tests de cohérence sont nécessités avant de réaliser la cohérence d'arc de la contrainte.

En dépit des améliorations qu'il apporte par rapport à l'Algorithme 9, l'Algorithme 10 n'est pas suffisamment efficace pour être appliqué à des contraintes impliquant de nombreuses variables dont les domaines ne sont pas de taille très réduite. Plus précisément, pour une contrainte impliquant 8 variables dont le domaine contient environ 10 éléments (une telle contrainte apparaît dans un problème aussi simple que SEND + MORE = MONEY !) on aura potentiellement 10^8 tests pour obtenir la cohérence d'arc. Ce coût est évidemment rédhibitoire.

Il existe dans BACKTALK plusieurs moyens d'améliorer l'efficacité du filtrage de ces contraintes en fonction de leur nature, et de leur importance dans le problème à résoudre. Le premier moyen d'améliorer l'efficacité de la gestion de ces contraintes est de ne pas les faire réagir à tous les démons. Il suffit le plus souvent de ne déclarer que le démon associé à l'instanciation d'une variable. Ainsi le filtrage n'est entrepris que lorsqu'une modification très significative est survenue.

Un autre moyen d'améliorer la gestion des contraintes de type block est de fixer, pour la taille du produit cartésien des domaines, un seuil au-delà duquel la contrainte ne sera pas filtrée. Ce seuil est une variable d'instance de la classe **BTBlockCt**, et peut être fixé à la création de la contrainte. Par exemple, si l'on veut interdire le déclenchement de l'algorithme de filtrage quand la taille du produit cartésien des domaines est supérieure à 100.000, oninstanciera la contrainte en utilisant le message suivant :

```
BTBlockCt
  on: Rect1
  and: Rect2
  block: [:R1 :R2 | R1 surface = 2*(R2 surface)]
  thresholdForFiltering: 100000
```

Ce seuil ne sera pas considéré lorsque toutes les variables sont instanciées sauf une, sous peine de risquer de perdre la complétude de la résolution.

B) UTILISER LA SÉMANTIQUE DES CONTRAINTES DE TYPE BLOCK

Une autre amélioration du filtrage des contraintes de type block consiste à prendre en compte la sémantique de la relation encapsulée dans le block définissant la contrainte. De nombreuses propriétés de l'expression constituant le block peuvent, si l'on sait les détecter, permettre de d'améliorer sensiblement la complexité du filtrage. De nombreux cas triviaux peuvent être inventés, tels le cas des contraintes définies par un des blocks suivants :

```
[:a :b :c :d | true]
[:a :b :c | false]
```

qui ne requièrent aucun filtrage. On peut également réduire la complexité du filtrage des contraintes définies par des blocks encapsulant une expression qui n'implique pas tous ses arguments, par exemple :

```
[:a :b :c :d :e | a + b < c * c]
```

Les contraintes de type “ $P(X_i | i \in I)$ et $Q(X_j | j \in J)$ ”

Dans le cas où les ensembles d'indices I et J ne sont pas identiques, plutôt que de poser une seule contrainte, il est préférable d'en poser deux. En effet, on peut déclarer une seule contrainte : $C(X_i | i \in I \cup J)$ équivalent à la conjonction de P et Q . Mais il est préférable de poser une contrainte sur les variables d'indice dans I , d'expression P et une autre contrainte correspondant à J et Q . C'est le principe même de la programmation par contraintes de diviser les problèmes en sous problèmes, les contraintes, afin de les résoudre plus facilement. C'est d'autant plus crucial que le coût du filtrage de la contrainte dépend de façon exponentielle du nombre de variables.

Donnons tout de suite un exemple : au lieu de définir la contrainte suivante qui est d'arité 6 :

BTBlockCt

```
on: (v1 v2 v3 v4 v5 v6)
block: [:a :b :c :d :e :f |
        (a + b + c + d = 0) and: [c + d + e + f = 0]]
```

il vaut mieux définir les deux contraintes d'arité 4 suivantes :

BTBlockCt

```
on: (v1 v2 v3 v4)
block: [:a :b :c :d | (a + b + c + d = 0)]
```

BTBlockCt

```
on: (v3 v4 v5 v6)
block: [:a :b :c :d | (a + b + c + d = 0)]
```

La détection de ce type de contraintes se fait par analyse de l'expression encapsulée dans le block définissant la contrainte. Grâce à la réflexivité du langage SMALLTALK, on peut analyser le contenu du block relativement aisément. On réalise cette détection avant la création de la contrainte. Si l'utilisateur demande la création d'une telle contrainte, alors le système ne la créera pas, mais ilinstanciera plusieurs contraintes plus simples. Cette manipulation est transparente pour l'utilisateur. Nous allons maintenant donner un exemple un peu plus complexe de ce qu'on peut faire en analysant les contenus des blocks.

Les contraintes de type "Si $P(X_1)$ alors $Q(X_1, \dots, X_n)$ "

Un exemple simple mais intéressant dans la mesure où il n'est pas trivial et où il apparaît souvent est celui des expressions de type :

Si $P(X_1)$ alors $Q(X_1, \dots, X_n)$

Considérons par exemple la contrainte exprimant que deux rectangles doivent avoir la même surface si le premier des deux a une surface plus petite que 10. Cette contrainte est définie par l'expression BACKTALK suivante :

BTBlockCt

```
on: Rect1
and: Rect2
block: [:R1 :R2 |
        (R1 surface < 10)
        ifTrue: [R1 surface = R2 surface]
        ifFalse: [true]]
```

De telles contraintes peuvent être filtrées bien plus efficacement que par application de l'Algorithme 10. Il suffit en effet, dans ce cas, d'appliquer le traitement préalable suivant avant de déclencher le filtrage proprement dit par l'Algorithme 10 ci-dessus : *ne déclencher le filtrage que s'il n'existe dans le domaine de **Rect1** aucun rectangle **R1** dont la surface est supérieure ou égale à 10.*

Ainsi, le parcours du domaine de R1, avec le test surface de chacun de ses éléments, permet d'éviter de déclencher le filtrage proprement dit de la contrainte lorsque c'est possible. Remarquons que ce traitement ne modifie pas le résultat du filtrage, mais seulement son efficacité. Plus généralement, le filtrage des contraintes de type block exprimant une relation de la forme

si $P(X_1)$ alors $Q(X_1, \dots, X_n)$

s'effectue de la façon suivante :

```
∀x1 ∈ D1 {  
  si ¬P(x1)  
  alors {  
    “déclencher le filtrage”  
    (Algorithme 10)  
  }  
}
```

La difficulté est de détecter le type de l'expression encapsulée dans le block définissant la contrainte. En BACKTALK, ceci est effectué lors de la création de la contrainte par instanciation de la classe **BTBlockCt**. La seule difficulté est que les expressions dont la sémantique est du type Si $P(X_1)$ alors $Q(X_1, \dots, X_n)$ peuvent avoir des formes différentes, dont certaines sont difficiles à reconnaître. Nous avons choisi de nous limiter aux expressions SMALLTALK suivantes, qui sont faciles à reconnaître :

```
Condition ifTrue: [expression] ifFalse: [true]  
Condition ifTrue: [true] ifFalse: [expression]  
Condition ifFalse: [expression] ifTrue: [true]  
Condition ifFalse: [true] ifTrue: [expression]  
Condition or: [expression]
```

Pour la reconnaissance on utilise ici encore le décompilateur et l'analyseur syntaxique standards de SMALLTALK. On vérifie que l'arbre d'analyse à l'un des formes précédentes, et, le cas échéant, que la condition n'implique qu'un des paramètres du block. Lorsqu'une telle contrainte est reconnue, on instancie alors une contrainte de la classe **BTConditionalBlockCt**, dont le mécanisme de filtrage est celui présenté plus haut. Cette opération est transparente pour l'utilisateur.

2.2.2 LES CONTRAINTES DE TYPE “PERFORM”

Un autre type de contraintes exprimant des relations entre objets est implémenté dans BACKTALK. Ces contraintes permettent de représenter la relation **perform** de SMALLTALK entre variables contraintes.

Reprenons l'exemple précédent : on souhaite avoir deux rectangles vérifiant que la surface du premier est le double de celle du second. Dans la partie qui précède, nous avons exprimé cette relation à l'aide d'une contrainte de type block, qui encapsule une expression entre les rectangles. L'utilisation des contraintes de type perform revient à réifier la surface de chaque rectangle sous la forme d'une nouvelle variable contrainte. La contrainte exprimant la relation entre les surfaces sera alors définie entre les variables représentant les surfaces des deux rectangles. Le schéma suivant donne une représentation graphique du problème de contraintes ainsi créé :

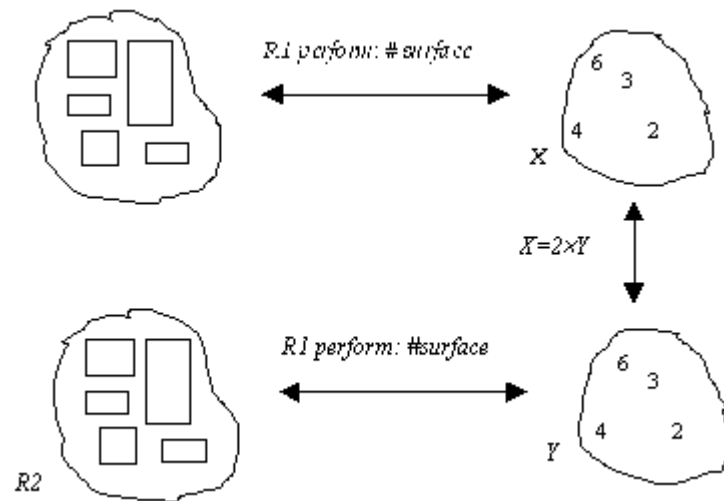


Figure 32 Le CSP correspondant au problème des deux rectangles exprimé à l'aide de contraintes de type perform. La contrainte exprimant la relation entre les surfaces des deux rectangles est ici une contrainte arithmétique entre les variables numériques représentant les aires.

CINQUIÈME PARTIE

DEUX APPLICATIONS

Le premier chapitre de cette partie est consacré à la construction de grilles de mots croisés. Jusqu'à maintenant, seules des approches procédurales ont donné de bons résultats sur ce problème. Nous montrons que l'utilisation des techniques de satisfaction de contraintes permet d'obtenir de bonnes performances. De plus, nous illustrons sur cet exemple les possibilités de BACKTALK pour l'expression de connaissances sur le problème. Les résultats finalement atteints sont très supérieurs à ceux des meilleures approches existantes.

Un des objets principaux de cette thèse était la résolution d'exercices d'harmonie musicale automatique. En dépit des nombreux travaux qui ont été consacrés à cet exercice, plusieurs problèmes restent à résoudre [Ballesta, 1994, Del-lacherie, 1998, Ebcioglu, 1992a, Ebcioglu, 1992b, Ovans, 1992, Ovans and Davison, 1992, Steels, 1986, Tsang and Aitken, 1991]. Nous avons développé un système fondé sur une intégration particulière des objets et des contraintes qui surpasse les systèmes existants en ce qui concerne l'efficacité de la résolution, la représentation des connaissances et la réutilisation de bibliothèques de classes prédéfinies. Le second chapitre de cette partie est consacré à cette application.

Chapitre 1

La création de grilles de mots croisés

Depuis les travaux de Feger et Mazlack [Feger, 1975, Mazlack, 1976a, Mazlack, 1976b] dans les années 1970, et même ceux de Spiegenthal en 1960, la production de grilles de mots croisés a été employée à plusieurs reprises comme domaine d'application et de test pour les techniques de résolution de problèmes. Comme dans toutes les disciplines de résolution de problèmes, on retrouve ici l'opposition entre deux types de stratégies : les approches *procédurales* versus les approches *déclaratives*. (Nous ne nous aventurons pas plus dans la discussion sur ces notions et les considérons comme évidentes !) Comme c'est souvent le cas lorsque l'on compare ces deux manières de résoudre un problème donné, l'efficacité est du côté des algorithmes procéduraux, la souplesse et la puissance expressive du côté des connaissances déclaratives. Nous allons le montrer en faisant présenter les algorithmes existants.

Comme l'avait prouvé en son temps Jean-Louis Laurière avec ALICE, la déclarativité peut permettre, si l'on emploie des techniques bien adaptées, d'atteindre une meilleure efficacité que des algorithmes spécialisés. C'est ce que nous allons illustrer avec application du système BACKTALK à la construction de grilles de mots croisés avec des techniques de satisfaction de contraintes [Roy, et al., 1997].

1.1 LE PROBLÈME

Nous n'aborderons ici que ce qui concerne le remplissage de la grille avec des mots sans considérer la création de définitions qui est étudiée par exemple par Du Boulay [Boulay and Smith, 1986], et pour laquelle on pourra lire avec plaisir certains chapitres de George Perec [Perec, 1979] ou de Roger La Ferté [Ferté and Capelovici, 1975]. Ainsi, notre problème consiste à remplir une grille de taille prédéfinie avec des mots issus d'un dictionnaire. La forme de la grille est rectangulaire. Il existe au moins deux variantes principales du problème selon que les cases noires sont disposées *a priori* ou non. Voici un exemple de problème. La liste de mots est la suivante : {ÉTÉ,

TÊT, ACE, ARE, CAR, ERG} et la grille à remplir est de dimensions 3×3 sans case noire. Voici deux solutions à ce problème :

E	T	E
T	E	T
E	T	E

A	C	E
C	A	R
E	R	G

Voici probablement l’une des premières grilles de mots croisés jamais construites. Elle date du II^e ou III^e siècle de notre ère :

S	A	T	O	R
A	R	E	P	O
T	E	N	E	T
O	P	E	R	A
R	O	T	A	S

Cette grille est symétrique et de plus, la phrase constituée par ses cinq mots est un palindrome (SATOR AREPO TENET OPERA ROTAS). Sa signification serait “*Le semeur, avec sa charrue, tient fermement les roues*” [Ferté and Capelovici, 1975].

1.2 LES APPROCHES PRÉCÉDENTES

Nous avons puisé de nombreux renseignements sur les systèmes précédents dans le rapport de D.E.A. de Sik Cambon Jensen [Jensen, 1997] qui est un document inestimable pour qui s’intéresse à tous les aspects de la fabrication de grilles de mots croisés par ordinateur.

A) LES APPROCHES PROCÉDURALES

N’ayant pas de référence pour le travail de Spiegenthal, nous commencerons cette revue des approches existantes par celle de Mazlack [Mazlack, 1976a]. Sa méthode consiste à disposer les lettres une par une dans la grille. L’algorithme consiste à choisir une case vide et à choisir une lettre possible pour cette case. La sélection de la case se fait en fonction de l’appartenance à un groupe de cases constituant un mot dont certaines lettres sont déjà fixées. La sélection de la lettre se fait en fonction de probabilités d’occurrence. Avec un dictionnaire contenant environ 2 000 mots de 4 lettres au plus, Mazlack a pu construire des grilles, de taille environ 7×7, contenant de nombreuses cases noires, mais n’a pas pu remplir une grille de 4×4 sans aucune case noire.

À l’instigation de Mazlack lui-même [Mazlack, 1976b] d’autres méthodes ont été employées qui manipulent directement des mots entiers au lieu de disposer des lettres. L’algorithme de Feger [Feger, 1975] se décompose en 4 étapes :

- 1) Il détermine un sens de parcours pour les cases,
- 2) Il détermine ensuite un sens pour le placement d’un mot dans chaque case,
- 3) Il insère un mot, avec la possibilité de déclencher un retour arrière si aucun ne convient,
- 4) Il ajoute des nouvelles informations dues à la présence d’un nouveau mot.

Cette approche a permis de construire des grilles plus complexes que celles de Mazlack, notamment des grilles sans case noire de taille 4×4 et 5×5.

Smith et Steen [Smith and Steen, 1981] ont mis au point le premier algorithme capable de fabriquer de “vraies” grilles de mots croisés. Leur algorithme est constitué des étapes suivantes :

- 1) Pour chaque emplacement de mot de la grille, ils calculent le nombre de mots du dictionnaire qui ont la bonne taille,
- 2) La sélection des mots commence par l'emplacement ayant le moins de valeur possible (on trouve ici une intuition du principe du "plus petit domaine d'abord"),
- 3) Calculent une *estimation* du nombre de mots pouvant être placés dans chacun des emplacements de mot qui croisent celui qui vient d'être rempli.

L'estimation calculée à l'étape 2) n'étant pas nécessairement un majorant du nombre exact de mots pouvant convenir, l'algorithme n'est pas complet. Nous verrons que cette approche anticipe assez précisément celle que nous employons avec les techniques de satisfaction de contraintes.

Matthew Ginsberg [Ginsberg, et al., 1990] a utilisé plusieurs techniques d'Intelligence Artificielle pour résoudre le problème. Ici encore, on voit poindre un schéma de résolution similaire à celui d'un algorithme de satisfaction de contraintes.

- 1) À chaque pas de la résolution, on continue l'énumération en choisissant l'emplacement de mot qui peut contenir le plus petit nombre de mots. C'est encore le principe du plus petit domaine d'abord.
- 2) De plus, on choisit un emplacement de mot qui coupe un emplacement dont le mot est déjà fixé. Sous ce principe apparemment arbitraire se cache en fait l'idée du retour arrière intelligent. En effet, si l'on procède à l'énumération de façon connexe, chaque retour arrière implique nécessairement une variable voisine, qui est probablement impliquée dans l'erreur.
- 3) À l'instanciation d'un mot, l'influence sur les autres mots de la grille est prise en compte immédiatement. On peut prendre en compte l'influence sur les mots qui coupent, et c'est alors l'algorithme de forward-checking qui est appliqué, ou aller plus loin dans la recherche, ce qui revient à appliquer un algorithme de type look-ahead voire de full look-ahead !
- 4) Parmi les mots qui peuvent convenir, on choisit ceux qui conviennent le mieux selon des critères tels que : "après un Q, il y a souvent un U".

Harris [Harris, 1990] a mis au point un système qui permet de produire des grilles de mots croisés dont les cases noires ne sont pas connues *a priori*. L'idée, (trop) simple, consiste à calculer *a priori* toutes les grilles possibles, soit 2^{mn} pour une dimension de $m \times n$, et à résoudre ensuite en utilisant une des techniques précédentes. Afin de raffiner la méthode il propose d'entrelacer le placement des cases noires avec la sélection des mots. Ainsi, lorsqu'une case noire est placée, on vérifie qu'il est encore probable qu'une solution existe.

Les mêmes auteurs [Smith, et al., 1990] ont proposé une version dynamique de la méthode de Smith et Steen qui permet une prévision des erreurs à la manière de Ginsberg.

Enfin signalons la contribution de Sik Cambon Jensen, qui propose une approche mixte dans son mémoire de D.E.A. [Jensen, 1997] et qui présente dans les plus petits détails les travaux existants concernant à la fois la gestion du dictionnaire, la construction des grilles ou encore la fabrication de définitions.

B) LES APPROCHES DÉCLARATIVES

À notre connaissance, il existe peu de systèmes employant une approche déclarative pour la construction de grilles de mots croisés. Comme on pouvait s'y attendre, la première approche repose sur la programmation logique [Berghel, 1987, Yi and Berghel, 1989] avec une implémentation en PROLOG. Leur contribution est la création d'une méthode qui compile le problème correspondant à une grille donnée en une série de clauses de Horn. Wilson a quant à lui testé la programmation en nombres

entiers [Wilson, 1989] sans parvenir à des résultats convaincants. Ces deux approches sont arrivées aux résultats attendus : de bonnes performances pour de petites grilles, mais l'impossibilité de construire de grandes grilles avec un dictionnaire important.

1.3 NOTRE APPROCHE

Nous proposons ici une approche fondée sur la satisfaction de contraintes pour la construction de grilles de mots croisés, dont les cases noires sont fixées à l'avance. Cette approche semble prometteuse pour les raisons suivantes :

- 1) Nous avons remarqué que les meilleures approches procédurales, qui obtiennent de bonnes performances, reposent sur des principes assez similaires à ceux de la satisfaction de contraintes.
- 2) Les approches déclaratives sont peu efficaces, mais on sait que la satisfaction de contraintes offre, pour ce type de problèmes, des performances nettement supérieures à la programmation logique ou à la programmation en nombres entiers.
- 3) La satisfaction de contraintes est certes un paradigme déclaratif, mais nous avons vu que la définition des contraintes en utilisant des méthodes de filtrage spécifiques permet en quelque sorte d'introduire, là où c'est nécessaire, des connaissances procédurales.
- 4) Il est assez facile d'exprimer des connaissances sur le problème sous forme de contraintes, ce qui permet d'améliorer la résolution. Nous illustrerons ce point dans ce qui suit.

Ainsi, nous pensons pouvoir allier les avantages des approches procédurales existantes à la souplesse et l'efficacité de la satisfaction de contraintes.

A) LA REPRÉSENTATION DU PROBLÈME

Comme la plupart des approches précédentes, nous employons une représentation dans laquelle les inconnues sont les mots et non pas les lettres. Ainsi, le problème de contraintes que nous définissons aura pour chaque emplacement de mot une variable contrainte dont le domaine contient les mots du dictionnaire. En l'occurrence, le dictionnaire utilisé comprends de l'ordre de 150 000 mots en français, et 180 000 en anglais (à l'adresse <ftp://gatekeeper.dec.com/pub/micro/msdos/misc/crossword-archive> on trouve les fichiers ukacd14.zip et ucacd15.zip qui contiennent le UK Advanced Cryptic Dictionary dans ses versions 1.4 et 1.5). Pour chaque case blanche de la grille qui est à l'intersection de deux mots, nous définissons une simple contrainte qui assure que cette case aura une seule valeur. Autrement dit, deux variables contraintes qui correspondent à des emplacements de mots qui se croisent ont, à leur intersection, une lettre commune. Ceci assure que les solutions du problème sont des grilles valides (i.e. avec une et une seule lettre par case). Voici un premier énoncé en BACKTALK pour ce problème dans laquelle les variables contraintes correspondent aux emplacements de mots de la grille et les contraintes correspondent aux intersections entre mots :

```

fromCrosswordgrid: grid
  | allwordVars horizontalVars verticalVars |
  pbm := (self new) grid: grid.
  horizontalVars := OrderedCollection new.
  verticalVars := OrderedCollection new.
  grid listOfwordSlots do:
    [:word |
      wordVar := pbm newwordVariableFromword: word.
      word isvertical
        ifTrue: [verticalVars add: wordVar]
        ifFalse: [horizontalVars add: wordVar]].
  allwordVars := horizontalVars , verticalVars.
  horizontalVars do: [:hv | verticalVars do: [:vv | | int |
    (int := vv intersectionwith: hv) notNil
      ifTrue: [CrosswordIntersectionCt
        on: hv and: vv
        hPos: (int at: #hPos)
        vPos: (int at: #vPos)
        square: (int at: #square)]]].
  ^pbm variablesToInstantiate: allwordVars

```

B) LES CONNAISSANCES SUR LE PROBLÈME

Un des intérêts, mais pas le seul, de la construction de grilles de mots croisés est la richesse de connaissances qu'on peut y trouver. Ces connaissances concernent à la fois le dictionnaire, la grille et les stratégies de résolutions. Ces différentes connaissances dépendent étroitement les unes des autres, mais nous allons les présenter séparément.

Connaissances sur les contraintes d'intersection

Dans la représentation que nous avons adoptée, qui exclut toute représentation des lettres/cases individuellement, le problème a une topologie bien particulière. En effet, il est peu contraint au sens où chaque variable, qui représente un emplacement de mot, est contrainte uniquement avec les mots qui croisent. L'application de l'algorithme de base de BACKTALK, qui consiste à propager toutes les réductions de domaine, se révèle inutilement coûteuse. Ceci est dû à la nature particulière des contraintes d'intersection entre mots. Ces contraintes sont assez faibles, au sens où elles excluent assez peu de couples de valeur, ce qui rend inutile la propagation d'événements autres que l'instanciation d'une variable. Prenons un exemple : considérons deux variables *M* et *N* qui se coupent en une case commune *C*. Dès lors que la case *C* peut prendre comme valeur l'ensemble des lettres de l'alphabet, la contrainte associée est arc cohérente. Cette situation se produit chaque fois que la projection sur la case *C* des domaines de chacune des deux variables contient l'alphabet en entier. Ainsi, les événements autres que l'instanciation, c'est-à-dire ceux qui correspondent à la suppression d'un ou plusieurs mots du domaine, ne remettent en cause la cohérence de la contrainte que si de nombreux mots ont été supprimés. Prenons tout de suite un exemple avec une grille très simple :

25	25	24	L	24	24	24
26	24	24	U	24	22	18
26	26	26	E	26	23	25

Le mot LUE n'a que peu d'influence sur les domaines des variables correspondant aux mots qui lui sont parallèles. En effet, dans chaque case, nous avons reporté le nombre total de lettres de l'alphabet possibles après propagation complète. La première case de la grille affiche 25, ce qui signifie que dans le dictionnaire, la projection sur la première lettre de l'ensemble des mots de 7 lettres ayant un L en 4^{ème} position, contient 25 lettres (en l'occurrence le X seul est impossible). Tous les nombres du tableau sont proches de 26, qui est le maximum possible avec l'alphabet latin.

Le mécanisme de propagation de BACKTALK procède ainsi : après instantiation du mot LUE, il propage toutes les contraintes d'intersection impliquant LUE. Ainsi, les domaines des mots horizontaux sont réduits, ce qui active les démons de type domaine des variables horizontales. Si l'on propage ces démons, alors toutes les contraintes d'intersection du problème seront filtrées, ce qui est coûteux. C'est inutile dans la mesure où puisque les domaines suffisent à garantir que pour chaque case, il existe dans le dictionnaire des mots pour presque chaque lettre de l'alphabet, les domaines des autres mots horizontaux ne seront pas réduits.

Prenons par exemple le premier mot horizontal. Tout ce que l'on peut déduire par propagation de la contrainte entre le premier mot vertical et le premier mot horizontal est que les mots de trois lettres ayant un X en première position sont incompatibles avec le mot LUE. Cette déduction ne supprime aucun mot du domaine de cette variable qui en contient pourtant 1 140. En clair, aucun mot de trois lettres ne commence par un X dans notre dictionnaire.

Ainsi, il apparaît inutile de propager les réductions de domaines autres que l'instanciation d'une variable. Cette constatation plaide pour l'utilisation d'un algorithme de type forward-checking. L'implémentation en BACKTALK consiste simplement à définir une classe pour les contraintes d'intersection, dont seuls les démons correspondant à l'instanciation d'une variable sont actifs. Voici une première implémentation de cette contrainte :

```
BTBinaryCt subclass: #CrosswordIntersectionCt
  instanceVariableNames: 'hPos vPos square '
```

“hPos est l’indice de la case commune dans le mot horizontal, vPos dans le mot vertical. Square désigne la case commune”

postDemons

“rien de plus que les démons de type value”

issatisfied

```
^(self hword value at: hPos) = (self vword value at: vPos)
```

hValue

```
| commonChar toKeep |
commonChar := self hword value at: hPos.
toKeep := y select: [:w | (w at: vPos) = commonChar].
toKeep isEmpty ifTrue: [V domainwipedOut raise].
self vword newDomain: toKeep
```

vValue

```
| commonChar toKeep |
commonChar := self vword value at: vPos.
toKeep := x select: [:w | (w at: hPos) = commonChar].
toKeep isEmpty ifTrue: [V domainwipedOut raise].
self hword newDomain: toKeep
```

value: var

```
var == x
  ifTrue: [self hValue]
  ifFalse: [self vValue]
```

Dans le cas où le mot vertical est instancié, le filtrage consiste simplement à supprimer du domaine de la variable correspondant au mot horizontal les mots n’ayant pas de caractère commun avec le mot fixé.

Connaissances sur les mots du dictionnaire

Un dictionnaire est une simple liste de mots construits sur un alphabet donné. En général, on ne peut pas décrire un dictionnaire en compréhension. On peut le faire pour certains types de dictionnaires, par exemple l’ensemble de tous les mots possibles construits sur un alphabet donné, mais pas pour un dictionnaire français ou anglais. Cependant, on peut utiliser certaines propriétés des mots qui sont contenus dans le dictionnaire afin d’améliorer l’efficacité de la construction des grilles.

Nous ne ferons ici aucune remarque sur la structuration du dictionnaire car dans notre approche les mots du dictionnaire sont utilisés comme valeurs pour les domaines des variables contraintes. Ainsi, une fois la déclaration du problème achevée, il n’y a plus d’accès direct au dictionnaire et sa structure ne change rien à l’efficacité du système. Nous pouvons simplement noter que nous utilisons un dictionnaire de français contenant 135 000 mots de longueurs allant de 3 à 25 et un dictionnaire anglais de 180 000 mots.

A priori, le dictionnaire est une liste de mots, tous construits sur un alphabet donné, que l'on ne peut pas décrire en compréhension, mais seulement en extension. Cette remarque valant pour les dictionnaires correspondant à un langage naturel mais pas pour des dictionnaires moins "réalistes" comme la liste de tous les mots d'une taille donnée que l'on peut construire sur un alphabet donné. Cependant, même si l'on ne peut pas décrire l'ensemble du dictionnaire par un ensemble restreint de propriétés sur les mots qui le constituent, il est possible de dégager certaines propriétés intéressantes. Par exemple, on peut remarquer que dans le dictionnaire français, il n'existe aucun mot contenant une consonne triple, ni aucune voyelle à l'exception du E. De même, certaines consonnes ne sont jamais doubles, comme Q, J ou W. Ou encore, la lettre Q est systématiquement suivie d'un U, sauf dans les mots IRAQ, CINQ et COQ dans lesquels le Q est à la fin. De telles propriétés peuvent aider à éviter de commencer des grilles qui ne pourront jamais être achevées.

Ainsi, si l'on reconsidère l'exemple plus haut dans lequel on remplace le L de Lue par un Q, les valeurs de la première ligne deviennent :

15	10	11	Q	1	4	8
----	----	----	---	---	---	---

La valeur 1 dans la 5^{ème} case désigne évidemment la lettre U. On voit ici qu'il devient intéressant de propager plus en avant les contraintes. En effet, le mot vertical à droite de QUE devra nécessairement commencer par un U, ce qui réduit très sensiblement le domaine.

De façon générale, il est intéressant de propager les contraintes d'intersection correspondant à des cases ayant un petit "domaine". Pour ce faire, il suffit de déclencher les démons associés aux autres réductions de domaines, et à ne réellement procéder au filtrage que lorsque peu de lettres sont possibles pour la case concernée.

A priori, le filtrage de la contrainte d'intersection peut se faire en appliquant la méthode la plus simple : calculer tous les couples de mots, et retenir uniquement, pour chaque variable, les mots appartenant à un couple cohérent. Cette méthode présente l'inconvénient d'être quadratique. Il existe une méthode de filtrage plus efficace (linéaire). Cette méthode est basée sur le fait que l'intersection de deux mots est une propriété locale (elle ne concerne qu'une seule lettre). L'idée est simple : une contrainte d'intersection est cohérente par arc si les projections des domaines des deux variables sur la case commune sont identiques. Ceci peut se faire avec la méthode suivante qui requiert 6 parcours de domaines, ce qui est linéaire :

postDemos

“On ajoute les démons correspondant aux réductions quelconques du domaine”

```
x addDomainDemonOn: self.
```

```
y addDomainDemonOn: self
```

domain: var

```
| xCars yCars |
```

```
“xCars = projection de x sur la case commune”
```

```
xCars := x collect: [:w | w at: hPos].
```

```
“on supprime de y les mots qui ne peuvent pas croiser x”
```

```
y removeAllSuchThat:
```

```
[:w | (xCars includes: (w at: vPos)) not].
```

```
“on calcule la projection de y sur la case commune”
```

```
yCars := y collect: [:w | w at: vPos].
```

```
“on supprime de x les mots qui ne peuvent pas croiser y”
```

```
x removeAllSuchThat:
```

```
[:w | (yCars includes: (w at: hPos)) not]]].
```

```
“on fait un second passage, ce qui est suffisant pour  
assurer la cohérence d’arc”
```

```
xCars := x collect: [:w | w at: hPos].
```

```
y
```

```
removeAllSuchThat:
```

```
[:w | (xCars includes: (w at: vPos)) not].
```

Afin de prendre en considération la remarque que nous venons de faire sur les intersections, nous allons nous contenter de déclencher le filtrage dans le cas où **xCars** ou **yCars** est un petit ensemble. Ce cas signifie que l’intersection entre les deux mots n’admet qu’un petit nombre de lettres possibles (ici le seuil est fixé à 15 lettres). Voici la méthode de filtrage de la contrainte d’intersection :

domain: var

```
| xCars yCars |
"xCars = projection de x sur la case commune"
xCars := x collect: [:w | w at: hPos].

"on supprime de y les mots qui ne peuvent pas croiser x"
xCars size < 15 [y removeAllSuchThat:
[:w | (xCars includes: (w at: vPos)) not]].

"on calcule la projection de y sur la case commune"
yCars := y collect: [:w | w at: vPos].

"on supprime de x les mots qui ne peuvent pas croiser y"
yCars size < 15 [X removeAllSuchThat:
[:w | (yCars includes: (w at: hPos)) not]]].

"on fait un second passage"
xCars := x collect: [:w | w at: hPos].
xCars size < 15 [y removeAllSuchThat:
[:w | (xCars includes: (w at: vPos)) not]].
```

Une dernière optimisation consiste à prendre en considération la taille maximale de l'alphabet latin (26 lettres pour les mots croisés, puisqu'on ne considère pas les symboles diacritiques). Ainsi, on peut arrêter le parcours des domaines, pour le calcul des listes **xCars** et **yCars**, lorsque celles-ci atteignent la taille de 26.

Ce filtrage, qui ne réalise pas la cohérence d'arc de la contrainte, est cependant une représentation sous forme de contrainte de l'idée précédemment exposée que certaines lettres, qui impose des restrictions pour les lettres voisines, méritent d'être propagées.

Connaissances sur la résolution

Un des intérêts des mots croisés est que l'on peut visualiser en temps réel la construction de la grille. Cela permet d'avoir une impression précise de la manière dont se passe la résolution. Ainsi on peut avoir de bonnes intuitions sur les causes d'inefficacité de la résolution. Nous en soulignerons deux, que nous avons pu constater en observant de multiples résolutions. La première est une tendance du système à construire une partie de la grille puis à la détruire avant de la reconstruire à l'identique. C'est ce que nous appellerons l'*effet Pénélope* du nom de la femme mythique qui défaisait chaque nuit le voile qu'elle avait tissé dans la journée. Une seconde cause d'inefficacité que l'on peut facilement observer est le remplacement par le solveur d'un mot incompatible avec ses voisins par un mot quasiment identique. Ce comportement conduit à refaire plusieurs fois une exploration similaire. De plus, cela arrive fréquemment, puisque les mots sont considérés dans l'ordre lexicographique et sont donc choisis de proche en proche.

Dans notre contexte, la géométrie de la grille est connue à l'avance : on connaît sa dimension et l'emplacement de ses cases noires. Cette information est très importante puisqu'elle permet d'orienter la résolution afin d'éviter certaines causes de l'explosion combinatoire. Par exemple, si la grille est clairement séparée en plusieurs "composantes connexes" de cases blanches, séparées entre elles par des rangées de

cases noires, il est judicieux de séparer le problème en deux sous problèmes distincts afin d'éviter d'entrelacer la résolution des différents sous problèmes, ce qui provoque une exploration inutilement coûteuse que nous avons appelé l'*effet Pénélope*. En voici une illustration sur une grille simple :

M	A	L	C	O	N	N	U	E	S

La partie grisée et la partie blanche sont deux composantes connexes distinctes du problème. Cela signifie que l'instanciation d'un mot dans la partie blanche n'a pas d'influence sur les variables de la partie grisée et inversement. L'effet Pénélope est la pathologie qui consiste à alterner les instanciations de mots dans chacune des deux parties. Par exemple, si la résolution se poursuit avec les étapes suivantes :

- 1) Le système instancie un mot dans la partie blanche, disons BANAL au 3^{ème} emplacement vertical ;
- 2) Il poursuit en instanciant les mots de la partie grisée ;
- 3) Aucune solution n'est trouvée dans la partie grisée (aucun de ses emplacements n'est donc instancié) ;
- 4) Le système remet en question la valeur BANAL en la remplaçant par CANAL ;
- 5) Il recommence une résolution de la partie grisée etc.

La seconde résolution de la composante grisée de la grille conduira obligatoirement à un nouvel échec, puisque le changement de BANAL en CANAL n'a aucune influence sur cette partie de la grille. Il faut remarquer que la séparation de la grille en plusieurs composantes connexes peut intervenir durant la résolution au fil de l'instanciation des variables. Sur notre exemple, ce sont les mots MALCONNUES et NOIR qui ont provoqué cette séparation.

Pour tenir compte dynamiquement de la topologie du problème, nous employons un mécanisme de retour arrière intelligent, qui consiste simplement à interdire après chaque échec de retourner en arrière à une situation dans laquelle la composante connexe de la variable responsable de l'échec est inchangée. Sur une grille telle que celle qui suit, l'utilisation de ce mécanisme rend le problème facile à résoudre, alors qu'une résolution directe nécessite plusieurs heures de calcul. En effet, cette grille comportant de nombreuses cases noires, l'instanciation de quelques mots suffit à créer des composantes connexes distinctes, ce qu'il est nécessaire de prendre en considération pour éviter une résolution extrêmement longue.

S	C	U	B	A		A	B	E	A	M		S	C	E	N	A
E	R	N		S		R		D		I	D	Y	L	L		R
T	O	A	D	Y		R	A	D	O	N		N	A	S	I	K
A	C	C	O	M	M	O	D	A	T	I	V	E	N	E	S	S
		Q		P	A	W				B	A	R			O	
C	O	U	N	T	E	R	I	N	S	U	R	G	E	N	C	Y
A	L	A		O		O	B	O	E	S		I		A	H	A
S	E	I	S	M		O	O	N	T	S		S	E	R	O	W
C	I	N	E	A	S	T		B	A	E	D	E	K	E	R	S
O	C	T	E	T		S	M	E	E	S			I			
		E		I	C			L					S	N	E	A
A	S	D	I	C		H	A	I	F	A		A	G	A	I	N
K	O	N		A	B	U	S	E	R	S		L		R		O
A	R	E	A	L		L	A	V	I	S	H	M	E	N	T	S
B	A	S	I	L		M		E	L	E	G	I	T		A	P
A	S	S	A	Y		E	A	R	L	S		S	A	F	E	S

Cette solution a été trouvée en utilisant le dictionnaire d'anglais, en 1 minute et 62 échecs, sur un Pentium 300 MHz.

Une autre cause d'inefficacité, que nous avons mentionné au début de ce paragraphe, est le remplacement, après un échec, d'un mot par un mot trop similaire. Voici un exemple très simple qui illustre ce problème :

J	O	L	I	E

S'il se trouve que cette instanciation partielle conduit à un échec, le mot JOLIE sera remplacé. Vraisemblablement, si les mots sont rangés selon l'ordre lexicographique dans les domaines des variables, le prochain choix sera JOLIS. Or la dernière lettre du mot n'est pas importante puisqu'elle ne porte aucune contrainte. Pour supprimer ce phénomène, il suffit de modifier la sélection des mots en tenant compte de la topologie de la grille. En BACKTALK, nous pouvons par exemple modifier la méthode **nextValue** implémentée dans la classe des variables contenant des mots. Voici le code de la méthode modifiée :

nextValue

```
| val diffs tmp |  
"oldValue contient la dernière valeur connue pour la variable"  
val := domain first.  
oldValue == nil ifTrue: [^val].  
  
"L contient les cases sur lesquelles val et oldValue ont des  
lettres différentes"  
L := self listOfDifferentSquaresBetween: val and: oldValue .  
  
"Si au moins une case de L est une intersection, val est une  
valeur vraiment différente de oldValue. On la retourne"  
L detect: [:sq | sq isConstrained not] ifNone: [^val].  
  
"Sinon, on supprime val du domaine, et on recommence, en  
ayant préservé oldValue"  
tmp := oldValue.  
self size > 1 ifTrue: [self remove: val] ifFalse: [^val].  
oldValue := tmp.  
  
"On recommence"  
^self nextValue
```

En outre, aucune grille ne peut contenir le mot JOLIE ainsi placé si l'on utilise un dictionnaire français. En effet, aucun mot de la langue française ne finit par un J. L'instanciation de JOLIE va ainsi provoquer un échec, et ce mot sera remplacé (par JOLIS comme nous l'avons dit). Or, le problème ne vient en aucune façon de la dernière lettre E, mais de la lettre J. Une solution à ce problème est d'utiliser la gestion dynamique des contraintes de BACKTALK en créant après chaque échec dû à une lettre particulière d'un mot, comme le J de JOLIE, une nouvelle contrainte qui interdit de mettre à cet emplacement un mot ayant la même lettre au même endroit. Cette contrainte sera automatiquement supprimée du problème si la résolution revient à un état antérieur de la résolution. Dans notre exemple, les instanciations du dernier emplacement horizontal avec des mots commençant par un J seront toutes évitées.

C) L'ÉVALUATION DES RÉSULTATS

Voici quelques résultats obtenus avec notre système en comparaison avec ceux obtenus par d'autres chercheurs.

Le benchmark de Berghel et Rankin

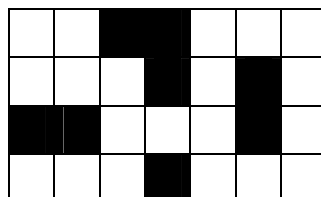
Nous avons testé le système en utilisant deux "benchmarks". L'un donné par Berghel et Rankin dans [Rankin and Berghel, 1990] propose de calculer toutes les grilles de dimensions 5×5 sans case noire, avec un dictionnaire comprenant les 134 mots suivants :

aaron	abase	abbas	abdal	abeam	abele	abner	abram
abrim	acana	acara	acate	acoin	addie	aegle	alada
alate	alban	allow	alosa	alula	amply	anana	anele
anent	ankle	araca	arain	aramu	arara	ardeb	areek
arena	arete	aroma	arulo	award	bacon	badon	baton
befit	benin	betis	blore	bosun	brace	brava	bravo
braze	breba	breme	breva	broma	brose	buret	cline
clite	crore	demit	denim	emend	enate	eurus	idiot
inert	itala	lader	laser	layer	manny	maral	marka
metal	monal	namer	nanny	nasal	natal	nathe	needs
neeld	neese	nenta	netty	newel	nodal	nonet	nonyl
notal	noter	obese	ochna	omina	onset	oraon	oriel
osela	ottar	ovile	ovoid	ovula	ozark	rabat	racon
ramed	range	raspy	rated	rater	rebud	rebut	recon
redan	refel	reges	renes	renky	repew	rerow	rerun
retan	riata	robin	rodge	roper	rotal	rudas	salal
salay	sandy	sayal	skeet	sowel	tates		

Il y a exactement 72 solutions à ce problème. Les temps d'exécution de notre système sur ce problème sont équivalents à ceux de Jensen ou de Berghel. Cette comparaison n'est pas précise, en raison de la différence des machines utilisées, mais l'ordre de grandeur est le même. Jensen met une seconde (entre 0,4 et 1,8 seconde) sur SUN SPARC Station 4 pour calculer les 72 solutions. L'approche de Berghel requiert 1 minute sur un PC AT à 10 MHz. Notre système demande environ 1,3 secondes pour trouver les 72 solutions sur une machine plus rapide (Pentium 300 MHz).

Le benchmark de Spring

Spring [Spring, et al., 1992] a proposé un autre benchmark pour tester les algorithmes de construction de grilles de mots croisés. Ils utilisent un dictionnaire contenant tous les mots possibles construits sur l'alphabet **{a, b}** de tailles 2, 3 et 4 et considèrent la grille suivante :

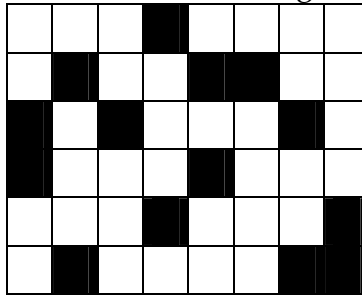


*Grille de mots croisés 1 La première grille de test de Spring. Le dictionnaire utilisé est la liste de tous les mots construisible sur l'alphabet **{a, b}**, il y a un million de solutions*

Dans ce cas, puisque toute suite de lettres sur cet alphabet est un mot du dictionnaire, n'importe quelle disposition possible des lettres **a** et **b** dans la grille est une solution. Il y a ainsi 2^{20} solutions, c'est-à-dire $1\,024 \times 1\,024$, soit plus d'un million de solutions. Le système de Jensen trouve toutes les solutions en un peu plus d'une **minute** sur SUN SPARC Station 4, alors que celui de Spring requiert environ le double de temps sur un PC à base de processeur 386 (fréquence inconnue). Notre approche est bien moins efficace sur ce problème.

Ceci s'explique aisément par le fait que beaucoup de travail de recherche de cohérence est mené, ce qui est ici inutile. Sur Pentium 300 MHz, il nous faut environ **13 minutes** pour calculer l'ensemble des solutions.

Spring a proposé un second benchmark : prendre comme dictionnaire les mots de 2 et 3 lettres construits sur l'alphabet {**a, b, c**} et les mots de 4 lettres construits sur l'alphabet {**d, e, f**}. la grille étant la suivante :



*Grille de mots croisés 2 La seconde grille de test de Spring. Le dictionnaire est la liste des mots de 2 et 3 lettres sur l'alphabet {**a, b, c**} et des mots de 4 lettres sur {**d, e, f**}. Il n'y a pas de solution à cette grille, ce qui est évident puisque les intersections entre mots de 3 et 4 lettres ne peuvent pas être occupées par une lettre commune*

Il n'y a aucune solution à ce problème, qui reste cependant facile. Jensen prouve l'absence de solution en 0,02 secondes en utilisant l'approche la plus mauvaise sur les grilles difficiles, et 141 secondes avec l'approche la meilleure sur les grilles difficiles. Spring trouvait ce résultat en 2 secondes sur PC 386. Nous mettons 0,035 secondes sur Pentium 300 MHz pour prouver que le problème est insatisfiable.

En revanche sur des problèmes difficiles, la tendance devient largement en notre faveur. La grille suivante par exemple n'a pas été résolue, ni par Berghel, ni par Jensen (en 24 heures de calculs). Nous l'avons résolue en moins de 40 secondes, en utilisant le même dictionnaire que Jensen (UK Advanced Cryptic Dictionary de 180 000 mots anglais). La propagation active des contraintes, en utilisant une représentation du problème permettant d'utiliser les connaissances que nous avons présentées ci-dessus, permet de trouver une solution de façon quasiment directe. Il n'y a en effet que 5 retours arrière provoqués. En utilisant le dictionnaire français de 150 000 mots, nous arrivons à un résultat similaire.

B	A	B	A		C	H	O	T	A		S	C	R	Y
A	B	I	B		H	O	G	A	N		T	H	A	E
L	A	M	A		E	L	A	N	D			I	G	A
A	C	E		B	R	E	M	E	R	H	A	V	E	N
S	A	S	H	A	Y	S			O	A	F			
		T	A	A	L		S	A	P	H	E	A	D	S
B	O	R	D	E		D	A	D	O		D	E	E	K
A	L	I		D	V	A	N	D	V	A		R	A	I
B	A	A	L		I	C	E	S		B	A	I	L	S
A	F	L	U	T	T	E	R		C	A	B	A		
			B	A	A			B	E	F	A	L	L	S
B	R	A	B	B	L	E	M	E	N	T		I	A	N
A	I	D	A		I	B	E	R	T		A	S	C	I
S	T	A	R		S	O	G	E	R		S	T	E	P
E	A	R	D		E	N	A	T	E		P	S	T	S

Grille de mots croisés 3 Cette grille n'a pas été résolue par l'approche de Sik Cambon Jensen en 24 heures (sur SPARC 4). Notre approche donne cette première solution, avec le même dictionnaire, en 40 secondes avec aucun retour arrière

I	M	A	M		S	A	B	M	E		C	A	P	A
N	O	M	A		A	B	R	I	N		A	B	I	B
C	O	P	S	E	W	O	O	D	S		P	E	L	A
A	C	H	O	N	D	R	O	P	L	A	S	T	I	C
S	H	I	N	N	E	D		O	A	H	U			
		B	I	E	R		B	I	V	A	L	E	N	T
B	A	R	C	A		S	A	N	E		E	L	E	A
A	B	A		D	R	O	S	T	D	Y		A	R	E
B	A	C	H		E	M	E	S		E	U	S	O	L
A	C	H	A	E	A	N	S		G	A	N	T		
			T	A	L	I		C	O	S	T	I	N	G
M	A	T	E	R	I	A	L	I	D	T	I	C	A	L
O	L	A	F		S	T	A	P	H	Y	L	I	N	E
R	A	H	U		T	E	M	P	E		E	S	N	E
A	N	A	L		S	D	E	I	N		D	E	A	D

Grille de mots croisés 4 Une grille plus difficile, constituée de la grille précédente, avec 6 cases noires en moins. Cette solution a été trouvée en 100 secondes et 64 échecs en utilisant le dictionnaire UK Advanced Cryptic Dictionary

C	H	A	P	A	R	R	A	L
A	E	R	O	B	E		C	E
C	C		S	A	P	P	E	D
T	A	T	T		R	O	T	A
A	T	H	E	R	O	M	A	
C	O	R	R	I	V	A	L	S
E	M	E	N	D	E	D		E
A	B	A		G	R	E	A	T
E	S	P	R	E	S	S	O	S

Figure 33 Une grille difficile car elle contient peu de cases noires (6 case noires pour une grille de taille 6×6). Solution trouvée avec 77 échecs en 1 minute et 20 secondes. Le dictionnaire est le UK Advanced Cryptic Dictionary

En conclusion, il faut remarquer que notre approche se montre assez peu efficace pour des problèmes faciles. Ceci est dû à deux facteurs. D’une part, nous n’avons pas de représentation efficace du dictionnaire, puisque nous stockons simplement les mots dans les domaines des variables, et que nous ne pouvons pas tirer parti de techniques sophistiquées telles que la compression du dictionnaire. D’autre part, nous réalisons systématiquement des propagations qui se révèlent inutilement coûteuses dans le cas de problèmes très simples. En revanche, sur des problèmes complexes, nous arrivons à des résultats que les approches existantes ne permettent pas d’obtenir. Ceci est un des avantages de l’utilisation d’un paradigme déclaratif dans lequel il est aisé de représenter des connaissances sur le problème.

D) VARIANTES DU PROBLÈME

Voici quelques grilles de mots croisées construites en ajoutant des contraintes au problème précédent. Nous allons montrer des grilles symétriques, en forme de palindrome ou encore thématiques. Ce style de grilles donne à une approche déclarative telle que la nôtre un avantage décisif sur les approches procédurales. En effet, définir un problème différent revient simplement à définir des contraintes supplémentaires et à relancer une résolution. Aucune modification essentielle de l’algorithme de résolution n’est nécessaire.

Les grilles symétriques

Une variante du problème est la construction de grilles symétriques, c’est-à-dire invariantes par transposition. Ceci peut se faire en ajoutant simplement des contraintes d’égalités entre mots verticaux et horizontaux. Voici deux grilles symétriques de taille 7×7 sans case noire.

A	B	C	E	D	E	R
B	E	R	C	E	R	A
C	R	O	U	P	I	S
E	C	U	M	A	G	E
D	E	P	A	R	E	R
E	R	I	G	E	R	A
R	A	S	E	R	A	S

F	A	B	L	I	E	R
A	M	I	A	N	T	E
B	I	E	N	T	O	T
L	A	N	C	E	U	R
I	N	T	E	G	R	A
E	T	O	U	R	D	I
R	E	T	R	A	I	T

La construction de la première des deux grilles a nécessité 25 secondes de calculs sur Pentium 300 MHz avec 398 échecs en utilisant le dictionnaire français. La seconde a été calculée en imposant la lettre F dans la première case (contrainte unaire sur le premier mot vertical).

Les grilles non rectangulaires

Voici deux grilles de forme non rectangulaire, qui sont en fait définies en dessinant une grille rectangulaire avec un “masque” de cases noires qui dessine un losange. La seconde grille est symétrique. Ces deux grilles sont des problèmes très faciles nécessitant moins d’une seconde de résolution.

			E					
			A	C	E			
		G	L	A	C	A		
	D	E	L	I	A	I	T	
M	O	R	A	L	I	S	E	E
	S	M	I	L	L	E	R	
		E	T	A	L	E		
			A	G	E			
			E					

			E					
			A	C	E			
		E	L	U	D	E		
	A	L	I	M	E	N	T	
E	C	U	M	A	N	T	E	S
	E	D	E	N	T	E	S	
		E	N	T	E	R		
			T	E	S			
			S					

Les grilles en forme de palindrome

Voici quelques grilles qui sont en même temps des palindromes à la fois dans le sens horizontal et dans le sens vertical. Pour obtenir de telles grilles il suffit de définir des contraintes supplémentaires du type : le premier mot horizontal est le renversement du dernier. La résolution de chacune des grilles suivantes est instantanée (i.e. moins d’une seconde). En effet, le problème est très contraint, ce qui le rend facile.

A	R	E	S
R	I	M	E
E	M	I	R
S	E	R	A

E	R	R	E
R	E	E	R
R	E	E	R
E	R	R	E

A	I	R	E	S
I		O		A
N	A	B	A	B
E		E		R
S	E	R	I	A

A	I	R	E	S
D		A		A
M	O	D	A	L
I		A		U
S	E	R	I	A

A	R	R	E	T
S		A		I
S	A	G	A	S
I		E		S
T	E	R	R	A

A	R	R	E	T
R	O	S	S	E
R	S		S	R
E	S	S	O	R
T	E	R	R	A

K	A	Y	A	K
A	V	I	V	A
Y	I		I	Y
A	V	I	V	A
K	A	Y	A	K

R	E	V	E	R
E	P	A	T	E
V	A		A	V
E	T	A	P	E
R	E	V	E	R

S	E	M	E	S
E	P	A	T	E
V	A		A	V
E	T	A	P	E
S	E	M	E	S

C	A	S	S	E	R
A		E	E		E
S	E	R	V	E	S
S	E	V	R	E	S
E		E	E		A
R	E	S	S	A	C

E	T	A	L	E	R
T		N	A		E
A	N	I	M	A	L
L	A	M	I	N	A
E		A	N		T
R	E	L	A	T	E

R	E	L	A	T	E
E		A	N		T
L	A	M	I	N	A
A	N	I	M	A	L
T		N	A		E
E	T	A	L	E	R

S	E	R	A	I	T
E		E	M		I
R	E	S	U	M	A
A	M	U	S	E	R
I		M	E		E
T	I	A	R	E	S

Les grilles thématiques

Un problème plus intéressant et aussi plus difficile est la construction de grilles thématiques. On souhaite que la grille construite contienne un nombre important de mots d'une liste donnée. Un thème est donc une liste de mots, appartenant ou non au dictionnaire. Voici par exemple un thème contenant des noms de compositeurs :

```

Albeniz albinoni bach bartok beethoven berg berio
berlioz bocherini borodin boulez brahms bruckner
buxtehude byrd cage chabrier chailley charpentier
chausson chimarosa chopin clementi couperin debussy
defalla delassus diego dowland duthilleux falla frank
fux gabrieli gesualdo gibbons glass glinka granados
grieg gurlit haendel haydn henze honegger jenkins
jolivet kramer kreneck kulhau lalo lassus leclair
lekeu ligeti liszt locke lully lutotavsky machaud
malher massenet mendelssohn milhaud mozart murail nono
nonpapa part pergolese poulenc prokofiev purcell
rameau ravel reger reich rodrigo rossini saintsaens
scarlati schoenberg schonberg schubert schumann
sibelius spohr stockausen strauss stravinsky takemitsu
tallis telemann villalobos vivaldi wagner weber webern
xenakis zemlinsky

```

La construction d'une grille thématique revient simplement à ajouter les mots du thème dans les domaines des variables et à ajouter une contrainte exprimant que beaucoup de mots doivent appartenir à ce thème. On peut faire ça en utilisant une contrainte de cardinalité du type :

$$|\{ v \in V \mid value(v) \in \text{thème} \}| \geq |V|/10$$

qui signifie qu'au moins 10% des mots de la grille sont des mots du thème. On peut aussi, résoudre le problème d'optimisation pour maximiser dans la grille le nombre de mots du thème. Évidemment, ce problème d'optimisation est plus difficile à résoudre. Voici deux grilles de mots croisés thématiques, construites avec le dictionnaire d'anglais et la liste des noms de compositeurs. Ces deux solutions ont été obtenues en exigeant respectivement 10 et 15 noms du thème. Le temps de résolution est d'environ 30 secondes pour 10 backtracks.

O	L	A	F		A	L	B	I	N	O	N	I		U
D	E	B	U	S	S	Y		K	E	T	O	N	E	S
D		I	X		C	O	H	A	B		C	A	S	E
E	B	B		D	E	N	A	T	U	R	A	N	T	S
R	A		B	I	N	S			L	E	K	E	U	
	C	L	I	E	D		A	B	I	D	E		A	B
B	H	A	N	G		N	A	Y	S		S	E	R	A
E		L		O	H	O		R	E	C		R	Y	A
R	H	O	S		E	N	I	D		L	A	Y		L
G	A		E	L	B	O	W		B	O	R	T	S	
	E	S	T	E	R			O	R	A	C	H	E	S
C	N	U	T		I	P	E	C	A	C		R	A	P
A	D	D	E	N	D	A		C	H	A	M	I	S	O
G	E		E	N	E	R	G	U	M	E	N	S		H
E	L	D	S		S	T	A	R	S		A	M	I	R

Grille de mots croisés 5 Une grille de mots croisés thématiques. La grille comporte 15 noms de compositeurs, qui sont issu de la liste présentée plus haut. La résolution a duré 30 secondes pour 10 échecs. Ceci n'est pas une solution du problème d'optimisation, mais une solution du problème de décision avec la contrainte de cardinalité: "au moins 10 mots de la grille sont des noms de compositeurs". Le problème d'optimisation est bien plus difficile à résoudre, et nous n'avons pas eu de solution en 1 heure de recherche

1.4 LA VALEUR DE L'APPROCHE PAR CONTRAINTES

Nous avons présenté dans ce chapitre une approche nouvelle pour la construction de grilles de mots croisés. Ce n'est pas la première tentative d'utilisation d'un paradigme déclaratif pour la résolution de ce problème combinatoire. Cependant, les approches déclaratives précédentes étaient peu performantes et ne permettaient pas de construire des grilles de taille réaliste sur un dictionnaire de plus de 100 000 mots.

Notre approche consiste à utiliser les techniques de satisfaction de contraintes pour résoudre ce problème. Nous avons montré comment ce paradigme permet d'exprimer des connaissances procédurales et déclaratives sur le problème à résoudre. Ces connaissances, qui portent à la fois sur la topologie de la grille, sur la répartition des lettres dans les mots du dictionnaire et sur la gestion des contraintes d'intersection entre les mots, permettent d'améliorer de façon spectaculaire l'efficacité de la résolution. Ainsi, nous avons pu construire des grilles réputées difficiles dans un temps de recherche raisonnable (de l'ordre de la minute) et avec une exploration limitée (aucun échec pour la Grille de mots croisés 3 qui n'a pas été résolue par la meilleure approche procédurale existante). Nous rejoignons ainsi l'opinion exprimée par Jean-Louis Laurière [Laurière, 1976] selon qui l'utilisation d'un mécanisme de résolution général et déclaratif ne conduit pas nécessairement à de mauvaises performances.

Chapitre 2

L'harmonisation musicale automatique

L'ordinateur s'impose aujourd'hui comme un outil incontournable dans tous les aspects de la création musicale. Dans *the computer music tutorial* [Roads, 1995], Curtis Roads mentionne de nombreux domaines comme la production de sons, le mixage, l'analyse sonore, la notation musicale, les divers aspects de la "performance" comme l'accompagnement, l'interprétation ou l'improvisation, la composition ou encore la psycho-acoustique. À ces aspects, nous pouvons ajouter l'analyse, la représentation de concepts, la construction de séquences de morceaux ou encore la musicologie. La composition est un des objectifs majeurs de l'informatique musicale car elle représente, à l'instar de la compréhension du langage naturel, du jeu d'échec ou de la résolution de problèmes, un défi particulièrement difficile pour l'informatique en général et l'intelligence artificielle en particulier.

On peut distinguer plusieurs types de systèmes de composition musicale en fonction de leur nature (e.g. langages de programmation, systèmes interactifs, systèmes fermés) et de leur domaine d'utilisation (e.g. aide à la composition, composition automatique ou algorithmique). Certains systèmes d'aide à la composition sont destinés au compositeur et ont pour objet de lui éviter certaines tâches pénibles ou répétitives de la composition. Mais ils sont aussi conçus dans le but d'offrir au compositeur des possibilités nouvelles permises par les capacités de la machine. L'ordinateur permet de créer et d'organiser des sons nouveaux, et surtout, d'explorer des espaces de paramètres de très grande taille afin d'y trouver des solutions intéressantes. Les systèmes de composition automatique ou algorithmique ne sont, en revanche, pas nécessairement destinés au compositeur, même si Philippe Ballesta considère explicitement son système d'harmonisation automatique comme un outil d'aide à la composition. Ils sont le plus souvent des exercices de style, visant à illustrer ou à faire évoluer une technique informatique particulière, comme la programmation logique ou avec contraintes [Ebcioglu, 1992a, Ebcioglu, 1992b, Ovans, 1992, Ovans and Davison, 1992, Tsang and Aitken, 1991], les réseaux de neurones [Hörnel and Dengenhart, 1997], les grammaires [Barbar, et al., 1998, Roads, 1995] ou bien à mettre en œuvre certaines connaissances musicologiques [Ebcioglu, 1992a]. Le système d'harmonisation automatique que nous présentons dans cette partie n'est, au moins dans son état actuel, pas destiné au compositeur. Nous y voyons plutôt

une application particulièrement intéressante de l'intégration de la satisfaction de contraintes avec les objets.

Le problème auquel nous nous intéressons est la réalisation d'exercices d'harmonie tonale. La réalisation d'un exercice d'harmonie vise à produire un morceau à partir d'un matériau musical incomplet. Les deux types d'exercices d'harmonie que l'on résout traditionnellement, et qui diffèrent par le matériau donné initialement, sont l'harmonisation d'une ligne de basse chiffrée et l'harmonisation d'une mélodie sans information supplémentaire. La résolution, on parle souvent de *réalisation*, d'un tel exercice consiste à écrire des voix supplémentaires, qui seront jouées en même temps que la voix initiale. La difficulté et l'intérêt de l'exercice est que les mélodies supplémentaires sont soumises à des règles qui visent à garantir à la fois leur cohérence individuelle et la cohérence de leur exécution simultanée. D'une part, les notes jouées simultanément doivent produire ce que l'on appelle un *accord*, c'est-à-dire une masse sonore "harmonieuse". D'autre part, les transitions entre notes et accords successifs doivent respecter certains principes. Si l'on se réfère à la notation musicale traditionnelle, on peut dire que les premières règles sont verticales et les secondes horizontales. L'essence de l'harmonie est de concilier à tout instant ces deux aspects orthogonaux desquels naît la combinatoire.



Figure 34 Une harmonisation à quatre voix de "La marseillaise". Cette harmonisation a été obtenue par notre application en 0,5 secondes et 30 échecs sur Pentium 300 MHz

Le problème précis auquel on s'intéresse ici est la réalisation d'exercices d'harmonie tonale non modulante à quatre voix, sans emprunt de notes étrangères à la tonalité, même de façon passagère, qui intéresse les informaticiens de puis plus de trente ans. Les techniques de résolution les plus variées ont été utilisées pour le résoudre avec des fortunes diverses. Lejaren Hiller fut le précurseur et utilisa des techniques aléatoires associées à une méthode *Generate-and-Test* pour résoudre un problème similaire de réalisation de contrepoint [Hiller and Isaacson, 1959]. Pierre Barbaud utilisa des chaînes de Markov et des automates finis. Plus proche de nous, Kemal Ebcioglu a implémenté le système CHORAL de composition de chorales dans le style de Jean-Sébastien Bach. CHORAL est un système expert à base de règles de production, implémenté dans un langage de programmation logique (BSL) qui utilise un mécanisme de retour arrière intelligent. Les règles musicales de CHORAL sont représentées comme des contraintes mais manipulées de façon passive par BSL, ce qui le rend très inefficace. En revanche, un mécanisme d'alternance entre six points de vue différents sur la composition assure une qualité musicale remarquable des solutions. À côté de ces systèmes académiques, on trouve aujourd'hui, par exemple sur Internet, de nombreux logiciels qui sont capables d'harmoniser des lignes de basse chiffrée, des mélodies ou de faire de l'accompagnement. Le système *tonica*, que l'on peut trouver sur Internet à l'adresse www.softpart.co.uk, est capable de réaliser des chorals de qualité intéressante dans le style de

Jean Sébastien Bach et de Max Reger. Ce programme repose sur des réseaux de neurones auxquels on a appris un corpus d'œuvre de chacun des compositeurs. Un système très proche est Harmonet de Hörnel et Dengenhart [Hörnel and Dengenhart, 1997].

L'avènement de la programmation par contraintes a donné naissance à de nombreux systèmes d'harmonisation automatique basés sur les techniques de satisfaction. La première tentative d'utiliser des techniques de contraintes pour l'harmonisation automatique est CHORAL de Kemal Ebcioglu en 1988. Cependant, le langage d'implémentation de CHORAL, BSL, est un langage de logique qui gère les contraintes de façon passive. Tsang et Aitken [Tsang and Aitken, 1991] sont les premiers à avoir, utilisé un véritable langage de programmation logique avec contraintes. Leur système comprend une trentaine de règles musicales et résout le problème de l'harmonisation d'un chant donné. Les solutions du système sont de bonne qualité, mais les *performances sont très insatisfaisantes* (i.e. plusieurs minutes et plus de 70 Mega-octets de mémoire sont nécessaires à l'harmonisation d'un chant d'une dizaine de notes). Russel Ovans a réalisé un système d'harmonisation de mélodies à deux voix, lui aussi implémenté dans un langage de programmation logique avec contraintes, dont les performances sont bonnes, mais peu significatives compte tenu de la simplicité du problème résolu. Enfin, Philippe Ballesta, a réalisé un système d'harmonisation de lignes de basse chiffrée qui combine à la fois les techniques de satisfaction de contraintes et la programmation par objets. Ce système est implémenté en PECOS, la version LISP de ILOGSOLVER. Ce système est le plus proche de notre travail, mais aussi le plus intéressant puisqu'il comprend une représentation explicite des structures musicales par des objets, un peu dans la lignée des travaux de Francis Courtot avec son système CARLA [Courtot, 1992]. Cependant, selon la terminologie introduite dans la partie précédente, cette implémentation suit l'approche par attributs, qui permet une représentation riche et dynamique des structures musicales présente plusieurs défauts. Elle rend ardue la déclaration des règles musicales sous forme de contraintes, empêche la réutilisation de structures objets prédéfinies et grève les performances du système. Nous illustrerons soigneusement ces trois points à l'aide d'exemples. De nombreux autres systèmes existent, parmi lesquels celui de Jean-Louis Guénégau à l'IRCAM, implémenté dans le langage de programmation visuel OPENMUSIC [Agon, et al., 1998] à l'aide du solveur de contraintes SITUATION de Camilo Rueda et Antoine Bonnet [Bonnet and Rueda, 1993, Bonnet and Rueda, 1998]. On citera également la reconstruction partielle de CHORAL de Kemal Ebcioglu, avec le système ECLIPSE de programmation par contraintes, par Pierre Dellacherie [Dellacherie, 1998]. Cette nouvelle réalisation de CHORAL a pour objectif d'évaluer l'apport des outils modernes de satisfaction de contraintes, fondées sur les techniques de cohérences d'arc, par rapport aux techniques de programmation logique avec retour arrière intelligent utilisées à l'époque par Ebcioglu. Les résultats sont pour l'instant décevants, ce qui est dû à une représentation très inadaptée du problème.

2.1 LE SYSTÈME DE PHILIPPE BALLESTA

Nous avons donné une description sommaire des principaux systèmes d'harmonisation automatique implémentés à ce jour. Tous ont représenté l'avènement d'idées ou de pratiques nouvelles pour ce problème, et sont intéressants à ce titre. Cependant, nous nous contenterons de décrire dans les détails celui de Philippe Ballesta, qui représente une solution pertinente au problème qui nous intéresse dans ce document : *comment définir résoudre des problèmes de satisfaction de contraintes dans lesquels les inconnues sont des objets complexes ?* Dans le système de Philippe Ballesta les contraintes servent à la fois à la définition des règles d'harmonie, mais aussi à la définition des structures musicales manipulées comme les notes ou les accords.

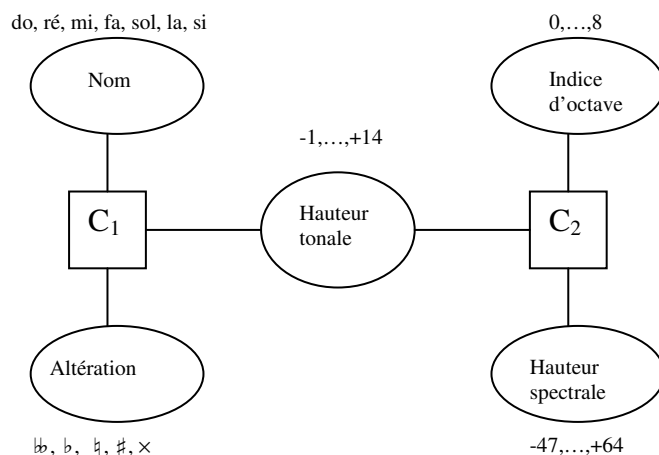


Figure 35 La structure d'une note dans le système de Philippe Ballesta. Les contraintes C_1 et C_2 sont des contraintes structurelles qui garantissent la cohérence entre les valeurs des différents attributs constituant la note

Donnons tout de suite la définition d'une *note* dans son système. Une note est un objet, instance de la classe note, et constitué des six attributs suivants : un *nom* choisi parmi *do*, *ré*, *mi*, *fa*, *sol*, *la* ou *si*. L'*altération* qui caractérise la modification de hauteur d'une note par rapport à la note naturelle portant le même nom. On compte cinq altérations possibles qui sont : le *bécarre* ($\natural$), le *bémol* (b), le *dièse* ($\sharp$), le *double bémol* (bb) et le *double dièse* (x) ; l'*indice d'octave* qui permet, avec le nom et l'altération, de déterminer la hauteur exacte (la fréquence) d'une note ; la *hauteur spectrale*, qui est un nombre caractérisant la fréquence de la note dans une échelle entière, allant de -47 à $+64$, et dans laquelle le 0 correspond au *La 3* dont la fréquence est 440 Hertz ; la *hauteur tonale* qui est la combinaison du nom de la note avec son altération, et qui caractérise sa hauteur spectrale, modulo l'indice d'octave (les valeurs vont de -1 pour *do double bémol* à $+14$ pour *si double dièse*) ; enfin, le *degré*, fort important en harmonisation, qui est l'indice de la note dans la tonalité à laquelle elle appartient (e.g. en *do* majeur, *do* a pour degré 1 et *sol* a pour degré 5). La Figure 35 montre la structure d'une note dans cette représentation.

Cette représentation d'une note, qui peut sembler excessivement complexe et redondante, permet en fait une richesse expressive intéressante, notamment par l'usage de contraintes. Plus précisément, on peut remarquer que pour déterminer une note, il n'est pas nécessaire de connaître la valeur de chacun de ses attributs. Par exemple, on peut déduire le nom d'une note si l'on connaît la hauteur tonale et l'altération (e.g. si hauteur tonale est 4 et altération est *dièse* alors le nom est *ré*). Plus généralement, la connaissance de la valeur de deux attributs parmi la hauteur tonale, le nom et l'altération, permet de déduire la valeur du troisième. Bien évidemment ce type de relation entre les différents attributs d'un objet sont représentées idéalement par des contraintes. L'intérêt de ces *contraintes structurelles* est qu'elles permettent de construire les notes à partir d'informations partielles. Par exemple, lors de la résolution d'un problème, la découverte du nom et de l'altération d'une note permet de déterminer sa hauteur tonale par propagation de la contrainte liant les trois attributs. La hauteur tonale peut alors à son tour être utilisée par d'autres contraintes pour trouver d'autres informations sur le problème.

La représentation des intervalles dans ce système participe de la même démarche. Un intervalle est défini par : un *nom* (e.g. *seconde*, *neuvième*, *soixantième*) ; un *nom simple*, qui est le nom modulo une octave ; la *complexité*, qui caractérise le nombre d'octave séparant les extrémités de l'intervalle (la complexité permet de retrouver le nom à partir du nom simple) ;

la *proximité* qui indique si les extrémités de l'intervalle sont des notes conjointes ou non ; la *qualification*, qui permet de distinguer des intervalles différents ayant le même nom (e.g. tierce *majeure* ou *mineure*) ; l'*étendue absolue* et l'*étendue*, qui mesurent la distance entre les deux notes de l'intervalle ; le *sens* et le *sens absolu*, qui permettent de distinguer les intervalles ascendants et descendants (les paramètres absolus se réfèrent aux hauteurs spectrales, alors que les paramètres non absolus se réfèrent aux noms) ; deux attributs supplémentaires *note-1* et *note-2* qui désignent les deux notes constituant l'intervalle.

Cependant, cette représentation présente trois inconvénients majeurs qui concernent à la fois la définition du problème et sa résolution. D'une part, elle est coûteuse, puisque tous les objets musicaux, même les plus simples, comportent des variables et des contraintes. Le Tableau 9 donne la taille de chaque objet du système et permet d'avoir une idée précise du coût de la représentation. Pour un exercice comportant n notes à la basse, le problème de contraintes à résoudre comportera $126 \times n - 28$ variables. Ainsi, pour un exercice de 16 accords, il y aura près de 2.000 variables. En conséquence la résolution prends plusieurs minutes en utilisant ILOGSOLVER qui est pourtant l'un des systèmes de satisfaction de contraintes les plus rapides disponibles. En outre, les variables symboliques utilisées dans cette représentation ne sont pas, dans ce système, exploitables directement pour la définition des contraintes, ce qui conduit à créer, pour chaque variable symbolique, et pour chaque contrainte impliquant cette variable, une variable numérique équivalente. La variable symbolique est reliée à chaque variable numérique équivalente par une contrainte **ct-element** assurant la cohérence entre leurs domaines respectifs. On a ainsi, une représentation sensiblement plus complexe, mais qui permet de bénéficier à la fois du confort expressif des variables symboliques et de la possibilité de définir les contraintes sur des variables numériques. Philippe Ballesta a estimé que cette représentation redondante des variables symboliques augmente de 20% le temps de résolution. Il propose une amélioration qui consiste à ne déclarer, pour chaque variable symbolique qu'une seule variable numérique, qui sera commune à toutes les contraintes. Ce problème est illustré par la Figure 36.

Objet	Variables	Notes	Intervalles	Transitions	Accords	Tonalités	Total
Note	6						6
Intervalle	9						9
Transition			4				36
Accord	3	4	7				90
Tonalité	2	1					8
Exercice		5	7	0	1	1	98
		13	29	2	3	1	350
		33	84	7	8	1	980
		65	172	15	16	1	1.988
		$4 \times n + 1$	$7 \times n + 4 \times (n - 1)$	$n - 1$	n	1	$126 \times n - 28$

Tableau 9 Le coût de la représentation dans le système de Philippe Ballesta

D'autre part, la représentation des objets à l'aide de contraintes structurelles est spécifique au sens ou elle dépend du problème qu'on souhaite résoudre. Plus précisément, on définira les objets notes de façons radicalement différentes selon que l'on souhaite faire de l'édition de partition ou bien résoudre des exercices d'harmonie. Il en résulte l'impossibilité, en suivant cette approche, de réutiliser pour un problème donné des classes d'objets définies dans un autre but. La nécessité de déclarer une variable numérique pour chaque variable symbolique illustre bien la spécificité des objets manipulés.

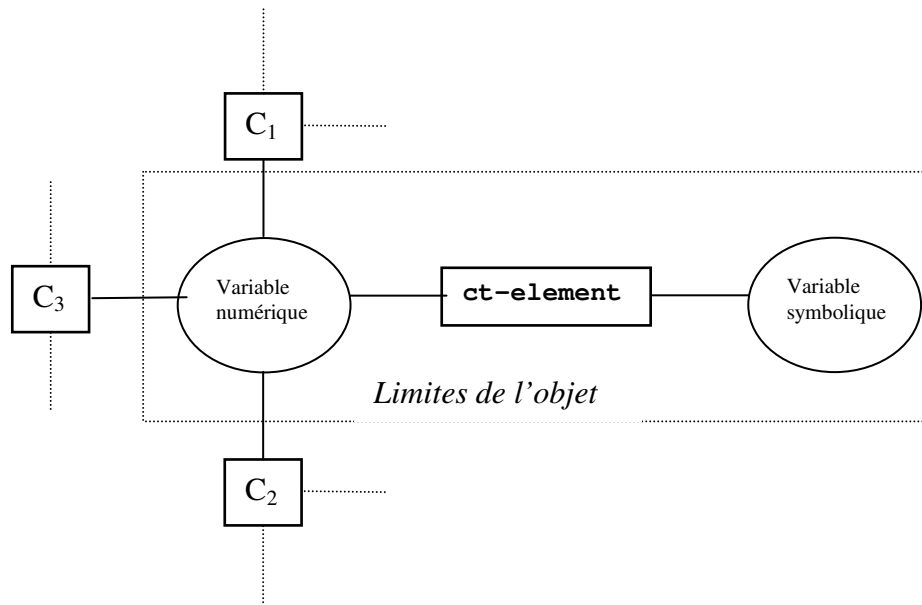
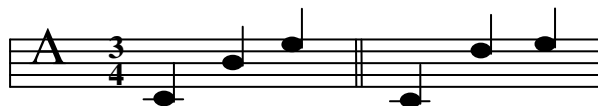


Figure 36 La déclaration d'une variable numérique pour chaque variable symbolique. Les deux variables sont reliées entre elles par une contrainte de type **ct-element** qui assure qu'elles sont équivalentes

Finalement, la représentation des objets musicaux telle qu'elle a été choisie par Philippe Ballesta conditionne la définition des contraintes musicales, i.e. non structurales, pour deux raisons : d'une part, les relations exprimées par les contraintes sont déclarées entre les attributs des objets musicaux et, d'autre part, pour des raisons d'efficacité dues au coût de la représentation, les contraintes doivent être implémentées de façon *ad hoc*. Plus précisément, la structure des objets musicaux conditionne la manière dont on exprime les contraintes. Voici l'exemple de la règle des neuvièmes franchies en deux sauts. Cette règle, valable pour les quatre voix, dit que si trois notes successives sont telles que la première et la troisième forment un intervalle de neuvième, alors la deuxième note doit former un intervalle de seconde avec la première ou avec la dernière note. Dans ce système, cette règle est définie par une contrainte portant sur les variables correspondant aux noms et les sens des deux intervalles consécutifs. Il est clair qu'une représentation différente des objets intervalles conduirait à une expression différente des contraintes.



Règle musicale 1 La neuvième (et la septième) franchies en deux sauts doivent comporter un intervalle de seconde. La solution de gauche n'est pas admise, car do-mi est une neuvième et ni do-si ni si-mi ne sont des secondes. En revanche, la solution de droite est correcte car ré-mi est une seconde

Mais la difficulté majeure concerne l'implémentation proprement dite des méthodes de filtrages des contraintes. Le coût excessif de la représentation utilisant des contraintes structurales rend le problème extrêmement difficile à résoudre par les techniques de résolution par défaut d'ILOGSOLVER. Il est nécessaire de définir des méthodes de filtrages spécifiques à chaque contrainte, de façon à ce que le filtrage des contraintes soit suffisamment rapide pour compenser leur nombre. Ces méthodes de filtrages sont implémentées en ILOGSOLVER par des schémas, qui sont des équivalents des méthodes de filtrages associées aux démons dans

BACKTALK. Ainsi, chaque contrainte est implémentée par une collection de schémas, correspondant aux différents événements susceptibles de réduire les domaines de ses variables. Cette approche garantit de bonnes performances pour le filtrage de chaque contrainte, mais en rend l'implémentation délicate. Pour définir de nouvelles règles musicales dans ce système, il est nécessaire pour l'utilisateur de connaître à la fois la structure interne des objets musicaux et le fonctionnement d'ILOGSOLVER. Voici, par exemple, la définition complète d'une contrainte simple : celle qui concerne la neuvième franchise en deux sauts.

```
(de ct-7-9-deux-sauts (nom-simple-i1 sens-i1 nom-simple-i2 sens-i2)
  ; liaison entre variables contraintes numériques et symboliques
  (ct-element rang-nom-simple-i1 nom-simple-i1 liste-noms-simples)
  (ct-element rang-nom-simple-i2 nom-simple-i2 liste-noms-simples)
  (ct-7-9-deux-sauts-aux
    rang-nom-simple-i1
    sens-i1
    rang-nom-simple-i2
    sens-i2))

(defctconstraint ct-7-9-deux-sauts-aux
  (
    (rang-i1 ct-fix-var)
    (sens-i1 ct-fix-var)
    (rang-i2 ct-fix-var)
    (sens-i2 ct-fix-var))

schemas
(
  (when      (rang-i1 = ri1) (rang-i2 = ri2) (sens-i1 = si1)
    test (and (eqn (add ri1 ri2) 6)
      ; quand l'intervalle obtenu par somme de i1 et i2 est une septième...
      (neqn ri1 1)
      (neqn ri2 1)
      ; et que ni i1, ni i2 ne sont des secondes...
      (neqn ri1 0)
      (neqn ri2 0)
      ; ni des unissons...
      assert (sens-i2 <> si1))
      ; alors le sens de i2 doit être différent de celui de i1.
      ; schéma similaire pour la neuvième, puis schémas inverses.
      ...
  ))
```

Contrainte 1 La contrainte correspondant à la règle de la septième et de la neuvième franchises en deux sauts, exprimée dans le système de Philippe Ballesta

2.2 LA REPRÉSENTATION DES OBJETS MUSICAUX EN MUSES

Nous avons souligné trois problèmes du système de Philippe Ballesta qui sont l'impossibilité de réutiliser des classes prédéfinies pour la définition des objets musicaux, la difficulté d'expression des contraintes d'harmonie à cause de leur dépendance avec la struc-

ture des objets et enfin la faiblesse des performances. Ces remarques nous ont conduit à employer l'approche par classes sur le problème de la résolution d'exercices d'harmonisation. Dans cette approche, on n'utilise pas de contraintes structurelles, la cohérence des attributs des objets étant assurée par leur classe dès leur création. La différence la plus visible entre les deux approches est que l'approche par classe permet de limiter de façon spectaculaire le nombre de variables contraintes du problème, car au lieu de représenter les objets musicaux comme des collections de variables contraintes, elles sont représentées par des objets en bonne et due forme. Avant de détailler notre approche pour la définition et la résolution des exercices d'harmonie, nous allons présenter MusES [Pachet, 1994], une bibliothèque SMALL-TALK de classes d'objets musicaux, dont l'objet est la représentation des connaissances nécessaires à l'analyse d'accords en musique tonale. MusES comprend une centaine de classes représentant les concepts principaux de la musique tonale : les notes, les intervalles, les accords, les gammes et les mélodies, et dispose en outre de modules permettant d'éditer et de jouer des partitions.

Il existe en MusES cinq classes concrètes de notes, i.e. **NaturalNote**, **SharpNote**, **DoubleSharpNote**, **FlatNote** et **DoubleFlatNote**, correspondant aux différents types de notes utilisées par l'analyse tonale. La classe **NaturalNote** a sept instances, qui sont *do*, *ré*, *mi*, *fa*, *sol*, *la* et *si*. Les quatre dernières classes, qui représentent les notes altérées sont sous classes d'une classe abstraite commune **AlteredNote**. **NaturalNote** et **AlteredNote** ont une super-classe abstraite commune appelée **Note**. Les méthodes correspondant aux opérations traditionnelles sur les notes, comme leur altération, sont implémentées différemment dans chacune des classes de la hiérarchie, ce que permet le polymorphisme des langages à objets. Voici quelques exemples d'expressions SMALL-TALK permettant de construire des notes.

Note C	-> C
Note C sharp	-> C#
Note C sharp sharp flat	-> C#
Note C flat flat flat	-> flat not understood by DoubleFlatNote

La représentation des intervalles en MusES est très différente de celle de Philippe Ballesta. Un intervalle est une instance de la classe **Interval**, ayant deux attributs, un **type** et un nombre de demi-tons, appelé **semiTones**. Une première différence avec le système de représentation de Philippe Ballesta est qu'un intervalle de MusES n'est associé à aucune note. En outre, on ne peut pas faire avec les intervalles de MusES toutes les opérations que le système de Philippe Ballesta permet de faire. Par exemple, dans le système de Philippe Ballesta, il est possible de calculer les expressions suivantes :

- Quels sont tous les intervalles possibles entre une note dont le nom est *do*, et une note dont le nom est *sol*, les deux notes ayant le même indice d'octave ?
- Quels sont tous les couples de notes d'indice d'octave 3 définissant une quinte juste ascendante ?

Les intervalles de MusES ne permettent pas d'effectuer ce genre de calcul. Pour obtenir les mêmes résultats, il est nécessaire d'écrire un algorithme en SMALLTALK qui fasse le travail. Les seules opérations que les intervalles de MusES permettent de faire sont les trois opérations suivantes :

- Connaissant un intervalle et la note à son origine (resp. extrémité), on peut calculer la note qui est à son extrémité (resp. origine).
- Calculer un intervalle dont on connaît la note d'origine et la note extrême.

3. On peut calculer la somme de deux intervalles et le renversement d'un intervalle.

Voici une session d'utilisation de MusES qui illustre chacune des trois opérations sur les intervalles que nous avons décrites ci-dessus. Le mot connu est écrit en gras pour insister sur le fait que MusES ne permet pas directement de manipuler de l'information partielle (i.e. des objets partiellement instanciés), au contraire du système de Philippe Ballesta. Il s'agit ici de programmation par objets standard, et non de programmation par contraintes et objets :

"1. Calcul de la note obtenue par translation d'une note **connue** par un intervalle **connu**"

```
Note C flatFifth -> Gb
Note C augmentedFourth -> F#
Note C majorThird majorThird -> G#
Note C flat minorSeventh -> Bbb
Note C flat diminishedSeventh -> error: illegal interval
```

```
Interval diminishedFifth bottomIfTopIs: (Note F sharp) -> C
Interval diminishedFifth bottomIfTopIs: (Note G flat) -> Db
```

"2. Calcul de l'intervalle entre deux notes **connues**"

```
Note C intervalwith: Note F sharp -> augmented fourth
Note C sharp intervalwith: Note G -> diminished fifth
```

"3. Somme et renversement d'intervalles **connus**"

```
Interval perfectFifth + Interval majorSecond -> majorSixth
Interval majorThird reverse -> minor sixth
```

La relative pauvreté du modèle de représentation des objets musicaux de MusES n'est pas un réel handicap, dans la mesure où d'une part, il est toujours possible, et facile, d'ajouter des fonctionnalités nouvelles. Par exemple, l'écriture d'une nouvelle méthode dans la classe des intervalles pour chacun des deux points a. et b. plus haut, serait très facile. D'autre part, la conception de MusES a été menée avec l'objectif de représenter les connaissances "consensuelles" de la musique tonale occidentale pour permettre l'élaboration d'applications "à tendance analytique" selon les termes de son auteur. Ainsi, il est suffisant de permettre les trois types de manipulation d'intervalles exposés ci-dessus, sachant que ces opérations sont les seules qui "ont du sens" dans ce contexte. La gestion du problème des notes enharmoniques (e.g. *ré#* et *mi♭*) est un autre exemple de la différence de philosophie entre les deux systèmes. Les deux approches permettent de gérer parfaitement ce problème délicat, mais si dans l'approche de Ballesta, il est possible de calculer toutes les notes enharmoniques d'une note donnée, cela n'est pas possible directement dans MusES. La philosophie sous-jacente à MusES est qu'une telle requête est inutile dans son contexte d'utilisation.

2.3 L'HARMONISATION AUTOMATIQUE AVEC MUSES ET BACKTALK : L'APPROCHE PAR CLASSES

Nous avons souligné les défauts majeurs des approches existantes pour la résolution d'exercices d'harmonie avec des techniques de contraintes. En outre, nous avons présenté une méthodologie pour définir les problèmes de contraintes et objets, *l'approche par classes*, qui permet de s'affranchir de ces difficultés. MusES et BACKTALK, présentés précédemment, sont

les briques de bases nécessaires à la mise en œuvre de cette approche pour le problème de l'harmonisation automatique [Pachet and Roy, 1995b].

Le choix de l'approche par classes pour le problème de l'harmonisation automatique se justifie par la nature des règles musicales exposées dans les traités. Comme nous l'avons déjà écrit, il existe à la fois des règles mélodiques ou “horizontales”, et des règles harmoniques ou “verticales”. La difficulté combinatoire vient de ce que ces règles sont en conflit les unes avec les autres. Une spécificité supplémentaire des exercices d'harmonie est que certaines règles portent sur des structures musicales simples comme les notes, par exemple la règle interdisant le croisement entre les voix, alors que d'autres portent sur les accords qui sont des structures complexes, par exemple les règles concernant la succession des degrés des accords.

Dans un exercice d'harmonie on doit donc à la fois considérer les aspects horizontaux (temporels) et verticaux (harmoniques) de la musique, mais on doit de plus faire cohabiter des structures de niveaux différents comme les notes et les accords. Les approches existantes, basés sur *l'approche par attributs*, s'accommodent naturellement et efficacement des aspects combinatoires venant de la cohabitation de contraintes horizontales et verticales. En revanche, aucune ne prend en compte explicitement la gestion de structures de niveaux de complexité différents. Dans les approches précédentes, le problème est exprimé uniquement en termes de structures atomiques, qui sont les notes dans le cas de Tsang ou Ovans ou bien des attributs de notes, d'intervalles ou d'accords. C'est ce qui constitue la cause essentielle des trois défauts soulignés au début de cette partie. Nous allons montrer comment l'approche par classe, appliquée aux accords, permet d'obtenir un système plus efficace, dans lequel il est possible d'utiliser des objets prédéfinis et où l'écriture de règles musicales sous forme de contrainte est directe.

Utiliser une approche par classe pour ce problème consiste simplement à considérer les *accords* comme des inconnues du problème, au même titre que les notes. En termes de satisfaction de contraintes, cela signifie que l'on va définir des variables contraintes dont le domaine contient des objets *notes* (resp. *accords*), qui sont en l'occurrence des instances de la classe des *notes* (resp. *accords*) de MusES. Les règles musicales seront alors exprimées simplement par des contraintes entre ces variables. La figure Figure 37 illustre la structure du problème en utilisant cette approche [Pachet and Roy, 1995a, Pachet and Roy, 1998].

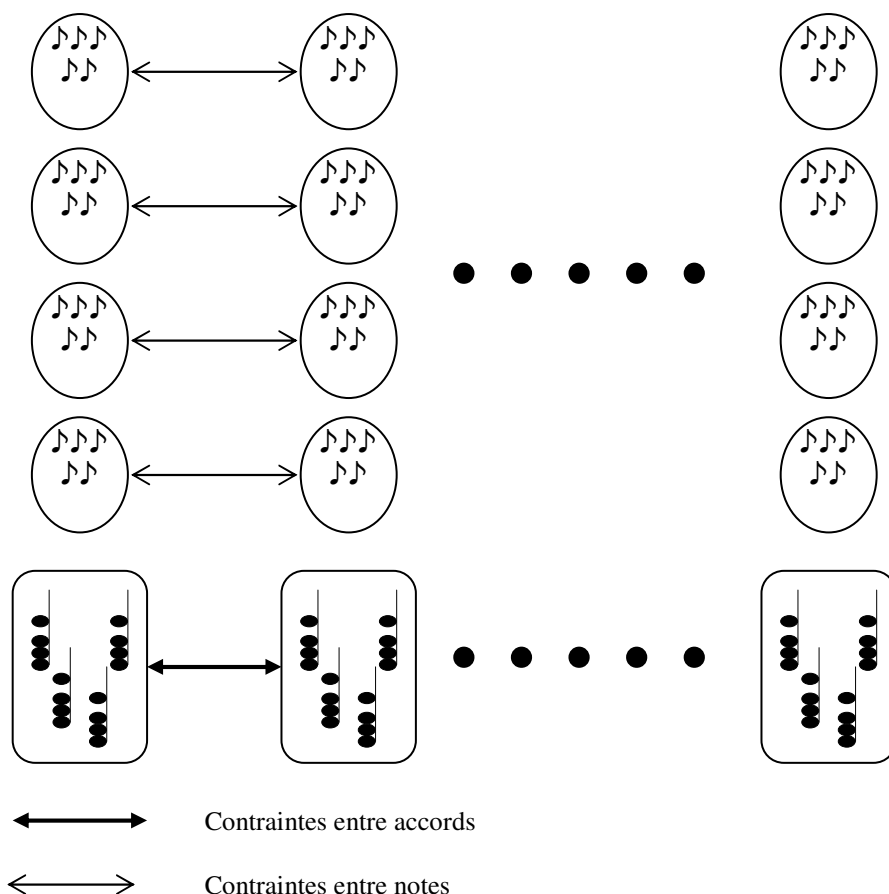


Figure 37 La structure simplifiée du problème d'harmonisation automatique dans l'approche par classes. Il y a deux catégories de variables. Les variables encadrées sont celles dont le domaine contient des notes, alors que les variables encadrées contiennent des accords. Les règles d'harmonies exprimant des relations entre notes sont définies par des contraintes posées entre les variables de la première catégorie, alors que les règles concernant les accords sont définies par des contraintes entre les variables de la seconde catégorie

On voit déjà, sur la Figure 37 que la présence de variables contenant des notes, ainsi que de variables contenant des accords permet de définir les règles musicales par des contraintes entre variables dont le domaine contient des objets complexes. Il est ainsi possible de définir les règles concernant les notes directement par des contraintes posées entre les variables dont le domaine contient des notes, alors que les règles concernant les accords sont définies par des contraintes entre variables contenant des accords.

2.3.1 LA STRATÉGIE DE RÉOLUTION

Cette approche pose cependant deux problèmes majeurs. Premièrement, dans la définition d'un exercice d'harmonie, on donne simplement une mélodie, c'est-à-dire une suite de notes, mais aucun accord n'est fourni. Afin de pouvoir appliquer l'approche par classe, il nous faut d'abord construire les accords nécessaires à la constitution des variables d'accords. Nous allons voir comment régler ce problème. Deuxièmement, nous avons l'intention de représenter les règles musicales comme des contraintes entre variables de notes ou d'accords. Cependant, dans la majeure partie des cas, ces règles ne peuvent pas s'exprimer avec des contraintes prédéfinies dans le système BACKTALK. Nous allons montrer que la définition de ces contraintes peut néanmoins se faire aisément en utilisant les contraintes de type *block* et de type *perform* de BACKTALK et qui ont été présentées précédemment.

Le problème de la création des accords vient de ce qu'il est très coûteux de créer *a priori* tous les accords possibles, même dans une tonalité donnée où leur nombre est de

l'ordre de 15^3 , soit environ 2.000 accords. La complexité de résolution d'un problème de contraintes dépendant de la taille des domaines des variables, cette approche directe conduirait à un problème particulièrement difficile. Afin d'éviter cet écueil, nous allons procéder à une séparation de la résolution en deux phases distinctes, comme nous l'avons fait pour la résolution du problème de rectangles. Ici, la première phase consiste simplement à déclarer les variables et les contraintes concernant les notes, c'est-à-dire les variables correspondant aux notes des différentes mélodies, et les contraintes correspondant aux règles mélodiques et aux règles harmoniques entre notes. Ensuite, nous appliquons à ce problème préliminaire l'algorithme de propagation de BACKTALK, qui effectue les filtrages des contraintes et réduit sensiblement les domaines des variables. L'étape suivante consiste à créer les variables accords et à déclarer les contraintes sur les accords. Finalement, c'est le problème contenant à la fois les variables et les contraintes correspondant aux notes et aux accords que l'on résout. Voici le détail de ces différentes étapes sur l'exemple de l'exercice de chant donné suivant :



Figure 38 La mélodie proposée par Tsang et Aitken

- 1) Le problème contenant les notes et les contraintes correspondant aux règles sur les notes. Il comporte 14×3 variables notes (14 pour la mélodie chantée à l'alto, 14 pour le ténor et 14 pour la basse). Le domaine initial de chaque variable contient 12 éléments pour les 14 variables de la voix d'alto, et 13 notes pour les 28 variables restantes (ténor et basse). Ces domaines sont fonction de la tessiture de chaque voix.
- 2) Les contraintes sont celles qui expriment les règles suivantes : pas de croisement entre voix ; la septième et la neuvième franchies en deux sauts doivent comporter une seconde ; quatre notes simultanées doivent être en harmonie ; certains intervalles mélodiques sont interdits (e.g. les tritons) ; les distances entre les voix n'excèdent pas l'octave, sauf pour le ténor et la basse, où elle peut atteindre la douzième. La pose des contraintes entraîne des réductions de domaines immédiates. Les tailles des domaines qui restent sont les suivantes : les variables du soprano contiennent maintenant 8 notes, les variables de l'alto et du ténor contiennent respectivement 12 et 13 notes.
- 3) On applique l'algorithme de propagation de contraintes de BACKTALK à ce problème. Il en résulte une importante réduction des domaines. Les variables de la voix d'alto contiennent maintenant 6 notes, et même 5 seulement pour l'une d'entre elles. Les variables de la voix de ténor contiennent 8 notes, celles de la basse environ 9 notes.
- 4) En faisant une énumération sur les groupes de 4 variables simultanées, on construit les accords, qui sont en nombre restreint. Le produit cartésien des domaines des notes a une taille d'environ $6 \times 8 \times 9$ soit 432 accords.
- 5) On pose les contraintes sur les accords. Parmi ces contraintes, certaines sont unaires, comme la règle qui régit les doublures de note. D'autre impliquent plusieurs accords, comme les contraintes sur les mouvements directs. Il reste à ce stade entre 23 et 64 accords dans le domaine des variables correspondant aux accords. On ajoute à cela les contraintes liant chaque accord aux quatre variables qui représentent les notes le constituant. Ces contraintes sont de type **BTPerformCt**.
- 6) On résout ensuite ce problème en employant le mécanisme de résolution standard de BACKTALK.

La structure exacte du problème final, contenant à la fois les notes *et* les accords, est légèrement plus complexe. Pour permettre une définition encore plus aisée des règles d'harmonie en termes de contraintes, nous ajoutons des variables représentant les degrés et les positions des accords. Ces variables sont reliées aux accords par des contraintes de type **BTPerformCt**. La Figure 39 montre la structure, comprenant les variables représentant les quatre notes, l'accord, le degré et la position, définie pour chaque note de la mélodie.

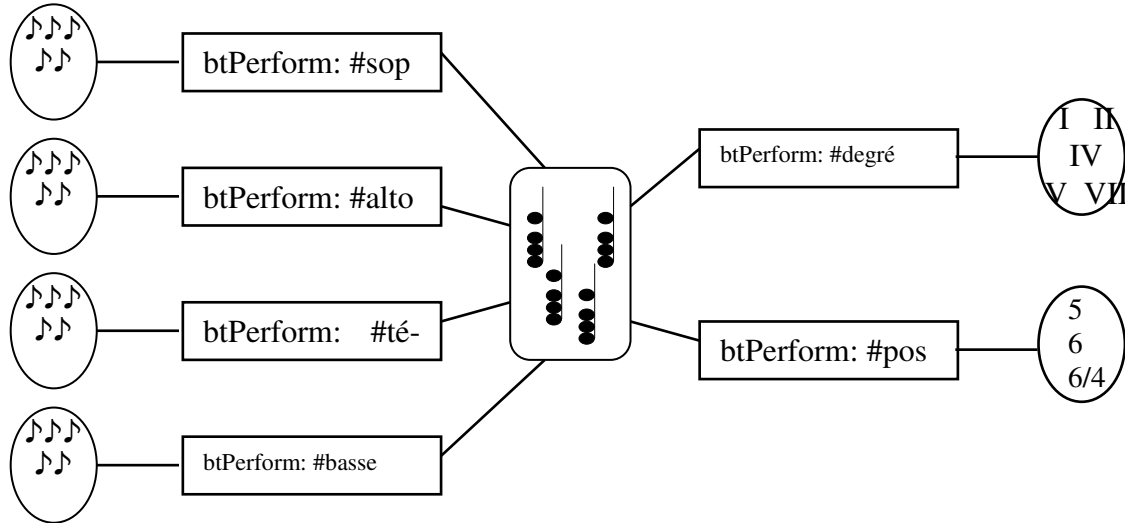


Figure 39 La structure définie pour chaque note de la mélodie à harmoniser. Cette structure comprends les quatre variables notes, correspondant à chacune des quatre voix, ainsi qu'une variable représentant l'accord et deux variables additionnelles pour le degré et la position. Ces différentes variables sont reliées à l'accord pour des contraintes de type **BTPerformCt**

Cette représentation semble finalement assez proche de celle de Philippe Ballesta, puisque chaque accord est en fait représenté par ses notes constitutives, son degré et sa position, liées entre elles par des contraintes. On peut y voir une version simplifiée de la structure utilisée par Ballesta. Néanmoins, il existe une différence essentielle entre les deux représentations : dans celle de Philippe Ballesta, l'accord n'existe pas vraiment, mais est *uniquement* représenté par les variables qui le constituent, et les contraintes structurelles associées. Dans notre approche, au contraire, les notes, le degré et la position sont des variables supplémentaires, qui ne constituent pas l'accord à proprement parler, mais qui en émanent. L'accord lui-même est représenté par une variable à part entière. Dans notre représentation, les variables supplémentaires sont créées pour faciliter la définition de règles musicales portant sur les notes, les degrés ou les positions des accords.

2.3.2 LES RÈGLES ET LES CONTRAINTES

Dans ce paragraphe, nous présentons les règles musicales que nous avons retenues pour notre système. Cette liste peut être étendue aisément par l'utilisateur afin de définir un style de composition différent de celui qui est adopté ici. Les différentes sources d'information que nous avons utilisées sont les suivantes : le traité d'harmonie de Marcel Bitsch [Bitsch, 1957] et celui d'Arnold Schönberg [Schönberg, 1978]. Nous avons également retenu quelques règles données par Tsang [Tsang and Aitken, 1991]. Enfin, nous avons implémenté l'ensemble des règles relatives à la réalisation de basses chiffrées qui sont présentées en détail dans la thèse de Philippe Ballesta [Ballesta, 1994]. Nous donnons, pour chaque règle, une implémentation détaillée sous forme de contraintes BACKTALK. En revanche, nous ne fournissons volontairement aucune justification musicologique aux règles.

Pour chaque extrait du code présenté, nous avons mis en caractères gras les instructions qui représentent à proprement parler la contrainte posée. Les autres instructions ne sont

que des parcours de listes ou des instanciations de variables temporaires, qui ne font pas partie de la contrainte.

Remarques sur l'expression des règles sous forme de contrainte :

La remarque essentielle à faire sur ces contraintes est que leur expression est purement déclarative au sens suivant : chaque règle musicale est définie par une contrainte (éventuellement une série) de type **BTBlockCt**. Cela signifie que nous n'avons utilisé *aucune technique spécifique à l'implémentation efficace des contraintes*, comme la définition de méthodes de filtrage spécialisées ou de schémas de contraintes comme dans le système de Philippe Ball-esta. Au contraire, chaque contrainte est définie par un prédicat SMALLTALK, dont la valeur est *faux* si la règle correspondante est violée, *vrai* sinon.

La représentation explicite des concepts de l'analyse tonale, y compris les plus complexes comme les accords, permet d'exprimer ces prédicats en utilisant directement le langage de ces objets. Par exemple, les contraintes sur les quintes, octaves et unissons directs, pourtant complexes, sont définies par un simple envoi de message aux accords. Chacune de ces contraintes est la traduction directe d'une méthode de la classe des accords. Plus précisément, ceci est applicable pour chaque règle pour laquelle il existe une méthode à valeur booléenne, dont l'évaluation donne la valeur de vérité de la règle.

La majorité des règles est représentée par une série de contraintes. Par exemple, pour la règle interdisant les octaves parallèles on pose une contrainte pour chaque paire d'accords successifs.

A) RÈGLES RELATIVES AUX NOTES

Chaque règle musicale concernant les notes est représentée par une contrainte posée entre les différentes variables du premier problème (celui qui contient les notes). Chaque règle est en fait associée à une méthode qui pose les contraintes correspondantes. À la création du problème, la méthode d'initialisation suivante déclenche l'exécution de chacune de ces méthodes.

```
makeVariableConstraints
    self makeNoCrossingConstraints.
    self makeHarmonicNoteConstraints.
    self makeNoteDistanceConstraints.
    self makeMelodicNoteConstraints.
    self make7thAnd9thNoteConstraints.
    self makeParallel15thAnd8veConstraints
```

Les tessitures des différentes voix

Chaque voix est contrainte à rester dans une tessiture donnée. Les tessitures des quatre voix, sont les suivantes (de gauche à droite : soprano, alto, ténor et basse) :



Pour représenter cette règle, on pose deux contraintes sur chaque variable note. Pour chaque variable **V** correspondant à une note de la voix de soprano, on déclare les contraintes :

$V \leq (N \text{ ré octave: } 4)$
 $V \geq (N \text{ fa octave: } 2)$

On procède de même pour les trois autres voix.

Pas de croisement entre voix

Les voix ne doivent pas se croiser, ce qui se représente simplement par des contraintes d'ordre entre les différentes notes jouées simultanément. C'est une des seules règles pour laquelle on a utilisé une contrainte prédéfinie de BACKTALK (la contrainte exprimant la relation "supérieur ou égal") plutôt qu'une définition en termes de blocks.

makeNoCrossingConstraints

```
1 to: length do: [:i |
  | s a t b |
  s := sopranoVars at: i.
  a := altoVars at: i.
  t := tenorVars at: i.
  b := bassVars at: i.
  s >= a.
  a >= t.
  t >= b]
```

Distances minimales et maximales entre voix

La distance entre le soprano et l'alto, ainsi que la distance entre l'alto et le ténor sont limités à l'octave. En revanche, la distance entre le ténor et la basse peut atteindre la douzième juste. En outre, si la basse est plus grave que le *do3*, on exige une distance supérieure à la tierce entre la basse et le ténor.

makeNoteDistanceConstraints

"Maximal AND minimal distance between voices"

```
1 to: length do: [:i | | s a t b |
  s := sopranoVars at: i.
  a := altoVars at: i.
  t := tenorVars at: i.
  b := bassVars at: i.
  BTBlockCt
    on: (Array with: s with: a)
    block: [:ss :aa | (aa intervalTypeBetween: ss) <= 8].
  BTBlockCt
    on: (Array with: a with: t)
    block: [:aa :tt | (tt intervalTypeBetween: aa) <= 8].
  BTBlockCt
    on: (Array with: t with: b)
    block: [:tt :bb | | interval |
      interval := bb intervalTypeBetween: tt.
      (interval <= 12) and: [interval > 3 or: [bb >= (N do @ 3)]]]]]
```

Septièmes et neuvièmes franchies en deux sauts

Si un intervalle de septième ou de neuvième est franchi en deux sauts, l'un des deux sauts doit être un intervalle de seconde. Cette règle est représentée par un ensemble de contraintes : dans chaque voix, pour tout triplet de notes consécutives, on pose une contrainte qui interdit la septième ou la neuvième entre les notes extrêmes si la note intermédiaire ne forme pas une seconde avec l'une des deux notes extrêmes.

make7thAnd9thNoteConstraints

```
| b1 |
b1 := [:x :y :z | | interval |
    interval := x intervalTypeBetween: z.
    (interval = 7 or: [interval = 9])
    ifTrue: [(x intervalTypeBetween: y)
        = 2 or: [(y intervalTypeBetween: z) = 2]]
    ifFalse: [true]].
1 to: length - 2 do: [:i | | s1 a1 t1 b1 s2 a2 t2 b2 s3 a3 t3 b3 |
    s1 := sopranoVars at: i. a1 := altoVars at: i.
    t1 := tenorVars at: i. b1 := bassVars at: i.
    s2 := sopranoVars at: i + 1. a2 := altoVars at: i + 1.
    t2 := tenorVars at: i + 1. b2 := bassVars at: i + 1.
    s3 := sopranoVars at: i + 2. a3 := altoVars at: i + 2.
    t3 := tenorVars at: i + 2. b3 := bassVars at: i + 2.
    BTBlockCt
    on: (Array with: s1 with: s2 with: s3) block: b1;
    on: (Array with: a1 with: a2 with: a3) block: b1;
    on: (Array with: t1 with: t2 with: t3) block: b1;
    on: (Array with: b1 with: b2 with: b3) block: b1]
```

Intervalles mélodiques autorisés

L'intervalle entre deux notes successives est limité à une octave. On interdit de plus les intervalles de quarte diminuée et de quinte diminuée.

makeMelodicNoteConstraints

```
| b1 |
b1 := [:x :y | | interval b1 b2 |
    interval := x intervalTypeBetween: y.
    (#(1 2 3 4 5 6 8) includes: interval)
    and: [(#(6 -6) includes: (x semiToneswith: y)) not]].
1 to: length - 1 do: [:i | | s1 a1 t1 b1 s2 a2 t2 b2 |
    s1 := sopranoVars at: i. a1 := altoVars at: i.
    t1 := tenorVars at: i. b1 := bassVars at: i.
    s2 := sopranoVars at: i + 1. a2 := altoVars at: i + 1.
    t2 := tenorVars at: i + 1. b2 := bassVars at: i + 1.
    BTBlockCt
    on: (Array with: s1 with: s2) block: b1;
    on: (Array with: a1 with: a2) block: b1;
    on: (Array with: t1 with: t2) block: b1;
    on: (Array with: b1 with: b2) block: b1]
```

Contraintes harmoniques entre notes

Cette contrainte assure que les notes jouées simultanément appartiennent à un même accord parfait de trois sons. Pour définir cette règle, nous posons naturellement une contrainte pour chaque groupe de quatre notes simultanées. À l'instar de la règle des croisements entre voix, nous n'utilisons pas ici de contrainte block, mais une contrainte de type **AHPHarmonicNoteCt**. Cette contrainte est filtrée dès que l'une de ses variables est instanciée (i.e. elle réagit au démon de type valeur), et le filtrage consiste alors à supprimer des domaines des trois autres variables les notes qui ne font avec elle ni un unisson, ni un intervalle de tierce, de quarte, de quinte, de sixte ou d'octave.

makeHarmonicNoteConstraints

```
1 to: length do: [:i |
    AHPHarmonicNoteCt
        s: (sopranoVars at: i)
        a: (altoVars at: i)
        t: (tenorVars at: i)
        b: (bassVars at: i) scale: scale]
```

Quintes et octaves parallèles

Cette règle stipule qu'on ne peut pas faire se succéder deux octaves ou deux quintes entre les mêmes voix d'un accord. Ici encore, cette règle est fidèlement représentée par une série de contraintes impliquant quatre notes : deux notes consécutives de deux voix différentes. Il y a ainsi six séries de telles contraintes : une pour les quintes et octaves parallèles entre les voix extrêmes, une concernant les voix de soprano et d'alto, etc.

makeParallel5thAnd8veConstraints

```
| b1 |
b1 := [:v1n1 :v1n2 :v2n1 :v2n2 | | int1 int2 |
    int1 := (v2n1 intervalTypeBetween: v1n1) \\ 7.
    int2 := (v2n2 intervalTypeBetween: v1n2) \\ 7.
    ((int1 = 5 and: [int2 = 5])
    or: [int1 = 1 and: [int2 = 1]]) not].
1 to: length - 1 do: [:i | | s1 a1 t1 b1 s2 a2 t2 b2 |
    s1 := sopranoVars at: i. a1 := altoVars at: i.
    t1 := tenorVars at: i. b1 := bassVars at: i.
    s2 := sopranoVars at: i + 1. a2 := altoVars at: i + 1.
    t2 := tenorVars at: i + 1. b2 := bassVars at: i + 1.
    BTBlockCt
        on: (Array with: s1 with: s2 with: a1 with: a2) block: b;
        on: (Array with: s1 with: s2 with: t1 with: t2) block: b1;
        on: (Array with: s1 with: s2 with: b1 with: b2) block: b1;
        on: (Array with: a1 with: a2 with: t1 with: t2) block: b1;
        on: (Array with: a1 with: a2 with: b1 with: b2) block: b1;
        on: (Array : t1 with: t2 with: b1 with: b2) block: b1]
```

B) RÈGLES RELATIVES AUX ACCORDS

De la même façon que pour les notes, les règles musicales concernant les accords sont représentées par une ou plusieurs contraintes, posées entre les différentes variables du second problème (celui qui contient les notes et les accords). Voici la méthode d'initialisation des

contraintes. Chaque message fait référence à une règle donnée. Les méthodes correspondantes sont décrites dans les paragraphes qui suivent.

makeChordConstraints

```
self makeDegreeConstraints.  
self makePositionConstraints.  
self makeCadenceConstraints.  
self makeBeginingConstraints.  
self makeDoubleNoteConstraints.  
self makeLeadingNoteConstraints.  
self makeDegreeConstraints.  
self makeDirect8veConstraints.  
self makeDirect5thConstraints.  
self makeGlobalMotionConstraints.  
self makeDirectUnissonConstraints
```

Degré et position du premier accord

Cette règle stipule que le premier accord du morceau doit être un accord de tonique (i.e. du premier degré), en position fondamentale. Cette règle ne donne évidemment lieu à aucune déclaration de contrainte si l'on réalise un exercice de basse chiffrée.

makeBeginingConstraints

```
degrees == nil  
if True:  
    [degreeVars first value: 1.  
    positionVars first value: 0]
```

Mouvement de la sensible

Cette règle régit le mouvement de la note sensible (i.e. le septième degré de la gamme, par exemple, *si* en *do majeur*). L'idée générale est que la note sensible doit monter à la tonique. Plus précisément, si la sensible apparaît à une voix donnée dans un accord, l'accord suivant doit comporter la note tonique immédiatement au-dessus et dans la même voix. Cette règle souffre plusieurs exceptions :

- Si le premier accord n'est pas un accord de dominante, alors la sensible n'est pas tenue de monter à la tonique.
- Si l'accord qui suit ne contient pas la note tonique, alors la sensible n'est pas tenue de monter.
- Dans un enchaînement d'accords de degrés V et IV, on interdit la présence de la sensible au soprano du premier accord et de la sous-dominante à la basse du second. C'est la règle dite de la *fausse relation de triton*.
- Dans un enchaînement d'accords de degrés V et VI, la sensible située dans une voix intermédiaire est dispensée de monter à la tonique, si celle-ci apparaît dans l'accord suivant à la voix placée juste au-dessus.

makeLeadingNoteConstraints

```
| leadingNote subDominant b11 tonic b12 |
leadingNote := scale notes at: 7.
subDominant := scale notes at: 4.
tonic := scale notes at: 1.
b11 := [:x :y | "fausse relation de triton"
  (x s pitchClass = leadingNote
  and: [y b pitchClass = subDominant])
  or: [x degree ~= 5 or: [y degree ~= 4]]].
b12 := [:x :y | | shouldMove |
  shouldMove := true.
  x degree ~= 5 ifTrue: [shouldMove := false].    "Pas de degré v"
  shouldMove ifTrue:                               "Pas de tonique"
    [(y notes
      detect: [:n | n pitch = tonic]
      ifNone: [nil]) isNil ifTrue: [shouldMove := false]].
  shouldMove ifTrue:                               "Enchaînement V-VI"
    [y degree = 6 ifTrue: [| leadingVoice |
      x s pitch = leadingNote ifTrue: [leadingVoice := 1].
      x a pitch = leadingNote ifTrue: [leadingVoice := 2].
      x t pitch = leadingNote ifTrue: [leadingVoice := 3].
      x b pitch = leadingNote ifTrue: [leadingVoice := 4].
      (leadingVoice = 4 or: [leadingVoice = 1]) ifFalse:
        [leadingVoice = 2 ifTrue:
          [y s pitch = tonic ifTrue:
            [shouldMove := false]].
          leadingVoice = 3 ifTrue:
            [y a pitch = tonic ifTrue:
              [shouldMove := false]]]]].
    shouldMove ifTrue: [| leadingVoice bool | "Cas général"
      x s pitch = leadingNote ifTrue: [leadingVoice := #s].
      x a pitch = leadingNote ifTrue: [leadingVoice := #a].
      x t pitch = leadingNote ifTrue: [leadingVoice := #t].
      x b pitch = leadingNote ifTrue: [leadingVoice := #b].
      bool := (y perform: leadingVoice) pitchClass = tonic.
    bool]
  ifFalse: [true]].
1 to: length - 2 do: [:i | | ch1 ch2 |
  ch1 := chordVars at: i.
  ch2 := chordVars at: i + 1.
  BTBlockCt on: (Array with: ch1 with: ch2) block: b11.
  BTBlockCt on: (Array with: ch1 with: ch2) block: b12]
```

Cadence finale

Les deux derniers accords du morceau doivent constituer une cadence parfaite. Le premier accord de la cadence est un accord de dominante (i.e. du cinquième de gré), et le second est un accord de tonique. Les deux accords apparaissent en position fondamentale. En outre, on applique ici une légère modification de la règle du mouvement de la sensible exposée ci-dessus. Si, dans une cadence parfaite, la sensible dans une voix intermédiaire ne peut pas monter à la tonique, elle peut descendre d'une tierce. Cette règle s'applique notamment

aux mélodies qui finissent par un enchaînement entre le second degré de la gamme et la tonique. Par exemple en *do majeur* :



On ne peut pas mettre de cadence parfaite sous une telle mélodie puisque les notes de la basse sont respectivement *sol* et *do*. Ainsi, la sensible apparaît à une des voix intermédiaires du premier accord. Sa montée à la tonique provoquerait donc le triplement de celle-ci dans le second accord. La solution consiste à faire descendre la sensible d'une tierce afin de conserver un accord de trois sons, comme ceci (extrait d'un choral en *la majeur* de Bach) :



makeCadenceConstraints

```
| s1 s2 a1 a2 t1 t2 b1 b2 leadingNote tonic tierce b1 |
degrees == nil ifFalse: [^self].
degreeVars last value: 1.
(degreeVars at: length - 1) value: 5.
length >= 3 ifTrue:
    "Une cadence n'est pas précédé par un de ses deux accords"
    [(degreeVars at: length - 2) remove: 1; remove: 5].
positionVars last value: 0.
(positionVars at: length - 1) value: 0.

"Leading note move"
a1 := altoVars at: length - 1. a2 := altoVars last.
t1 := tenorVars at: length - 1. t2 := tenorVars last.
leadingNote := scale notes at: 7. tonic := scale notes at: 1.
b1 := [:x :y | x pitch = leadingNote
    ifTrue:
        [y pitch = tonic
        or: [y < x and: [(y intervalTypeBetween: x) = 3]]]
    ifFalse: [true]].
BTBlockCt on: (Array with: a1 with: a2) block: b1.
BTBlockCt on: (Array with: t1 with: t2) block: b1
```

Mouvement mélodique global

Cette règle interdit de faire se succéder des accords dont toutes les voix ont un mouvement semblable. Plus précisément, au moins deux voix doivent être en mouvement oblique ou contraire.

makeGlobalMotionConstraints

```
1 to: length - 1 do: [:i |  
    BTBlockCt  
        on: (Array with:(chordVars at: i) with:(chordVars at: i+1))  
        block: [:ch1 :ch2 |  
            ((ch1 s > ch2 s  
             and: [ch1 t > ch2 t  
             and: [ch1 a > ch2 a  
             and: [ch1 b > ch2 b]]))  
            or: [ch1 s < ch2 s  
             and: [ch1 t < ch2 t  
             and: [ch1 a < ch2 a  
             and: [ch1 b < ch2 b]]]]) not]]
```

Quintes, octaves et unissons directes

La règle concernant les quintes et les octaves directes entre deux accords successifs interdit dans la majeure partie des occasions d'avoir une quinte ou une octave au second accord sur laquelle on arrive par mouvement direct. Voici deux exemples de situations interdites par cette règle.



Figure 40 À gauche, une quinte directe interdite, à droite, une octave directe interdite

makeDirect5thConstraints

```
1 to: length - 1 do: [:i |  
    BTBlockCt  
        on: (Array with:(chordVars at: i) with:(chordVars at: i+1))  
        block: [:ch1 :ch2 | (ch1 hasADirect5thwith: ch2) not]]
```

makeDirect8veConstraints

```
1 to: length - 1 do: [:i |  
    BTBlockCt  
        on: (Array with:(chordVars at: i) with:(chordVars at: i+1))  
        block: [:ch1 :ch2 | (ch1 hasADirect8vewith: ch2) not]]
```

makeDirectUnissonConstraints

```
| b11 b12 |
b11 := [:ch1 :ch2 | (ch1 hasADirectUnissonWith: ch2) not].
b12 := [:ch1 :ch2 | | bool |
  "Après unisson, les voix sont en mouvement oblique ou contraire"
  bool := true.
  ch1 b = ch1 t ifTrue:
    [bool := (ch1 b >= ch2 b and: [ch1 t <= ch2 t])
     or: [ch1 b <= ch2 b and: [ch1 t >= ch2 t]]].
  ch1 t = ch1 a ifTrue:
    [bool := (ch1 t >= ch2 t and: [ch1 a <= ch2 a])
     or: [ch1 t <= ch2 t and: [ch1 a >= ch2 a]]].
  ch1 a = ch1 s ifTrue:
    [bool := (ch1 a >= ch2 a and: [ch1 s <= ch2 s])
     or: [ch1 a <= ch2 a and: [ch1 s >= ch2 s]]].
  bool].
1 to: length - 1 do: [:i | BTBlockCt
  on: (Array with:(chordVars at: i) with:(chordVars at: i+1))
  block: b11;
  on: (Array with:(chordVars at: i) with:(chordVars at: i+1))
  block: b12]
```

Doublure des notes au sein des accords

Puisque nous utilisons des accords de trois sons pour faire de l'harmonie à quatre voix, deux voix exactement ont le même son. Ce son qui apparaît deux fois dans un accord est sa *doublure*. Cette doublure est soumise à plusieurs règles, qui dépendent de la position et du degré de l'accord. Voici quelques-unes de ces règles :

- Une règle générale est que l'on ne double jamais la sensible.

À l'état fondamental :

- Sur les accords de degré VII, on ne double pas la quinte.
- Sur les degrés I et IV, on ne double pas la tierce, sauf entre le soprano et l'alto.

À l'état de premier renversement :

- On ne double jamais la basse à l'alto.

- Sur les degrés I, IV, VI et VII, on ne double pas la basse sauf si l'accord est un simple changement de position du précédent.
 - Etc.
-

makeDoubleNoteConstraints

```
| leadingNote b1 b12 b13 b14 b15 |
leadingNote := scale notes at: 7.
b1 := [:x | | count |
    count := 0.
    x s pitchClass = leadingNote ifTrue: [count := count + 1].
    x a pitchClass = leadingNote ifTrue: [count := count + 1].
    x t pitchClass = leadingNote ifTrue: [count := count + 1].
    x b pitchClass = leadingNote ifTrue: [count := count + 1].
    count <= 1].
b12 := [:x | ((#(1 4 6 7) includes: x degree) and: [x position = 1])
    ifTrue: [x b pitchClass ~= (x t pitchClass)]
    ifFalse: [true]].

(...etc...)
```

Utilisation des différents degrés et positions

La consécution des degrés est un des éléments les plus importants de la résolution d'un exercice d'harmonie. Nous avons choisi d'implémenter les règles suivantes, qui ne sont pas appliquées lors de la résolution d'un exercice de basse chiffrée :

- Les accords autorisés sont : I *a*, I *b*, I *c*, II *a*, II *b*, IV *a*, IV *b*, IV *c*, V *a*, V *b*, VI *a*, VI *b*, VII *b* (*a* désigne l'état fondamental, *b* et *c* les premier et second reversements).
 - Les transitions interdites sont : VII *b*—IV, V—IV et VI *b*—V
 - Le second degré n'est employé que pour les successions : II—V et II—IV—V
-

makeDegreeConstraints

```
| b1 b12 b13 |
degrees = nil ifFalse: [^self].
b1 := [:d :p | | r | "Degrés et positions autorisés"
    r := false.
    d = 1 ifTrue: [r := true].
    d = 2 ifTrue: [r := p ~= 2].
    d = 3 ifTrue: [r := false].
    d = 4 ifTrue: [r := true].
    d = 5 ifTrue: [r := p ~= 2].
    d = 6 ifTrue: [r := p ~= 2].
    d = 7 ifTrue: [r := p = 1].
    r].
1 to: length do: [:i |
    BTBlockCt
        on:(Array with:(degreeVars at: i)with:(positionVars at: i))
        block: b1].
```

```

b12 := [:d1 :p1 :d2 :p2 | | r | "Transitions autorisées"

    r := true.
    d1 = 2 ifTrue: [r := d2 ~= 1].
    d1 = 7 ifTrue: [p1 = 1 ifTrue: [r := d2 ~= 4]].
    d1 = 5 ifTrue: [r := d2 ~= 4].
    d1 = 6 ifTrue: [p1 = 1 ifTrue: [r := d2 ~= 5]].
    r].
b13 := [:d1 :d2 :d3 | "Utilisation de II"
    d1 = 2
        ifFalse: [true]
        ifTrue: [d2 = 5 or: [d2 = 4 and: [d3 = 5]]]].
1 to: length - 1 do: [:i | BTBlockCt on: (Array
    with: (degreeVars at: i)
    with: (positionVars at: i)
    with: (degreeVars at: i + 1)
    with: (positionVars at: i + 1))
    block: b12].
1 to: length - 2 do: [:i | BTBlockCt on: (Array
    with: (degreeVars at: i)
    with: (degreeVars at: i + 1)
    with: (degreeVars at: i + 2))
    block: b13]

```

2.3.3 QUELQUES EXEMPLES MUSICAUX

Le système que nous venons de décrire est adapté aussi bien à la réalisation de lignes de basses chiffrées, ainsi qu'à l'harmonisation de chant choral. Plus généralement, il peut être employé pour la réalisation d'harmonisations à partir d'un matériau musical incomplet quelconque tel qu'une voix de ténor ou bien une voix d'alto avec quelques informations de chiffrage. L'utilisation d'une approche de type satisfaction de contraintes permet, sans modification aucune du système, de fixer n'importe quel paramètre représenté par une variable contrainte, avant d'entamer la résolution. Ce mode de fonctionnement permet une utilisation interactive du système. En effet, l'utilisateur peut faire calculer une première solution au système et y apporter des modifications, par exemple en imposant une valeur différente pour une note donnée de la solution ou en interdisant d'utiliser un certain type d'accord, avant de relancer une résolution. Ce processus peut être poursuivi jusqu'à l'obtention d'une solution jugée musicalement acceptable par l'utilisateur. Nous avons donc utilisé notre système à la fois pour la résolution d'exercices de basse chiffrée, comme celui proposé par Philippe Ballista, et pour des exercices de chant donné, comme ceux fournis par Tsang ou Ebcioglu.

A) LA BASSE CHIFFRÉE DE PHILIPPE BALLESTA

Voici tout d'abord deux résolutions successives de l'exercice proposé par Philippe Ballesta. La basse chiffrée à réaliser est en *fa majeur* et comporte 14 notes. Les degrés sont fixés, ce qui a pour effet de fixer les positions puisque les notes de la basse sont connues.



Figure 41 La basse chiffrée proposée par Philippe Ballesta

Nous avons résolu ce problème une première fois, en utilisant fidèlement les règles musicales exposées par Philippe Ballesta. La solution que nous trouvons n'est pas la même que celle de son auteur. Ceci est dû à la différence entre les deux stratégies de résolution adoptées. La Figure 42 montre la première solution obtenue avec notre système :



Figure 42 Notre première solution pour la basse chiffrée de Philippe Ballesta

La Figure 43 montre la solution de Philippe Ballesta pour le même problème :



Figure 43 La première solution de Philippe Ballesta

On peut remarquer que, malgré la différence de stratégie de résolution, les deux solutions sont très proches. En effet, les cinq premiers accords, ainsi que les cinq derniers, sont identiques dans les deux solutions. Nous pouvons cependant tenter d'améliorer notre solution en interdisant la répétition de notes au soprano. Ceci peut être spécifié par une simple liste de contraintes binaires de différence, posées sur les variables représentant les notes de la voix de soprano. La solution que l'on obtient ainsi est la suivante :



Figure 44 La seconde solution obtenue

B) LA MÉLODIE DE TSANG ET AITKEN

Tsang et Aitken ont illustré le fonctionnement de leur système sur la mélodie suivante, en *sol majeur*, que nous avons montré à la Figure 38 :



Figure 45 La solution de Tsang et Aitken, dont les chiffres sont : I, V, I, VI, II, VI, II, V, I, IV, I, IV, V, I



Figure 46 Notre solution, dont les chiffres (I, V, I, VI, IV, I, V, VII, V, VII, I, IV, V, I) sont différents, ce qui est nécessaire puisque nous interdisons, par exemples, l'utilisation de l'accord du second degré s'il n'est pas suivi d'un accord de dominante éventuellement précédé d'un accord de degré IV



Figure 47 Une solution, trouvée par notre système, en harmonisant la mélodie de Tsang et Aitken, et en imposant les degrés des accords. La solution trouvée n'est pas exactement la même que celle de Tsang et Aitken

La ligne de basse est plus conjointe que celle de Tsang, ce qui constitue une amélioration. Cependant, la conduite des voix intermédiaires est moins intéressante, notamment la voix de ténor. Pour remédier à ce problème, nous pouvons par exemple ajouter une série de contraintes exigeant que les voix intermédiaires ne présentent pas d'intervalles supérieurs à la tierce majeure. La solution que l'on obtient est la suivante :



Figure 48 Une autre solution pour la mélodie de la mélodie de Tsang, obtenue avec les degrés de Tsang, et en ajoutant une contrainte pour la conduite des voix intermédiaires : on interdit les sauts plus grands que la tierce majeure au ténor et à l'alto

C) UNE BASSE CHIFFRÉE DE J.-S. BACH

Présentons maintenant un exemple plus intéressant musicalement. L'un des chorals de Bach, celui issu de la Cantate B.W.V. 67, commence par la phrase suivante :

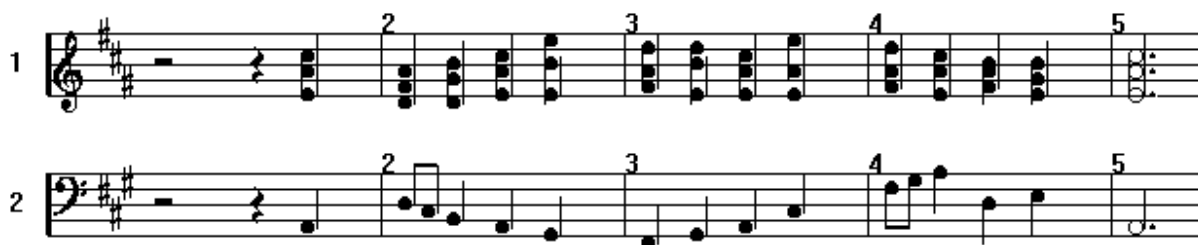


Figure 49 La première phrase du choral de la Cantate B.W.V. 67 de Jean-Sébastien Bach. On remarquera particulièrement l'allure générale de la ligne de basse, ainsi que celle de la voix de soprano, qui sont quasi systématiquement en mouvement contraire

L'harmonisation de cette basse, sans aucune contrainte particulière et sans information sur les degrés des accords conduit à une solution de piètre qualité. De nombreuses notes sont répétées à la voix de soprano et le dessin général de l'original de Bach, avec cette opposition entre le mouvement de la voix de soprano et celui de la basse n'est pas reproduit ici. En fait, la voix supérieure, dans notre première solution, évolue en mouvement descendant durant toute la phrase. Afin de nous rapprocher de la solution plus intéressante de Bach, nous avons simplement ajouté la série de contraintes suivante : la voix de soprano doit commencer par un mouvement ascendant et finir par un mouvement descendant. Le mouvement des notes médianes étant lassé libre.



Figure 50 Une première harmonisation de cette basse

Les partitions qui suivent montrent deux solutions possibles pour l'harmonisation de la basse du choral de la Cantate B.W.V. 67. La première solution a été obtenue sans imposer sur les degrés aucune contrainte supplémentaire à celles que nous avons présentées au début de cette partie. En revanche, la seconde solution reproduit les degrés.



Figure 51 Une harmonisation de cette basse en imposant le dessin général de la voix de soprano

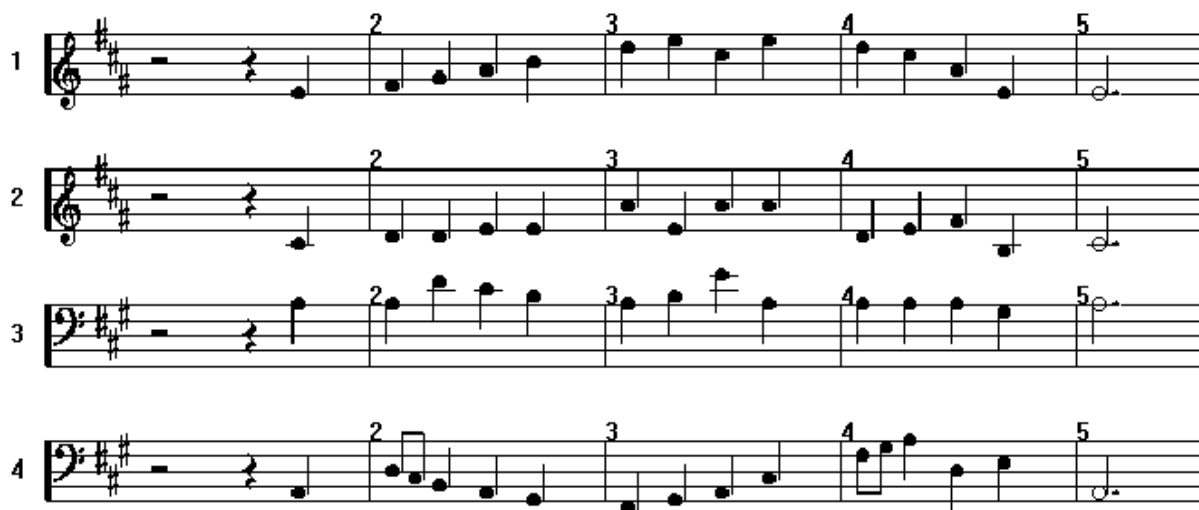


Figure 52 Une autre harmonisation de cette basse avec le même dessin pour la voix de soprano et avec les mêmes degrés que l'original de Bach

Dans ces deux exemples, on peut remarquer une certaine maladresse dans le dessin de la mélodie. D'une part, les deux notes répétées à la cadence finale, et d'autre part, dans la première partie, la mélodie est conjointe et ascendante, avec une exception : la tierce entre la 5^{ème} note et la 6^{ème} note. On peut proscrire ce style de mouvement, en interdisant ou bien de faire suivre ce genre de progression diatonique d'un intervalle plus important dans le même sens ou bien de continuer dans le même sens après un tel intervalle. Nous pouvons mettre cette règle en œuvre simplement en définissant une contrainte globale sur l'ensemble des variables contraintes représentant les notes de la mélodie. Voici une solution obtenue avec cette contrainte supplémentaire :



Notes	Ballesta (ILOGSOLVER) SPARC Station 10			BACKTALK + MusES Pentium 300 MHz (estimation SPARC 10)		
	Échecs	Tps CPU	Tps sol n°1	Échecs	Tps CPU	Tps sol n°1
1	100	6,9s	1,9s	6	0,01s (0,08s)	0s (0s)
2	341	26s	9,1s	7	0,1s (0,8s)	0,015s (0,1s)
3	1.044	2mn 1s	44,7s	46	0,8s (6,5s)	0,5s (4s)
4	4.460	6mn 13s	2mn 7s	54	1,1s (9s)	0,9s (8s)
5	1.543	2mn 43s	1mn 36s	30	0,5s (4s)	0,3s (2,4s)
6	9.360	17mn 33s	2mn 41s	38	0,52s (4s)	0,3s (2,4s)
7	39.676	110mn	2mn 59s	72	0,8s (6,5s)	0,35s (3s)
8	232.347	430mn	2mn 10s	426	2,9s (24s)	0,4s (3,2s)
12	?	>38h23mn*	3mn 11	1.255	>10s (90s)*	0,6s (5s)
14	?	?	3mn 04	?	?	0,65s (5s)

*Tableau 10 Comparaison des temps de résolutions et du nombre d'échecs dans le système de Philippe Ballesta et le nôtre pour un même exercice. La machine de Ballesta est environ 8 fois plus lente. *Pour la mélodie de 12 notes, on ne calcule que les 520 premières solutions*

Les informations contenues dans le tableau permettent d'estimer, de façon approximative seulement à cause de la différence de machine employée, la performance relative des deux systèmes. Pour ce qui est du temps de calcul de la première solution, le rapport oscille entre 10 et 90 en notre faveur. Pour ce qui est du calcul de l'espace des solutions (ou d'une partie seulement dans le cas de 12 notes), le rapport est de l'ordre de 20 à 90 pour les petites mélodies (moins de 6 notes), pour la mélodie de 6 notes, il est de 200 environ, et pour les mélodies de 7, 8 et 12 notes, ce rapport est entre 850 et 1.500. Ce rapport étant calculé compte tenu de la différence de performance entre les machines utilisées.

Outre que notre approche est bien plus performante, l'analyse de ces données montre que le temps de calcul de la première solution est, dans notre approche, beaucoup plus lent que le temps de calcul des solutions suivantes. Le temps de calcul, sur le PC Pentium 300MHz, de cette première réalisation pour les exercices de 8 à 12 notes est d'environ une demi-seconde. Pour certaines mélodies particulièrement difficiles, il peut aller jusqu'à 10 secondes pour 10 à 15 notes. En revanche, le temps moyen d'obtention des solutions suivantes est d'environ 0,015 secondes. Chez Ballesta, toutes les solutions requièrent un temps de calcul identique (entre 2 et 4 minutes sur SPARC 10). En tenant compte de la différence de machine, la recherche des solutions suivant la première est dans notre système 1.500 fois plus rapide en moyenne. En d'autres termes, nous avons un surcroît de calculs à faire en début de résolution, dû au calcul et à la présence de nombreux accords qui sont rapidement éliminés pour la suite de la résolution qui devient alors peu combinatoire.

Une telle amélioration des performances, dans le calcul des solutions autres que la première, rend possible de nombreuses expériences qui n'ont jamais été menées. À ce sujet, nous rejoignons ici l'opinion de Myriam Dessainte-Catherine [Barbar, et al., 1998]. En effet, cette amélioration intrinsèque des performances, ajoutée à l'augmentation d'efficacité des machines, qui sont aujourd'hui bien plus rapide qu'à l'époque de la thèse de Philippe Ballesta, et peut-être 50 fois plus qu'à celle des travaux de Kemal Ebcioglu, permet d'envisager un travail plus complexe. On peut, par exemple, réaliser des harmonisations de mélodies beaucoup plus longues ou s'attacher à améliorer la qualité musicale des solutions produites en ajoutant des contraintes complexes, par exemple, des contraintes globales sur le dessin de la mélodie. On peut aussi employer, ce qui avait été suggéré par Philippe Ballesta, des méthodes

d'optimisation pour favoriser un critère particulier comme le nombre de mouvements contraires entre la basse et le soprano ou le caractère conjoint des voix intermédiaires. Seule une obtention rapide des solutions intermédiaires permet d'envisager sérieusement l'optimisation de critères aussi globaux.

Considérons par exemple la mélodie suivante (première phrase de La Marseillaise) en Ut majeur et une première harmonisation.



Figure 54 La première phrase de La Marseillaise



Figure 55 Une première harmonisation de La Marseillaise. On peut remarquer que la conduite des voix intermédiaires n'est pas très satisfaisante

La conduite des voix intermédiaires de cette première harmonisation contient beaucoup d'intervalles importants. Afin de réduire ces sauts, nous pouvons lancer une recherche qui optimise le critère "taille cumulée des intervalles apparaissant aux voix intermédiaires". Ce critère peut s'exprimer aisément en définissant pour chaque succession de notes, au ténor et à l'alto, une variable numérique, et de lier ces trois variables par la contrainte de type block suivante :

BTBlockCt

```
on: (Array
  with: (sautsAlto at: i)
  with: (altos at: i)
  with: (altos at: i + 1))
block:
  [ :saut :note1 :note2 |
    saut = (notes1 intervalTypewith: note2)]
```

Et de définir la variable **sautsCumulés** par une contrainte arithmétique linéaire et de déclencher un résolution avec optimisation du critère :

```

sautsCumulés := sautsAlto sum + sautsTenor sum.
...
p
minimize: sautsCumulés “minimise les sauts des voix internes”
do: [:sol | self displaySolution] “affiche chaque solution”

```



Figure 56 La solution optimale pour l'harmonisation de La Marseillaise avec minimisation des sauts aux voix intermédiaires. Cette solution est obtenue en environ 7 minutes, et son optimalité est prouvée en 15 minutes environ sur Pentium 300 MHz

2.3.5 BILAN

Nous avons présenté dans cette partie une nouvelle approche pour l'utilisation des techniques de satisfaction de contraintes pour l'harmonisation musicale automatique. Les résultats obtenus représentent une nette amélioration par rapport aux systèmes existants en ce qui concerne trois points fondamentaux. Cette approche permet d'utiliser directement des classes prédéfinies pour représenter les objets musicaux utiles à la définition du problème, comme nous l'avons fait avec les classes de la bibliothèque MusES. En outre, il est possible de définir les contraintes représentant les règles musicales de façon déclarative, c'est-à-dire par de simples formules de satisfaction, au lieu d'avoir à implémenter des méthodes de filtrage spécialisées pour chaque contrainte. De plus, ces formules de satisfaction peuvent être écrites en utilisant directement le langage implémenté dans les classes des objets manipulés. Finalement, le problème qui en résulte est beaucoup plus facile à résoudre par les techniques de satisfaction de contraintes, comme le prouvent les temps d'exécution affichés.

Le Tableau 11 donne quelques éléments importants de comparaison entre trois systèmes d'harmonisation automatique fondés sur les techniques de contraintes. Nous avons comparé notre approche avec CHORAL de Kemal Ebcioglu, dans sa version initiale ainsi que dans sa version rénovée par Pierre Dellacherie, et avec le système de Philippe Ballesta.

	CHORAL (Ebcioglu)	CHORAL (Dellacherie)	Philippe Ballesta	BACKTALK + MusES
Problème	Harmonisation de choral à la manière de J.-S. Bach	Idem	Harmonisation de basses chiffrées	Harmonisation de basses chiffrées ou de mélodies
Résolution	350 règles musicales représentées par des règles de production, des contraintes et des heuristiques. Utilisation de techniques de retour arrière intelligent, pas de propagation ni de techniques de cohérence d'arc	Un corpus de règles plus restreint (50 règles environ). Utilisation de techniques de cohérences dans un système de programmation logique avec contraintes (ECLIPSE)	Utilisation d'un corpus de règles issue des traités d'harmonie tonale. Résolution avec le système ILOG-SOLVER (dans sa version LISP)	Même corpus que chez Ballesta, avec ajout de règles concernant les degrés des accords. Résolution avec le système BACKTALK fondés sur les techniques de cohérences d'arc
Les objets	Variables logiques	Variables uniquement numériques	Objets partiellement instanciés	Objets standards
Représentation des contraintes	Contraintes passives, seulement vérifiées <i>a posteriori</i>	Contraintes arithmétiques et contraintes de disjonction pour représenter les contraintes non linéaires	Contraintes de structures interne aux objets, et contraintes entre attributs d'objets différents	Contraintes de haut niveau entre objets composites
Efficacité	De quelques minutes à plusieurs heures pour une page	En retrait par rapport à Ebcioglu	Plusieurs minutes pour chaque solution pour 12 notes	1 sec. Pour la 1 ^{ère} solution puis 0,05 sec. par solution
Qualité musicale	Excellente	Bonne, mais moins riche : ni ornementation ni de notes de passage	Rigoureuse mais scolaire	Rigoureuse, mais scolaire. Peut être optimisée selon des critères globaux

Tableau 11 Synthèse des différences majeures entre quatre systèmes d'harmonisation automatique basés sur la programmation par contraintes

CONCLUSION

Dans cette thèse nous avons abordé l'intégration des contraintes avec les objets sous deux angles différents. Dans un premier temps, nous avons proposé une architecture logicielle de type *framework* [Johnson and Foote, 1988] pour l'implémentation d'un système de satisfaction de contraintes dans un langage à objets. Nous avons illustré ce propos à l'aide du système de satisfaction de contraintes BACKTALK [Roy and Pachet, 1997b] implémenté en SMALLTALK. Un framework est une application semi-complète constituée de classes abstraites liées entre elles par un schéma de contrôle prédéfini. Cette approche permet de fournir des classes abstraites pour représenter les concepts essentiels de la discipline : les variables contraintes, les contraintes, les algorithmes de résolution, les objets permettant la sauvegarde et la restauration des états successifs du problème et les heuristiques de sélection.

- Le contrôle dans ce système est constitué par des *démons* qui sont des objets intermédiaires entre les variables, les contraintes et les algorithmes. Les démons déterminent le schéma de propagation des contraintes.
- Les classes abstraites implémentent des primitives qui en déterminent le comportement. Ces primitives sont constituées d'une suite d'opérations élémentaires qui sont définies dans les classes concrètes.

Cette architecture permet de proposer un système dont le fonctionnement est prédéterminé dans ses grandes lignes tout en étant modifiable dans les détails. Cette approche offre une souplesse et une ouverture nécessaires dans ce contexte tout en garantissant une certaine fiabilité.

BACKTALK a été utilisé dans plusieurs applications dans des domaines divers. Thierry Goubier [Goubier, 1997] l'a utilisé pour la construction d'interfaces graphiques avec des contraintes. Nicolas Ramaux [Ramaux and Fontaine, 1996] a employé BACKTALK pour la reconnaissance de scénarios temporels dans le contexte d'assistance médicale. BACKTALK est également l'un des deux outils utilisés dans le projet FIBOF développé au [Lesueur, et al., 1998]. Dans ce contexte, BACKTALK est employé à la résolution de problèmes d'analyse financière.

La seconde contribution de cette thèse est d'ordre méthodologique. Nous avons proposé une nouvelle approche pour la définition de problèmes combinatoires structurés. Cette méthodologie, appelée *approche par classes*, s'oppose à la démarche consistant à définir le problème comme un problème numérique en aplanissant ses structures. Nous appelons cette dernière démarche l'*approche par attributs*. Intuitivement, dans l'approche par attributs, les structures complexes sont décomposées en structures atomiques/numériques (les attributs). Les relations que l'on souhaite satisfaire entre les structures complexes sont exprimées par des contraintes entre les variables contraintes représentant ces attributs numériques. L'approche par classe consiste au contraire à utiliser des variables contraintes dont les valeurs potentielles sont des objets quelconques du langage. Les relations entre ces objets sont exprimées directement par des contraintes. Cette méthodologie exige de pouvoir définir des contraintes impliquant des objets complexes, ce qui nécessite un langage pour leur définition et des techniques pour leur résolution. Ceci est rendu possible par la définition de classes de contraintes génériques pour la définition de relation sur les objets.

Nous avons illustré ces deux aspects de notre travail sur deux applications d'Intelligence Artificielle.

- La construction de grilles de mots croisés nous a permis de montrer l'utilisation de l'ensemble des possibilités du système BACKTALK pour la résolution d'un problème difficile et très spécifique. Pour sa résolution, nous avons défini une stratégie de résolution particulière utilisant un mécanisme de retour arrière intelligent. En outre, nous avons représenté par des contraintes un ensemble de connaissances sur les divers composants du problème : la topologie de la grille, les mots du dictionnaire et les croisements entre mots. L'application qui en résulte n'est pas aussi efficace que les meilleures approches procédurales pour les grilles simples (i.e. une grille est simple si elle est de petite dimension et que le dictionnaire comporte peu de mots). En revanche, notre approche donne d'excellents résultats comme la résolution en quelques secondes et sans échec d'une grille ayant résisté aux meilleures approches existantes.
- La résolution d'exercices d'harmonisation musicale, présentée à la fin de ce mémoire, a été mise en œuvre suivant l'approche par classes. Notre système présente plusieurs avantages essentiels par rapport aux approches précédentes, toutes fondées sur l'approche par attributs. Premièrement, les objets musicaux (e.g. les notes, les mélodies, les accords) utilisés dans la représentation du problème sont issus de classes prédéfinies n'ayant pas été implémentées dans cette perspective. Deuxièmement, la formulation du problème a été faite de façon purement déclarative avec une formalisation minimale. Plus précisément, le problème est simplement constitué de variables contraintes représentant les objets musicaux à calculer et d'un ensemble de contraintes représentant les règles musicales. Troisièmement, la formulation des règles musicales par des contraintes est purement *déclarative* au sens suivant : chaque règle musicale a été traduite par une simple formule logique écrite dans le langage des objets musicaux prédéfinis. Pour chaque règle nous avons utilisé les classes de contraintes génériques de BACKTALK sans qu'il ait été nécessaire de définir des contraintes spécifiques. Dernièrement, les performances obtenues sont très largement supérieures aux systèmes existants. Nous présentons une comparaison détaillée avec le système de Philippe Ballesta [Ballesta, 1994] qui est implémenté suivant l'approche attributs afin de mettre en évidence les différences entre les deux approches.

Ces deux applications représentent une amélioration nette des approches existantes, tant sur le plan des performances que concernant la formulation des problèmes. Ces résultats sont suffisamment encourageants pour envisager des applications plus ambitieuses qui restent encore hors de portée des ordinateurs. Il existe de nombreux jeux de mots croisés (au sens large) posant des difficultés colossales aux algorithmes. Un de ces jeux est la création de

grilles de “biffe tout”. Un biffe tout est constitué d’une grille contenant des lettres et d’une liste de mots. Le joueur doit localiser les mots de la liste dans la grille. Ces mots peuvent être écrits verticalement, horizontalement ou en diagonale. Pour chaque direction, ils peuvent apparaître à l’endroit ou bien à l’envers. La fabrication de ces grilles est extrêmement combinatoire. Nous donnons ici une petite grille calculée avec BACKTALK. S’il est facile d’obtenir, avec une représentation naïve du problème, des grilles de cette taille, il est difficile de calculer des grilles de taille intéressante.

M	R	A	L	A
A	O	B	A	R
E	R	A	C	S
R	I	B	K	O
S	P	A	I	N

Figure 57 Une grille de biffe tout contenant les mots: AAAI AI ALARM ARE ARS BRA CAR CARE ERA KO LAC LACK OK PAIN REAM RIB SCARE SON SPAIN et quelques autres

Dans le domaine de l’harmonisation, nos résultats nous permettent d’envisager sérieusement une production de bien meilleure qualité musicale. Le corpus de règles de Kémal Ebcioglu, par exemple, a été implémenté dans un environnement moderne de satisfaction de contraintes, mais selon une approche par attributs. Nous pensons qu’une implémentation selon les principes présentés dans cette thèse donnerait des résultats bien supérieurs. En outre, nous avons expérimenté l’introduction de quelques contraintes globales sur la conduite des voix. Cette expérience a été très limitée. L’accroissement des performances permet d’envisager l’expressions de règles de très haut niveau d’abstraction, afin d’améliorer la qualité musicale des solutions.

BIBLIOGRAPHIE

- [Agon, et al., 1998] C. Agon, G. Assayag, O. Delerue, and C. Rueda, "Objects, Time and Constraints in OpenMusic," presented at ICMC, Ann Arbor, MI, 1998.
- [Ballesta, 1994] P. Ballesta, "Contraintes et objets : clefs de voûte d'un outil d'aide à la composition musicales ?," . Le Mans: Université du Maine, 1994, pp. 198.
- [Baptiste and Pape, 1995] P. Baptiste and C. L. Pape, "Theoretical and Experimental Comparison of Constraint Propagation Techniques for Disjunctive Scheduling," presented at International Joint Conference on Artificial Intelligence, IJCAI'95, Montréal, Canada, 1995.
- [Barbar, et al., 1998] K. Barbar, A. Beurive, and M. Dessainte-Catherine, "Structures hiérarchiques pour la composition assistée par ordinateur," in *Recherche et applications en Informatique musicale, informatique musicale*, M. Chemillier and F. Pachet, Eds., Hermès ed. Paris, 1998, pp. 109-124.
- [Beldiceanu and Contejean, 1994] N. Beldiceanu and E. Contejean, "Introducing Global Constraints in CHIP," *Mathematical Computer modeling*, vol. 20(12), pp. 97-123, 1994.
- [Berghel, 1987] H. Berghel, "Crossword Compilation with Horn Clauses," *The Computer Journal*, vol. 30(2), pp. 182-188, 1987.
- [Berlandier, 1992] P. Berlandier, "Étude de mécanismes d'interprétation de contraintes et de leur intégration dans un système à base de connaissances," . Nice: Université de Nice Sophia-Antipolis, 1992, pp. 223.
- [Bessière, 1994] C. Bessière, "Arc Consistency and Arc Consistency Again," *Artificial Intelligence*, vol. 65, pp. 179-190, 1994.
- [Bessière and Régin, 1995] C. Bessière and J.-C. Régin, "Using Bidirectionality to Speed Up Arc Consistency Processing," in *Constraint Processing Selected Papers*, vol. 923, *LCNS*, G. Goos, J. Hartmanis, and J. v. Leeuwen, Eds., Springer-Verlag ed, 1995, pp. 157-169.
- [Bessière, et al., 1995] I. C. Bessière, E. Freuder, and J.-C. Régin, "Using Inference to Reduce Arc Consistency Computation," presented at International Joint Conference on Artificial Intelligence, IJCAI'95, Montréal, Canada, 1995.

- [Bistarelli, et al., 1995] S. Bistarelli, U. Montanari, and F. Rossi, "Constraint Solving over Semirings," presented at International Joint Conference on Artificial Intelligence, IJ-CAI'95, Montréal, Canada, 1995.
- [Bitsch, 1957] M. Bitsch, *Précis d'harmonie tonale*, Alphonse Leduc ed, 1957.
- [Bonnet and Rueda, 1993] A. Bonnet and C. Rueda, "Situation : manuel de référence," IR-CAM, Paris, Technical Report 1993.
- [Bonnet and Rueda, 1998] A. Bonnet and C. Rueda, "Un langage visuel basé sur les contraintes pour la composition musicale," in *Recherche et applications en Informatique musicale, informatique musicale*, M. Chemillier and F. Pachet, Eds., Hermès ed. Paris, 1998, pp. 65-74.
- [Borning, 1981] A. Borning, "The Programming Language Aspects of ThingLab, a Constraint-oriented Simulation Laboratory," *ACM Transactions on Programming Languages and Systems*, vol. 3(4), pp. 353-387, 1981.
- [Borning, et al., 1996] A. Borning, R. Anderson, and B. Freemann-Benson, "Indigo : A Local Propagation Algorithm for Inequality Constraints," presented at ACM Symposium on User Interface Software and Technology, 1996.
- [Borning and Freeman-Benson, 1995] A. Borning and B. Freeman-Benson, "The OTI Constraint Solver: A Constraint Library for Constructing Interactive Graphical User Interface," presented at Principles and Practice of Constraint Programming, CP'95, Cassis, France, 1995.
- [Boulay and Smith, 1986] J. B. H. D. Boulay and G. W. Smith, "The Generation of Cryptic Crossword Clues," *The Computer Journal*, vol. 29(3), pp. 282-284, 1986.
- [Caseau, 1991] Y. Caseau, "Abstract Interpretation of Constraints over an Order-Sorted Domain," presented at International Logic Programming Symposium, San Diego, CA, 1991.
- [Caseau, 1994] Y. Caseau, "Constraint Satisfaction with an Object-Oriented Knowledge Representation Language," *Journal of Applied Intelligence*, vol. 4, pp. 157-184, 1994.
- [Caseau and Laburthe, 1994] Y. Caseau and F. Laburthe, "Disjunctive Scheduling with Task Intervals," presented at 11th International Conference on Logic Programming, 1994.
- [Caseau and Laburthe, 1995] Y. Caseau and F. Laburthe, "Improving Branch and Bound for Job-shop Scheduling with Constraint Propagation," presented at Combinatorics and Computer Science, CCS'95, 1995.
- [Caseau and Laburthe, 1996] Y. Caseau and F. Laburthe, "CLAIRE: Combining Objects and Rules for Problem Solving," presented at JICSLP'96 workshop on multi-paradigm logic programming, TU berlin, 1996.
- [Codognet, 1995] P. Codognet, "Programmation logique avec contraintes : une introduction," *Technique et Science Informatique*, vol. 14, 1995.
- [Codognet, et al., 1986] P. Codognet, C. Codognet, and G. Filé, "Backtracking intelligent en programmation logique," presented at Séminaire sur la programmation en logique, Trégastel, France, 1986.
- [Codognet and Nardiello, 1994] P. Codognet and G. Nardiello, "Enhancing the Constraint-solving Power of clp(FD) by Means of Path-consistency Methods," in *Constraint Programming : Basics and Trends*, A. Podelski, Ed.: LCNS Springer-Verlag, 1994, pp. 39-61.
- [Colmerauer, 1990] A. Colmerauer, "An Introduction to PROLOG III," *CACM*, vol. 33, pp. 70-80, 1990.
- [Courtot, 1992] F. Courtot, "CARLA : acquisition et induction sur le matériau compositionnel," . Rennes, France: Université de Rennes I, 1992.
- [Cox, 1977] P. T. Cox, "Deduction Plans, a Graphical Proof Procedure for First-order Predicate Calculus," in *Department of Computer Science*. Waterloo, Canada: University of Waterloo, 1977.

- [Crawford, et al., 1996] J. D. Crawford, M. Ginsberg, E. M. Luks, and A. Roy, "Symmetry-Breaking Predicates for Search Problems," presented at Knowledge Representation, Cambridge, MA, 1996.
- [David, 1994] P. David, "Consistance de pivot et CSP fonctionnels. Une constance partielle pour des solutions globales," *Revue d'Intelligence Artificielle*, vol. 8, pp. 145-185, 1994.
- [Dellacherie, 1998] P. Dellacherie, "(Tentatives de) Modélisation et optimisation d'un harmoniseur automatique de chorals en PCL(FD)," in *D.E.A. I.A.A.* Caen: Université de Caen, 1998, pp. 21.
- [Deville and Hentenryck, 1991] Y. Deville and P. v. Hentenryck, "An Efficient Arc Consistency Algorithm for a Class of CSP Problems," presented at International Joint Conference on Artificial Intelligence, IJCAI'91, Sydney, Australia, 1991.
- [Diaz, 1995] D. Diaz, "Étude de la compilation des langages logiques de programmation par contraintes sur les domaines finis: le système clp(FD)," . Orléans: Université D'Orléans, 1995, pp. 270.
- [Dincbas, et al., 1990] M. Dincbas, H. Simonis, and P. V. Hentenryck, "Solving Large Combinatorial Problems in Logic Programming," *Journal of Logic Programming*, vol. 7 (1), 1990.
- [Ducourneau and Quinqueton, 1986] R. Ducourneau and J. Quinqueton, "YAFOOL : encore un langage à objets à base de frames," INRIA, Rocquencourt 72, 1986.
- [Ebcioglu, 1992a] K. Ebcioglu, "An Expert System for Harmonizing Chorales in the Style of J. S. Bach," in *Understanding Music with AI: Perspectives on Music Cognition*. Menlo Park, CA: AAAI Press, 1992a.
- [Ebcioglu, 1992b] K. Ebcioglu, "An Expert System for Harmonizing Four-part Chorales," *Computer Music Journal*, vol. 12, pp. 43-51, 1992b.
- [Fargier, et al., 1995] H. Fargier, D. Dubois, and H. Prade, "Problèmes de satisfaction de contraintes flexibles : une approche égalitariste," *Revue d'Intelligence Artificielle*, vol. 9(3), pp. 311-354, 1995.
- [Fayad and Schmidt, 1997] M. Fayad and D. Schmidt, "Object-Oriented Application Frameworks," *Communications of the ACM, Special Issue on Object-Oriented Application Framework*, vol. 40(10), 1997.
- [Feger, 1975] O. Feger, "A Program for the Construction of Crossword Puzzles," *Data Information*, vol. 17(5), pp. 189-195, 1975.
- [Ferté and Capelovici, 1975] R. L. Ferté and J. Capelovici, *Pratique des mots croisés*, Presses Universitaires de France, puf ed. Paris, France, 1975.
- [Freuder and Sabin, 1995] E. C. Freuder and D. Sabin, "Interchangeability Supports Abstraction and Reformulation for Constraint Satisfaction," presented at SARA, 1995.
- [Fron, 1994] A. Fron, *Programmation par contraintes*: Addison-Wesley France, 1994.
- [Gaschnig, 1977] J. Gaschnig, "A General Backtrack Algorithm that Eliminates most Redundant Tests," presented at IJCAI'77, Cambridge MA, 1977.
- [Gensel, 1998] J. Gensel, "Objets et contraintes," in *Langages et modèles à objets, Collection didactique*, INRIA, Ed. Rocquencourt: R. Ducournau, J. Euzenat, G. Masini, A. Napoli, 1998, pp. 257-289.
- [Georget and Codognet, 1998] Y. Georget and P. Codognet, "Compiling Semiring-based Constraints in clp(FD,S)," presented at Principles and Practice of Constraint Programming, CP'98, Pisa, Italy, 1998.
- [Ginsberg, 1993] M. Ginsberg, "Dynamic Backtracking," *Journal of Artificial Intelligence Research*, 1993.
- [Ginsberg, et al., 1990] M. L. Ginsberg, M. Frank, M. P. Halpin, and M. C. Torrance, "Search Lessons Learned from Crossword Puzzles," presented at AAAI'90, Boston, MA, 1990.

- [Glover, 1989] F. Glover, "Tabu Search Part I," *O.R.S.A. Journal on Computing*, vol. 1, pp. 190-206, 1989.
- [Goubier, 1997] T. Goubier, "Interface de modélisation d'interfaces sémantiques," . Brest, France: ENST de Bretagne, 1997.
- [Haralick and Elliott, 1980] R. Haralick and G. Elliott, "Increasing Tree Search Efficiency for Constraint Satisfaction Problems," *Artificial Intelligence*, vol. 14, pp. 263-313, 1980.
- [Harris, 1990] G. Harris, "Generation of Solution Sets for Unconstrained Crossword Puzzles," presented at Symposium on Applied Computry, 1990.
- [Hentenryck, 1989] P. V. Hentenryck, *Constraint Satisfaction in Logic Programming*. Cambridge, MA: MIT Press, 1989.
- [Hiller and Isaacson, 1959] L. Hiller and L. Isaacson, *Experimental Music*, McGraw-Hill Book Company ed. New York, 1959.
- [Hörnel and Dengenhardt, 1997] D. Hörnel and P. Dengenhardt, "A Neural Organist improvising baroque-style melodic variations," presented at International Conference on Computer Music, ICMC'97, Thessaloniki, GR, 1997.
- [ILOGSOLVER, 1996] ILOGSOLVER, "ILOG SOLVER 3.1 User's Manual," ILOG, Gentilly, France 1996 1996.
- [Ingalls, 1978] D. H. H. Ingalls, "The Smalltalk-76 Programming System Design and Implementation," , Tucson, AR, 1978.
- [Jaffar and Lassez, 1987] J. Jaffar and J.-L. Lassez, "Constraint Logic Programming," presented at Principle of Programming Languages, POPL'87, Munich, Germany, 1987.
- [Jeavons, et al., 1995] P. Jeavons, D. Cohen, and M. Gyssens, "A Unifying Framework for Tractable Constraints," presented at Principles and Practice of Constraint Programming, CP'95, Cassis, France, 1995.
- [Jensen, 1997] S. C. Jensen, "Design and Implementation of Crossword Compilation Programs Using Sequential Approaches," in *IMADA*. Odense: Odense University, 1997.
- [Johnson and Foote, 1988] R. Johnson and B. Foote, "Designing Reusable Classes," *Journal of Object-oriented Programming, JOOP*, vol. 1(2), pp. 22, 1988.
- [Kask and Dechter, 1995] K. Kask and R. Dechter, "GSAT and Local Consistency," presented at International Joint Conference on Artificial Intelligence, IJCAI'95, Montréal, Canada, 1995.
- [Kökeny, 1993] T. Kökeny, "CSPOO : un système à résolution de contraintes orienté objet," presented at 2ème Journées Représentations Par Objets, La Grande Motte, France, 1993.
- [Kökeny, 1994] T. Kökeny, "Yet Another Object-Oriented Constraint Resolution System (YAFCRS) : an Open Architecture Approach," presented at *TOOLS USA 94*, Santa Barbara, 1994.
- [Laburthe, 1998] F. Laburthe, "Contraintes et algorithmes en optimisation combinatoire," . Paris: Université Paris VII, 1998, pp. 177.
- [Laburthe and Caseau, 1996] F. Laburthe and Y. Caseau, "Introduction to the CLAIRE Programming Language," École Normale Supérieure, Paris, Technical Report LIENS - 96 - 15, 1996.
- [Laburthe and Caseau, 1998] F. Laburthe and Y. Caseau, "SALSLA : A Language for Search Algorithms," presented at Principles and Practice of Constraint Programming, CP'98, Pisa, Italy, 1998.
- [Lalonde and Gulik, 1988] W. Lalonde and M. V. Gulik, "Building a Backtracking Facility for Smalltalk without Kernel Support," presented at OOPSLA'88, 1988.
- [Laurière, 1976] J.-L. Laurière, "Un langage et un programme pour énoncer et résoudre des problèmes combinatoires," in *LAFORIA (LIP6)*. Paris: Université Paris 6, 1976, pp. 227.
- [Laurière, 1978] J.-L. Laurière, "A Language and a Program for Stating and Solving Combinatorial Problems," *Artificial intelligence*, vol. 10(1), pp. 29-127, 1978.

- [Laurière, 1996] J.-L. Laurière, "Programmation de contraintes ou programmation automatique ?," LAFORIA (LIP6), Paris, Technical Report 96-19, Juin 1996 1996.
- [Lesueur, et al., 1998] B. Lesueur, N. Revaux, G. Sunye, and M. Ziane, "Using the MétaGen Modeling and Development Environment in the FIBOF Esprit Project," presented at Automating the Object-oriented Software Development Workshop at ECOOP'98, Bruxelles, Belgique, 1998.
- [Liret, et al., 1997a] A. Liret, P. Roy, and F. Pachet, "Combining Formal Reasoning Techniques and CSP," presented at ERCIM Workshop on constraint Programming and Processing (in conjunction with CP'97), Linz, Austria, 1997a.
- [Liret, et al., 1997b] A. Liret, P. Roy, and F. Pachet, "Conception par objets d'un système pour combiner raisonnement formel et satisfaction de contraintes," presented at Journées PRC/GDR "programmation", Pôle Programmation par Objets, Rennes, France, 1997b.
- [Liret, et al., 1998] A. Liret, P. Roy, and F. Pachet, "Conception par objets d'un système pour combiner raisonnement formel et satisfaction de contraintes," presented at JFLA'98, Lac de Coma, Italy, 1998.
- [Mackworth, 1977] A. Mackworth, "Consistency in Networks of Relations," *Artificial Intelligence*, vol. 8, pp. 99-118, 1977.
- [Mazlack, 1976a] L. J. Mazlack, "Computer Construction of Crossword Puzzles Using Precedence Relationships," *Artificial Intelligence*, vol. 7(1), pp. 1-19, 1976a.
- [Mazlack, 1976b] L. J. Mazlack, "Machine Selection of Elements in Crossword Puzzles," *SIAM Journal of Computing*, vol. 5(1), pp. 67-72, 1976b.
- [Michel and Hentenryck, 1997] L. Michel and P. V. Hentenryck, "LOCALIZER : A Language for Local Search," presented at Principles and Practice of Constraint Programming, CP'97, Linz, Austria, 1997.
- [Minton, 1996] S. Minton, "Automatically Configuring Constraint Satisfaction Programs : A Case Study," *Constraints*, vol. 1, pp. 7-43, 1996.
- [Mohr and Henderson, 1986] R. Mohr and T. Henderson, "Arc and Path Consistency Revisited," *Artificial Intelligence*, vol. 28, pp. 225-233, 1986.
- [Montanari, 1974] U. Montanari, "Networks of Constraints: Fundamental Properties and Applications to Picture Processing," *Information Sciences*, vol. 7, pp. 95-132, 1974.
- [Nadel, 1988] B. A. Nadel, "Tree Search and Arc Consistency in Constraint Satisfaction Algorithms," *Search in Artificial Intelligence*, vol. Springer-Verlag, pp. 287-342, 1988.
- [Ovans, 1992] R. Ovans, "Efficient Music Composition via Consistency Techniques," Simon Fraser University, Burnaby, B.C., Canada, Technical Report CSS-IS TR 92-02, 1992 1992.
- [Ovans and Davison, 1992] R. Ovans and R. Davison, "An Interactive Constraint-based Expert Assistant for Music Composition," presented at 9th Canadian Conference on Artificial Intelligence, Vancouver, B.C., 1992.
- [Pachet, 1994] F. Pachet, "The MusES System: An Environment for Experimenting with Knowledge Representation Techniques in Tonal Harmony," presented at 1st Brazilian Symposium on Computer Music, SBC&M'94, Caxambu, Minas Gerais, Brazil, 1994.
- [Pachet and Delerue, 1998] F. Pachet and O. Delerue, "MidiSpace: A Temporal Constraint-based Music Spatializer," presented at The 6th ACM Multimedia Conference, ACM Multimedia'98, 1998.
- [Pachet and Roy, 1995a] F. Pachet and P. Roy, "Integrating Constraint Satisfaction Techniques with Complex Object Structures," presented at 15th Annual Conference of the British Computer Society Specialist Group on Expert Systems, Cambridge, GB, 1995a.
- [Pachet and Roy, 1995b] F. Pachet and P. Roy, "Mixing Constraints and Objects: a Case Study in Automatic Harmonization," presented at Technology of Object-Oriented Languages and Systems, TOOLS Europe'95, Versailles, France, 1995b.

- [Pachet and Roy, 1998] F. Pachet and P. Roy, "Reifying chords in automatic harmonization," presented at European Conference on Artificial Intelligence, ECAI'98, Workshop on Constraints for Artistic Applications, Brighton, England, 1998.
- [Perec, 1979] G. Perec, *Les mots croisés, précédés de considérations de l'auteur sur l'art et la manière de croiser les mots*, Mazarine ed. Paris, France, 1979.
- [Prosser, 1993a] P. Prosser, "Domain Filtering Can Degrade Intelligent Backtracking Search," presented at International Joint Conference on Artificial Intelligence, IJCAI'93, Chambéry, France, 1993a.
- [Prosser, 1993b] P. Prosser, "Hybrid Algorithms for the Constraint Satisfaction Problem," *Computational Intelligence*, vol. 9, pp. 268-299, 1993b.
- [Prosser, 1995] P. Prosser, "MAC-CBJ: Maintaining Arc Consistency with Conflict-Directed Backjumping," Department of Computer Science, University of Starthclyde, Technical Report 95-177, 1995 1995.
- [Puget, 1992] J.-F. Puget, "PECOS : a High Level Constraint Programming Language," presented at SPICIS, Singapore, 1992.
- [Puget, 1993] J.-F. Puget, "On The Satisfiability of Symmetrical Constraint Satisfaction Problems," presented at ISMIS, Norway, 1993.
- [Puget, 1994] J.-F. Puget, "Programmation sous contraintes en C++," presented at LMO'94, 1994.
- [Puget and Leconte, 1995] J.-F. Puget and M. Leconte, "Beyond the Glass Box: Constraints as Objects," presented at ILPS'95, Portland, Oregon, 1995.
- [Ramaux and Fontaine, 1996] N. Ramaux and D. Fontaine, "Recognizing Scenarios by Calculating a Proximity Index Between Constraint Graphs," presented at International Conference on Tools in Artificial Intelligence, ICTAI'96, Toulouse, France, 1996.
- [Rankin and Berghel, 1990] R. Rankin and H. Berghel, "A Proposed Standard for Measuring Crossword Compilation Efficiency," *The Computer Journal*, vol. 33(2), pp. 181-184, 1990.
- [Régis, 1994] J.-C. Régis, "A Filtering Algorithm for Constraints of Difference in CSPs," presented at AAAI'94, Seattle, WA, 1994.
- [Régis, 1996] J.-C. Régis, "Generalized Arc Consistency for Global Cardinality Constraints," presented at AAAI'96, Seattle, WA, 1996.
- [Roads, 1995] C. Roads, *the computer music tutorial*. London, England: The MIT Press, 1995.
- [Roy and Pachet, 1997a] P. Roy and F. Pachet, "Conception de Problèmes par Objets et Contraintes," presented at JFLA'97, Dolomieu, Isère, 1997a.
- [Roy and Pachet, 1997b] P. Roy and F. Pachet, "Reifying Constraint Satisfaction in Small-talk," *Journal of Object-Oriented Programming*, 1997b.
- [Roy and Pachet, 1998] P. Roy and F. Pachet, "The Framework Approach for Constraint Satisfaction," *ACM Computing Surveys Symposium*, 1998.
- [Roy, et al., 1997] P. Roy, J.-F. Perrot, and F. Pachet, "A Framework for Expressing Knowledge about Constraint Satisfaction Problems," presented at FLAIRS'97, Daytona Beach, FL, 1997.
- [Sannella, 1994] M. Sannella, "SkyBlue : A Multi-way Local Propagation Constraint Solver," presented at ACM Symposium on User Interface Software and Technology, 1994.
- [Schönberg, 1978] A. Schönberg, *Theory of Harmony*. Berkeley and Los Angeles, California: University of California Press, 1978.
- [Selman, et al., 1992] B. Selman, H. Levesque, and D. Mitchell, "A New Method for Solving Hard Satisfiability Problems," presented at AAAI'92, 1992.
- [Smith, et al., 1990] P. D. Smith, H. Berghel, and G. H. Harris, "Dynamic Crossword Slot Table Implementation," presented at Symposium on Applied Computry, 1990.

- [Smith and Steen, 1981] P. D. Smith and S. Y. Steen, "A Prototype Crossword Compiler," *The Computer Journal*, vol. 24(2), pp. 107-111, 1981.
- [Spring, et al., 1992] L. J. Spring, G. H. Harris, J. J. H. Forster, and H. Berghel, "A Proposed Benchmark for Testing Implementations of Crossword Puzzle Algorithms," presented at ACM/SIGAPP Symposium on Applied computing (vol. I): technological challenges of the 1990's, SAC '92, 1992.
- [Steels, 1986] L. Steels, "Learning the Craft of Musical Composition," presented at ICMC, 1986.
- [Tsang and Aitken, 1991] C. P. Tsang and M. Aitken, "Harmonizing Music as a Discipline of Constraint Logic Programming," presented at ICMC, Montréal, Canada, 1991.
- [Wallace, 1993] R. Wallace, "Conjunctive Width Heuristics for Maximal Constraint Satisfaction," presented at AAAI'93, Washington D.C., 1993.
- [Waltz, 1975] D. Waltz, "Understanding Line Drawing of Scenes with Shadows," in *The Psychology of Computer Vision*, P. Winston, Ed. Cambridge, MA: McGraw-Hill, 1975, pp. 19-91.
- [Wilson, 1989] J. M. Wilson, "Crossword Compilation Using Integer Programming," *The Computer Journal*, vol. 32(3), pp. 273-275, 1989.
- [Yi and Berghel, 1989] C. Yi and H. Berghel, "Crossword Compiler-Compilation," *The Computer ²Journal*, vol. 32(3), pp. 276-280, 1989.